CRASH COURSE PLAN — v4 (thêm Part 27–31)

NestJS Crash Course

> Lộ trình học chi tiết từng phần — kèm giải thích cơ chế hoạt động bên trong

Repo: crash-course / nestjs-crash-course

Bản audit: đối chiếu với code thật (node_modules) và docs.nestjs.com — Part P + Part 00–26, nhắm NestJS 11

Mục lục

P TypeScript cho người viết NestJS

00 Kiến trúc tổng thể & Request Lifecycle

01 Khởi tạo & cấu trúc dự án

02 Controllers & Routing

03 Providers, Services & Dependency Injection

04 Modules

05 DTO & Validation

06 Pipes & Exception Filters

07 Middleware, Guards, Interceptors

08 Kết nối Database với TypeORM

09 Quan hệ dữ liệu (Relations)

10 Authentication (JWT + Passport)

11 Testing (Jest)

12 Pagination

13 Transactions

14 Config & biến môi trường

15 Serialization — định dạng dữ liệu trả về

16 Chuẩn hoá REST API cho production

17 Authorization chuyên sâu: Roles & Policy

18 Queue & Jobs với BullMQ

19 Events: EventEmitter & TypeORM Subscribers

20 Cache & Lock

21 Task Scheduling (cron)

22 File Upload & Storage

23 Mail & Notifications

24 Kiến trúc dự án lớn

25 Production: build, deploy, quan sát hệ thống

26 Lộ trình master NestJS

27 Refresh token xoay vòng, cookie HttpOnly MỚI

28 Chuyển sang Postgres và chống race condition MỚI

29 WebSocket với gateway và xác thực MỚI

30 CI với GitHub Actions và sửa bộ test MỚI

31 Prisma: đọc hiểu và so sánh với TypeORM MỚI

Cách dùng tài liệu này: Bạn tự code 100% từng phần. Mỗi phần có mục "Việc cần làm" (task) — không có code giải sẵn để copy, chỉ có ví dụ cú pháp minh hoạ để hiểu cách viết. Khi xong một phần, quay lại chat để mình review và giải thích tiếp phần sau.

Bản v4 thêm gì: Part 27–31 gồm refresh token xoay vòng, chuyển sang Postgres và chống race condition, WebSocket có xác thực, CI bằng GitHub Actions, so sánh Prisma. Code lấy từ bản sao dự án của bạn đã chạy **14 test e2e trên Node 24 và Postgres thật**. **Lưu ý quan trọng khi rà lại:** máy đang dùng Node 20 thì `npm test` của dự án bị lỗi "Must use import to load ES Module", còn `npm run test:e2e` thiếu `moduleNameMapper`. Xem cách sửa ở Part 30.

Bản audit này khác gì bản gốc: mọi phần đã được đối chiếu lại với code THẬT đã chạy qua trong quá trình học (Part 1-9), sửa các chỗ sai do phiên bản package đổi khác (đặc biệt Part 08), bổ sung các bài học rút ra từ lỗi thực tế đã gặp (đánh dấu bằng khung màu xanh này), và thêm 2 phần mới: Pagination, Transactions.

Bản cập nhật lần 2 (đối chiếu docs.nestjs.com): toàn bộ nội dung đã được đối chiếu thêm 1 lần nữa với tài liệu chính thức docs.nestjs.com (thuật ngữ, API còn hiện hành hay đã đổi) — sửa 1 lỗi sai thực sự (Part 05/12, cấu hình `transform` mặc định), bổ sung cảnh báo về các trang docs đã tái cấu trúc (Part 08, Part 10), và thêm 2 mục "so sánh 2 cách" mới ở Part 06 và Part 10, nâng cấp mục so sánh ở Part 07 — theo đúng format đã dùng ở mục 8.4/13.4. Khi tài liệu chính thức và phát hiện từ code thật (node_modules) mâu thuẫn nhau, phát hiện từ code thật luôn được giữ nguyên vì đó là sự thật cho đúng phiên bản package đang cài.

Bản cập nhật lần 3 (mở rộng ngang Laravel): thêm Part P (TypeScript cho NestJS) và Part 14–26 (Config, Serialization, REST production, Authorization, Queue, Events, Cache, Scheduling, Upload, Mail, Kiến trúc, Production, Lộ trình), làm sâu Part 09/11/12/13, sửa mục 8.4 (Repository API làm được OR bằng `where` dạng mảng). Mọi lệnh `npm i` đã kiểm tra peerDependencies với **NestJS 11** — dự án đang dùng bản 11, trong khi npm `latest` và docs.nestjs.com đã chuyển sang NestJS 12 (ESM). Đừng chép đuôi `.js` trong import hay `await` ở top-level từ docs mới vào dự án này.

PART P — NỀN TẢNG NGÔN NGỮ

TypeScript cho người viết NestJS

> **Mục tiêu:** Nắm đúng phần TypeScript mà NestJS thực sự dựa vào. Hầu hết "phép màu" của Nest (DI tự inject, `ValidationPipe` biết DTO là class nào, `@InjectRepository`) đều là hệ quả của 1 sự thật: **type của TypeScript biến mất khi chạy, chỉ class, enum và lời gọi decorator còn lại**. Hiểu câu này thì phần lớn lỗi "code đúng kiểu mà chạy sai" sẽ tự có lời giải.

P.1 Mô hình cốt lõi: TypeScript chỉ tồn tại lúc compile

TypeScript = JavaScript + hệ thống kiểu. `tsc` (hoặc `nest build`, bên dưới cũng gọi `tsc`) làm 2 việc: (1) kiểm tra kiểu, báo lỗi; (2) **xoá sạch kiểu** để sinh ra JavaScript thuần trong `dist/`. Node.js chỉ chạy `dist/main.js` và không biết TypeScript là gì. Handbook chính thức ghi rõ: type annotation, type alias, interface, type assertion đều bị compiler xoá và không ảnh hưởng hành vi lúc chạy.

Cú pháp TypeScript	Còn lại sau compile?	Hệ quả trong NestJS
`class PostsService {}`	✅ Thành class JS thật	Dùng làm token cho DI, làm "metatype" cho `ValidationPipe`
`interface`, `type X = {...}`	❌ Biến mất	Không inject được bằng interface, không validate được DTO khai bằng interface (P.3)
Generic `Repository<Post>`	❌ Chỉ còn `Repository`	Vì thế mới cần `@InjectRepository(Post)` (P.4)
`enum Role { Admin = 'admin' }`	✅ Thành object JS	Dùng được với `@IsEnum(Role)` (P.8)
Union `'draft' | 'published'`	❌ Biến mất	Muốn validate phải liệt kê lại giá trị: `@IsIn([...])`
`private` / `protected` / `readonly`	❌ Chỉ kiểm tra lúc compile	Field `private` vẫn xuất hiện trong JSON response (P.2)
Decorator `@Get()`, `@IsString()`	✅ Thành lời gọi hàm	Gắn metadata lên class/method/field
Kiểu tham số của method/constructor có decorator	✅ Một phần — thành metadata `design:paramtypes`	Nền tảng của DI và `ValidationPipe` (P.5)

Đây là đoạn thật trong `dist/posts/posts.controller.js` của project — xem TypeScript "để lại" gì:

```
__param(0, (0, common_1.Body)()),
__metadata("design:paramtypes", [create_post_dto_1.CreatePostDto]),
...
__param(0, (0, common_1.Param)('id', common_1.ParseIntPipe)),
__metadata("design:paramtypes", [Number]),
```

`@Body() createPostDto: CreatePostDto` để lại đúng 1 thứ: tham chiếu tới **class** `CreatePostDto`. `ValidationPipe` đọc chính mảng này để biết cần `plainToInstance` thành class nào rồi chạy validate. Nếu `CreatePostDto` là interface, ở chỗ đó chỉ còn `Object` — và Pipe không còn gì để làm.

> **Bẫy "khai kiểu là xong":** `page: number` trong `PaginationQueryDto` KHÔNG biến chuỗi `"2"` từ query string thành số `2` — dòng đó biến mất sau compile. Thứ thật sự ép kiểu là `@Type(() => Number)` (một lời gọi hàm, còn sống lúc chạy) cộng với việc `ValidationPipe` chạy `class-transformer`. Tương tự, `configService.get<number>('PORT')` không biến `"3000"` thành số (Part 14.4). Quy tắc: **type chỉ là lời hứa với compiler, không phải lệnh chuyển đổi**.

P.2 Class, access modifier & parameter property

Dòng xuất hiện ở mọi Service/Controller trong project:

```
constructor(private readonly postsService: PostsService) {}
```

là **parameter property**: gắn modifier (`public` / `private` / `protected` / `readonly`) trước tham số constructor thì TypeScript tự khai báo field cùng tên và tự gán `this.postsService = postsService`. Viết đầy đủ tương đương:

```
export class PostsController {
  private readonly postsService: PostsService;

  constructor(postsService: PostsService) {
    this.postsService = postsService;
  }
}
```

Modifier	Ý nghĩa	Dùng trong Nest khi nào
`private`	Chỉ dùng bên trong class (kiểm tra lúc compile)	Mặc định cho dependency được inject — không ai bên ngoài nên gọi `controller.postsService`
`readonly`	Chỉ gán được trong constructor	Luôn kèm với dependency: chống lỡ tay gán đè `this.repo = ...`
`protected`	Class con dùng được	Khi viết class cha để kế thừa (vd custom Guard kế thừa `AuthGuard('jwt')`)
`public`	Mặc định nếu không ghi	Hiếm khi cần ghi tường minh
`#field` (private của JS)	Private thật, cả lúc chạy	Hiếm dùng trong Nest; không dùng được làm parameter property

> **Bẫy: `private` không che được dữ liệu khỏi JSON.** Handbook gọi `private` / `protected` của TypeScript là "soft private" — chỉ kiểm tra lúc compile, lúc chạy vẫn là property bình thường. Khai `private password: string` trong entity rồi `return user` thì `JSON.stringify` vẫn in ra `password`. Muốn ẩn field khỏi response phải dùng cơ chế chạy lúc runtime — Part 15.

Tham số KHÔNG có modifier thì không thành field: trong `JwtStrategy` của project, `constructor(config: ConfigService, private readonly usersService: UsersService)` — `config` chỉ dùng được bên trong constructor (để truyền cho `super()`), không có `this.config`. Đó là cố ý, không phải lỗi.

P.3 `interface` hay `class` hay `type` — vì sao DTO bắt buộc là class

Cả 3 đều mô tả "hình dạng" object lúc compile. Khác nhau ở chỗ **chỉ class còn sống lúc chạy**. Thử nghiệm thật với `tsc` 5.9.3 của project (bật `emitDecoratorMetadata`):

```
interface Logger { log(m: string): void }
class Repo {}
type Opts = { a: number };

@Injectable()
export class Svc {
  constructor(repo: Repo, logger: Logger, opts: Opts, p: Promise<Repo>, arr: Repo[]) {}
}
// Reflect.getMetadata('design:paramtypes', Svc)
// → [Repo, Object, Object, Promise, Array]
```

`Logger` (interface) và `Opts` (type) đều thành `Object` — Nest không thể biết phải inject cái gì, và `ValidationPipe` gặp metatype `Object` thì **bỏ qua validate hoàn toàn** (trong source `validation.pipe.js`, hàm `toValidate()` trả `false` khi metatype là `String`, `Boolean`, `Number`, `Array`, `Object`, `Buffer`, `Date`).

Dùng làm	`class`	`interface` / `type`
DTO nhận body/query (cần validate)	✅ Bắt buộc	❌ Validation bị bỏ qua âm thầm, `whitelist` cũng không lọc field lạ
Provider để inject	✅ Class chính là token	❌ Phải tự tạo token: `@Inject('LOGGER')`
Entity TypeORM	✅ Bắt buộc (decorator gắn lên class)	❌
Kiểu dữ liệu nội bộ, chỉ để compiler kiểm tra (`PaginationMeta`, `ApiResponse<T>`, `JwtPayload`)	Được, nhưng thừa	✅ Nên dùng — nhẹ, không sinh code

`type` hay `interface`? Handbook: khác biệt chính là interface "mở lại" được (declaration merging), type thì không; và gợi ý "dùng `interface` cho tới khi cần tính năng chỉ `type` có" — ví dụ union (`type Status = 'draft' | 'published'`) hay kiểu tổ hợp. Project đang làm đúng tinh thần này: `PaginationMeta` / `Paginated<T>` là interface, `JwtPayload` trong `jwt.strategy.ts` là `type`.

Bẫy âm thầm nhất: viết `@Body() dto: { title: string; body: string }` (object type inline) hoặc `@Body() dto: Partial<CreatePostDto>` — code compile sạch, request chạy được, nhưng **không có dòng validate nào chạy**, và `forbidNonWhitelisted` cũng không chặn field lạ. Không có lỗi nào báo cho bạn biết. Luôn là class có decorator.

P.4 Generics — `Repository<Post>`, `Promise<T>`, `Paginated<T>`

Generic là "tham số cho kiểu": viết 1 lần, dùng cho nhiều kiểu cụ thể. Project dùng khắp nơi:

```
export interface Paginated<T> {
  data: T[];
  meta: PaginationMeta;
}

async findAll(query: PaginationQueryDto): Promise<Paginated<PostEntity>> { ... }

private readonly postsRepository: Repository<PostEntity>;
this.postsRepository.findOneBy({ id });  // TS biết trả Promise<PostEntity | null>
```

Generic	Đọc là	Lợi ích
`Promise<PostEntity>`	"Một Promise, khi resolve sẽ ra PostEntity"	Quên `await` thì TS báo lỗi khi bạn truy cập `.title`
`Repository<Post>`	"Repository thao tác trên entity Post"	`where`, `order`, `relations` được gợi ý đúng tên cột, sai tên là lỗi compile
`Paginated<T>`	"Trang dữ liệu của bất kỳ T nào"	1 interface dùng chung cho Post, Product, Category
`TransformInterceptor<T>`	Interceptor bọc response kiểu T	Kiểu của `data` được giữ xuyên qua interceptor
`ConfigService<Env, true>`	ConfigService biết danh sách biến môi trường	`get('PORT', { infer: true })` tự suy ra kiểu (Part 14.4)

Vì sao cần `@InjectRepository(Post)`? Đây là hệ quả trực tiếp của P.1. Xem `dist/users/users.service.js` thật của project:

```
__param(0, (0, typeorm_1.InjectRepository)(user_enity_1.User)),
__metadata("design:paramtypes", [typeorm_2.Repository])
```

Metadata chỉ còn `Repository` — phần `<User>` đã bị xoá. Nếu chỉ dựa vào kiểu, Nest không phân biệt được `Repository<User>` với `Repository<Post>`. `@InjectRepository(User)` là decorator tham số, đưa class `User` (còn sống lúc chạy) vào làm token — "cần đúng repository của User".

P.5 Decorator & metadata — cơ chế bên dưới toàn bộ NestJS

Decorator là **một hàm** chạy lúc class được định nghĩa (khi file được `require`), không phải lúc request tới. Nó nhận class/method/tham số và gắn metadata bằng `Reflect.defineMetadata()` (thư viện `reflect-metadata`). Để có điều này, `tsconfig.json` của project bật 2 cờ:

```
{
  "compilerOptions": {
    "experimentalDecorators": true,
    "emitDecoratorMetadata": true
  }
}
```

Cờ / thư viện	Làm gì	Thiếu thì sao
`experimentalDecorators`	Dùng decorator kiểu "legacy" (stage 2) — kiểu duy nhất cho phép decorator trên **tham số** (`@Body()` , `@Param()` , `@InjectRepository()`)	TS dùng decorator chuẩn TC39 (từ TS 5.0), không có decorator tham số → Nest không chạy
`emitDecoratorMetadata`	Sinh `design:type` , `design:paramtypes` , `design:returntype` cho member có decorator	DI không biết inject gì ("Nest can't resolve dependencies"), `ValidationPipe` không biết DTO
`reflect-metadata`	Cung cấp `Reflect.defineMetadata` / `getMetadata`	Metadata không được lưu. Nest tự import sẵn — bạn hiếm khi phải import tay

Thứ tự chạy khi xếp chồng nhiều decorator (handbook): các biểu thức decorator được *tính* từ trên xuống, nhưng *áp dụng* từ dưới lên — như hàm lồng `f(g(x))` . Với decorator của `class-validator` thì thứ tự không quan trọng (chúng chỉ đăng ký rule), nhưng khi tự viết decorator gộp (`applyDecorators`) thì nên nhớ.

Bẫy TS1272 — vì sao project có dòng `import type { Request } from 'express'` : project bật đồng thời `isolatedModules` và `emitDecoratorMetadata` . Khi một kiểu chỉ là type (interface `Request` của Express) xuất hiện trong chữ ký method có decorator, phải import bằng `import type` . Viết `import { Request } from 'express'` ở `auth.controller.ts` , `tsc` báo đúng nguyên văn (đã chạy thử): `error TS1272: A type referenced in a decorated signature must be imported with 'import type' or a namespace import when 'isolatedModules' and 'emitDecoratorMetadata' are enabled.` Sửa: `import type { Request } from 'express'` — Nest sẽ nhận metadata `Object` cho tham số đó, không sao vì `@Req()` không cần validate.

Đã đối chiếu typescriptlang.org + chạy thử: Handbook "Decorators" ghi rõ đây là "experimental stage 2 decorators implementation", bản chuẩn stage 3 có từ TS 5.0; ghi chú phát hành TS 5.0 xác nhận decorator chuẩn "not compatible with `--emitDecoratorMetadata` " và "does not allow decorating parameters", đồng thời `--experimentalDecorators` "will continue to exist for the foreseeable future". Chạy thử với `typescript` **5.9.3** (bản đang cài) và cả **7.0.2** (bản mới nhất trên npm, compiler native): cả hai vẫn sinh `design:paramtypes` đúng như trên khi bật 2 cờ này.

P.6 async/await & Promise — các bẫy hay gặp trong Service

Mọi thao tác DB của TypeORM đều trả `Promise` . Part 08.3 đã có quy tắc "khi nào cần `async/await` " và bẫy quên `await` ở Controller. Dưới đây là những bẫy còn lại, đều đã chạy thử trên Node 24:

Viết thế này	Chuyện gì xảy ra	Viết đúng

`try { return this.repo.save(x); } catch { ... }`	Lỗi DB **không bị catch** — Promise được trả ra ngoài trước khi reject, `catch` đã qua rồi	`try { return await this.repo.save(x); } catch { ... }`
`ids.forEach(async (id) => { await this.repo.delete(id); })`	`forEach` không chờ callback async — hàm trả về khi chưa xoá xong cái nào	`for (const id of ids) { await ... }` (tuần tự) hoặc `await Promise.all(ids.map(...))` (song song)
`const post = this.repo.findOneBy({ id }); if (!post) ...`	`post` là Promise, luôn "truthy" — check `null` không bao giờ đúng	Thêm `await`. TS 5.9 đã bắt được: `TS2801: This condition will always return true since this 'Promise<...>' is always defined.`
`seed();` ở cuối file (không await, không `.catch`)	"Floating promise" — nếu reject ngoài `try`, thành unhandled rejection	`seed().catch(...)` như `main.ts` đang làm với `bootstrap()`, hoặc `void seed()` nếu chắc chắn đã tự bắt lỗi bên trong
2 query độc lập viết nối tiếp `await a(); await b();`	Chạy tuần tự, chậm gấp đôi	`const [x, y] = await Promise.all([a(), b()]);` — chỉ khi 2 việc thật sự độc lập

Lưới an toàn đã có sẵn trong project: `eslint.config.mjs` bật `'@typescript-eslint/no-floating-promises': 'warn'` (kèm bộ `recommendedTypeChecked`). Chạy `npm run lint` sẽ chỉ ra các Promise bị bỏ rơi. Đừng tắt rule này — nó bắt đúng loại bug làm sập server ở Part 08.3.

P.7 Null safety: `strictNullChecks`, `?`, `!` — và cấu hình thật của project

`tsconfig.json` của project **không** bật `strict`, mà bật lẻ từng cờ: `strictNullChecks: true`, `noImplicitAny: false`, `strictBindCallApply: false`. Hệ quả cụ thể:

Cờ	Giá trị trong project	Nghĩa là
`strictNullChecks`	✅ bật	`findOneBy()` trả `User | null` — dùng `user.email` khi chưa check null là lỗi compile `TS18047: 'user' is possibly 'null'`. Đây là lý do mọi `findOne` trong project đều có `if (!post) throw new NotFoundException()`
`strictPropertyInitialization`	❌ tắt (vì `strict` không bật)	Entity/DTO khai `id: number;` không cần giá trị khởi tạo mà không báo lỗi
`noImplicitAny`	❌ tắt	Tham số không khai kiểu thành `any` âm thầm — nên tự khai kiểu dù không bị bắt

```ts
export class ProductQueryDto extends PaginationQueryDto {
  search?: string;        // ? = có thể không có: kiểu là string | undefined
}

export class User {
  id!: number;            // ! = "definite assignment": tôi hứa TypeORM sẽ gán
}

const port = process.env.PORT ?? 3000;      // ?? chỉ thay khi null/undefined
const label = product.description ?? '';
const price = dto.price || 100;              // SAI nếu price = 0 hợp lệ: || thay cả 0 và ''
```

Hai dấu `!` khác nhau: `id!: number` (sau tên field, definite assignment) nói "field này sẽ được gán ở nơi khác" — hợp lý cho entity vì TypeORM gán. `user!.email` (sau biểu thức, non-null assertion) nói "tin tôi, không null đâu" — handbook cảnh báo nó "doesn't change the runtime behavior", sai là `TypeError` lúc chạy. Với kết quả `findOneBy()`, luôn `if (!user) throw ...` thay vì `user!`.

Đã đối chiếu typescriptlang.org/tsconfig + chạy thử 2 phiên bản compiler: `strictPropertyInitialization` "requires `strictNullChecks`" và bật theo `strict`. Với **typescript 5.9.3** (đang cài, `^5.7.3` trong package.json), class `export class E { id: number; }` compile sạch với tsconfig của project. Với **typescript 7.0.2** (mới nhất trên npm), cùng file báo `TS2564: Property 'id' has no initializer...` vì từ TS 6.0 `strict` mặc định là `true`. Khi nào nâng TypeScript lên 6/7: hoặc thêm `!` vào field của entity/DTO, hoặc khai tường minh `"strictPropertyInitialization": false`. Ngoài ra, vì `target` là `ES2023`, `useDefineForClassFields` mặc định bật: field khai báo mà chưa gán vẫn thành own property có giá trị `undefined` (`Object.keys(new Entity())` trả về đủ tên field).

P.8 `enum` hay union string type — khi nào dùng cái nào

Sắp tới project sẽ có trạng thái kiểu `Post.status` (draft/published) hay `User.role` (admin/user). TypeScript cho 2 cách mô tả "một trong vài giá trị cố định", và người mới hay chọn theo thói quen. Câu hỏi quyết định: **giá trị này có cần tồn tại lúc chạy (để validate, để TypeORM tạo cột) hay chỉ cần compiler kiểm tra?**

Cách 1 — `enum` (có mặt lúc chạy)

```ts
export enum PostStatus {
  Draft = 'draft',
  Published = 'published',
}

@Column({ type: 'simple-enum', enum: PostStatus, default: PostStatus.Draft })
status: PostStatus;

@IsEnum(PostStatus)
status: PostStatus;
```

Enum compile thành object JS thật (`{ Draft: 'draft', Published: 'published' }`), nên `@IsEnum()` và option `enum` của TypeORM đọc được danh sách giá trị. Luôn dùng **string enum** (gán chuỗi tường minh)

— handbook nhấn mạnh string enum "serialize well": trong DB và JSON bạn thấy `"draft"` chứ không phải số `0` khó hiểu.

Cách 2 — union string type (chỉ có lúc compile)

```
export type SortOrder = 'ASC' | 'DESC';

@IsIn(['ASC', 'DESC'])
order: SortOrder = 'DESC';
```

Gọn, không sinh code, TypeORM tự dùng kiểu này ở nhiều chỗ (`order: { id: 'DESC' }`). Nhược điểm: phải liệt kê lại giá trị cho `@IsIn` vì type đã biến mất lúc chạy. Có thể gộp 2 ưu điểm bằng object `as const` (handbook gợi ý "you may not need an enum when an object with `as const` could suffice").

> **Đã đối chiếu:** typescriptlang.org/docs/handbook/enums ("Enums are real objects that exist at runtime"; `isolatedModules` "fundamentally incompatible with ambient const enums" — project bật `isolatedModules` , nên **tránh** `const enum`). `'simple-enum'` có trong `ColumnTypes.d.ts` của typeorm 1.1.1 và được xử lý ở `AbstractSqliteDriver` (dùng được với better-sqlite3); `IsEnum` / `IsIn` có trong class-validator 0.15.1.

Nhu cầu	Cách 1: `enum`	Cách 2: union type
Lưu vào cột DB, cần TypeORM biết danh sách giá trị	✅ `enum: PostStatus`	❌ Phải tự viết mảng
Validate input bằng class-validator	✅ `@IsEnum(PostStatus)`	Được, nhưng lặp giá trị trong `@IsIn([...])`
Giá trị chỉ dùng nội bộ, không đi qua API/DB	Thừa	✅ Gọn, không sinh code
Dùng với `isolatedModules`	✅ enum thường; ❌ tránh `const enum`	✅

Quy tắc thực dụng: trạng thái nghiệp vụ lưu DB và đi qua API (status, role) → string `enum` . Tập giá trị nhỏ chỉ dùng trong code hoặc tham số query đơn giản (`'ASC' | 'DESC'`) → union type + `@IsIn` .

P.9 Utility types của TS vs `PartialType` của Nest — giống tên, khác bản chất

Utility type (TS, compile-time)	Kết quả	Tương ứng của `@nestjs/mapped-types` (runtime)
`Partial<T>`	Mọi field thành optional	`PartialType(Dto)`
`Pick<T, 'a' | 'b'>`	Chỉ giữ field a, b	`PickType(Dto, ['a', 'b'] as const)`
`Omit<T, 'a'>`	Bỏ field a	`OmitType(Dto, ['a'] as const)`
`A & B`	Gộp 2 kiểu	`IntersectionType(A, B)`
`Record<K, V>` , `ReturnType<F>`	Object key K giá trị V; kiểu trả về của hàm	Không có — chỉ dùng cho kiểu nội bộ (`ConfigType` ở Part 14 là một biến thể của `ReturnType`)

Khác biệt then chốt: `Partial<CreatePostDto>` chỉ là kiểu — biến mất sau compile, không có class nào để `ValidationPipe` dùng (P.3). `PartialType(CreatePostDto)` là **hàm chạy lúc runtime, trả về một class mới**. Mở source `node_modules/@nestjs/mapped-types/dist/partial-type.helper.js` (bản 12.0.0) thấy nó làm đúng 3 việc: tạo `class PartialClassType`, gọi `inheritValidationMetadata()` (chép rule class-validator), `inheritTransformationMetadata()` (chép `@Type()` / `@Transform()` của class-transformer), rồi gắn thêm `@IsOptional()` cho mọi field. Nhờ vậy `UpdatePostDto` của project vẫn giữ `@IsString()` / `@IsNotEmpty()` — chỉ là không bắt buộc có mặt.

```
export class UpdatePostDto extends PartialType(CreatePostDto) {}                // đúng
export class RegisterResponse extends OmitType(RegisterDto, ['password'] as const) {}

update(@Body() dto: Partial<CreatePostDto>) {}   // SAI: không validate gì cả
```

Option ít người biết: `PartialType(Dto, { skipNullProperties: false })` — mặc định field optional chấp nhận cả `null` lẫn `undefined` (vì `@IsOptional()` bỏ qua cả hai). Truyền `skipNullProperties: false` thì source chuyển sang `ValidateIf((_, value) => value !== undefined)` — chỉ bỏ qua khi `undefined` — gửi `"title": null` sẽ bị validate và báo lỗi. Hữu ích khi cột DB là NOT NULL.

P.10 Module system: project này là CommonJS, nhưng đang nạp package ESM

JavaScript có 2 hệ module: **CommonJS** (`require` / `module.exports` , truyền thống của Node) và **ESM** (`import` / `export` thật). Viết `import` trong file `.ts` không có nghĩa là đang dùng ESM — quyết định nằm ở cấu hình:

Nơi quyết định	Giá trị trong project	Hệ quả
`package.json` → `"type"`	Không khai	Với `module: nodenext` , TS coi mọi file `.ts` là CommonJS → `dist/main.js` toàn `require()` (mở ra là thấy)
`tsconfig.json` → `module` / `moduleResolution`	`nodenext`	TS mô phỏng đúng luật module của Node bản mới
`tsconfig.json` → `baseUrl: "./"`	Có	Cho phép `import ... from 'src/users/users.service'` ; Jest cần thêm `moduleNameMapper` `^src/(.*)$` (đã có trong package.json)
`esModuleInterop`	`true`	Import được package CommonJS kiểu default; project dùng `import * as bcrypt from 'bcrypt'` — vẫn đúng

Điểm đặc biệt của project này: `@nestjs/jwt` 12.0.2, `@nestjs/passport` 12.0.0, `@nestjs/typeorm` 12.0.1, `@nestjs/mapped-types` 12.0.0 đều là **ESM-only** (`"type": "module"` , không có bản build CommonJS) — trong khi code của bạn compile ra CommonJS và `require()` chúng. Việc này chạy được nhờ tính năng `require(esm)` của Node: bỏ cờ từ v22.12.0 / v20.19.0, và máy đang chạy Node v24.16.0. Jest thì dùng module loader riêng chứ không dùng `require` của Node, nên script test phải có `NODE_OPTIONS=--experimental-vm-modules` : bỏ biến này đi, chạy `npx jest` báo ngay `SyntaxError: Unexpected token 'export'` tại `node_modules/@nestjs/typeorm/dist/index.js` (đã chạy thử).

Bẫy khi đổi môi trường: (1) Chạy project trên Node < 20.19 (hoặc 22.0–22.11) → lỗi `ERR_REQUIRE_ESM` ngay lúc khởi động. Nếu CI/Docker báo lỗi này mà máy bạn chạy bình thường, kiểm tra version Node trước tiên. (2) `require(esm)` không nạp được module có top-level `await` (`ERR_REQUIRE_ASYNC_MODULE`). (3) Đừng tự thêm `"type": "module"` vào package.json "cho hiện đại" — toàn bộ import tương đối sẽ phải có đuôi `.js` , `__dirname` biến mất, Jest phải cấu hình lại. (4) docs.nestjs.com hiện hiển thị NestJS 12 với code mẫu kiểu ESM (import có đuôi `.js` , `await bootstrap()` ở top-level). Course này nhắm **NestJS 11 + CommonJS** như project đang cài (`@nestjs/common` 11.2.3) — chép nguyên các chi tiết đó vào project sẽ lỗi: top-level `await` báo `TS1309: The current file is a CommonJS module and cannot use 'await' at the top level` (đã chạy thử), còn đuôi `.js` chỉ thừa và làm code lệch khỏi phần còn lại của project.

Đã đối chiếu: typescriptlang.org/docs/handbook/modules/reference — file `.ts` "are ES modules if the nearest ancestor package.json file contains `"type": "module"` , otherwise CommonJS modules"; ở `nodenext` , " `require` can reference an ES module" để phản ánh Node.js v22.12.0+. nodejs.org/api/modules ("Loading ECMAScript modules using require()"): hết cần cờ từ v23.0.0 / v22.12.0 / v20.19.0, hết "experimental" từ v25.4.0. Loại module của từng package kiểm tra trực tiếp trong `node_modules/*/package.json` . **Cảnh báo phiên bản:** chạy thử typescript 7.0.2 với tsconfig có `baseUrl` → `error TS5102: Option 'baseUrl' has been removed` (TS 6.0 đánh dấu deprecated, TS 7 bỏ hẳn; compiler gợi ý thay bằng `"paths": {"*": ["./*"]}`). Với TS 5.9.3 hiện tại chưa ảnh hưởng.

P.11 Bảng tra nhanh: TypeScript ↔ NestJS dùng nó vào việc gì

Tính năng TypeScript	NestJS / thư viện dùng nó để	Ví dụ trong project
Class	Controller, Provider, Module, DTO, Entity — class là token DI và là metatype cho Pipe	`PostsService` , `CreatePostDto` , `Post`
Parameter property	Constructor injection gọn 1 dòng	`constructor(private readonly authService: AuthService)`
Decorator (legacy)	Khai báo route, DI, validation, mapping ORM	`@Get()` , `@Injectable()` , `@IsEmail()` , `@Column()`
`emitDecoratorMetadata`	Nest đọc kiểu tham số constructor/handler lúc chạy	`design:paramtypes` trong `dist/`
Generic	Type-safe cho repository, Promise, response wrapper	`Repository<Product>` , `Paginated<T>`
Interface / type	Kiểu nội bộ không cần tồn tại lúc chạy	`PaginationMeta` , `JwtPayload`
Type guard (`value is X`)	Thu hẹp kiểu sau khi kiểm tra lúc chạy	`isPaginated(value): value is Paginated<unknown>`
`strictNullChecks`	Bắt quên kiểm tra kết quả `findOneBy()`	`Promise<User | null>`
String enum	Giá trị cố định, validate + lưu DB	(sẽ dùng cho status/role)
`import type`	Import chỉ-kiểu, bắt buộc trong chữ ký có decorator khi bật `isolatedModules`	`import type { Request } from 'express'`

async/await	Mọi I/O: DB, bcrypt, JWT	`await bcrypt.hash()`, `await this.jwtService.signAsync()`

Việc cần làm:

1. Chạy `npm run build`, mở `dist/posts/posts.controller.js` và `dist/posts/posts.service.js`: tìm mọi dòng `design:paramtypes`, giải thích bằng lời vì sao `PostsService` chỉ có `Repository` mà không có `<Post>`, và vì sao chỗ đó cần `@InjectRepository(PostEntity)`
2. Thí nghiệm P.3: tạm đổi `create(@Body() dto: CreatePostDto)` thành `create(@Body() dto: { title: string; body: string })`, gửi body `{ "title": 123, "hack": true }` — xác nhận request lọt qua không lỗi 400. Hoàn tác lại
3. Thí nghiệm P.5: trong `auth.controller.ts`, đổi `import type { Request }` thành `import { Request }`, chạy `npx tsc --noEmit`, đọc lỗi TS1272, rồi hoàn tác
4. Chạy `npm run lint`, tìm cảnh báo `no-floating-promises` (gợi ý: `seed.ts`), sửa theo bảng P.6
5. Thêm `status` cho `Post` bằng string enum `PostStatus` (cột `simple-enum`, default `draft`) và `@IsEnum(PostStatus)` + `@IsOptional()` trong `CreatePostDto` — xác nhận `UpdatePostDto` tự nhận luôn rule mới nhờ `PartialType`
6. Trả lời được: nếu `npm i typescript@7` thì 2 thứ gì trong `tsconfig.json`/entity của project sẽ báo lỗi (P.7 và P.10), và sửa thế nào

PART 00 — NỀN TẢNG

Kiến trúc tổng thể & Request Lifecycle

Đây là phần quan trọng nhất để hiểu "mọi thứ hoạt động thế nào" trước khi đi vào từng phần nhỏ. NestJS không phải một framework hoàn toàn mới — nó là một lớp kiến trúc (architecture layer) xây trên nền HTTP server có sẵn (mặc định là **Express**, có thể đổi sang **Fastify**). NestJS lấy cảm hứng từ Angular: dùng **TypeScript decorators**, **Dependency Injection (DI)**, và tổ chức code theo **Module**.

1. Decorator & Reflection Metadata — "phép màu" đằng sau NestJS

Toàn bộ NestJS hoạt động dựa trên một cơ chế của TypeScript gọi là **decorator** kết hợp thư viện `reflect-metadata`. Khi bạn viết:

```
@Controller('posts')
export class PostsController { }
```

Decorator `@Controller('posts')` thực chất là một hàm chạy tại thời điểm class được định nghĩa (compile/load time). Nó không "chạy logic nghiệp vụ" — nó chỉ **gắn metadata** (dữ liệu mô tả) vào class đó, kiểu như dán nhãn: "class này là một Controller, prefix route là `/posts`". NestJS lưu metadata này bằng `Reflect.defineMetadata()`.

Khi ứng dụng khởi động (`bootstrap()`), NestJS quét toàn bộ metadata đã được gắn (dùng `Reflect.getMetadata()`) để biết: class nào là Controller, class nào là Provider, route nào ứng với method nào, cần inject dependency gì vào constructor... Đây chính là lý do NestJS gọi là "declarative" — bạn *khai báo* ý định bằng decorator, framework tự đọc và ráp nối lại.

> **Bài học thực tế:** vì decorator/class chỉ là 1 identifier bình thường trong TypeScript, việc đặt tên Entity/DTO trùng tên với 1 decorator import từ `@nestjs/common` (ví dụ đặt Entity tên `Post` , trùng decorator `@Post()`) là cái bẫy rất dễ gặp — xem cảnh báo chi tiết ở Part 08.

2. Module Graph & Dependency Injection Container (IoC Container)

Khi app khởi động, NestJS xây dựng một **cây module** (module tree) bắt đầu từ `AppModule` (root module). Với mỗi module, nó đọc metadata trong `@Module({...})` :

- `imports` : các module khác mà module này cần dùng
- `controllers` : các controller thuộc module này
- `providers` : các service/class sẽ được đăng ký vào DI container
- `exports` : các provider mà module này "cho phép" module khác dùng lại

NestJS tạo ra một **IoC Container** (Inversion of Control) — một "kho chứa instance". Mỗi provider (mặc định là **singleton**) chỉ được khởi tạo **một lần duy nhất** trong toàn bộ vòng đời app. Khi class A cần class B trong constructor:

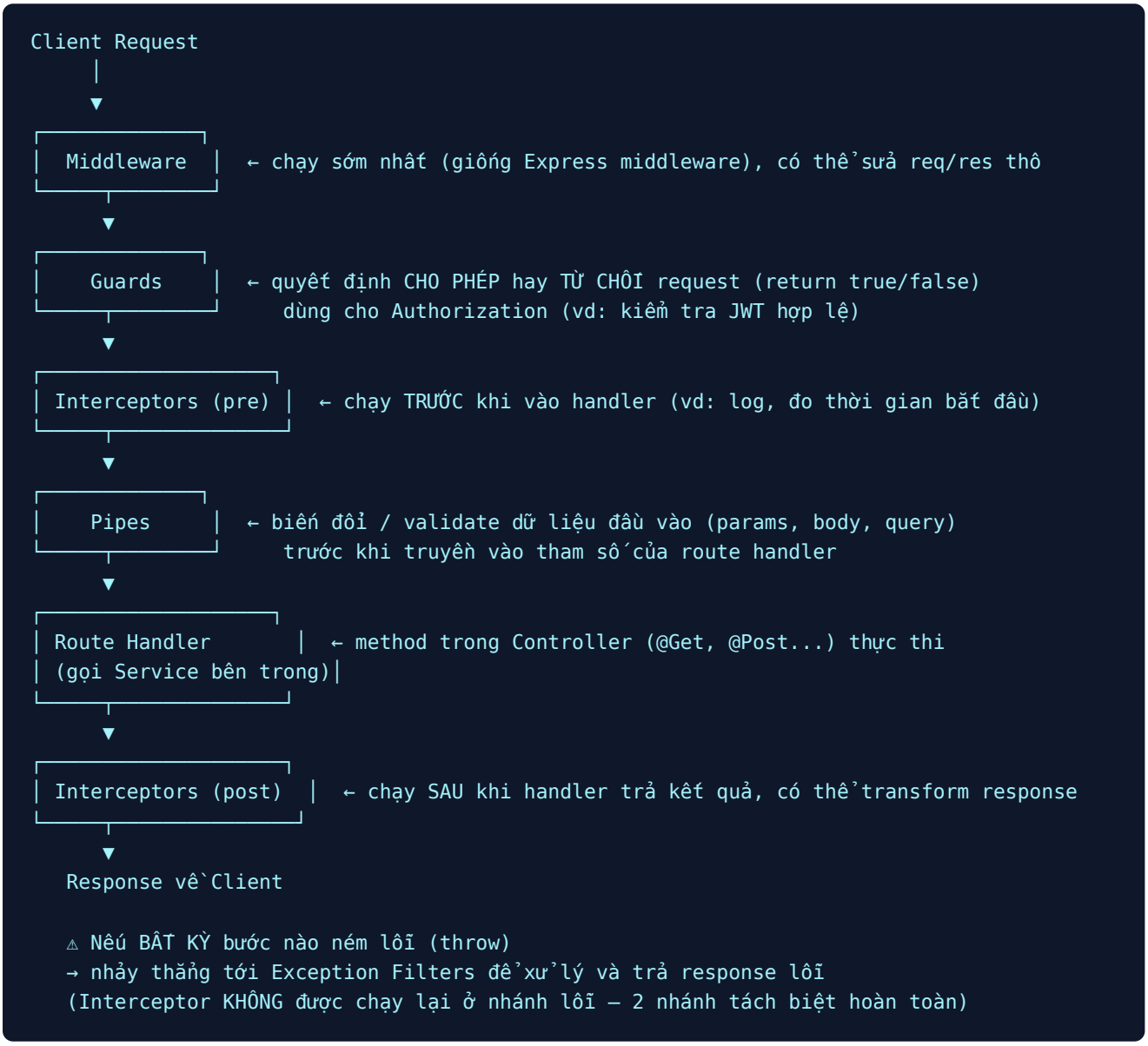
```
@Injectable()
export class PostsController {
  constructor(private readonly postsService: PostsService) {}
}
```

NestJS nhìn vào **type** của tham số constructor (nhờ TypeScript compile ra metadata kiểu dữ liệu), tra trong container xem đã có instance của `PostsService` chưa — nếu có thì tái sử dụng, nếu chưa thì tạo mới rồi lưu lại. Đây gọi là **constructor-based injection**. Bạn không bao giờ tự viết `new PostsService()` — NestJS làm việc đó cho bạn. Lợi ích: dễ test (mock được), dễ thay thế implementation, giảm coupling giữa các class.

> **Encapsulation theo module:** Provider trong module A *không* tự động dùng được ở module B — trừ khi A `export` nó ra và B `import` module A vào. Đây là ranh giới đóng gói (giống package-private), giúp kiến trúc lớn không bị rối.

3. Request Lifecycle — thứ tự một request đi qua hệ thống

Đây là sơ đồ quan trọng nhất cần thuộc lòng. Khi một HTTP request đến, nó đi qua các lớp theo đúng thứ tự sau:



Sơ đồ này trả lời hầu hết câu hỏi "tại sao": Guard chạy trước Controller để chặn truy cập trái phép sớm; Pipe chạy trước handler để data luôn sạch/hợp lệ; Interceptor "bọc" được cả input lẫn output vì nó chạy cả trước và sau.

4. Provider Scope (phạm vi sống của instance)

99% trường hợp trong crash course này bạn sẽ dùng Singleton (mặc định) — không cần khai báo gì thêm. Bảng dưới để biết còn 2 lựa chọn khác khi cần:

Scope	Mặc định?	Giải thích
`DEFAULT` (Singleton)	✅ Có	Một instance duy nhất, dùng chung cho toàn bộ app suốt vòng đời server
`REQUEST`	Không	Tạo instance mới cho **mỗi request** — dùng khi cần lưu state riêng theo request (vd: multi-tenant)
`TRANSIENT`	Không	Tạo instance mới **mỗi lần được inject** — hiếm dùng

---

PART 01

Khởi tạo & cấu trúc dự án

> **Mục tiêu:** Hiểu Nest CLI tạo ra những gì, và luồng khởi động (bootstrap) của một app NestJS.

1.1 Cài đặt & tạo project

```
npm i -g @nestjs/cli
nest new nestjs-crash-course
```

Nest CLI dùng schematics (khuôn mẫu code) để sinh ra cấu trúc chuẩn, tương tự `create-react-app` hay `vue create`.

1.2 Cấu trúc thư mục mặc định

File	Vai trò
`src/main.ts`	Entry point — nơi app thực sự khởi động
`src/app.module.ts`	Root Module — gốc của cây module
`src/app.controller.ts`	Controller mẫu, xử lý route `/`
`src/app.service.ts`	Service mẫu, chứa logic nghiệp vụ
`nest-cli.json`	Cấu hình Nest CLI (build, schematics...)
`tsconfig.json`	Cấu hình TypeScript — chú ý `experimentalDecorators` và `emitDecoratorMetadata` phải bật (đây là 2 cờ TS bắt buộc để decorator + DI hoạt động)

Lưu ý về `tsconfig.json`: nếu không khai báo tường minh mảng `"lib"`, TypeScript sẽ tự nạp thêm cả bộ type của **DOM** (dành cho browser) bên cạnh target ES bạn chọn. Điều này gây ra 1 bug rất khó phát hiện ở Part 09 (biến toàn cục `Comment` của DOM trùng tên với Entity tự viết) — cứ để mặc định lúc học, nhưng nhớ lại phần này khi gặp lỗi "tsc sạch nhưng chạy vẫn lỗi".

1.3 Giải phẫu main.ts

```
async function bootstrap() {
  const app = await NestFactory.create(AppModule);
  await app.listen(process.env.PORT ?? 3000);
}
bootstrap();
```

`NestFactory.create(AppModule)` chính là bước NestJS đọc `AppModule`, duyệt toàn bộ `imports` đệ quy, đăng ký hết Controllers/Providers vào IoC container, resolve toàn bộ dependency injection — tất cả xảy ra **trước** khi `listen()` được gọi. Nếu có lỗi thiếu dependency hoặc circular dependency, app sẽ crash ngay tại bước `create()` này (rất hữu ích để debug).

Việc cần làm:

1. Tạo project bằng `nest new nestjs-crash-course`
2. Chạy `npm run start:dev`, mở `http://localhost:3000`, xác nhận thấy "Hello World!"
3. Đọc kỹ 4 file: main.ts, app.module.ts, app.controller.ts, app.service.ts — thử tự vẽ lại sơ đồ ai gọi ai
4. Thử đổi cổng bằng biến môi trường `PORT=4000 npm run start:dev`

PART 02

Controllers & Routing

Mục tiêu: Hiểu cách NestJS map một HTTP request tới đúng method trong Controller, và cách lấy dữ liệu từ request.

2.1 Cơ chế map route

`@Controller('posts')` đăng ký prefix `/posts`. Mỗi method bên trong gắn thêm decorator HTTP method:

```
@Controller('posts')
export class PostsController {
  @Get()          // GET /posts
  @Get(':id')     // GET /posts/:id
  @Post()         // POST /posts
  @Put(':id')     // PUT /posts/:id
  @Delete(':id')  // DELETE /posts/:id
}
```

Khi bootstrap, NestJS quét toàn bộ Controller đã đăng ký trong `controllers: []` của module, đọc metadata của từng method, rồi đăng ký route đó thẳng vào Express router bên dưới. Về bản chất, NestJS chỉ là một lớp "khai báo" đặt lên trên `app.get()`, `app.post()` của Express.

2.2 Param Decorators — lấy dữ liệu từ request

Decorator	Lấy gì	Ví dụ
`@Param('id')`	Route param	`/posts/5` → `id = "5"`
`@Query('search')`	Query string	`/posts?search=abc` → `"abc"`
`@Body()`	Toàn bộ request body (JSON)	dùng cho POST/PUT
`@Headers('authorization')`	HTTP header	lấy token
`@Req()`	Object request thô của Express	ít dùng, phá vỡ tính "framework-agnostic"

Các decorator này thực chất cũng chỉ là metadata — NestJS đọc metadata đó, biết phải lấy giá trị nào từ `req` gốc của Express, rồi **tự động truyền đúng thứ tự** vào tham số hàm của bạn khi gọi handler. Đây là lý do bạn không cần viết `req.params.id` thủ công như Express thuần.

> **Đúc kết từ thực tế:** `@Param('id')` LUÔN trả về `string`, kể cả khi route param là số. Route `GET /posts/:id` viết đơn giản nhất chỉ nhận `id: string` rồi tự `Number(id)` — nhưng cách chuẩn hơn (Part 06) là gắn `ParseIntPipe` ngay tại decorator để Nest tự ép kiểu, tự trả 400 nếu id không phải số. Nếu bạn thấy so sánh `post.id === id` luôn `false` dù nhìn "giống nhau" trên console.log — 99% là do 1 bên `number` 1 bên `string`.

> ⚠ **Bug thật rất dễ mắc — thiếu dấu `:`** `@Get('id')` và `@Get(':id')` nhìn gần giống nhau nhưng khác hoàn toàn. `@Get('id')` đăng ký route **tĩnh**, chỉ khớp đúng URL `/posts/id` (chữ "id" theo nghĩa đen) — gọi `GET /posts/5` sẽ nhận **404** vì không route nào khớp. Dấu `:` đứng trước mới biến nó thành route **động** (route param). Đây là lỗi gõ thiếu 1 ký tự nhưng rất khó nhận ra bằng mắt — cách phát hiện nhanh nhất là chạy `nest start` và đọc log `Mapped {/posts/id, GET} route` (thiếu dấu `:` trong log = bug), hoặc đơn giản là thấy 404 dù chắc chắn route đã viết.

> **Việc cần làm:**
>
> 1. Tạo `PostsController` bằng `nest g controller posts`
> 2. Viết 5 route CRUD cơ bản (GET all, GET one, POST, PUT, DELETE) — trả về mảng cứng trong bộ nhớ (chưa cần service/database)
> 3. Test bằng Postman/curl/Thunder Client cho từng route

PART 03

Providers, Services & Dependency Injection

Mục tiêu: Tách logic nghiệp vụ ra khỏi Controller, hiểu sâu cách DI container resolve dependency.

3.1 Tại sao tách Service?

Nguyên tắc **Separation of Concerns**: Controller chỉ nên lo việc "nhận request → gọi service → trả response". Toàn bộ logic nghiệp vụ (tính toán, thao tác dữ liệu, gọi database) nằm trong Service. Điều này giúp: Controller dễ đọc, Service tái sử dụng được ở nơi khác (vd: một cron job cũng gọi lại service đó), và quan trọng nhất — **dễ test** (test Service không cần giả lập HTTP request).

3.2 @Injectable() nghĩa là gì?

```
@Injectable()
export class PostsService {
  private posts = [ ... ];
  findAll() { return this.posts; }
}
```

`@Injectable()` đánh dấu class này **có thể được đưa vào DI container** để inject cho class khác. Bản thân từ "Injectable" nghĩa là "có thể tiêm vào" — không phải class này tự tiêm, mà là nó *đủ điều kiện để bị tiêm vào constructor của class khác*.

3.3 Luồng resolve dependency chi tiết

```
1. App bootstrap → NestJS đọc @Module({ providers: [PostsService] })
2. NestJS thấy PostsController có constructor(private postsService: PostsService)
3. NestJS tra IoC container: "đã có instance PostsService chưa?"
   → Chưa có → tạo mới new PostsService() → LƯU VÀO container (singleton)
   → Đã có → lấy lại instance cũ, KHÔNG tạo mới
4. NestJS gán instance đó vào this.postsService của PostsController
5. Toàn bộ app dùng CHUNG 1 instance PostsService này
```

Vì sao phải khai báo providers trong @Module? Nếu bạn quên thêm `PostsService` vào mảng `providers` của module, NestJS sẽ báo lỗi `Nest can't resolve dependencies` ngay lúc khởi động — vì nó không biết class đó tồn tại trong container.

Việc cần làm:

1. Tạo `PostsService` bằng `nest g service posts`
2. Chuyển toàn bộ logic (mảng posts, các hàm CRUD) từ Controller sang Service
3. Inject Service vào Controller qua constructor, Controller chỉ gọi lại các method của Service

PART 04

Modules

> **Mục tiêu:** Tổ chức app lớn theo feature module thay vì nhồi hết vào AppModule.

4.1 Feature Module là gì?

Khi app lớn dần, bạn không nhét tất cả Controller/Service vào `AppModule`. Thay vào đó, mỗi "tính năng" (feature) có module riêng — vd `PostsModule`, `UsersModule`, `AuthModule` — rồi `AppModule` chỉ `import` các module con này lại.

```
@Module({
  controllers: [PostsController],
  providers: [PostsService],
  exports: [PostsService], // cho module khác dùng lại PostsService
})
export class PostsModule {}
```

`nest g resource posts` là lệnh "tất cả trong một" — tự sinh Controller, Service, Module, DTO, Entity cùng lúc và tự động import vào AppModule. Rất hay dùng trong thực tế.

Việc cần làm:

1. Refactor `PostsController` / `PostsService` đã làm ở Part 2-3 vào một `PostsModule` riêng
2. Import `PostsModule` vào `AppModule`, xoá bớt khai báo trùng lặp ở AppModule
3. Thử tạo thêm 1 module mới bằng `nest g resource users` để quen tay với CLI

PART 05

DTO & Validation

> **Mục tiêu:** Đảm bảo dữ liệu client gửi lên luôn đúng định dạng trước khi vào tới Service.

5.1 DTO (Data Transfer Object) là gì?

DTO là một class mô tả "hình dạng" dữ liệu được truyền giữa client và server (khác với Entity — mô tả bảng trong database, sẽ học ở Part 8). Ví dụ `CreatePostDto` mô tả body hợp lệ khi tạo post mới.

5.2 class-validator + ValidationPipe hoạt động thế nào

```
export class CreatePostDto {
  @IsString() @IsNotEmpty()
  title: string;
  @IsString() @IsNotEmpty()
  body: string;
}
```

Decorator như `@IsString()` gắn metadata validation lên từng field (thư viện `class-validator`). Khi request tới, **`ValidationPipe`** (chạy ở bước "Pipes" trong request lifecycle — xem Part 0) sẽ:

1. Lấy raw JSON body, `class-transformer` biến nó thành instance thật của class DTO (`plainToInstance`)
2. `class-validator` chạy tất cả rule decorator đã khai báo, thu thập lỗi
3. Nếu có lỗi → tự động throw `BadRequestException` (HTTP 400) kèm danh sách lỗi chi tiết, request **không bao giờ chạm tới Controller**
4. Nếu hợp lệ → object sạch được truyền vào `@Body()` của handler

Đăng ký global (áp dụng cho toàn bộ app) trong `main.ts`:

```
app.useGlobalPipes(new ValidationPipe({
  whitelist: true,        // tự động loại bỏ field lạ không khai báo trong DTO
  forbidNonWhitelisted: true, // field lạ → throw lỗi thay vì âm thầm bỏ qua
  transform: true,        // tự convert primitive (vd string "2" → number 2) trước khi vào handler
}));
```

Đối chiếu docs.nestjs.com/techniques/validation: `transform` **KHÔNG** bật mặc định — `new ValidationPipe()` không truyền option gì thì `transform` mặc định là `false`. Phải khai báo tường minh `transform: true` như trên thì `class-transformer` mới thực sự convert kiểu dữ liệu (string query param → number, v.v.) trước khi handler nhận được — đây là điều kiện bắt buộc để `@Type(() => Number)` ở Part 12 hoạt động, không phải hành vi có sẵn.

Ghi chú cho sau này — `PartialType`: mục "Việc cần làm" bên dưới dùng `PartialType` từ `@nestjs/mapped-types` — đúng cho course này (chưa dùng Swagger). Nếu sau này bạn thêm `@nestjs/swagger` để tự sinh tài liệu API, tài liệu chính thức cảnh báo: import `PartialType` từ `@nestjs/mapped-types` lúc đó có thể gây "undocumented side effects" — phải đổi sang import `PartialType` từ chính `@nestjs/swagger`.

Việc cần làm:

1. Cài `class-validator` `class-transformer`
2. Tạo `CreatePostDto` và `UpdatePostDto` (dùng `PartialType` từ `@nestjs/mapped-types` để field đều optional)
3. Gắn `ValidationPipe` global trong main.ts
4. Test gửi body thiếu field / sai kiểu dữ liệu → xác nhận nhận về lỗi 400 rõ ràng

PART 06

Pipes & Exception Filters

> **Mục tiêu:** Tự viết pipe/filter riêng, hiểu cách NestJS xử lý lỗi tập trung.

6.1 Custom Pipe / Pipe có sẵn

Pipe là class implement interface `PipeTransform` với method `transform(value, metadata)` — nhận giá trị đầu vào, trả về giá trị đã biến đổi (hoặc throw lỗi nếu không hợp lệ). Pipe có sẵn hay dùng nhất: `ParseIntPipe` — ép `:id` từ string sang number ngay tại tầng Pipe, tự throw `BadRequestException` nếu không parse được:

```
@Get(':id')
findOne(@Param('id', ParseIntPipe) id: number) {
  // id đã là number thật, không cần Number(id) nữa
  return this.postsService.findOne(id);
}
```

6.2 Exception hierarchy

NestJS có sẵn class `HttpException` và các class con tương ứng mã HTTP: `NotFoundException` (404), `BadRequestException` (400), `UnauthorizedException` (401), `ForbiddenException` (403)... Khi bạn `throw new NotFoundException('Post not found')` ở bất kỳ đâu (Service hay Controller), NestJS có sẵn một **Exception Filter mặc định** bắt lấy, format thành JSON response chuẩn và trả về client — bạn không cần try/catch thủ công trong từng route.

6.3 Custom Exception Filter — bản đã audit

```
@Catch(HttpException)
export class HttpExceptionFilter implements ExceptionFilter {
  catch(exception: HttpException, host: ArgumentsHost) {
    const ctx = host.switchToHttp();
    const response = ctx.getResponse<Response>();
    const request = ctx.getRequest<Request>();
    const status = exception.getStatus();

    // getResponse() trả về`string (khi throw new NotFoundException('msg'))
    // HOẶC object { message, error, statusCode } (khi Nest tự tạo payload).
    // Phải bóc ra, nếu không client sẽ phải đọc body.message.message.
    const payload = exception.getResponse();
    const message =
      typeof payload === 'string'
        ? payload
        : (payload as { message?: string }).message ?? exception.message;

    response.status(status).json({
      success: false,   // khớp key "success" với TransformInterceptor ở Part 07
      statusCode: status,
      path: request.url,
      timestamp: new Date().toISOString(),
      message,
    });
  }
}
```

> **Bài học từ thực tế:** phiên bản đầu tiên (chỉ nhét nguyên `exception.getResponse()` vào field `message`) tạo ra response lồng 2 lớp `message: { message: '...', error, statusCode }` — rất khó dùng ở client. `getResponse()` có thể trả về `string` (khi bạn tự `throw new NotFoundException('text')`) hoặc `object` (khi Nest tự sinh payload lỗi mặc định) — luôn phải kiểm tra `typeof` trước khi dùng. Field `success: false` ở đây cũng nên khớp với `success: true` mà Interceptor gắn ở Part 07, để client chỉ cần check 1 field duy nhất cho cả 2 nhánh thành công/lỗi.

`@Catch(HttpException)` nghĩa là filter này chỉ bắt lỗi thuộc loại `HttpException` (và các class con) — nếu bạn `throw new Error(...)` thường (không kế thừa `HttpException`), filter này **không bắt được**, sẽ rơi vào handler lỗi 500 mặc định của Nest. Đây là lý do luôn dùng `NotFoundException` / `BadRequestException`... thay vì tự `throw new Error()`.

6.4 Dùng thẳng `HttpException` có sẵn hay tự viết Exception Filter — khi nào dùng cái nào

Người mới hay rơi vào 1 trong 2 thái cực: hoặc `try/catch` thủ công ở mọi route (thừa — Nest đã có sẵn cơ chế bắt lỗi tập trung), hoặc ngược lại nghĩ phải tự viết `@Catch()` filter cho mọi loại lỗi. Thực ra 2 công cụ giải quyết 2 tầng khác nhau, không loại trừ nhau — dự án thật thường dùng CẢ HAI cùng lúc, như course này đang làm.

Cách 1 — Ném thẳng `HttpException` con có sẵn (`NotFoundException` , `BadRequestException` ...)

Đủ dùng cho hầu hết lỗi nghiệp vụ thông thường (đã minh hoạ ở mục 6.2). Nest tự có 1 exception filter mặc định bắt mọi `HttpException` , tự format response chuẩn — không cần viết filter nào nếu chấp nhận format mặc định của Nest.

Cách 2 — Tự viết `@Catch()` filter riêng (như mục 6.3)

Cần khi bạn muốn **toàn quyền kiểm soát tầng exception**: đổi cấu trúc JSON response (chuẩn hoá field `success` như mục 6.3), bắt cả những lỗi **không** kế thừa `HttpException` (vd lỗi `QueryFailedError` ném thẳng từ TypeORM khi vi phạm ràng buộc unique ở DB), hoặc thêm logging tập trung (gửi Sentry, ghi file log) cho mọi lỗi của app.

> **Đối chiếu docs.nestjs.com/exception-filters:** tài liệu chính thức nêu đúng lý do này: *"you may want full control over the exceptions layer, e.g., to add logging or to use a different JSON schema based on dynamic factors"* — tức là chỉ cần tự viết filter khi có 1 trong các nhu cầu cụ thể đó, không phải yêu cầu bắt buộc cho mọi lỗi HTTP thông thường.

Tình huống	Cách 1: `HttpException` có sẵn	Cách 2: Custom Filter
Lỗi nghiệp vụ chuẩn (404, 400, 401...)	✅ Đủ dùng, không cần filter riêng	Thừa nếu chỉ để bắt loại lỗi này
Cần format response lỗi nhất quán toàn app (vd luôn có field `success` , `timestamp`)	❌ Không kiểm soát được format mặc định của Nest	✅ Bắt buộc (đã làm ở mục 6.3)
Bắt lỗi KHÔNG kế thừa `HttpException` (vd lỗi ném thẳng từ TypeORM, thư viện ngoài)	❌ Filter mặc định của Nest không nhận diện, rơi vào 500 chung chung	✅ Tự `@Catch()` đúng loại lỗi đó để xử lý riêng
Cần logging tập trung mọi lỗi	❌ Không có chỗ để chèn logic	✅ Chèn logging ngay trong `catch()`

Quy tắc thực dụng: luôn `throw` đúng `HttpException` con có sẵn cho lỗi nghiệp vụ (không tự viết `Error` thường). Chỉ viết thêm custom filter khi cần 1 trong 3 việc: chuẩn hoá format response toàn app, bắt lỗi không phải `HttpException` , hoặc logging tập trung — không viết filter chỉ vì "thấy ví dụ trên mạng làm vậy" khi chưa có nhu cầu đó.

> **Việc cần làm:**
>
> 1. Trong `PostsService` , khi không tìm thấy post theo id → throw `NotFoundException`
> 2. Gắn `ParseIntPipe` vào mọi route nhận `:id`
> 3. Viết exception filter như trên, đăng ký global bằng `app.useGlobalFilters()`

PART 07

Middleware, Guards, Interceptors

> **Mục tiêu:** Nắm 3 lớp còn lại trong request lifecycle, biết dùng đúng công cụ cho đúng việc.

7.1 Middleware — chạy sớm nhất, gần Express nhất

```
@Injectable()
export class LoggerMiddleware implements NestMiddleware {
  use(req: Request, res: Response, next: NextFunction) {
    const start = Date.now();
    res.on('finish', () => {
      const ms = Date.now() - start;
      console.log(`${req.method} ${req.originalUrl} ${res.statusCode} - ${ms}ms`);
    });
    next(); // bắt buộc gọi next() để đi tiếp, nếu không request sẽ treo
  }
}
```

> **Tại sao log trong `res.on('finish')` chứ không log ngay sau `next()` :** ngay sau `next()`, response chưa thực sự được gửi xong — `res.statusCode` lúc đó vẫn chưa phải giá trị cuối cùng. Lắng nghe event `'finish'` đảm bảo log đúng status code thật và đo được thời gian xử lý toàn bộ request.

Middleware KHÔNG khai báo trong `@Module({...})` như Controller/Provider — nó được gắn thủ công qua method `configure()` trong module, dùng `MiddlewareConsumer` để chỉ định áp dụng cho route nào (`consumer.apply(LoggerMiddleware).forRoutes('*')` = áp dụng toàn app).

7.2 Guards — trả lời câu hỏi "có được vào không?"

```
@Injectable()
export class AuthGuard implements CanActivate {
  canActivate(context: ExecutionContext): boolean {
    const request = context.switchToHttp().getRequest();
    if (!request.headers.authorization) {
      throw new UnauthorizedException('Missing Authorization header');
    }
    return true;
  }
}
```

Guard implement `CanActivate`, trả về `true` / `false` (hoặc `Promise<boolean>`) — mặc định trả `false` sẽ bị Nest tự trả 403; ở đây chủ động `throw UnauthorizedException` để kiểm soát status 401 rõ ràng hơn. Đây là nơi chuẩn để đặt logic **Authorization** — khác với Middleware ở chỗ Guard có quyền truy cập `ExecutionContext` (biết được đang gọi tới Controller/handler nào, hữu ích để đọc thêm metadata custom qua `Reflector`, ví dụ decorator `@Roles('admin')` tự viết).

7.3 Interceptors — bọc quanh cả input lẫn output

```
@Injectable()
export class TransformInterceptor implements NestInterceptor {
  intercept(context: ExecutionContext, next: CallHandler): Observable<any> {
    return next.handle().pipe(
      map(data => ({ success: true, data }))
    );
  }
}
```

Interceptor dùng **RxJS Observable**. `next.handle()` trả về Observable đại diện cho kết quả của route handler — bạn có thể `.pipe(map(...))` để biến đổi response trước khi trả về client. Đăng ký global bằng `app.useGlobalInterceptors(new TransformInterceptor())` trong `main.ts`.

> **Đã kiểm chứng bằng test thật:** sau khi bật Interceptor này, response LỖI (từ `HttpExceptionFilter` Part 06) **không** bị bọc theo format `{success, data}` — vì Interceptor chỉ bọc nhánh `next.handle()` (thành công), nhánh lỗi đi thẳng qua Exception Filter, bỏ qua Interceptor hoàn toàn (đúng như sơ đồ Part 00). Đây là lý do `HttpExceptionFilter` phải tự thêm field `success: false` bằng tay — không có Interceptor nào làm việc đó cho nhánh lỗi.

7.4 Middleware, Guard, Interceptor — cùng "chen" vào request nhưng khác việc, chọn nhầm là code rối

Cả 3 đều là chỗ đặt logic dùng lại được (logging, auth, transform...) mà không nhét trực tiếp vào Controller, nên người mới rất dễ phân vân "cái này viết Guard hay Interceptor hay Middleware cũng ra kết quả giống nhau mà?". Kỹ thuật thì đôi khi làm được cả 3 cách thật, nhưng mỗi công cụ được thiết kế để trả lời đúng 1 câu hỏi khác nhau — chọn đúng câu hỏi trước, công cụ sẽ tự lộ ra.

Middleware — trả lời "tôi cần đọc/sửa request thô, càng sớm càng tốt, không cần biết sắp gọi route nào"

Middleware KHÔNG có `ExecutionContext` — nó không biết (và không cần biết) request sẽ rơi vào Controller/method nào. Phù hợp cho việc chạy giống hệt nhau ở MỌI route (logging, đọc IP...). Đăng ký ngoài `@Module()`, qua `configure()` (mục 7.1).

Guard — trả lời "request này có được đi tiếp không?"

Guard có `ExecutionContext` (biết chính xác sắp gọi Controller/method nào — dùng được với `Reflector` để đọc metadata custom kiểu `@Roles('admin')`). Việc DUY NHẤT của Guard là trả lời true/false — không phải chỗ để transform dữ liệu.

Interceptor — trả lời "tôi cần bọc quanh cả input lẫn output của handler"

Interceptor cũng có `ExecutionContext`, nhưng khác Guard ở chỗ nó không quyết định pass/fail — nó luôn cho request đi tiếp (trừ khi chủ động không gọi `next.handle()`), và chạy được logic cả trước lẫn sau khi handler xong (transform response, đo thời gian, cache).

> **Đối chiếu docs.nestjs.com/guards & docs.nestjs.com/interceptors:** tài liệu chính thức xác nhận đúng thứ tự ở sơ đồ Part 00: *"Guards are executed after all middleware, but before any interceptor or pipe."* Tài liệu cũng xác nhận đúng phát hiện thực tế ở mục 7.3: operator `map()` trong Interceptor *"is not called if the route handler throws"* — nhánh lỗi tách biệt hoàn toàn (chỉ `catchError()` mới "chạm" được exception từ trong Interceptor — kỹ thuật nâng cao ngoài phạm vi course này).

Công cụ	Dùng khi nào	Chạy ở đâu trong lifecycle
Middleware	Logging, đọc/sửa raw request sớm	Sớm nhất, trước routing
Guard	Authorization (được phép truy cập không?)	Sau routing, trước Interceptor
Interceptor	Transform response, logging có ngữ cảnh handler, cache	Bọc quanh route handler
Pipe	Validate/transform input	Ngay trước khi vào handler
Exception Filter	Xử lý lỗi tập trung	Bất cứ khi nào có exception

Quy tắc thực dụng: tự hỏi theo đúng thứ tự sau. (1) Logic này có cần biết route/handler cụ thể không? — không cần → Middleware. (2) Nếu cần: việc này có phải là QUYẾT ĐỊNH cho đi tiếp hay chặn lại? — đúng → Guard. (3) Nếu không phải quyết định pass/fail mà là biến đổi dữ liệu → Interceptor (dữ liệu đi ra) hoặc Pipe (dữ liệu đi vào, mục 6.1).

> **Việc cần làm:**
>
> 1. Viết `LoggerMiddleware` log method + url + status + thời gian, áp dụng cho toàn bộ route
> 2. Viết `AuthGuard` đơn giản kiểm tra header `Authorization` có tồn tại, áp lên route tạo/sửa/xoá post
> 3. Viết `TransformInterceptor` bọc mọi response thành công theo format `{ success: true, data }`, đăng ký global

---

PART 08

Kết nối Database với TypeORM

> **Mục tiêu:** Thay mảng dữ liệu cứng trong bộ nhớ bằng database thật (SQLite để đơn giản, có thể nâng cấp Postgres sau).

8.1 TypeORM là gì

TypeORM là một **ORM** (Object-Relational Mapping) — cho phép thao tác database bằng object/class TypeScript thay vì viết SQL tay. NestJS tích hợp sẵn qua package `@nestjs/typeorm`.

> **Về nguồn tài liệu chính thức (đối chiếu lần 2 với docs.nestjs.com):** trang `docs.nestjs.com/techniques/database` hiện chỉ còn là trang tổng quan liệt kê các ORM được hỗ trợ (TypeORM, Prisma, Mongoose...) — toàn bộ nội dung chi tiết (Repository pattern, Relations, Transactions) đã chuyển sang `docs.nestjs.com/data/typeorm`. Cú pháp `TypeOrmModule.forRoot()` / `forFeature()` / `@InjectRepository()` ở mục 8.3 và Part 13 khớp đúng với trang hiện hành này.

> ⚠ **Đã sửa so với bản gốc:** bản trước hướng dẫn dùng `type: 'sqlite'` + package `sqlite3` — đây là API TypeORM CŨ. Bản TypeORM hiện tại (kiểm tra bằng `npm view typeorm version`) đã **bỏ hẳn driver `"sqlite"`**, chỉ còn `"better-sqlite3"` cho SQLite. Luôn kiểm tra `node_modules/typeorm/driver/types/DatabaseType.d.ts` nếu nghi ngờ danh sách driver hợp lệ đã đổi.

8.2 Entity — class ánh xạ tới bảng

```
@Entity()
export class Post {
  @PrimaryGeneratedColumn()
  id: number;

  @Column()
  title: string;

  @Column()
  body: string;
}
```

`@Entity()` đánh dấu class này ánh xạ tới một bảng — TypeORM tự đặt tên bảng bằng cách lowercase tên class (`Post` → bảng `post`, KHÔNG tự thêm "s"). Mỗi `@Column()` là một cột. TypeORM đọc metadata này để tự sinh câu SQL `CREATE TABLE` (nếu bật `synchronize: true` — chỉ nên dùng khi dev, không dùng ở production vì có thể mất dữ liệu khi đổi schema).

> ⚠ **Bẫy trùng tên — RẤT dễ gặp:** Entity thường được đặt tên số ít khớp domain (`Post`, `Comment`, `User`) — nhưng `Post` lại trùng tên với decorator `@Post()` import từ `@nestjs/common`! Trong Controller/Service cần dùng cả 2, phải **alias khi import**:
>
> ```
> import { Post as PostEntity } from './entities/post.entity';
> ```
>
> Nếu bạn thấy TypeScript báo `error TS2749: 'Post' refers to a value, but is being used as a type here` — chính xác là bug này. Alias theo chiều nào (đổi tên decorator hay Entity) tuỳ bạn, chỉ cần nhất quán trong 1 file.

8.3 Repository Pattern

```
constructor(
  @InjectRepository(PostEntity)
  private readonly postsRepository: Repository<PostEntity>,
) {}

// find() ĐÃ tự trả Promise → chỉ forward ra, KHÔNG cần async
findAll(): Promise<PostEntity[]> {
  return this.postsRepository.find();
}

// BẮT BUỘC async/await: cần giá trị thật của post để kiểm tra if (!post)
async findOne(id: number): Promise<PostEntity> {
  const post = await this.postsRepository.findOneBy({ id });
  if (!post) {
    throw new NotFoundException(`Post with id ${id} not found`);
  }
  return post;
}

create(dto: CreatePostDto): Promise<PostEntity> {
  // create() ĐỒNG BỘ: chỉ tạo object JS trong RAM, chưa chạm DB
  const post = this.postsRepository.create(dto);
  // save() BẤT ĐỒNG BỘ: trả Promise → return thẳng, không cần await
  return this.postsRepository.save(post);
}
```

`@InjectRepository(PostEntity)` là một biến thể đặc biệt của DI — thay vì inject trực tiếp một class do bạn viết, bạn inject một **Repository object** mà TypeORM tự sinh ra cho Entity `PostEntity`. Repository này có sẵn các method `find`, `findOneBy`, `save`, `remove`... — về bản chất đây vẫn là DI container, chỉ là "provider" ở đây được đăng ký bởi `TypeOrmModule.forFeature([Post])` thay vì bạn tự viết `@Injectable()`.

> **Quy tắc chung khi nào cần `async/await`:** chỉ cần khi bạn phải *dùng giá trị đã resolve* của 1 Promise ngay trong thân hàm (so sánh `if`, `Object.assign`, truyền làm tham số cho hàm khác cần entity thật...). Nếu chỉ nhận 1 Promise rồi trả thẳng ra ngoài (không đụng vào giá trị) — không cần `async/await`, cứ `return` Promise đó, người gọi tự lo. Ví dụ cụ thể: `findAll()` / `create()` không cần async (chỉ forward Promise), `findOne()` / `update()` / `remove()` bắt buộc cần (phải `if (!post)`, hoặc cần entity thật để gọi `repository.remove(entity)`).

Bug nguy hiểm nếu quên await ở Controller:

```
// SAI — không await, không return:
remove(id: number) {
  this.postsService.remove(id); // Promise bị "bỏ rơi"
  return { message: 'deleted' };  // trả response NGAY, chưa chắc đã xoá xong
}
```

Nếu `id` không tồn tại, `service.remove()` sẽ `throw` **bên trong 1 Promise không ai await** — thành "unhandled promise rejection", Nest không bắt được, và tuỳ phiên bản Node.js có thể **crash cả server**. Luôn `await` Promise trước khi return trong Controller.

8.4 Repository hay QueryBuilder — 2 cách viết query, chọn đúng lúc nào

Đây là chỗ người mới hay bối rối nhất: cùng một việc "lọc dữ liệu theo điều kiện", TypeORM cho **2 cách viết** hoàn toàn khác nhau. Không phải cách nào "cao cấp hơn" — mỗi cách có việc riêng, dùng sai chỗ là code dài dòng vô ích hoặc bị thiếu tính năng.

Cách 1 — Repository API (`find` / `findOne` / `findAndCount`)

Truyền một **object mô tả điều kiện** (`where`), TypeORM tự dịch thành SQL. Không cần biết cú pháp SQL, không cần tự quản lý JOIN.

```
import { Between, Like, MoreThanOrEqual, LessThanOrEqual, FindOptionsWhere } from
'typeorm';

async findAll(query: ProductQueryDto) {
  const { page, limit, search, categoryId, minPrice, maxPrice } = query;
  const where: FindOptionsWhere<Product> = {};

  if (search) where.name = Like(`%${search}%`);
  if (categoryId) where.category = { id: categoryId }; // filter qua quan hệ 1 cấp,
không cần JOIN thủ công
  if (minPrice && maxPrice) where.price = Between(minPrice, maxPrice);
  else if (minPrice) where.price = MoreThanOrEqual(minPrice);
  else if (maxPrice) where.price = LessThanOrEqual(maxPrice);

  const [data, total] = await this.productsRepository.findAndCount({
    where,
    relations: { category: true },
    order: { id: 'DESC' },
    skip: (page - 1) * limit,
    take: limit,
  });
}
```

`Like` , `Between` , `MoreThanOrEqual` , `LessThanOrEqual` , `In` ... là các **operator** import từ `typeorm` — thay cho việc tự viết chuỗi SQL. Gán thẳng `where.category = { id: categoryId }` lọc qua bảng quan hệ mà **không cần** tự viết `leftJoin` — TypeORM tự JOIN ngầm dựa vào field bạn khai trong `where` .

Điều kiện OR — Repository API cũng làm được: truyền **mảng** object vào `where` . Mỗi object là 1 nhóm điều kiện AND, các object trong mảng được nối với nhau bằng OR:

```
// tìm theo tên HOẶC mô tả: WHERE (name LIKE ?) OR (description LIKE ?)
this.productsRepository.find({
  where: [
    { name: Like(`%${search}%`) },
    { description: Like(`%${search}%`) },
  ],
});

// (a OR b) AND c — phải LẶP LẠI c trong từng nhánh:
// WHERE (name LIKE ? AND price < 20) OR (description LIKE ? AND price < 20)
this.productsRepository.find({
  where: [
    { name: Like(`%${search}%`), price: LessThan(maxPrice) },
    { description: Like(`%${search}%`), price: LessThan(maxPrice) },
  ],
});
```

Dạng "1 cụm OR lồng trong AND" như ví dụ thứ 2 vẫn viết được nhưng phải nhân bản điều kiện chung vào mọi nhánh — thêm 1 bộ lọc nữa là số nhánh phình ra. Đó là lúc QueryBuilder với `Brackets` (bên dưới) gọn và rõ hơn.

> **Đối chiếu typeorm.io/docs/working-with-entity-manager/find-options (mục OR) + chạy thật:** type của option là `where?: FindOptionsWhere<Entity>[] | FindOptionsWhere<Entity>` (`FindOneOptions.d.ts` , TypeORM 1.1.1) — nhận cả mảng. Docs ví dụ `where: [{ firstName: "Timber", lastName: "Saw" }, { firstName: "Stan", lastName: "Lee" }]` sinh `WHERE (...) OR (...)` ; chạy thật 2 đoạn trên cho đúng SQL ghi trong comment.

Cách 2 — QueryBuilder (`createQueryBuilder`)

Viết gần giống SQL thật bằng chuỗi method, kiểm soát chi tiết hơn nhưng dài hơn và mất type-safety ở phần điều kiện (chuỗi `'product.name LIKE :search'` không được TypeScript kiểm tra).

```
const qb = this.productsRepository
  .createQueryBuilder('product')
  .leftJoin('product.category', 'category')
  .loadRelationIdAndMap('product.tagIds', 'product.tags') // Repository KHÔNG làm được
việc này
  .orderBy('product.id', 'DESC')
  .skip((page - 1) * limit)
  .take(limit);

if (search) qb.andWhere('product.name LIKE :search', { search: `%${search}%` });
```

Nhóm điều kiện OR bên trong AND bằng `Brackets` (import từ `typeorm`) — điều kiện chung chỉ viết 1 lần:

```
// WHERE product.price < :max AND (product.name LIKE :s OR product.description LIKE :s)
qb.where('product.price < :max', { max: maxPrice })
  .andWhere(
    new Brackets((sub) => {
      sub.where('product.name LIKE :s', { s: `%${search}%` })
         .orWhere('product.description LIKE :s', { s: `%${search}%` });
    }),
  );
```

> **Đối chiếu typeorm.io/docs/query-builder/select-query-builder (mục Brackets):** *"You can add a complex WHERE expression into an existing WHERE using Brackets"*. `Brackets` (và `NotBrackets`) được export từ `typeorm/index.d.ts` bản 1.1.1; chạy thật đoạn trên sinh đúng `WHERE "product"."price" < 20 AND ("product"."name" LIKE ? OR "product"."description" LIKE ?)`.

> **Bảng quyết định — dựa đúng trên bug thật đã gặp khi build project này:** ban đầu code lọc sản phẩm (search + categoryId) được viết bằng QueryBuilder với `leftJoin` thủ công — chạy đúng, nhưng dài hơn cần thiết cho một việc chỉ là "vài điều kiện AND đơn giản". Đây chính xác là trường hợp nên dùng Cách 1.

Nhu cầu	Cách 1: Repository	Cách 2: QueryBuilder
Vài điều kiện AND đơn giản (bằng, LIKE, khoảng giá trị)	✅ Dùng cách này — gọn, type-safe	Chạy được nhưng dài dòng không cần thiết
Lọc qua quan hệ 1 cấp (`where.category = { id }`)	✅ Tự JOIN ngầm, không cần khai `leftJoin`	Phải tự viết `leftJoin` + đặt alias tay
Đếm/gán id quan hệ vào từng dòng (`loadRelationIdAndMap`, Part 9.4)	❌ Không có API này	✅ Bắt buộc
OR đơn giản giữa vài nhóm điều kiện (`a OR b`)	✅ Truyền mảng vào `where`	Chạy được, dài hơn
OR lồng trong AND (`(a OR b) AND c`), subquery, GROUP BY, SELECT cột tính toán	❌ Phải lặp điều kiện chung vào mọi nhánh OR; subquery/GROUP BY thì không diễn tả được	✅ `Brackets` / QueryBuilder
Tra cứu 1 field đơn giản (`findOneBy({ email })`)	✅ Luôn dùng cách này	Thừa hoàn toàn

Quy tắc thực dụng: luôn bắt đầu bằng Cách 1 (Repository). Chỉ chuyển sang Cách 2 khi gặp dòng `loadRelationIdAndMap` hoặc dòng "OR lồng trong AND / subquery / GROUP BY" của bảng trên — tức là khi bạn *thử* viết bằng Repository trước và thấy nó không diễn tả được, chứ không phải chọn QueryBuilder "cho chắc" ngay từ đầu.

> **Việc cần làm:**
>
> 1. Cài `@nestjs/typeorm typeorm better-sqlite3` (KHÔNG dùng `sqlite3` — xem cảnh báo ở trên)
> 2. Cấu hình `TypeOrmModule.forRoot({ type: 'better-sqlite3', database: 'db.sqlite3', autoLoadEntities: true, synchronize: true })` trong AppModule

3. Tạo `Post` entity thay cho mảng cứng, nhớ alias khi import vào Controller/Service (mục 8.2)
4. Đăng ký `TypeOrmModule.forFeature([Post])` trong PostsModule, inject Repository vào PostsService
5. Chuyển toàn bộ hàm CRUD trong service sang gọi Repository, thêm `async/await` đúng chỗ (mục 8.3)
6. Thêm `db.sqlite3` vào `.gitignore` — không commit file database lên git
7. Kiểm chứng persistence: tạo 1 post, restart hẳn server (`Ctrl+C` rồi chạy lại), `GET` lại — post phải còn nguyên
8. Viết filter (search + khoảng giá) cho 1 danh sách bất kỳ bằng Cách 1 (mục 8.4) — tự hỏi trước khi viết: "điều kiện này Repository diễn tả được không?"

PART 09

Quan hệ dữ liệu (Relations)

Mục tiêu: Thêm entity thứ 2 (Comment) và liên kết với Post — hiểu cách TypeORM quản lý khoá ngoại, nhận diện và sửa N+1 bằng log SQL, load/lọc quan hệ lồng nhau, ManyToMany, và các bẫy của `cascade` / `onDelete`.

9.1 One-to-Many / Many-to-One

```
// Trong Post entity
@OneToMany(() => Comment, (comment) => comment.post)
comments: Comment[];

// Trong Comment entity
@ManyToOne(() => Post, (post) => post.comments, { onDelete: 'CASCADE' })
post: Post;
```

Một `Post` có nhiều `Comment` (OneToMany), mỗi `Comment` thuộc về đúng một `Post` (ManyToOne) — hai decorator này luôn đi thành cặp, tham số thứ 2 là hàm trỏ ngược lại field ở phía bên kia để TypeORM biết đây là 2 đầu của cùng 1 quan hệ. **Khoá ngoại thật (`postId`) chỉ nằm ở bảng phía "Many"** (`comment`) — `@OneToMany` không tạo cột nào cả. `onDelete: 'CASCADE'` — xoá `Post` thì mọi `Comment` của nó tự bị xoá theo, tránh dữ liệu "mồ côi".

⚠ **Bẫy cực tinh vi — global `Comment` của DOM:** nếu bạn `import { Comment } from './entities/comment.entity'` nhưng lỡ **quên import** ở nơi khác dùng đến (ví dụ trong `@Module({ imports: [TypeOrmModule.forFeature([Comment, Post])] })`), TypeScript **sẽ không báo lỗi gì**! Lý do: khi `tsconfig.json` không giới hạn `"lib"` , TypeScript tự nạp thêm type của DOM — trong đó có sẵn 1 biến toàn cục tên `Comment` (constructor của "comment node" trong HTML, giống `Document`). Compile "sạch", nhưng chạy trong Node.js sẽ `ReferenceError: Comment is not defined` ngay khi module được nạp — vì Node.js không có global đó. **Bài học:** "tsc không báo lỗi" không đồng nghĩa "chắc chắn đúng" khi có tên trùng global ambient type/value có sẵn trong lib mặc định. Luôn để ý gợi ý "Add import" của IDE khi gõ tên class — nếu nó không hiện gợi ý dù bạn chưa import, hãy nghi ngờ.

9.2 Eager vs Lazy loading

`eager: true` trong decorator quan hệ nghĩa là mỗi lần query Post, TypeORM tự động JOIN kèm luôn Comment. Nếu không bật, bạn phải chủ động yêu cầu bằng `relations: { comments: true }` khi query — cách này kiểm soát hiệu năng tốt hơn (tránh JOIN thừa khi không cần, ví dụ `GET /posts` danh sách không cần tải hết comments của từng post).

> **Đã sửa so với bản gốc:** bản trước viết `relations: ['comments']` (mảng chuỗi) — đây là API TypeORM CŨ. Bản TypeORM hiện tại chỉ chấp nhận **object**: `relations: { comments: true }`, lồng sâu thì `{ comments: { author: true } }`. Dùng mảng chuỗi sẽ báo lỗi compile `Type 'string[]' has no properties in common with type 'FindOptionsRelations<Post>'`.

9.3 Kiểm tra parent tồn tại trước khi tạo con

```
async create(postId: number, dto: CreateCommentDto): Promise<Comment> {
  const post = await this.postsRepository.findOneBy({ id: postId });
  if (!post) {
    throw new NotFoundException(`Post with id ${postId} not found`);
  }
  // gán cả object post vào field quan hệ, KHÔNG phải postId
  const comment = this.commentsRepository.create({ ...dto, post });
  return this.commentsRepository.save(comment);
}
```

Gán trực tiếp **object** `post` (không phải `postId`) vào field quan hệ — TypeORM tự hiểu cần lưu `post.id` vào cột khoá ngoại bên dưới. Luôn kiểm tra parent tồn tại trước — nếu không sẽ tạo ra comment "mồ côi" ngay từ đầu ở tầng nghiệp vụ (dù DB không cấm).

9.4 Đếm số bản ghi liên quan — không tải cả mảng chỉ để lấy `.length`

Bài toán rất hay gặp: category có bao nhiêu product, post có bao nhiêu comment... **Cách sai** — tải nguyên bảng con rồi đếm:

```
// SAI — kéo hết dữ liệu product (name, price, description...) chỉ để đếm
const category = await this.categoriesRepository.findOne({
  where: { id },
  relations: { products: true },
});
const count = category.products.length; // category có 10.000 product = kéo cả 10.000 dòng
```

Cách đúng: yêu cầu database tự đếm (`COUNT`), không kéo dữ liệu con về. TypeORM có 2 hàm cho việc này trên `QueryBuilder` — nhưng **chỉ 1 hàm còn tồn tại** ở bản bạn đang cài:

```
const [rows, total] = await this.categoriesRepository
  .createQueryBuilder('category')
  .loadRelationIdAndMap('category.productsCount', 'category.products')
  .skip((page - 1) * limit)
  .take(limit)
  .getManyAndCount();

const data = rows.map((category) => ({
  ...category,
  productsCount: (category.productsCount as unknown as number[])?.length ?? 0,
}));
```

> ⚠ **Đã sửa so với bản gốc:** hàm "chuẩn sách giáo khoa" cho việc này tên là `loadRelationCountAndMap` (tự chạy `COUNT`, trả thẳng về số) — đây là hàm hầu hết tutorial/docs TypeORM nói tới. Nhưng ở bản TypeORM đang cài, hàm đó **không còn tồn tại** — bị bỏ hẳn, TS báo `Property 'loadRelationCountAndMap' does not exist`. Chỉ còn `loadRelationIdAndMap` — trả về **mảng id**, không phải số, phải tự `.length` ở tầng code như ví dụ trên.

9.5 N+1 — lỗi hiệu năng phải hiểu (và cách NHÌN THẤY nó)

N+1 là lỗi hiệu năng phổ biến nhất của mọi ORM: lấy danh sách N bản ghi cha bằng **1** query, rồi trong vòng lặp lại chạy thêm **1 query cho mỗi bản ghi** để lấy dữ liệu con → tổng cộng 1 + N query. Với 3 post thì không ai để ý; với trang danh sách 100 post là 101 lượt đi-về database cho 1 request. Trước khi sửa, phải **nhìn thấy** được nó — bật log SQL của TypeORM:

```
// app.module.ts — CHỈ bật khi dev, tắt ở production (log rất nhiều)
TypeOrmModule.forRoot({
  type: 'better-sqlite3',
  database: 'db.sqlite3',
  autoLoadEntities: true,
  synchronize: true,
  logging: ['query', 'error'], // hoặc logging: true để log mọi loại
}),
```

Từ giờ mỗi câu SQL được in ra terminal dạng `query: SELECT ... -- PARAMETERS: [...]`. Giờ xem 2 cách viết cùng 1 việc "danh sách post kèm comment":

```ts
// ✘ SAI — N+1: 1 query lấy post + 1 query comment CHO MỖI post
async listWithComments() {
  const posts = await this.postsRepository.find();
  for (const post of posts) {
    post.comments = await this.commentsRepository.find({
      where: { post: { id: post.id } },
    });
  }
  return posts;
}

// ✔ ĐÚNG — để TypeORM JOIN 1 lần
listWithComments() {
  return this.postsRepository.find({ relations: { comments: true } });
}

// ✔ ĐÚNG — cùng kết quả, viết bằng QueryBuilder
listWithCommentsQB() {
  return this.postsRepository
    .createQueryBuilder('post')
    .leftJoinAndSelect('post.comments', 'comment')
    .getMany();
}
```

Log thật (đã rút gọn danh sách cột) khi có 3 post:

```
-- Cách SAI: 4 query (1 + 3)
query: SELECT "Post"."id", "Post"."title", "Post"."body" FROM "post" "Post"
query: SELECT ... FROM "comment" "Comment" LEFT JOIN "post" ... WHERE ... "id" = 1
query: SELECT ... FROM "comment" "Comment" LEFT JOIN "post" ... WHERE ... "id" = 2
query: SELECT ... FROM "comment" "Comment" LEFT JOIN "post" ... WHERE ... "id" = 3

-- Cách ĐÚNG: 1 query
query: SELECT "Post"."id", ..., "Post__Post_comments"."id", ... FROM "post" "Post"
       LEFT JOIN "comment" "Post__Post_comments" ON
"Post__Post_comments"."postId"="Post"."id"
```

Đã kiểm chứng bằng cách chạy thật (TypeORM 1.1.1 + better-sqlite3, DB `:memory:`, bật logger đếm query): vòng lặp `find` cho 3 post sinh đúng 4 query, `relations: { comments: true }` sinh đúng 1 query `LEFT JOIN`, `leftJoinAndSelect` cũng 1 query. Cú pháp `logging: true | "all" | ["query", "error", ...]` khớp type `LoggerOptions` trong `typeorm/logger/LoggerOptions.d.ts` và trang `typeorm.io/docs/logging` (logger mặc định là `advanced-console`, in có màu).

Khác Laravel: Laravel có `preventLazyLoading()` để tự throw khi quên eager load. TypeORM **không có** cơ chế tương đương — quan hệ không khai trong `relations` đơn giản là `undefined`, và N+1 chỉ xuất hiện khi chính bạn viết vòng lặp query như trên. Cách phát hiện duy nhất là đọc log SQL (hoặc viết test đếm query). Nếu chỉ cần **đếm** số comment thì đừng load cả mảng — xem lại mục 9.4.

9.6 Quan hệ lồng nhau & lọc theo quan hệ

Lồng nhiều cấp — object lồng object, đúng cú pháp đã sửa ở mục 9.2. Ví dụ dưới giả sử bạn đã thêm quan hệ `author` (ManyToOne tới `User`) vào `Comment` — project hiện tại chưa có field này, đây là 1 việc trong task-box:

```
// comment.entity.ts — thêm tác giả cho comment
@ManyToOne(() => User, { nullable: true, onDelete: 'SET NULL' })
author: User | null;

// post kèm comment, mỗi comment kèm tác giả — vẫn KHÔNG có N+1
this.postsRepository.findOne({
  where: { id },
  relations: { comments: { author: true } },
});
```

Lọc theo quan hệ — viết điều kiện lồng trong `where`, TypeORM tự `LEFT JOIN` (không cần khai `relations`):

```
// post nào có ít nhất 1 comment chứa chữ "nest"
this.postsRepository.find({
  where: { comments: { content: Like('%nest%') } },
});
// SQL: ... LEFT JOIN "comment" ... WHERE "Post__Post_comments"."content" LIKE ?
```

⚠ **Bẫy: lọc theo quan hệ + load chính quan hệ đó cùng lúc.** Nếu viết `find({ where: { comments: { content: 'abc' } }, relations: { comments: true } })`, điều kiện `WHERE` áp lên **cùng** JOIN dùng để load → mảng `post.comments` trả về **chỉ còn những comment khớp điều kiện**, không phải toàn bộ comment của post. Đã chạy thử: post có 2 comment, lọc theo 1 comment → response chỉ còn 1 comment. Muốn "lọc post theo comment nhưng vẫn trả đủ comment" thì phải dùng QueryBuilder với **2 alias khác nhau** — 1 alias để lọc, 1 alias để load:

```
this.postsRepository
  .createQueryBuilder('post')
  .innerJoin('post.comments', 'matched', 'matched.content LIKE :q', { q: `%${q}%` })
  .leftJoinAndSelect('post.comments', 'comment') // alias riêng → load đủ
  .getMany();
```

Đối chiếu typeorm.io/docs/working-with-entity-manager/find-options: tài liệu xác nhận cú pháp lồng cho cả `relations` (*"Sub-relations can also be loaded"*, ví dụ `videos: { videoAttributes: true }`) và `where` theo quan hệ. Hành vi "mảng quan hệ bị lọc theo" **không** được docs nhắc tới — đó là kết quả chạy thật trên TypeORM 1.1.1, kèm cách sửa bằng 2 alias đã chạy ra đủ comment.

9.7 Chỉ lấy cột cần — `select`

Mặc định TypeORM `SELECT` mọi cột của mọi bảng được JOIN. Với danh sách chỉ hiển thị tiêu đề, kéo cả `body` (có thể dài hàng chục KB mỗi post) là lãng phí. Chọn cột bằng `select` — cú pháp object, lồng được qua quan hệ:

```
// Repository API
this.postsRepository.find({
  select: { id: true, title: true, comments: { id: true, content: true } },
  relations: { comments: true },
});

// QueryBuilder: leftJoin (KHÔNG có "AndSelect") + tự chọn cột
this.postsRepository
  .createQueryBuilder('post')
  .leftJoin('post.comments', 'comment')
  .select(['post.id', 'post.title', 'comment.id', 'comment.content'])
  .getMany();
```

Cả 2 cách đã chạy thật: SQL chỉ còn `SELECT "post"."id", "post"."title", "comment"."id", ...` — không còn cột `body`, và kết quả vẫn là object lồng `{ id, title, comments: [{ id, content }] }`. Khác biệt cần nhớ: `leftJoinAndSelect` = JOIN + lấy **mọi cột** bảng con; `leftJoin` = chỉ JOIN (dùng để lọc/sắp xếp), bảng con **không** có trong kết quả trừ khi bạn tự `select` / `addSelect` cột của nó.

> **Object trả về là entity "thiếu field":** `post.body` lúc này là `undefined` dù type TypeScript vẫn khai `body: string` — compiler không cảnh báo. Chỉ dùng `select` cho dữ liệu **đọc để trả ra**; đừng truyền object đã cắt cột đó cho hàm khác đang mong 1 entity đầy đủ (nó sẽ đọc phải `undefined` mà không hề biết).

9.8 JOIN 1 câu hay tách thành nhiều câu — `relationLoadStrategy`

Đã hết N+1 rồi, nhưng vẫn còn 1 câu hỏi: load quan hệ bằng **1 câu JOIN lớn** hay **vài câu nhỏ riêng**? TypeORM cho chọn bằng option `relationLoadStrategy`. Cả 2 đều không phải N+1 — số query không tăng theo số bản ghi.

Cách 1 — `'join'` (mặc định): 1 câu `LEFT JOIN`

```
this.postsRepository.find({ relations: { comments: true } });
// 1 query: SELECT post.*, comment.* FROM post LEFT JOIN comment ON ...
```

Nhược điểm: JOIN 1-N **nhân bản dòng** — post có 50 comment thì dữ liệu của post đó lặp lại 50 lần trong kết quả SQL. JOIN 2 quan hệ 1-N cùng lúc (`comments` + `tags`) thì số dòng thành **tích** của chúng (50 × 10 = 500 dòng cho 1 post). Ngoài ra khi kết hợp với phân trang (`skip` / `take`), TypeORM phải làm thêm câu `SELECT DISTINCT` để lấy đúng N post chứ không phải N dòng — đã chạy thử `findAndCount({ relations: { comments: true }, skip, take })`: ra **3 query** (DISTINCT id, lấy dữ liệu, `COUNT(DISTINCT)`).

Cách 2 — `'query'`: tách query riêng cho từng quan hệ

```
this.postsRepository.find({
  relations: { comments: true, tags: true },
  relationLoadStrategy: 'query',
});
// query 1: SELECT ... FROM post
// query 2: SELECT ... FROM comment WHERE postId IN (1, 2, 3)
// (+ query cho tags, và 1 query nhỏ ghép id quan hệ)
```

Không nhân bản dòng, mỗi bảng chỉ đọc đúng dữ liệu của nó — đây chính là cách Laravel `with()` làm mặc định (`WHERE post_id IN (...)`). Đổi lại là nhiều lượt đi-về DB hơn (vẫn cố định, không phụ thuộc N).

> **Đã kiểm chứng:** option có trong `FindOneOptions.d.ts` của TypeORM 1.1.1 (`relationLoadStrategy?: "join" | "query"` , chú thích gốc: *"If you are loading too much data with nested joins it's better to load relations using separate queries"*), và cũng đặt được mặc định cho cả app qua option cùng tên của DataSource (`BaseDataSourceOptions.d.ts`). Chạy thật với 3 post: `'join'` = 1 query, `'query'` = 3 query (post, comment `WHERE "postId" IN (?, ?, ?)` , và 1 query ghép id).

Tình huống	Cách 1: `'join'`	Cách 2: `'query'`
Load 1 quan hệ ManyToOne (comment → post, product → category)	✅ Mỗi dòng con chỉ JOIN đúng 1 cha, không nhân bản	Thừa lượt query
1 quan hệ 1-N ít phần tử (post → vài comment)	✅ 1 query, đơn giản	Chạy được, không cần thiết
Nhiều quan hệ 1-N / ManyToMany cùng lúc (comments + tags)	❌ Dòng nhân theo tích, dữ liệu phình to	✅ Mỗi quan hệ 1 query riêng
Quan hệ 1-N rất nhiều phần tử + phân trang	❌ Thêm DISTINCT subquery, nặng	✅ Phân trang trên bảng cha, rồi `IN (...)`
Cần lọc/sắp xếp theo cột bảng con trong cùng câu	✅ Bắt buộc JOIN (hoặc QueryBuilder)	❌ Query tách không lọc chéo được

Quy tắc thực dụng: cứ để mặc định `'join'` . Khi đọc log SQL (mục 9.5) thấy 1 request JOIN từ 2 quan hệ 1-N trở lên, hoặc JOIN quan hệ 1-N có hàng trăm phần tử — chuyển đúng query đó sang `relationLoadStrategy: 'query'` . Đừng đổi mặc định toàn app khi chưa đo.

9.9 ManyToMany — Post ↔ Tag qua bảng trung gian

1 post có nhiều tag, 1 tag gắn cho nhiều post → cần bảng trung gian (pivot) chứa cặp `(postId, tagId)` . TypeORM tự tạo bảng này khi bạn đặt `@JoinTable()` :

```
// tag.entity.ts
@Entity()
export class Tag {
  @PrimaryGeneratedColumn()
  id: number;

  @Column({ unique: true })
  name: string;

  @ManyToMany(() => Post, (post) => post.tags)
  posts: Post[];
}

// post.entity.ts — phía "sở hữu" quan hệ đặt @JoinTable()
@ManyToMany(() => Tag, (tag) => tag.posts, { cascade: ['insert'] })
@JoinTable()
tags: Tag[];
```

Gắn tag cho post: load post **kèm** tags hiện có, gán mảng mới rồi `save`. Với `cascade: ['insert']`, tag chưa tồn tại (chưa có `id`) sẽ được INSERT luôn trong cùng lần save:

```
async addTags(postId: number, names: string[]) {
  const post = await this.postsRepository.findOne({
    where: { id: postId },
    relations: { tags: true }, // BẮT BUỘC load tags cũ, nếu không save sẽ coi như xoá hết
  });
  if (!post) throw new NotFoundException();

  const existing = await this.tagsRepository.findBy({ name: In(names) });
  const newTags = names
    .filter((n) => !existing.some((t) => t.name === n))
    .map((name) => this.tagsRepository.create({ name }));

  post.tags = [
    ...post.tags,
    ...existing.filter((t) => !post.tags.some((pt) => pt.id === t.id)),
    ...newTags,
  ];
  return this.postsRepository.save(post);
}
```

Log thật khi save: `BEGIN TRANSACTION` → `INSERT INTO "tag"` (cho từng tag mới) → `INSERT INTO "post_tags_tag"("postId", "tagId") VALUES (1, 1), (1, 2)` → `COMMIT`. Tên bảng trung gian tự sinh theo mẫu `<bảng>_<field>_<bảng kia>` = `post_tags_tag`. Gỡ 1 tag: `post.tags = post.tags.filter((t) => t.id !== tagId)` rồi `save` — TypeORM tự DELETE dòng tương ứng trong bảng trung gian (không xoá tag).

> **Đối chiếu typeorm.io/docs/relations/many-to-many-relations:** *"@JoinTable() is required for @ManyToMany relations. You must put @JoinTable on one (owning) side of relation."* — chỉ đặt ở **1 phía**. Cách gỡ quan hệ bằng `filter` rồi `save` và tên bảng tự sinh (`question_categories_category` trong ví dụ docs) khớp với kết quả chạy thật ở trên. Đoạn `addTags` đã type-check bằng `tsc` với `typeorm` 1.1.1 đang cài.

9.10 `onDelete` và `cascade` — 2 thứ khác nhau, cùng vài cái bẫy

	`onDelete`	`cascade`
Ai thực thi	**Database** (ràng buộc khoá ngoại `ON DELETE ...`)	**TypeORM** (code JS lúc bạn gọi `save` / `remove`)
Giá trị	`'RESTRICT' | 'CASCADE' | 'SET NULL' | 'DEFAULT' | 'NO ACTION'`	`true` hoặc mảng `'insert' | 'update' | 'remove' | 'soft-remove' | 'recover'`
Có tác dụng khi	Xoá cha bằng BẤT KỲ cách nào (kể cả `DELETE` SQL tay, DB viewer)	Chỉ khi đi qua `repository.save()` / `remove()` của TypeORM
Đặt ở đâu	Phía có cột khoá ngoại — `@ManyToOne`	Phía bạn gọi `save` (thường là cha)

⚠ **Bẫy 1 — đặt `onDelete` nhầm phía thì bị bỏ qua im lặng.** Đã chạy thử đặt `onDelete: 'CASCADE'` lên `@OneToMany` (phía Post): bảng `comment` sinh ra khoá ngoại `ON DELETE NO ACTION` — không báo lỗi gì, chỉ là không cascade. Phải đặt ở `@ManyToOne` như mục 9.1. (Với better-sqlite3, TypeORM tự bật `PRAGMA foreign_keys = ON` khi kết nối nên `CASCADE` hoạt động thật — xoá 1 post đã thấy comment của nó biến mất theo.)

⚠ **Bẫy 2 — "làm rỗng mảng con rồi save" KHÔNG xoá con.** Load post kèm `comments`, gán `post.comments = []` rồi `save(post)`: log thật là `UPDATE "comment" SET "postId" = NULL` — comment vẫn còn trong bảng, thành "mồ côi" (`post: null`). Lý do: option `orphanedRowAction` của phía `@ManyToOne` mặc định là `'nullify'`, và cột khoá ngoại `@ManyToOne` mặc định **nullable**. Hai cách chặn: khai `nullable: false` ở `@ManyToOne` (TypeORM 1.1.1 khi đó không set NULL được nên chuyển sang xoá — đã kiểm tra trong `OneToManySubjectBuilder.js` và chạy thử), hoặc khai tường minh `orphanedRowAction: 'delete'`. Muốn xoá 1 comment thì cách rõ ràng nhất vẫn là `commentsRepository.remove(comment)`.

⚠ **Bẫy 3 — `cascade: true` + DTO từ client.** Với `cascade: true` ở `Post.comments`, `postsRepository.save({ title, comments: [{ content: '...' }] })` sẽ INSERT luôn comment lồng trong object (đã chạy thử: có cascade → comment được tạo; không cascade → comment lồng bị bỏ qua, không báo lỗi). Nếu bạn truyền thẳng body request vào `save()`, client có thể "nhét" dữ liệu con bạn không định cho tạo. `whitelist` của `ValidationPipe` (Part 5) chặn được field lạ ở tầng DTO — nhưng quy tắc an toàn là chỉ bật cascade đúng loại cần (`['insert']` như ví dụ Tag ở 9.9), không bật `true` cho mọi quan hệ.

Đối chiếu typeorm.io/docs/relations/relations: `nullable` — *"By default it is nullable"*; về cascade docs cảnh báo nguyên văn: *"Cascades may seem like a good and easy way to work with relations, but they may also bring bugs and security issues when some undesired object is being saved into the database."* Danh sách giá trị `onDelete`, `cascade`, `orphanedRowAction` (`'nullify' | 'delete' | 'soft-delete' | 'disable'`) lấy từ `OnDeleteType.d.ts` và `RelationOptions.d.ts` của TypeORM 1.1.1 đang cài.

Việc cần làm:

1. Tạo `Comment` entity + `CommentsModule` (dùng `nest g resource comments`)
2. Thiết lập quan hệ OneToMany/ManyToOne giữa Post và Comment — nhớ import `Comment` đầy đủ ở MỌI nơi dùng đến (mục 9.1)
3. `CommentsModule` đăng ký `TypeOrmModule.forFeature([Comment, Post])` — cần cả `Post` vì Service phải tự tra Post cha có tồn tại
4. Viết route `GET /posts/:id` trả về post kèm danh sách comment của nó (`relations: { comments: true }`)
5. Viết route lồng `POST /posts/:postId/comments` để tạo comment cho 1 post cụ thể
6. Thêm field `commentsCount` vào `GET /posts` (danh sách) bằng `loadRelationIdAndMap` + `.length` — xác nhận response không kéo theo nội dung comment nào
7. Bật `logging: ['query', 'error']` trong `TypeOrmModule.forRoot()`, gọi `GET /posts/:id` và đọc SQL in ra terminal (mục 9.5)
8. Cố tình viết 1 hàm N+1 (vòng lặp `find` comment cho từng post), đếm số dòng `query:` trong log; sửa bằng `relations` rồi đếm lại — phải còn 1
9. Thêm quan hệ `author` (ManyToOne → `User`) cho `Comment`, trả post kèm comment kèm tác giả bằng `relations: { comments: { author: true } }` — đảm bảo KHÔNG lộ field `password` của user (dùng `select`, mục 9.7)

10. Viết `GET /posts?commentContains=...` lọc post theo nội dung comment nhưng vẫn trả đủ comment (2 alias, mục 9.6)
11. Tạo `Tag` + ManyToMany với `Post`, route `POST /posts/:id/tags` nhận `{ names: string[] }` (mục 9.9); mở DB viewer xem bảng `post_tags_tag`
12. Tái hiện bẫy 2 ở mục 9.10 (gán `post.comments = []` rồi save), xem comment thành `postId = NULL`; sửa bằng `nullable: false` hoặc `orphanedRowAction: 'delete'` và xác nhận lại

---

PART 10

Authentication (JWT + Passport)

Mục tiêu: Hiểu toàn bộ luồng đăng ký → đăng nhập → cấp token → bảo vệ route bằng token.

10.1 Luồng tổng quan

```
1. POST /auth/register
   → hash password bằng bcrypt (KHÔNG BAO GIỜ lưu plain text password)
   → lưu user vào database
2. POST /auth/login
   → tìm user theo email
   → so sánh password gửi lên với hash đã lưu (bcrypt.compare)
   → nếu khớp → tạo JWT token (chứa payload: userId, email...) ký bằng secret key
   → trả token về client
3. Client lưu token, gửi kèm mọi request sau đó:
   Header: Authorization: Bearer <token>
4. Route được bảo vệ gắn @UseGuards(AuthGuard('jwt'))
   → PassportStrategy tự động verify chữ ký token bằng secret key
   → nếu hợp lệ → giải mã payload, gắn vào request.user
   → nếu không hợp lệ/hết hạn → tự động trả 401 Unauthorized
```

10.2 Vì sao dùng Passport?

`@nestjs/passport` là lớp tích hợp giữa NestJS và thư viện Passport.js (chuẩn công nghiệp cho authentication ở Node.js). Bạn định nghĩa một `JwtStrategy` (kế thừa `PassportStrategy`) khai báo cách lấy token (từ header), secret key để verify, và hàm `validate(payload)` chạy sau khi token đã được xác minh hợp lệ — trả về gì trong `validate()` thì thứ đó sẽ được gán vào `request.user`.

Về nguồn tài liệu chính thức (đối chiếu lần 2 với docs.nestjs.com): trang `docs.nestjs.com/security/authentication` nay đã viết lại hoàn toàn quanh 1 module mới `@nestjs/authentication` (kiến trúc provider/guard toàn cục khác hẳn, dùng `scrypt` thay `bcrypt` để hash password). Luồng Passport + JWT dạy ở Part này — vẫn là cách phổ biến nhất trong thực tế và trong phần lớn tài liệu/dự án hiện có — nay nằm ở trang riêng `docs.nestjs.com/recipes/passport`. Trang này xác nhận đúng cách tiếp cận của course: *"Passport is the most popular Node.js authentication library, well known in the community and used in many production applications"*, và vẫn khuyến nghị dùng `bcrypt` để hash password — khớp đúng mục 10.3 bên dưới.

10.3 Vì sao hash password bằng bcrypt

Bcrypt là hash **một chiều** (không giải mã ngược được) có "salt" ngẫu nhiên tích hợp — cùng 1 password hash 2 lần ra 2 chuỗi khác nhau, chống rainbow table. Khi login không "giải mã" hash, mà hash lại password vừa nhập rồi so sánh (`bcrypt.compare` làm việc này).

> **Liên hệ với `AuthGuard` giả lập ở Part 07:** Guard ở Part 07 chỉ kiểm tra "có header Authorization hay không", không verify token. Part này thay nó bằng `AuthGuard('jwt')` của Passport (verify chữ ký JWT thật) — nên bỏ Guard giả đi khi đã có Guard thật, tránh nhầm lẫn.

10.4 Passport (`JwtStrategy` + `AuthGuard('jwt')`) hay tự viết Guard verify JWT tay

Vì `AuthGuard` ở Part 07 chỉ là 1 class tự viết implement `CanActivate`, người mới hay hỏi: sao không viết luôn 1 Guard tự `verify()` token bằng tay trong đó, cần gì phải học thêm cả bộ Passport (`PassportStrategy`, đăng ký strategy, module riêng...)? Cả 2 đều ra cùng kết quả (verify token, gắn `request.user`) — khác nhau ở độ chuẩn hoá và khả năng mở rộng, không phải ở việc "cái nào chạy đúng hơn".

Cách 1 — Passport `JwtStrategy` + `AuthGuard('jwt')` (mục 10.1-10.2, mặc định nên dùng)

Passport xử lý sẵn phần lặp đi lặp lại: đọc token đúng chuẩn header `Bearer`, verify chữ ký + hạn dùng, tự trả 401 nhất quán khi token thiếu/sai/hết hạn, và cho phép cắm thêm strategy khác (Google OAuth, Local username/password...) dùng chung 1 kiến trúc `@UseGuards(AuthGuard('...'))` — không phải viết lại Guard mới cho mỗi kiểu đăng nhập.

Cách 2 — Tự viết Guard verify JWT thủ công

```
@Injectable()
export class ManualJwtGuard implements CanActivate {
  constructor(private readonly jwtService: JwtService) {}

  async canActivate(context: ExecutionContext): Promise<boolean> {
    const request = context.switchToHttp().getRequest();
    const token = request.headers.authorization?.split(' ')[1];
    if (!token) throw new UnauthorizedException();
    try {
      request.user = await this.jwtService.verifyAsync(token);
      return true;
    } catch {
      throw new UnauthorizedException('Invalid token');
    }
  }
}
```

Vẫn dùng `@nestjs/jwt` để verify (không phải tự cài thuật toán JWT) — chỉ bỏ qua tầng Passport, tự viết lại phần đọc header + xử lý lỗi. Ngắn hơn cho 1 dự án nhỏ chỉ có đúng 1 kiểu đăng nhập, nhưng phải tự lo mọi edge case (header sai định dạng, thiếu chữ "Bearer", token rỗng...) mà Passport đã xử lý sẵn.

> **Đối chiếu docs.nestjs.com/recipes/passport:** toàn bộ tài liệu chính thức về JWT auth CHỈ trình bày cách dùng Passport — không có ví dụ "tự verify tay" nào. Cách 2 ở trên vẫn chạy đúng, không sai kỹ thuật, nhưng là tự viết lại phần Passport đã làm sẵn, đi ngược lời khuyên "dùng thư viện phổ biến, production-ready" mà chính tài liệu chính thức nêu ra cho Passport.

Tiêu chí	Cách 1: Passport	Cách 2: Guard tự viết
Độ dài code ban đầu	Nhiều file hơn (Strategy + Guard + Module)	✅ Gọn, 1 file
Xử lý edge case (header sai, token rỗng, hết hạn)	✅ Passport tự lo, đã kiểm chứng qua rất nhiều dự án	❌ Tự viết, tự chịu trách nhiệm bắt hết case
Thêm kiểu đăng nhập khác sau này (Google OAuth, Local)	✅ Chỉ thêm 1 Strategy mới, tái dùng kiến trúc cũ	❌ Phải viết lại từ đầu cho từng kiểu
Phù hợp	Dự án thật, production, hoặc học đúng chuẩn ngành	Học để hiểu rõ JWT hoạt động ra sao "dưới lớp vỏ" Passport

Quy tắc thực dụng: luôn dùng Passport (Cách 1) cho dự án thật — đây là lý do course này dạy Cách 1. Chỉ tự viết Guard verify tay (Cách 2) khi mục đích là *học* để hiểu rõ JWT/Passport đang làm gì bên trong, rồi quay lại dùng Cách 1 cho code thật.

> **Việc cần làm:**
>
> 1. Tạo `User` entity, `AuthModule`
> 2. Cài `bcrypt`, `@nestjs/jwt`, `@nestjs/passport`, `passport-jwt`
> 3. Viết route `register` (hash password trước khi lưu) và `login` (verify + trả JWT)
> 4. Viết `JwtStrategy` + `JwtAuthGuard`, áp dụng lên các route cần đăng nhập (vd tạo/xoá post phải đăng nhập)

PART 11 — TUỲ CHỌN

Testing với Jest

> **Mục tiêu:** Viết unit test cho Service/Controller với dependency được mock (không cần database), và e2e test gọi HTTP thật vào app chạy trên SQLite trong RAM — biết khi nào dùng loại nào.

11.1 Unit test Service

Nest CLI tích hợp sẵn Jest. Với unit test, bạn dùng `Test.createTestingModule()` để tạo một "module giả lập" chỉ chứa đúng provider cần test — có thể **mock** Repository/dependency khác thay vì dùng database thật, giúp test chạy nhanh và độc lập. Code cụ thể ở mục 11.4–11.5.

11.2 e2e test

e2e (end-to-end) test khởi động toàn bộ app thật (dùng `supertest` để gửi HTTP request thật tới app trong bộ nhớ), kiểm tra toàn bộ luồng từ HTTP request tới response — gần với hành vi thực tế nhất nhưng chạy

chậm hơn unit test. Code cụ thể và 2 cái bẫy lớn (ghi nhầm vào DB dev, thiếu cấu hình global của `main.ts`) ở mục 11.6–11.7.

11.3 Cách chạy test trong project này — và vì sao có `--experimental-vm-modules`

```
npm test                  # unit test: mọi file *.spec.ts trong src/
npm test -- posts         # chỉ chạy file có đường dẫn khớp "posts"
npm run test:watch        # tự chạy lại khi sửa code
npm run test:cov          # kèm báo cáo coverage (thư mục coverage/)
npm run test:e2e          # e2e: file *.e2e-spec.ts trong test/, config test/jest-
e2e.json
```

Cấu hình unit test nằm ngay trong `package.json` (khối `"jest"`: `rootDir: "src"`, `testRegex: ".*\\.spec\\.ts$"`, `transform` bằng `ts-jest`, và `moduleNameMapper` để import kiểu `src/...` chạy được trong test). e2e dùng file riêng `test/jest-e2e.json`.

> **Vì sao mọi script test đều có `NODE_OPTIONS=--experimental-vm-modules` — đã kiểm chứng bằng cách chạy thật:** `@nestjs/typeorm` 12.0.1 đang cài là package **ESM-only** (`"type": "module"` trong `package.json` của nó, file `dist/index.js` bắt đầu bằng `export * from ...`). Jest (30.4) mặc định chạy test ở chế độ CommonJS; bỏ cờ này đi và chạy `npx jest src/posts` thì nhận ngay `SyntaxError: Unexpected token 'export'` tại dòng `import { getRepositoryToken } from '@nestjs/typeorm'`. Có cờ thì chạy được, đổi lại mỗi lần chạy in thêm 1 dòng `ExperimentalWarning: VM Modules is an experimental feature` — vô hại, bỏ qua. **Đừng xoá cờ này** khi "dọn" `package.json`.

> **Nếu đọc docs.nestjs.com/fundamentals/testing thấy `vi.fn()`:** trang docs hiện nói *"Newly generated projects use Vitest by default"* — project mới tạo bằng CLI bây giờ dùng Vitest. Project này tạo từ trước nên dùng Jest. Mọi API của `@nestjs/testing` (`Test.createTestingModule`, `overrideProvider`...) giống hệt nhau (docs: *"the testing APIs exposed by Nest don't depend on a specific runner"*) — chỉ cần đọc `vi.fn()` thành `jest.fn()`.

11.4 Unit test Service — mock Repository bằng `getRepositoryToken()`

Service nhận Repository qua `@InjectRepository(Post)`. Bên dưới, decorator đó chỉ là `@Inject(token)` với 1 token do `@nestjs/typeorm` sinh ra — `getRepositoryToken(Post)` trả về **đúng token đó**. Đăng ký 1 provider với token này + `useValue` là object giả, Nest sẽ inject object giả thay vì Repository thật — không cần database. Đây là file `src/posts/posts.service.spec.ts` của project:

```ts
import { Test } from '@nestjs/testing';
import { getRepositoryToken } from '@nestjs/typeorm';
import { NotFoundException } from '@nestjs/common';
import { PostsService } from './posts.service';
import { Post } from './enities/post.entity';

describe('PostsService', () => {
  let service: PostsService;
  const repo = {
    find: jest.fn(), findOne: jest.fn(), findOneBy: jest.fn(),
    create: jest.fn(), save: jest.fn(), remove: jest.fn(),
  };

  beforeEach(async () => {
    jest.clearAllMocks(); // xoá lịch sử gọi của test trước
    const module = await Test.createTestingModule({
      providers: [
        PostsService,
        { provide: getRepositoryToken(Post), useValue: repo },
      ],
    }).compile();
    service = module.get(PostsService);
  });

  it('findOne ném NotFoundException khi không có', async () => {
    repo.findOne.mockResolvedValue(null);
    await expect(service.findOne(999)).rejects.toThrow(NotFoundException);
  });

  it('create gọi create rồi save', async () => {
    const dto = { title: 'New', body: 'Body' };
    repo.create.mockReturnValue(dto);                // create() ĐỒNG BỘ
    repo.save.mockResolvedValue({ id: 7, ...dto });  // save() trả Promise
    await expect(service.create(dto)).resolves.toEqual({ id: 7, ...dto });
    expect(repo.save).toHaveBeenCalledWith(dto);
  });
});
```

Chọn đúng hàm mock theo kiểu trả về của hàm thật: hàm trả Promise (`find` , `save`) → `mockResolvedValue` / `mockRejectedValue` ; hàm đồng bộ (`create`) → `mockReturnValue` . Dùng nhầm `mockReturnValue` cho `save` thì `await` vẫn chạy nhưng bạn đang test 1 hành vi không bao giờ xảy ra thật.

⚠ **Bug thật đang có trong project — test này hiện ĐỎ.** Chạy `npm test` ngay lúc viết tài liệu này, cả 3 test của `posts.service.spec.ts` đều fail với:

```
Nest can't resolve dependencies of the PostsService (PostRepository, ?).
Please make sure that the argument DataSource at index [1] is available
in the RootTestModule module.
```

Lý do: ở Part 13, `PostsService` được thêm tham số constructor thứ 2 là `DataSource` (để chạy transaction), nhưng file spec vẫn chỉ cung cấp Repository. Testing module **chỉ có đúng những provider bạn liệt kê** — thiếu 1 dependency là không compile được module. Sửa: thêm 1 provider giả cho `DataSource`. Token của nó chính là class `DataSource` (hàm `getDataSourceToken()` của `@nestjs/typeorm` trả về đúng class `DataSource` khi dùng kết nối mặc định — đã đọc source):

```
import { DataSource } from 'typeorm';

const manager = { save: jest.fn() };
const dataSource = {
  // giả lập transaction(): gọi luôn callback với manager giả
  transaction: jest.fn((cb: (m: typeof manager) => unknown) => cb(manager)),
};

providers: [
  PostsService,
  { provide: getRepositoryToken(Post), useValue: repo },
  { provide: DataSource, useValue: dataSource },
],
```

Có `dataSource` giả rồi thì test luôn được logic transaction mà không cần DB:

```
it('createWithWelcomeComment lưu post rồi comment gắn với post đó', async () => {
  manager.save
    .mockResolvedValueOnce({ id: 9, title: 'T', body: 'B' }) // lần 1: save Post
    .mockResolvedValueOnce({});                              // lần 2: save Comment

  await expect(service.createWithWelcomeComment({ title: 'T', body: 'B' }))
    .resolves.toMatchObject({ id: 9 });
  expect(manager.save).toHaveBeenNthCalledWith(
    2, Comment, expect.objectContaining({ post: { id: 9, title: 'T', body: 'B' } }),
  );
});

it('lỗi trong transaction được ném ra ngoài', async () => {
  manager.save.mockRejectedValueOnce(new Error('db down'));
  await expect(service.createWithWelcomeComment({ title: 'T', body: 'B' }))
    .rejects.toThrow('db down');
});
```

> **Đã chạy thật:** các đoạn trên (spec gốc + provider `DataSource` + 2 test transaction) được chạy bằng Jest của project, dùng đúng `PostsService` thật trong `src/` — tất cả xanh. `getRepositoryToken` được export từ `@nestjs/typeorm` 12.0.1 (`dist/common/typeorm.utils.d.ts`); docs.nestjs.com/data/typeorm mục Testing mô tả đúng cách này: *"The @nestjs/typeorm package exports the getRepositoryToken() function, which returns this token"*. Lưu ý: mock `transaction` chỉ kiểm tra **logic của bạn** (gọi save đúng thứ tự, đúng tham số) — nó KHÔNG chứng minh rollback thật hoạt động; việc đó cần DB thật (mục 11.6).

11.5 Test Controller — mock cả Service, và `overrideProvider`

Controller chỉ nên điều phối (nhận input → gọi Service → trả kết quả), nên khi test Controller thì mock luôn Service. Có 2 cách khai mock:

```
// Cách A — tự liệt kê provider giả (giống 11.4)
const postsService = { findOne: jest.fn().mockResolvedValue({ id: 3 }) };
const moduleRef = await Test.createTestingModule({
  controllers: [PostsController],
  providers: [{ provide: PostsService, useValue: postsService }],
}).compile();

const controller = moduleRef.get(PostsController);
await expect(controller.findOne(3)).resolves.toEqual({ id: 3 });
expect(postsService.findOne).toHaveBeenCalledWith(3);

// Cách B — nạp provider THẬT rồi "đè" bằng overrideProvider
const moduleRef2 = await Test.createTestingModule({
  controllers: [PostsController],
  providers: [PostsService],
})
  .overrideProvider(PostsService)
  .useValue({ findOne: jest.fn().mockResolvedValue({ id: 4 }) })
  .compile();
```

Cách A đủ cho unit test 1 class. Cách B có giá trị khi bạn `imports: [PostsModule]` nguyên cả module (không tự liệt kê được provider bên trong) mà chỉ muốn thay 1 provider — đây là cách docs dùng trong e2e. Bên cạnh `overrideProvider` còn `overrideGuard` , `overrideInterceptor` , `overridePipe` , `overrideFilter` (đều nhận `useValue` / `useClass` / `useFactory`) và `overrideModule(...).useModule(...)` .

> **Gọi thẳng method Controller = bỏ qua toàn bộ Pipe/Guard/Interceptor.** `controller.findOne(3)` ở trên là lời gọi hàm JS bình thường — `ParseIntPipe` , `@UseGuards(JwtAuthGuard)` , `ValidationPipe` **không chạy**. Test Controller kiểu này chỉ kiểm tra "Controller có chuyển đúng tham số cho Service không". Muốn test "gửi `id=abc` thì 400", "không token thì 401" → phải đi qua HTTP thật, tức e2e (11.6).

11.6 e2e với supertest + SQLite in-memory — đừng để test ghi vào DB dev

File `test/app.e2e-spec.ts` mà CLI sinh sẵn làm `imports: [AppModule]` . Trong project này `AppModule` kết nối `database: 'db.sqlite3'` với `synchronize: true` — nghĩa là **chạy e2e sẽ đọc/ghi thẳng vào database dev của bạn**: test tạo post rác, test xoá dữ liệu thật, và kết quả test phụ thuộc dữ liệu đang có. Cách sửa gọn nhất: e2e tự dựng module với 1 DB SQLite trong RAM (`':memory:'`) — mỗi lần chạy là 1 DB trống mới, chạy xong tự biến mất:

```
// test/posts.e2e-spec.ts
import { Test } from '@nestjs/testing';
import { INestApplication, ValidationPipe } from '@nestjs/common';
import { TypeOrmModule } from '@nestjs/typeorm';
import request from 'supertest';
import { PostsModule } from '../src/posts/posts.module';
import { CommentsModule } from '../src/comments/comments.module';
import { JwtAuthGuard } from '../src/auth/jwt-auth.guard';
import { TransformInterceptor } from
'../src/common/interceptors/transform.interceptor';

describe('Posts (e2e)', () => {
  let app: INestApplication;

  beforeAll(async () => {
    const moduleRef = await Test.createTestingModule({
      imports: [
        TypeOrmModule.forRoot({
          type: 'better-sqlite3',
          database: ':memory:',    // DB trong RAM, không đụng db.sqlite3
          autoLoadEntities: true,
          synchronize: true,
          retryAttempts: 0,        // lỗĩ câú hình thì fail ngay, không retry 9 lần
        }),
        PostsModule,
        CommentsModule,            // BẮT BUỘC: entity Post có quan hệ tới Comment
      ],
    })
      .overrideGuard(JwtAuthGuard)
      .useValue({ canActivate: () => true }) // bỏ qua đăng nhập trong test này
      .compile();

    app = moduleRef.createNestApplication();
    app.useGlobalPipes(new ValidationPipe({ whitelist: true, forbidNonWhitelisted:
true, transform: true }));
    app.useGlobalInterceptors(new TransformInterceptor());
    await app.init();
  });

  afterAll(async () => {
    await app.close();
  });

  it('POST /posts body sai → 400', () => {
    return request(app.getHttpServer()).post('/posts').send({ title: '' }).expect(400);
  });

  it('tạo rồi liệt kê', async () => {
    await request(app.getHttpServer()).post('/posts').send({ title: 'A', body: 'B'
}).expect(201);
    const res = await request(app.getHttpServer()).get('/posts?
page=1&limit=5').expect(200);
    expect(res.body).toMatchObject({ success: true, meta: { total: 1, page: 1, limit: 5
```

```
  } });
    });
});
```

> **Đã chạy thật** (Jest của project, import đúng `PostsModule` / `CommentsModule` / `JwtAuthGuard` / `TransformInterceptor` thật từ `src/`): cả 2 test xanh. Bỏ `CommentsModule` ra thì module không khởi động được: `TypeORMError: Entity metadata for Post#comments was not found` — vì `autoLoadEntities` chỉ nạp entity nào được `forFeature()` ở module đã import. Khung `createNestApplication()` → `app.init()` → `request(app.getHttpServer())` → `app.close()` khớp ví dụ e2e trên docs.nestjs.com/fundamentals/testing.

11.7 Bẫy lớn nhất của e2e: app test KHÔNG chạy `main.ts`

`main.ts` là nơi bạn gọi `app.useGlobalPipes(new ValidationPipe(...))`, `useGlobalFilters`, `useGlobalInterceptors`. Nhưng e2e tạo app bằng `moduleRef.createNestApplication()` — hàm `bootstrap()` trong `main.ts` **không bao giờ được gọi**. Kết quả: app trong test là 1 app "trần", thiếu hết cấu hình global. Đã chạy thử đúng test ở 11.6 nhưng **bỏ** dòng `useGlobalPipes`:

Request	Có ValidationPipe (như app thật)	Thiếu ValidationPipe (e2e quên)
`POST /posts` body `{ title: '' }`	**400** — `["title should not be empty", "body should not be empty", ...]`	**500** — body lọt xuống DB, SQLite báo `NOT NULL constraint failed: post.body`
`GET /posts?page=1&limit=5`	`meta.page = 1` (number)	`meta.page = "1"` (string — không có `transform`)

Tức là test có thể **xanh trong khi app thật hành xử khác**, hoặc ngược lại. Chính file `test/app.e2e-spec.ts` hiện tại là ví dụ: nó expect response `'Hello World!'` trần — chỉ đúng vì trong test không có `TransformInterceptor`; app thật (có interceptor từ `main.ts`) trả `{ success: true, data: 'Hello World!' }`. Hai cách sửa triệt để (bạn tự chọn và tự sửa code):

- **Tách hàm cấu hình dùng chung:** viết `export function configureApp(app: INestApplication) { app.useGlobalPipes(...); app.useGlobalFilters(...); app.useGlobalInterceptors(...); }` trong 1 file riêng; `main.ts` và mọi e2e đều gọi hàm này → không bao giờ lệch nhau.
- **Đăng ký global qua DI** trong `AppModule`: `providers: [{ provide: APP_PIPE, useValue: new ValidationPipe({...}) }]` (tương tự `APP_FILTER`, `APP_INTERCEPTOR` — 4 hằng số này export từ `@nestjs/core`). Khi đó cấu hình nằm trong module nên e2e `imports: [AppModule]` tự có. Docs có lưu ý: muốn `override` được enhancer global kiểu này trong test thì dùng `useExisting` thay vì `useClass`.

> **Phát hiện kèm theo khi chạy thử:** `main.ts` hiện tại của project khai `ValidationPipe({ whitelist: true, forbidNonWhitelisted: true })` — **chưa có** `transform: true` như mục 5.2 hướng dẫn. Vì vậy ngay cả app thật lúc này cũng trả `meta.page` dạng chuỗi `"1"`. Đây là việc cần sửa trong code (mục 5.2 / 12.2), tài liệu không sửa hộ.

11.8 Test Guard — 2 tầng

Unit test Guard: Guard chỉ là 1 class có `canActivate(context)`. Tạo 1 `ExecutionContext` giả chỉ có đúng phần Guard dùng tới, rồi gọi thẳng. Ví dụ với `AuthGuard` tự viết ở mục 7.2:

```
function contextWithHeaders(headers: Record<string, string>): ExecutionContext {
  return {
    switchToHttp: () => ({ getRequest: () => ({ headers }) }),
  } as unknown as ExecutionContext; // chỉ giả phần Guard thực sự gọi
}

it('thiếu header → 401', () => {
  expect(() => new AuthGuard().canActivate(contextWithHeaders({})))
    .toThrow(UnauthorizedException);
});

it('có header → cho qua', () => {
  expect(new AuthGuard().canActivate(contextWithHeaders({ authorization: 'Bearer x'
})))
    .toBe(true);
});
```

e2e với Guard thật (`JwtAuthGuard`): Guard của Passport cần `JwtStrategy` đã đăng ký → import `AuthModule` , mà `AuthModule` đọc `JWT_SECRET` qua `ConfigService` → cấp secret cho test bằng `ConfigModule` với `load` (không đọc file `.env` thật):

```
imports: [
  ConfigModule.forRoot({
    isGlobal: true,
    ignoreEnvFile: true,
    load: [() => ({ JWT_SECRET: 'test-secret' })],
  }),
  TypeOrmModule.forRoot({ type: 'better-sqlite3', database: ':memory:',
autoLoadEntities: true, synchronize: true }),
  PostsModule, CommentsModule, AuthModule,
],
// ...
it('không token → 401', () =>
  request(app.getHttpServer()).post('/posts').send({ title: 'a', body: 'b'
}).expect(401));
```

Để test luồng "có token hợp lệ → 201", gọi `POST /auth/register` rồi `POST /auth/login` ngay trong test, lấy token và gửi header `.set('Authorization', `Bearer ${token}`)` . `overrideGuard` (11.6) chỉ dùng khi test đó **không** nhằm kiểm tra auth.

> **Đã chạy thật:** cả unit test Guard (fake `ExecutionContext`) và e2e 401 với `JwtAuthGuard` + `AuthModule` thật đều xanh. `ignoreEnvFile` , `load` có trong `config-module-options.interface.d.ts` của `@nestjs/config` đang cài; `overrideGuard` / `overrideProvider` / `useMocker` có trong `testing-module.builder.d.ts` của `@nestjs/testing` 11.2.3.

11.9 Matcher / công cụ hay dùng

Viết	Kiểm tra
`expect(x).toBe(y)`	Bằng nhau tuyệt đối (`===`) — dùng cho số, chuỗi, boolean
`toEqual(obj)` / `toStrictEqual(obj)`	So sánh sâu từng field; `toStrictEqual` khắt khe hơn (phân biệt field `undefined` , class khác nhau)
`toMatchObject({ ... })`	Object **chứa** các field này (thừa field khác vẫn qua) — hợp với response API
`expect.objectContaining({...})` , `expect.any(Number)` , `expect.arrayContaining([...])`	"Khớp một phần" đặt bên trong matcher khác
`toHaveLength(n)` , `toContainEqual(item)` , `toBeNull()` , `toBeDefined()`	Độ dài mảng, mảng chứa phần tử, null/undefined
`await expect(p).resolves.toEqual(v)` / `.rejects.toThrow(NotFoundException)`	Promise resolve ra giá trị / reject với lỗi — **nhớ** `await` , thiếu thì test luôn xanh
`expect(fn).toThrow(...)`	Hàm **đồng bộ** ném lỗi (truyền hàm, không gọi hàm)
`toHaveBeenCalledWith(...)` , `toHaveBeenCalledTimes(n)` , `toHaveBeenNthCalledWith(i, ...)`	Mock được gọi với tham số nào, mấy lần, lần thứ i
`jest.fn()` , `.mockResolvedValue()` , `.mockRejectedValue()` , `.mockReturnValue()` , `...Once()`	Tạo mock và điều khiển giá trị trả về (`Once` = chỉ cho lần gọi kế tiếp)
`jest.clearAllMocks()`	Xoá lịch sử gọi giữa các test (đặt trong `beforeEach`)
supertest: `.expect(201)` , `res.body` , `.set('Authorization', ...)`	Status HTTP, body JSON đã parse, gửi header

11.10 Unit test với mock hay e2e với DB thật (in-memory) — viết loại nào

Người mới hay hỏi: "mock hết thì test chẳng chứng minh gì, mà e2e thì chậm — rốt cuộc viết cái nào?". Hai loại trả lời 2 câu hỏi khác nhau, dự án thật cần cả hai — câu hỏi đúng là "**logic này** nên test ở tầng nào".

Cách 1 — Unit test + mock (`getRepositoryToken` , `useValue`)

Kiểm tra **logic nghiệp vụ trong 1 class**: nhánh `if` , ném đúng exception, tính đúng `totalPages` , gọi Repository với đúng tham số. Chạy mili-giây, không cần DB, lỗi chỉ ra đúng hàm hỏng. Nhưng: mock trả gì là do bạn quyết định — nếu bạn hiểu sai cách TypeORM hoạt động (vd tưởng `where` theo quan hệ không lọc mảng con, mục 9.6), mock sẽ "đồng ý" với cái hiểu sai đó.

Cách 2 — e2e + SQLite `:memory:` (mục 11.6)

Kiểm tra **cả đường ống thật**: routing, Pipe/Guard/Interceptor, SQL do TypeORM sinh ra, ràng buộc DB (NOT NULL, FK, unique), transaction commit/rollback thật. Chậm hơn (mỗi file khởi động cả Nest app), lỗi đỏ thì phải tự dò xem hỏng ở tầng nào.

> **Đối chiếu docs.nestjs.com/fundamentals/testing + docs.nestjs.com/data/typeorm:** docs trình bày cả 2 loại song song — unit test với custom provider thay Repository (*"mockRepository is used as the User repository"*) và e2e với supertest — không coi loại nào thay thế loại nào. Lưu ý: SQLite in-memory không giống 100% Postgres/MySQL production (kiểu dữ liệu, isolation level — xem 13.5); nếu production dùng Postgres, e2e quan trọng nên chạy trên Postgres thật (Docker) ở CI.

Cần kiểm tra	Cách 1: Unit + mock	Cách 2: e2e + DB in-memory
Nhánh logic, exception, phép tính trong Service	✅ Nhanh, chỉ đúng chỗ hỏng	Được, nhưng chậm và khó dựng đủ mọi nhánh
Validation (400), auth (401/403), format response	❌ Gọi thẳng method bỏ qua Pipe/Guard	✅ Bắt buộc
Query TypeORM đúng (relations, where, phân trang)	❌ Mock không chạy SQL	✅ Bắt buộc
Rollback transaction, ràng buộc FK/NOT NULL	❌ Chỉ giả lập được	✅ Bắt buộc
Hàm tiện ích thuần (tính toán, format)	✅ Không cần Nest luôn — test hàm trực tiếp	Thừa

Quy tắc thực dụng: mỗi endpoint có ít nhất 1 e2e cho đường thành công + các mã lỗi chính (400/401/404) — đây là lưới an toàn thật sự. Viết thêm unit test cho Service nào có logic rẽ nhánh đáng kể. Không viết unit test chỉ để "test rằng mock trả đúng cái mình vừa bảo nó trả".

> **Việc cần làm (tuỳ chọn, làm sau khi xong Part 1-10):**
>
> 1. Viết unit test cho `PostsService.findAll()` và `findOne()` với repository được mock
> 2. Viết 1 e2e test đơn giản cho route `GET /posts`
> 3. Chạy `npm test`, xác nhận lỗi `Nest can't resolve dependencies of the PostsService (PostRepository, ?)`; sửa spec bằng provider giả cho `DataSource` (mục 11.4) cho tới khi xanh
> 4. Thêm 2 unit test cho `createWithWelcomeComment()`: thành công (thứ tự `save`) và lỗi được ném ra ngoài (mục 11.4)
> 5. Viết unit test cho `PostsController.findOne` với `PostsService` mock (mục 11.5)
> 6. Viết `test/posts.e2e-spec.ts` dùng SQLite `:memory:` (mục 11.6) — xác nhận file `db.sqlite3` không đổi sau khi chạy `npm run test:e2e`
> 7. Thử xoá dòng `useGlobalPipes` trong e2e, thấy body sai trả 500 thay vì 400 (mục 11.7); rồi tách hàm `configureApp()` dùng chung cho `main.ts` và e2e
> 8. e2e cho auth: không token → 401; register + login lấy token → tạo post được 201 (mục 11.8)

PART 12

Pagination

> **Mục tiêu:** Không trả về toàn bộ bảng cùng lúc — chia trang, giới hạn số lượng bản ghi mỗi lần trả về client; hiểu offset và cursor khác nhau ở tầng SQL, sort an toàn, và cách duyệt bảng lớn mà không nạp hết vào RAM.

12.1 Vì sao cần pagination

Ở Part 8-9, `GET /posts` luôn trả **toàn bộ** bảng. Với 20 post thì không vấn đề gì, nhưng với 200,000 post — mỗi request sẽ kéo hết dữ liệu, tốn băng thông, tốn RAM, chậm cả server và client. Pagination giải quyết bằng cách chỉ trả về 1 "trang" (page) nhỏ mỗi lần, kèm thông tin để client biết còn bao nhiêu trang.

12.2 Query DTO cho pagination

```
import { Type } from 'class-transformer';
import { IsInt, IsOptional, Max, Min } from 'class-validator';

export class PaginationQueryDto {
  @IsOptional()
  @Type(() => Number)
  @IsInt()
  @Min(1)
  page: number = 1;

  @IsOptional()
  @Type(() => Number)
  @IsInt()
  @Min(1)
  @Max(100)
  limit: number = 10;
}
```

> **Vì sao bắt buộc phải có `@Type(() => Number)` ở đây:** query string trong URL (`?page=2&limit=10`) luôn được Express/Nest parse thành **string**, kể cả khi bạn khai `page: number` trong DTO. Khác với body JSON (nơi `class-transformer` tự suy ra kiểu số nếu JSON gửi số), query string cần `@Type(() => Number)` tường minh để `class-transformer` biết ép `"2"` thành `2` trước khi `class-validator` chạy `@IsInt()` — thiếu dòng này, `@IsInt()` luôn fail vì nhận vào string.

`@Max(100)` ở `limit` — chặn client tự ý xin `?limit=999999` (biến pagination thành vô nghĩa, lại quay về vấn đề tải hết bảng).

12.3 Dùng `skip` / `take` với TypeORM

```
async findAll(query: PaginationQueryDto) {
  const { page, limit } = query;
  const [data, total] = await this.postsRepository.findAndCount({
    skip: (page - 1) * limit,
    take: limit,
    order: { id: 'DESC' },
  });

  return {
    data,
    meta: { total, page, limit, totalPages: Math.ceil(total / limit) },
  };
}
```

`findAndCount()` chạy 2 việc trong 1 lần gọi: lấy đúng `limit` bản ghi bắt đầu từ vị trí `skip` (SQL `LIMIT` / `OFFSET`), **và** đếm tổng số bản ghi thoả điều kiện (không tính `LIMIT`) — cần cả 2 số này để tính `totalPages`. Nếu chỉ dùng `find({ skip, take })` thường, bạn sẽ không biết tổng cộng có bao nhiêu trang.

Route Controller — nhớ dùng `@Query()`, không phải `@Param()`:

```
@Get()
findAll(@Query() query: PaginationQueryDto) {
  return this.postsService.findAll(query);
}
```

`ValidationPipe` global (Part 5) tự áp dụng luôn cho `@Query()`, không cần cấu hình thêm — nhưng phải đảm bảo `transform: true` đã được khai báo tường minh trong `main.ts` (mục 5.2 đã sửa lại để có dòng này — `transform` KHÔNG bật sẵn theo mặc định của `ValidationPipe`) để `@Type()` ở mục 12.2 thực sự chạy.

12.4 Tương tác với TransformInterceptor (Part 07)

`TransformInterceptor` đang bọc mọi response thành `{ success: true, data }`. Khi Service pagination tự trả về `{ data, meta }`, Interceptor sẽ bọc thêm 1 lớp thành `{ success: true, data: { data, meta } }` — lồng 2 lần chữ `data`, khó đọc. Không có đáp án "đúng tuyệt đối" — bạn tự chọn trade-off giữa "1 format nhất quán toàn app" (Part 07) và "pagination cần thêm field":

- **Cách 1:** Sửa Interceptor để nếu payload đã có sẵn field `meta` thì spread thẳng ra `{ success: true, data: result.data, meta: result.meta }` thay vì bọc thêm.
- **Cách 2:** Bỏ qua vấn đề, chấp nhận cấu trúc lồng — client đọc `response.data.data` và `response.data.meta`.

12.5 Offset và Cursor (keyset) — khác nhau ở câu SQL

Cách `skip` / `take` ở 12.3 gọi là **offset pagination**. Còn 1 kiểu nữa là **cursor pagination** (còn gọi keyset): thay vì "bỏ qua N dòng đầu", client gửi lên **id của bản ghi cuối cùng** đã thấy, server lấy tiếp các bản ghi "sau" nó.

```
-- Offset: trang 3, mỗi trang 10 (skip 20, take 10)
SELECT * FROM post ORDER BY id DESC LIMIT 10 OFFSET 20;

-- Cursor: "các post cũ hơn post id=481", mỗi lần 10
SELECT * FROM post WHERE id < 481 ORDER BY id DESC LIMIT 10;
```

Vì sao `OFFSET` lớn thì chậm: database không "nhảy" thẳng tới dòng thứ 250.000 được — nó phải đọc lần lượt 250.000 dòng đầu theo thứ tự sắp xếp rồi vứt đi, sau đó mới lấy 10 dòng bạn cần. Trang càng sâu càng chậm. Còn `WHERE id < 481` đi thẳng vào index của khoá chính, đọc đúng 10 dòng — trang 1 hay trang 50.000 tốn như nhau.

> **Đo thật** (better-sqlite3, bảng 500.000 post trong RAM, mỗi câu chạy 20 lần lấy trung bình, trên máy build tài liệu — con số tuyệt đối tuỳ máy, xu hướng mới là điều cần nhìn):
>
> | Vị trí | `LIMIT 10 OFFSET n` | `WHERE id < x LIMIT 10` |
> |---|---|---|
> | Đầu bảng (n = 0) | 0.03 ms | 0.01 ms |
> | n = 10.000 | 0.12 ms | 0.01 ms |
> | n = 250.000 | 4.30 ms | 0.01 ms |
> | n = 490.000 | 9.68 ms | 0.01 ms |
>
> `EXPLAIN QUERY PLAN` của câu cursor cho ra `SEARCH post USING INTEGER PRIMARY KEY (rowid<?)` — tức là tìm thẳng bằng khoá chính. Offset tăng tuyến tính theo `n`; cursor gần như không đổi. Trên Postgres/MySQL với bảng thật trên đĩa, chênh lệch còn lớn hơn nhiều.

Vấn đề thứ 2 của offset — trùng/sót khi dữ liệu thay đổi: client đang xem trang 1 (post 100→91), trong lúc đó có 1 post mới được tạo. Client bấm trang 2 (`OFFSET 10`) → mọi thứ bị đẩy lùi 1 vị trí → post 91 xuất hiện **lại** ở đầu trang 2. Ngược lại nếu có post bị xoá thì 1 post bị **sót**. Cursor không bị: "các post có id < 91" luôn đúng bất kể phía trên có thêm/bớt gì.

12.6 Cursor pagination với TypeORM — code & response

```
// common/dto/cursor-query.dto.ts
export class CursorQueryDto {
  @IsOptional()
  @Type(() => Number)
  @IsInt()
  @Min(1)
  cursor?: number;      // không gửi = trang đầu tiên

  @IsOptional()
  @Type(() => Number)
  @IsInt()
  @Min(1)
  @Max(100)             // chặn trần giống 12.2
  limit: number = 20;
}

// posts.service.ts
async feed(query: CursorQueryDto) {
  const { cursor, limit } = query;
  const rows = await this.postsRepository.find({
    where: cursor ? { id: LessThan(cursor) } : {},
    order: { id: 'DESC' },
    take: limit + 1, // lấy DƯ 1 dòng để biết còn trang sau không
  });

  const hasMore = rows.length > limit;
  const data = hasMore ? rows.slice(0, limit) : rows;
  return {
    data,
    meta: { limit, hasMore, nextCursor: hasMore ? data[data.length - 1].id : null },
  };
}
```

Mẹo `take: limit + 1`: nếu DB trả về nhiều hơn `limit` dòng thì chắc chắn còn trang sau — không cần chạy thêm `COUNT(*)` (thứ vốn cũng chậm trên bảng lớn). Client gọi lần đầu `GET /posts/feed?limit=20`, lần sau gửi lại `?cursor=<nextCursor>&limit=20`, dừng khi `hasMore = false`.

	Offset (12.3)	**Cursor (12.6)**
Client gửi	`?page=3&limit=10`	`?cursor=481&limit=10`
`meta` trả về	`{ total, page, limit, totalPages }`	`{ limit, hasMore, nextCursor }`
Số query	2 (`findAndCount` = dữ liệu + `COUNT`)	1
Nhảy tới trang bất kỳ	✅	❌ Chỉ "tiếp theo"

⚠ **Thứ tự khai route trong Controller:** `@Get('feed')` phải nằm **phía trên** `@Get(':id')`. Nest đăng ký route theo thứ tự khai báo, và `:id` khớp với mọi chuỗi — kể cả chữ `feed`. Đã chạy thử: khai `:id` trước thì `GET /posts/feed` rơi vào `findOne`, `ParseIntPipe` trả `400 Validation failed (numeric string is expected)`; đảo thứ tự thì trả đúng feed.

```
@Get('feed')     // route cố định — đặt TRƯỚC
feed(@Query() query: CursorQueryDto) { return this.postsService.feed(query); }

@Get(':id')      // route có tham số — đặt SAU
findOne(@Param('id', ParseIntPipe) id: number) { /* ... */ }
```

Với `TransformInterceptor` của project: type guard `isPaginated()` trong `src/common/pagination.ts` chỉ kiểm tra object có key `data` và `meta` → response cursor ở trên cũng được "trải phẳng" thành `{ success, data, meta }` đúng như offset. Nhưng interface `Paginated<T>` / `PaginationMeta` hiện khai cứng các field của offset (`total`, `totalPages`...) — nên khai thêm 1 type riêng cho meta của cursor thay vì ép kiểu.

Sắp xếp theo cột KHÔNG unique (vd `createdAt`): nhiều post có thể trùng `createdAt` → `WHERE createdAt < :c` sẽ bỏ sót các post cùng thời điểm với con trỏ. Phải thêm cột unique làm "trọng tài" (tie-breaker) và cursor mang cả 2 giá trị:

```
qb.where('post.createdAt < :c OR (post.createdAt = :c AND post.id < :id)', { c, id
})
  .orderBy('post.createdAt', 'DESC')
  .addOrderBy('post.id', 'DESC')
  .take(limit + 1);
```

Đã chạy thử với dữ liệu mỗi ngày 3 post trùng `createdAt`: không sót, không trùng. Sort theo `id` (như 12.6) thì không cần, vì `id` đã unique.

12.7 Sort do client chọn — bắt buộc whitelist

Cho client chọn `?sort=title` là nhu cầu rất thường gặp. Cách sai là đưa thẳng chuỗi từ query vào query:

```
// ✘ SAI — chuỗi của client đi thẳng vào SQL
qb.orderBy(`post.${query.sort}`);
// ✘ SAI — ít nguy hiểm hơn nhưng vẫn sai
repo.find({ order: { [query.sort]: 'ASC' } });
```

Cách 1 là **SQL injection**: `orderBy()` nhận chuỗi SQL thô, không được bind như tham số `:search` — đã chạy thử `?sort=id, (SELECT COUNT(*) FROM sqlite_master)` và câu subquery đó được ghép nguyên văn vào `ORDER BY` rồi thực thi. Cách 2 an toàn hơn vì TypeORM kiểm tra tên field — nhưng field không tồn tại thì TypeORM ném `EntityPropertyNotFoundError: Property "xyz" was not found in "Post"` (đã chạy thử) → client nhận **500** thay vì 400. Tệ hơn, field **có tồn tại** nhưng nhạy cảm vẫn lọt: `GET /users?sort=password` sẽ sắp xếp theo hash mật khẩu. Luôn validate bằng danh sách cho phép:

```ts
import { IsIn, IsInt, IsOptional, Max, Min } from 'class-validator';

const SORTABLE = ['id', '-id', 'title', '-title'] as const; // "-" = giảm dần
type SortKey = (typeof SORTABLE)[number];

export class PostListQueryDto extends PaginationQueryDto {
  @IsOptional()
  @IsIn(SORTABLE)       // ngoài danh sách → 400 do ValidationPipe
  sort: SortKey = '-id';
}

function toOrder(sort: SortKey): FindOptionsOrder<Post> {
  const direction = sort.startsWith('-') ? 'DESC' : 'ASC';
  const field = sort.replace('-', '') as 'id' | 'title';
  return { [field]: direction };
}

// trong findAll: order: toOrder(query.sort)
```

> **Đã kiểm chứng:** `IsIn(values: readonly any[])` có trong `class-validator` 0.15.1 đang cài (nhận được mảng `as const`); toàn bộ đoạn DTO + `toOrder` + `feed()` đã type-check bằng `tsc` với `typeorm` 1.1.1; lỗi `EntityPropertyNotFoundError` khi `order` chứa field lạ là kết quả chạy thật. Cùng nguyên tắc cho mọi tham số client điều khiển cấu trúc query (tên cột lọc, hướng sắp xếp): whitelist ở DTO, không bao giờ nối chuỗi.

12.8 Duyệt dữ liệu lớn KHÔNG phân trang — xử lý theo lô

Pagination là cho client. Khi **chính server** cần đi qua toàn bộ bảng (export CSV, gửi mail hàng loạt, sửa dữ liệu cũ), `repository.find()` không điều kiện sẽ nạp cả trăm nghìn entity vào RAM cùng lúc. Cách làm: lấy **từng lô**, xử lý xong lô này mới lấy lô sau — và dùng đúng kỹ thuật keyset ở 12.5 cho vòng lặp:

```ts
async *iterateInBatches(batchSize = 500): AsyncGenerator<Post[]> {
  let lastId = 0;
  while (true) {
    const batch = await this.postsRepository.find({
      where: { id: MoreThan(lastId) },
      order: { id: 'ASC' },
      take: batchSize,
    });
    if (batch.length === 0) return;
    yield batch;
    lastId = batch[batch.length - 1].id;
  }
}

// dùng: đọc như vòng lặp thường, bên dưới là từng lô 500
for await (const batch of this.iterateInBatches()) {
  for (const post of batch) { /* ghi 1 dòng CSV... */ }
}
```

Vì sao `WHERE id > lastId` mà không phải `skip: i * 500`: (1) lý do tốc độ ở 12.5 — lô cuối của bảng lớn sẽ rất chậm; (2) nếu trong lúc duyệt bạn **sửa hoặc xoá** chính các dòng đang duyệt, offset bị lệch và **bỏ sót** dòng — keyset theo id thì không.

> ⚠ **`QueryBuilder.stream()` KHÔNG dùng được với SQLite.** TypeORM có `.stream()` (trả về `ReadStream`, đọc từng dòng, RAM thấp nhất) và nhiều tutorial khuyên dùng cho export lớn. Nhưng với driver `better-sqlite3`, gọi nó ném ngay `TypeORMError: Stream is not supported by sqlite driver.` (đã chạy thử; source `AbstractSqliteQueryRunner.js` ném lỗi này vô điều kiện). Stream chỉ dùng được khi chuyển sang driver hỗ trợ như Postgres/MySQL — với project này, dùng vòng lặp lô ở trên.

	`find()` tất cả	Lô bằng `skip` / `take`	Lô bằng keyset (`id > lastId`)	`.stream()`
RAM	❌ Cả bảng	1 lô	1 lô	1 dòng
Lô cuối bảng lớn	—	❌ Chậm dần (OFFSET)	✅ Nhanh như lô đầu	✅
An toàn khi sửa/xoá dòng đang duyệt	✅	❌ Sót dòng	✅	Tuỳ DB
better-sqlite3 (project này)	✅	✅	✅	❌ Không hỗ trợ

> Việc duyệt cả bảng như thế này không nên chạy trong 1 request HTTP (client chờ lâu, dễ timeout). Chỗ đúng của nó là 1 script chạy tay (như `npm run seed` của project) hoặc background job ở phần Queue phía sau.

12.9 Offset (có số trang) hay Cursor (cuộn vô hạn) — chọn cái nào

Hai kiểu không phải "cũ và mới" — mỗi kiểu phục vụ 1 kiểu giao diện. Câu hỏi quyết định: **người dùng có cần nhảy tới "trang 7" hoặc biết "tổng cộng bao nhiêu" không?**

Cách 1 — Offset (`skip` / `take` + `findAndCount` , mục 12.3)

Có `total`, `totalPages`, nhảy trang tuỳ ý — đúng thứ bảng quản trị (admin) cần. Trả giá: 1 câu `COUNT` mỗi request, trang sâu chậm dần, dữ liệu thay đổi liên tục thì có thể trùng/sót giữa các trang.

Cách 2 — Cursor/keyset (`WHERE id < :cursor` , mục 12.6)

Tốc độ như nhau ở mọi độ sâu, không trùng/sót khi có bản ghi mới chen vào — đúng thứ feed/timeline/"tải thêm" cần. Trả giá: không có tổng số, không nhảy trang, cột sắp xếp phải unique (hoặc có tie-breaker), và đổi kiểu sắp xếp là phải đổi cả cách tạo cursor.

> **Căn cứ:** SQL của cả 2 cách là log thật từ TypeORM 1.1.1 (`findAndCount({ skip: 20, take: 10 })` → `... ORDER BY "Post"."id" DESC LIMIT 10 OFFSET 20` + `SELECT COUNT(1)` ; `LessThan(3)` + `take: 3` → `WHERE (("Post"."id" < 3)) ORDER BY "Post"."id" DESC LIMIT 3`), số đo tốc độ ở 12.5. Laravel (Part 12 tài liệu Laravel) tách đúng 2 kiểu này thành `paginate()` và `cursorPaginate()` với cùng ràng buộc "cột sắp xếp phải unique" — TypeORM không có hàm dựng sẵn cho cursor, nên phải tự viết như 12.6.

Giao diện / tình huống	Cách 1: Offset	Cách 2: Cursor
Bảng admin có số trang 1, 2, 3... và "Tổng: 1.234"	✅ Dùng cách này	❌ Không có tổng, không nhảy trang
Feed / bình luận / "Tải thêm" / cuộn vô hạn trên mobile	Chạy được nhưng dễ trùng bài khi có bài mới	✅ Dùng cách này
Bảng lớn (hàng trăm nghìn dòng trở lên), người dùng lướt rất sâu	❌ Trang sâu chậm dần	✅ Tốc độ không đổi
Sắp xếp theo nhiều kiểu do client chọn (12.7)	✅ Đổi `order` là xong	Phức tạp — mỗi kiểu sort cần cursor riêng

Quy tắc thực dụng: mặc định dùng offset cho mọi danh sách (đã có sẵn ở 12.3, giới hạn `limit ≤ 100`). Chỉ thêm 1 endpoint cursor riêng (vd `GET /posts/feed`) cho đúng màn hình cuộn vô hạn — không cần chuyển cả API sang cursor.

Việc cần làm:

1. Tạo `PaginationQueryDto` với `page`, `limit` có default + validate
2. Sửa `PostsService.findAll()` dùng `findAndCount()` với `skip` / `take`
3. Trả kèm `meta: { total, page, limit, totalPages }`
4. Tự quyết định và xử lý vấn đề lồng response với `TransformInterceptor` (mục 12.4)
5. Test `GET /posts?page=2&limit=5`, xác nhận đúng số lượng và đúng trang
6. Thêm `sort` có whitelist bằng `@IsIn` (mục 12.7); test `?sort=body` và `?limit=1000` đều phải nhận 400
7. Seed vài nghìn post (sửa `src/seed.ts`), bật logging, so sánh SQL của `?page=200` với endpoint cursor (mục 12.5)
8. Viết `GET /posts/feed?cursor=...&limit=...` theo mục 12.6 (khai `@Get('feed')` TRƯỚC `@Get(':id')`); gọi liên tiếp bằng `nextCursor` tới khi `hasMore = false`, xác nhận không trùng/sót post nào
9. Trong lúc đang "cuộn" feed, tạo thêm 1 post mới — xác nhận trang tiếp theo của feed không bị lặp bài, còn offset thì có
10. Viết script duyệt toàn bộ post theo lô 500 bằng `iterateInBatches()` (mục 12.8); thử gọi `.stream()` để tự thấy lỗi của driver SQLite

PART 13

Transactions

> **Mục tiêu:** Đảm bảo nhiều thao tác ghi database "thành công cùng nhau hoặc thất bại cùng nhau" (atomicity); hiểu isolation level, giới hạn của SQLite khi chạy đồng thời, deadlock & retry, và vì sao tác dụng phụ (event, mail, HTTP) không được nằm trong transaction.

13.1 Vấn đề transaction giải quyết

Hình dung: tạo 1 `Post` mới, đồng thời cũng cần tạo sẵn 1 `Comment` chào mừng đầu tiên gắn với post đó. Nếu `save(post)` thành công nhưng `save(comment)` thất bại (lỗi mạng, lỗi validate, crash giữa chừng...) — bạn còn lại 1 `Post` "mồ côi" không có comment đi kèm như thiết kế, dữ liệu ở trạng thái nửa vời. Transaction đảm bảo: **hoặc cả 2 lệnh cùng thành công (commit), hoặc cả 2 cùng bị hoàn tác (rollback)** như chưa từng xảy ra.

13.2 Cách đơn giản — `DataSource.transaction()`

```
import { DataSource } from 'typeorm';

@Injectable()
export class PostsService {
  constructor(private readonly dataSource: DataSource) {}

  async createWithFirstComment(dto: CreatePostDto, firstComment: string) {
    return this.dataSource.transaction(async (manager) => {
      const post = await manager.save(PostEntity, dto);
      await manager.save(Comment, { content: firstComment, post });
      // throw ở BẤT KỲ đâu trong callback này → toàn bộ tự rollback
      return post;
    });
  }
}
```

`DataSource` (inject trực tiếp qua constructor, không cần `@InjectRepository`) đại diện cho kết nối database gốc. `.transaction(callback)` tự động: mở transaction (SQL `BEGIN`) → chạy callback → nếu callback chạy xong không lỗi thì `COMMIT` (ghi thật vào DB) → nếu callback `throw` bất kỳ đâu thì tự `ROLLBACK` (hoàn tác mọi thay đổi, như chưa từng chạy).

> **Lưu ý bắt buộc:** mọi lệnh ghi database trong callback phải gọi qua `manager` được truyền vào (KHÔNG dùng `this.postsRepository.save()` như bình thường) — vì `manager` chính là "phiên làm việc" đang nằm trong transaction đó. Nếu lỡ tay dùng lại Repository thường bên ngoài, lệnh đó sẽ chạy trong 1 transaction/connection KHÁC, không được rollback cùng nhóm.

13.3 Cách kiểm soát tay — `QueryRunner`

```
async createWithFirstComment(dto: CreatePostDto, firstComment: string) {
  const queryRunner = this.dataSource.createQueryRunner();
  await queryRunner.connect();
  await queryRunner.startTransaction();

  try {
    const post = await queryRunner.manager.save(PostEntity, dto);
    await queryRunner.manager.save(Comment, { content: firstComment, post });
    await queryRunner.commitTransaction();
    return post;
  } catch (err) {
    await queryRunner.rollbackTransaction();
    throw err;
  } finally {
    // LUÔN release, kể cả khi lỗi — tránh rò rỉ connection
    await queryRunner.release();
  }
}
```

Cách này viết dài hơn `DataSource.transaction()` nhưng cho kiểm soát chi tiết hơn — cần khi bạn phải truyền **cùng 1** `queryRunner` xuyên qua nhiều Service khác nhau (ví dụ 1 luồng nghiệp vụ gọi cả `PostsService` và `CommentsService`, cả 2 phải nằm trong đúng 1 transaction). `DataSource.transaction()` ở mục 13.2 không làm được việc này vì `manager` chỉ tồn tại trong phạm vi callback, không truyền tự nhiên qua lời gọi Service khác được.

13.4 So sánh nhanh

	`DataSource.transaction()`	`QueryRunner` thủ công
Độ dài code	Ngắn	Dài hơn, nhiều bước tay
Tự rollback khi throw	✅ Tự động	Phải tự viết try/catch/rollback
Dùng chung qua nhiều Service	Khó (manager giới hạn trong callback)	✅ Dễ — truyền `queryRunner` làm tham số
Phù hợp	Transaction đơn giản, gọn trong 1 Service	Transaction phức tạp, nhiều Service tham gia

> **Đối chiếu docs.nestjs.com/data/typeorm (mục "Transactions"):** cả 2 ví dụ code ở mục 13.2 và 13.3 khớp gần như nguyên văn với ví dụ chính thức (cùng thứ tự gọi `connect()` → `startTransaction()` → `try/catch/finally` với `commitTransaction()`/`rollbackTransaction()`/`release()`). Tài liệu chính thức cũng nêu đúng trade-off ở bảng trên: khuyến nghị `QueryRunner` khi cần "full control over the transaction", đồng thời cảnh báo nhược điểm là khó test hơn vì phải mock nguyên `DataSource` — một lý do nữa (ngoài bảng trên) để ưu tiên `DataSource.transaction()` khi không thực sự cần kiểm soát tay.

13.5 Isolation level — transaction "nhìn thấy" gì của transaction khác

Atomicity (13.1) trả lời "cả nhóm cùng thành công hay cùng thất bại". Isolation trả lời câu khác: **khi 2 transaction chạy cùng lúc, cái này thấy được bao nhiêu thay đổi của cái kia**. Mức càng cao càng an toàn nhưng càng dễ phải chờ nhau/bị huỷ. TypeORM cho truyền mức isolation làm tham số **đầu tiên**:

```
// DataSource.transaction(isolationLevel, callback)
return this.dataSource.transaction('SERIALIZABLE', async (manager) => {
  return manager.save(PostEntity, dto);
});

// QueryRunner: truyền vào startTransaction()
await queryRunner.startTransaction('SERIALIZABLE');

// Hoặc đặt mặc định cho mọi transaction của DataSource
TypeOrmModule.forRoot({ /* ... */ isolationLevel: 'SERIALIZABLE' });
```

Mức	Chặn được	better-sqlite3 (project này)	Postgres / MySQL
`READ UNCOMMITTED`	Gần như không gì — có thể đọc dữ liệu chưa commit ("dirty read")	✅ Chấp nhận	✅ (Postgres nhận nhưng xử lý như `READ COMMITTED`)
`READ COMMITTED`	Dirty read. Đọc 2 lần cùng 1 dòng trong 1 transaction vẫn có thể ra 2 giá trị khác	❌ Ném lỗi	✅ (mặc định của Postgres)
`REPEATABLE READ`	Thêm: dòng đã đọc thì đọc lại vẫn như cũ	❌ Ném lỗi	✅ (mặc định của MySQL InnoDB)
`SERIALIZABLE`	Mọi hiện tượng — kết quả như thể các transaction chạy lần lượt	✅ Chấp nhận	✅ (Postgres có thể huỷ transaction với lỗi `40001`, xem 13.7)

Đã kiểm chứng: chữ ký `transaction(isolationLevel: IsolationLevel, runInTransaction)` có trong `DataSource.d.ts` / `EntityManager.d.ts`, `startTransaction(isolationLevel?)` trong `QueryRunner.d.ts`, option `isolationLevel` trong `BaseDataSourceOptions.d.ts` của TypeORM 1.1.1. Driver SQLite khai `supportedIsolationLevels = ["READ UNCOMMITTED", "SERIALIZABLE"]`; chạy thật: 2 mức này chạy được (TypeORM gửi `PRAGMA read_uncommitted = true/false` rồi `BEGIN TRANSACTION`), còn `'READ COMMITTED'` ném `TypeORMError: READ COMMITTED isolation level is not supported` (tương tự `'REPEATABLE READ'`). Trang typeorm.io/docs/transactions ghi đúng: SQLite chỉ hỗ trợ `READ UNCOMMITTED` và `SERIALIZABLE`. Cột Postgres/MySQL theo docs gốc: postgresql.org — *"Read Committed is the default isolation level in PostgreSQL"*, *"PostgreSQL's Read Uncommitted mode behaves like Read Committed"*; dev.mysql.com — *"The default isolation level for InnoDB is REPEATABLE READ"*.

Hệ quả khi đổi DB: code viết `transaction('READ COMMITTED', ...)` chạy tốt trên Postgres nhưng **crash** khi chạy e2e bằng SQLite in-memory (Part 11). Nếu cần mức isolation cụ thể, chỉ dùng những mức cả DB dev/test lẫn DB production đều hỗ trợ — với cặp SQLite + Postgres là `SERIALIZABLE`. Còn không cần thì đừng truyền: để DB dùng mặc định của nó.

13.6 better-sqlite3 chỉ có 1 kết nối — hệ quả với transaction chạy song song

Postgres/MySQL có **pool** nhiều connection: mỗi `createQueryRunner()` lấy 1 connection riêng, 2 request đồng thời mở 2 transaction độc lập. **better-sqlite3 thì không** — cả app dùng chung đúng 1 connection. Trong TypeORM 1.1.1, `BetterSqlite3Driver.createQueryRunner()` luôn trả về **cùng 1 object** QueryRunner (`this.queryRunner ??= new BetterSqlite3QueryRunner(this)`), và `release()` gần như không làm gì (chú thích trong source: *"sqlite do not support multiple connections thus query runners"*). Options của driver còn khai `poolSize?: never`.

Hệ quả khi 2 request cùng mở transaction chồng lên nhau (đã chạy thử: request A mở transaction, save rồi chờ 50ms rồi ném lỗi; trong lúc đó request B mở transaction, save rồi xong):

```
query: BEGIN TRANSACTION                ← A mở
query: INSERT INTO "post" ... ["A"]     ← A ghi
query: SAVEPOINT typeorm_1              ← B "mở" = chỉ là SAVEPOINT lồng trong A
query: INSERT INTO "post" ... ["B"]     ← B ghi
query: RELEASE SAVEPOINT typeorm_1      ← B "commit" — nhưng chưa ghi thật
query: ROLLBACK                         ← A lỗi → rollback CẢ post của B

Kết quả: A nhận lỗi (đúng), B nhận "thành công" — nhưng post B KHÔNG có trong DB.
```

⚠ **Điều này nghĩa là gì với bạn:** trên project học (1 người gọi API mỗi lúc) gần như không bao giờ gặp. Nhưng đừng dùng SQLite để "test tải" logic transaction, và đừng suy ra "transaction của mình an toàn khi đồng thời" từ việc nó chạy đúng trên SQLite — hành vi trên Postgres khác hẳn. Hai process khác nhau (vd server đang chạy + `npm run seed`, hoặc DB viewer đang ghi) mở 2 connection tới cùng file `db.sqlite3` thì lại là chuyện khác: SQLite khoá cả file khi ghi, bên kia chờ tối đa `timeout` (mặc định 5000ms của better-sqlite3) rồi nhận `SqliteError: database is locked` (mã `SQLITE_BUSY`) — đã chạy thử với `timeout: 200` : lỗi sau ~205ms, bọc trong `QueryFailedError` của TypeORM.

13.7 Deadlock & retry — chạy lại cả transaction, đúng loại lỗi

Deadlock xảy ra khi 2 transaction chờ khoá của nhau (A giữ dòng 1 chờ dòng 2, B giữ dòng 2 chờ dòng 1). Database tự phát hiện và **giết 1 trong 2**; transaction bị giết nhận lỗi và đã bị rollback. Ở mức `SERIALIZABLE`, Postgres còn chủ động huỷ transaction khi phát hiện xung đột ("serialization failure"). Cả 2 trường hợp, cách xử lý chuẩn là **chạy lại toàn bộ transaction từ đầu** vài lần. TypeORM không có sẵn tham số retry như Laravel (`DB::transaction(..., attempts: 5)`), nên tự viết 1 helper nhỏ:

```ts
import { QueryFailedError } from 'typeorm';

// 40001 = serialization_failure, 40P01 = deadlock_detected (Postgres)
// SQLITE_BUSY = file SQLite đang bị process khác khoá (13.6)
const RETRYABLE = new Set(['40001', '40P01', 'SQLITE_BUSY']);

export async function withRetry<T>(fn: () => Promise<T>, attempts = 3): Promise<T> {
  for (let attempt = 1; ; attempt++) {
    try {
      return await fn();
    } catch (err) {
      const code = err instanceof QueryFailedError
        ? (err.driverError as { code?: string }).code
        : undefined;
      if (!code || !RETRYABLE.has(code) || attempt >= attempts) throw err;
      await new Promise((resolve) => setTimeout(resolve, 50 * 2 ** attempt)); // chờ tăng dần
    }
  }
}

// dùng: bọc NGOÀI dataSource.transaction(), không phải bên trong callback
return withRetry(() =>
  this.dataSource.transaction(async (manager) => {
    const post = await manager.save(PostEntity, dto);
    await manager.save(Comment, { content: 'Chào mừng!', post });
    return post;
  }),
);
```

- **Chỉ retry đúng loại lỗi tạm thời.** Lỗi validate, `NotFoundException`, vi phạm unique... chạy lại bao nhiêu lần cũng vậy — retry chỉ làm chậm và che lỗi.
- **Bọc bên ngoài cả transaction.** Bắt lỗi rồi thử lại *bên trong* callback là vô nghĩa: transaction đó đã bị DB huỷ, mọi lệnh tiếp theo trong nó đều hỏng.
- **Callback phải chạy lại được an toàn (idempotent)**: không gửi mail, không gọi API ngoài, không đẩy job bên trong — nếu không, mỗi lần retry lại gửi thêm 1 lần (13.8).

> **Đã kiểm chứng:** `QueryFailedError` export từ `typeorm` với field `readonly driverError` (`QueryFailedError.d.ts`); mã `40001 serialization_failure` và `40P01 deadlock_detected` theo bảng mã lỗi chính thức postgresql.org/docs/current/errcodes-appendix.html; mã `SQLITE_BUSY` là giá trị `driverError.code` thật đã bắt được ở thí nghiệm 13.6. Helper đã type-check bằng `tsc`. Với MySQL, deadlock báo bằng mã khác (`ER_LOCK_DEADLOCK`) — nếu đổi sang MySQL phải bổ sung vào danh sách.

13.8 Bẫy lớn: gửi mail / bắn event / đẩy job từ BÊN TRONG transaction

```
// ✘ SAI
await this.dataSource.transaction(async (manager) => {
  const post = await manager.save(PostEntity, dto);   // ① INSERT — CHƯA commit
  this.events.emit('post.created', post);             // ② listener chạy NGAY
  await manager.save(Comment, { content: '...', post }); // ③ lỗi ở đây → ROLLBACK
});
```

Hai kiểu hỏng: (1) listener/job chạy ở ② đi tìm post trong DB qua 1 connection khác → **không thấy** vì chưa commit; (2) ③ lỗi, transaction rollback — nhưng mail "bài viết đã đăng" thì đã gửi đi, không thu hồi được. Với 13.7 còn tệ hơn: mỗi lần retry lại bắn thêm 1 event. Quy tắc: **transaction chỉ chứa lệnh ghi DB; mọi tác dụng phụ ra bên ngoài chạy SAU khi transaction đã commit xong**:

```
// ✔ ĐÚNG — await transaction xong (đã COMMIT) rồi mới phát event
const post = await this.dataSource.transaction(async (manager) => {
  const saved = await manager.save(PostEntity, dto);
  await manager.save(Comment, { content: '...', post: saved });
  return saved;
});
this.events.emit('post.created', post); // chỉ tới được dòng này khi đã commit
```

Cơ chế event trong Nest và cách listener/queue xử lý tình huống này sẽ học chi tiết ở phần sau — xem Part 19. Ở đây chỉ cần nhớ vị trí đặt: **sau** `await dataSource.transaction(...)`, không phải bên trong callback.

> **Vì sao trên project này bug (1) bị "giấu":** với better-sqlite3 chỉ có 1 connection (13.6), nên listener đọc DB lại đi qua **đúng connection đang mở transaction** và *thấy* dữ liệu chưa commit — đã chạy thử: trong callback transaction, đọc lại post vừa INSERT bằng repository thường (không qua `manager`) vẫn tìm thấy, rồi transaction ROLLBACK và post đó biến mất. Code sai vẫn chạy "đúng" trên SQLite — chuyển sang Postgres mới lộ. Đây là lý do nên học quy tắc, không dựa vào việc "chạy thử thấy ổn".

13.9 Transaction phải ngắn — không gọi HTTP, không chờ gì bên trong

Từ lúc `BEGIN` tới `COMMIT`, các dòng/bảng transaction đã ghi bị **khoá** — transaction khác muốn ghi vào đó phải chờ. Với SQLite còn nặng hơn: ghi là khoá **cả file database**. Transaction càng dài, người khác chờ càng lâu, khả năng deadlock (13.7) càng cao.

```
// ✘ SAI — gọi API ngoài (có thể mất vài giây, hoặc treo) khi đang giữ khoá
await this.dataSource.transaction(async (manager) => {
  const post = await manager.save(PostEntity, dto);
  const summary = await this.aiClient.summarize(post.body); // HTTP 3 giây...
  await manager.update(PostEntity, post.id, { summary });
});

// ✔ ĐÚNG — làm việc chậm TRƯỚC, transaction chỉ còn các lệnh ghi
const summary = await this.aiClient.summarize(dto.body);
await this.dataSource.transaction(async (manager) => {
  await manager.save(PostEntity, { ...dto, summary });
});
```

Checklist trước khi viết 1 transaction: validate input, đọc dữ liệu tham chiếu, gọi service ngoài, tính toán nặng — làm hết **trước**. Bên trong chỉ còn các lệnh `manager.save/update/delete` cần "cùng thành công hoặc cùng thất bại". (`aiClient` / `summary` ở trên chỉ là ví dụ minh hoạ, project không có.)

13.10 Quan sát transaction qua log SQL

Bật `logging: ['query', 'error']` như mục 9.5 là cách nhanh nhất để **chứng minh** transaction đang làm đúng việc. Log thật của `createWithWelcomeComment()` :

```
-- thành công
query: BEGIN TRANSACTION
query: INSERT INTO "post"("id", "title", "body") VALUES (NULL, ?, ?) -- PARAMETERS:
[...]
query: INSERT INTO "comment"("id", "content", "postId") VALUES (NULL, ?, 1) --
PARAMETERS: [...]
query: COMMIT

-- callback throw sau khi save comment
query: BEGIN TRANSACTION
query: INSERT INTO "post" ...
query: INSERT INTO "comment" ...
query: ROLLBACK
```

Những gì cần tự kiểm tra khi đọc log: (1) mọi lệnh ghi nằm **giữa** `BEGIN` và `COMMIT` — nếu thấy 1 `INSERT` nằm ngoài, bạn đã lỡ dùng repository thường thay vì `manager` (lưu ý bắt buộc ở 13.2); (2) thấy `SAVEPOINT typeorm_1` nghĩa là đang có transaction lồng trong transaction — cố ý (gọi `manager.transaction()` bên trong) hoặc do nhiều request chồng nhau trên SQLite (13.6); (3) khoảng thời gian giữa `BEGIN` và `COMMIT` ngắn (13.9). Thêm `maxQueryExecutionTime: 500` vào cấu hình để TypeORM tự log cảnh báo cho câu nào chạy quá 500ms.

> **Đã kiểm chứng:** định dạng dòng log `query: ... -- PARAMETERS: [...]` là output thật của logger mặc định với `logging: ['query']` ; chuỗi `BEGIN TRANSACTION` / `COMMIT` / `ROLLBACK` / `SAVEPOINT typeorm_N` là đúng các lệnh `AbstractSqliteQueryRunner.js` của TypeORM 1.1.1 gửi xuống SQLite. `maxQueryExecutionTime` có trong `BaseDataSourceOptions.d.ts` ; typeorm.io/docs/logging: đặt `maxQueryExecutionTime: 1000` để *log queries exceeding one second*.

Việc cần làm:

1. Viết method `createWithFirstComment()` trong `PostsService` dùng `DataSource.transaction()`
2. Test case thành công: gọi API, xác nhận cả `Post` và `Comment` đều xuất hiện trong database
3. Test case rollback: chủ động sửa tạm code để `throw new Error('test rollback')` ngay sau khi save comment — gọi lại API, xác nhận **cả Post cũng KHÔNG được lưu** (kiểm tra bằng DevDb/SQLite Viewer hoặc `GET /posts`) — đây là bằng chứng transaction hoạt động đúng
4. Xoá dòng `throw` test sau khi xác nhận xong
5. Bật logging, chạy lại 2 case trên và đối chiếu log với mục 13.10 (`BEGIN` … `COMMIT` / `ROLLBACK`)
6. Thử `dataSource.transaction('READ COMMITTED', ...)` để tự thấy lỗi của SQLite, rồi đổi sang `'SERIALIZABLE'` (mục 13.5)

7. Viết helper `withRetry()` (mục 13.7) và bọc quanh `createWithWelcomeComment()`; viết unit test: lần 1 ném `QueryFailedError` có `driverError.code = 'SQLITE_BUSY'`, lần 2 thành công → hàm trả kết quả; ném lỗi thường → không retry
8. Rà lại mọi transaction trong project: không có gọi HTTP, gửi mail, emit event nào nằm TRONG callback (mục 13.8, 13.9)

PART 14

Config & biến môi trường

Mục tiêu: Mọi thứ khác nhau giữa máy dev, máy test và server production (secret, đường dẫn DB, port, thời hạn token) phải nằm **ngoài code**, được **kiểm tra ngay lúc khởi động**, và được đọc qua **một cửa duy nhất** có kiểu rõ ràng. Cuối Part này, app phải từ chối khởi động nếu thiếu `JWT_SECRET`, thay vì chạy lên rồi lỗi ở request login đầu tiên.

14.1 Vì sao cấu hình phải nằm ngoài code — và `ConfigModule` làm gì bên trong

Cùng một bản build (`dist/`) phải chạy được ở nhiều môi trường: ở máy bạn dùng `db.sqlite3` và secret tuỳ ý, ở server dùng DB thật và secret thật. Nếu những giá trị đó nằm cứng trong code, muốn đổi môi trường là phải sửa code, và secret sẽ nằm trong lịch sử git mãi mãi. Giải pháp chuẩn: đọc từ **biến môi trường** (`process.env`). Ở máy dev, cho tiện, ta ghi chúng vào file `.env` và nhờ thư viện nạp vào.

Project đang có sẵn `ConfigModule.forRoot({ isGlobal: true })` trong `AppModule` và file `.env` chứa `PORT`, `JWT_SECRET`. Mở source `node_modules/@nestjs/config/dist/config.module.js` (bản 4.0.4 đang cài), hàm `forRoot()` làm đúng thứ tự sau:

```
ConfigModule.forRoot(options)   ← chạy khi file app.module.ts được nạp
  │
  ├─ 1. Đọc từng file trong envFilePath (mặc định: <cwd>/.env) bằng dotenv.parse
  │     nhiều file → giá trị ở file ĐỨNG TRƯỚC thắng
  │
  ├─ 2. Trộn với biến môi trường thật:  { ...giá trị từ file, ...process.env }
  │     → biến đặt từ shell/Docker/hosting THẮNG giá trị trong .env
  │
  ├─ 3. Nếu có validate / validationSchema → chạy kiểm tra
  │     lỗi → throw → app KHÔNG khởi động (fail fast)
  │     hợp lệ → giữ lại bản "đã validate" (có thể đã ép kiểu số, gán default)
  │
  ├─ 4. Ghi ngược vào process.env những key CHƯA có (luôn thành string)
  │
  └─ 5. Đăng ký provider ConfigService (global nếu isGlobal: true)
        ConfigService.get(key) tra theo thứ tự:
        config từ load/registerAs → bản đã validate → process.env → default
```

Bẫy đọc `process.env` ở top-level của file: `ConfigModule.forRoot()` chỉ chạy khi decorator `@Module()` của `AppModule` được thực thi — tức là **sau khi** mọi file mà `app.module.ts` import đã được nạp xong. Viết `export const jwtConstants = { secret: process.env.JWT_SECRET }` ở đầu 1 file thuộc `AuthModule` thì lúc dòng đó chạy, `.env` chưa được đọc. Đã chạy thử: hằng số module-level ra `undefined`, còn `process.env.FOO` đọc sau khi `forRoot()` chạy thì có giá trị. Đây là lý do project đọc `JWT_SECRET` bên trong `useFactory` và constructor của `JwtStrategy` (chạy muộn, lúc DI khởi tạo) — đúng cách.

Nếu bạn đến từ Laravel: `.env` + `config/*.php` + `config('app.debug')` bên đó tương ứng với `.env` + `registerAs()` + `ConfigService` bên này. Quy tắc vàng cũng giống nhau: **chỉ đọc biến môi trường thô ở tầng cấu hình, code nghiệp vụ đọc qua service cấu hình** (mục 14.7).

14.2 `ConfigModule.forRoot()` — các option cần biết

Option	Mặc định	Tác dụng
`isGlobal`	`false`	Đăng ký global: mọi module inject được `ConfigService` mà không cần `imports: [ConfigModule]`. Project đang bật — đó là lý do `AuthModule` dùng được `ConfigService` mà không import gì
`envFilePath`	`<cwd>/.env`	1 đường dẫn hoặc mảng. Mảng: "If a variable is found in multiple files, the first one takes precedence" (docs)
`ignoreEnvFile`	`false`	`true` = không đọc file nào, chỉ dùng biến môi trường thật (thường dùng ở production)
`cache`	`false`	Cache giá trị đọc từ `process.env` trong RAM, tăng tốc `get()`
`expandVariables`	`false`	Cho phép `APP_URL=http://localhost:${PORT}` trong `.env` (qua `dotenv-expand`)
`load`	`[]`	Danh sách factory cấu hình (`registerAs`, mục 14.5)
`validate`	—	Hàm tự viết nhận object env, trả object đã kiểm tra hoặc throw (mục 14.3)
`validationSchema` + `validationOptions`	—	Kiểm tra bằng schema Joi (mục 14.3)
`validatePredefined`	`true`	Có đưa cả biến môi trường sẵn có của hệ điều hành (`PATH`, `HOME`...) vào bước validate không
`skipProcessEnv`	`false`	`true` = `ConfigService.get()` không đọc `process.env`

Cấu hình gợi ý cho project, tách file theo môi trường:

```
const env = process.env.NODE_ENV ?? 'development';

@Module({
  imports: [
    ConfigModule.forRoot({
      isGlobal: true,
      cache: true,
      envFilePath: [`.env.${env}.local`, `.env.${env}`, '.env'],
      validate,
    }),
    ...
  ],
})
export class AppModule {}
```

Ở đây đọc thẳng `process.env.NODE_ENV` là hợp lệ và bắt buộc: đây là bài toán "con gà quả trứng" — cần biết môi trường để chọn file `.env` nào, nên `NODE_ENV` phải đến từ shell (`NODE_ENV=test npm run test:e2e`), không thể đến từ chính file `.env` .

File	Commit lên git?	Dùng cho
`.env`	❌ Không (project đã có trong `.gitignore`)	Giá trị mặc định ở máy bạn
`.env.example`	✅ Có	"Bản kê khai" mọi biến cần có, giá trị giả — người khác clone về copy thành `.env`
`.env.test`	Tuỳ (không chứa secret thật thì commit được)	DB riêng cho e2e test, không đụng dữ liệu dev
`.env.development.local` , `.env.test.local` ...	❌ Không (`.gitignore` của project đã liệt kê sẵn `.env.development.local` , `.env.test.local` , `.env.production.local` , `.env.local`)	Ghi đè riêng từng máy
Production	Không dùng file	Đặt biến môi trường thật trên hosting/Docker/CI — chúng tự thắng `.env` (bước 2 sơ đồ trên)

14.3 Validate biến môi trường lúc khởi động — fail fast

Hiện tại nếu quên `JWT_SECRET` , app vẫn khởi động được tới khi DI tạo `JwtStrategy` — nhờ project dùng `getOrThrow` nên may mắn là vẫn chết sớm. Nhưng với biến dùng `get()` thường (port, thời hạn token), giá trị sai sẽ lọt vào và chỉ lộ ra khi có request. Validate lúc khởi động gom mọi kiểm tra về một chỗ, báo lỗi rõ ràng, và **ép kiểu** luôn.

Cách A — hàm `validate` với class-validator (không cần cài thêm gì)

```
import { plainToInstance } from 'class-transformer';
import { IsEnum, IsInt, IsString, Max, Min, MinLength, validateSync } from 'class-
validator';

export enum Environment {
  Development = 'development',
  Production = 'production',
  Test = 'test',
}

export class EnvironmentVariables {
  @IsEnum(Environment)
  NODE_ENV: Environment = Environment.Development;

  @IsInt() @Min(0) @Max(65535)
  PORT: number = 3000;

  @IsString() @MinLength(32)
  JWT_SECRET: string;

  @IsInt() @Min(60)
  JWT_EXPIRES_IN: number = 3600;

  @IsString()
  DB_PATH: string = 'db.sqlite3';
}

export function validate(config: Record<string, unknown>) {
  const validated = plainToInstance(EnvironmentVariables, config, {
    enableImplicitConversion: true,
  });
  const errors = validateSync(validated, { skipMissingProperties: false });
  if (errors.length > 0) {
    throw new Error(errors.toString());
  }
  return validated;
}
```

`enableImplicitConversion: true` bảo class-transformer đọc kiểu khai báo của field (nhờ `emitDecoratorMetadata`, Part P.5) để ép `"4000"` thành `4000` — thiếu nó thì `@IsInt()` luôn fail vì mọi biến môi trường đều là string. Đã chạy thử với class-validator 0.15.1: thiếu `JWT_SECRET` đủ dài → throw `property JWT_SECRET has failed the following constraints: minLength`; hợp lệ → `typeof PORT === 'number'`, default được gán, và biến lạ như `PATH` vẫn được giữ nguyên (không bị lọc).

Cách B — schema Joi

```
npm i joi@^18
```

```
import * as Joi from 'joi';

ConfigModule.forRoot({
  validationSchema: Joi.object({
    NODE_ENV: Joi.string().valid('development', 'production',
'test').default('development'),
    PORT: Joi.number().port().default(3000),
    JWT_SECRET: Joi.string().min(32).required(),
  }),
});
```

Ngắn hơn, nhưng thêm 1 dependency và là cú pháp thứ hai phải học song song với class-validator. Với project này (đã có class-validator), **Cách A** là lựa chọn tự nhiên.

Bẫy `allowUnknown`: với Joi, `@nestjs/config` tự đặt `allowUnknown: true`, `abortEarly: false` nếu bạn không truyền (thấy trong hàm `getSchemaValidationOptions` của source). Nếu bạn tự truyền `validationOptions: { allowUnknown: false }` "cho chặt", app sẽ chết ngay vì `validatePredefined` mặc định `true` đưa cả `PATH`, `HOME`, `SHELL`... của hệ điều hành vào schema — toàn biến bạn không khai báo.

Đối chiếu docs.nestjs.com/techniques/configuration + source đang cài: hàm `validate` ở Cách A khớp nguyên văn ví dụ chính thức (`plainToInstance` + `enableImplicitConversion: true` + `validateSync` + `skipMissingProperties: false`). Kết quả trả về của `validate` được lưu làm "validated env" và `ConfigService.get()` tra nó **trước** `process.env` (hàm `getFromValidatedEnv` trong `config.service.js`) — nên `get('PORT')` ra số, trong khi `process.env.PORT` vẫn là chuỗi. Cách B đã chạy thử với **joi 18.2.9** (bản mới nhất): `PORT: '4000'` được ép thành `4000`, lỗi trả về dạng `"JWT_SECRET" length must be at least 32 characters long`, và `@nestjs/config` 4.0.4 bọc thành `Config validation error: ...`.

14.4 Đọc cấu hình qua `ConfigService` — và kiểu dữ liệu thật sự trả về

Cách gọi	Kiểu TypeScript	Khi không có key
`config.get<string>('JWT_SECRET')`	`string | undefined`	Trả `undefined`
`config.get<number>('PORT', 3000)`	`number`	Trả default `3000`
`config.getOrThrow<string>('JWT_SECRET')`	`string`	Throw `TypeError: Configuration key "JWT_SECRET" does not exist`
`ConfigService<Env>` + `get('PORT', { infer: true })`	`number | undefined` (tự suy từ `Env`)	Trả `undefined`; gõ sai tên key là lỗi compile
`ConfigService<Env, true>` + `get('PORT', { infer: true })`	`number`	Tham số `true` = "tôi cam đoan đã validate" — bỏ `undefined` khỏi kiểu
`config.get('jwt.secret')`	theo generic	Đọc lồng theo dấu chấm vào config namespace (mục 14.5)

```ts
interface Env {
  PORT: number;
  JWT_SECRET: string;
}

async function bootstrap() {
  const app = await NestFactory.create(AppModule);
  const config = app.get(ConfigService<Env, true>);
  const port = config.get('PORT', { infer: true });
  await app.listen(port);
}
```

Bẫy lớn nhất của Part này — generic chỉ là lời hứa: `config.get<number>('PORT')` **không** ép kiểu (Part P.1). Không có bước validate ép kiểu thì giá trị đến từ `process.env` luôn là string: `port + 1` ra `"30001"`, `limit > 100` so sánh chuỗi. Chính ví dụ `forRootAsync` trong docs TypeORM của Nest cũng phải viết `port: +configService.get('PORT')` (dấu `+` để ép số). Tương tự, `ConfigService<Env, true>` chỉ đúng khi bạn thật sự đã validate ở mục 14.3 — khai `true` mà không validate là tự lừa compiler.

Đã kiểm chứng bằng type-check thật (tsc 5.9.3 + `@nestjs/config` 4.0.4): với `ConfigService<Env>`, gán `const n: number = c.get('PORT', { infer: true })` báo `Type 'number | undefined' is not assignable to type 'number'`; với `ConfigService<Env, true>` thì hết lỗi; gọi `get('NOPE', { infer: true })` báo `No overload matches this call`. Thứ tự tra cứu và thông báo lỗi của `getOrThrow` đọc trực tiếp trong `dist/config.service.js`.

14.5 Config theo namespace — `registerAs()` & `ConfigType`

Khi số biến tăng lên, gom chúng theo nhóm (jwt, database...) thay vì rải chuỗi key khắp nơi. `registerAs(token, factory)` đăng ký 1 factory; factory chỉ chạy lúc DI khởi tạo — sau khi `.env` đã nạp — nên đọc `process.env` **bên trong factory** là an toàn.

```ts
import { registerAs } from '@nestjs/config';

export default registerAs('jwt', () => ({
  secret: process.env.JWT_SECRET,
  signOptions: { expiresIn: Number(process.env.JWT_EXPIRES_IN ?? 3600) },
}));
```

```ts
import jwtConfig from './config/jwt.config';

ConfigModule.forRoot({ isGlobal: true, load: [jwtConfig], validate });
```

3 cách dùng, từ lỏng tới chặt:

```
this.config.get<string>('jwt.secret');

import type { ConfigType } from '@nestjs/config';

constructor(
  @Inject(jwtConfig.KEY)
  private readonly jwt: ConfigType<typeof jwtConfig>,
) {}
this.jwt.secret;

JwtModule.registerAsync(jwtConfig.asProvider());
```

`ConfigType<typeof jwtConfig>` là utility type lấy kiểu trả về của factory (bản chất là `ReturnType`, bóc thêm `Promise` nếu factory async) — đổi factory là kiểu tự cập nhật. `jwtConfig.KEY` là token DI mà `registerAs` gắn lên chính hàm factory. `asProvider()` trả sẵn `{ imports, useFactory, inject }` để đưa thẳng vào `registerAsync` / `forRootAsync`.

Bẫy TS1272 với `ConfigType` — ví dụ trong docs không compile nguyên văn ở project này: `ConfigType` chỉ là type, lại nằm trong chữ ký constructor có decorator (`@Inject`), mà project bật `isolatedModules` + `emitDecoratorMetadata` (Part P.5). Viết `import { ConfigType, registerAs } from '@nestjs/config'` như docs → `tsc` báo `TS1272: A type referenced in a decorated signature must be imported with 'import type'...` (đã chạy thử). Sửa: tách riêng `import type { ConfigType } from '@nestjs/config'` như code trên. Không mất gì: token DI đã được `@Inject(jwtConfig.KEY)` chỉ định tường minh, Nest không cần metadata kiểu ở vị trí này.

Bẫy `asProvider()` sai hình dạng: `asProvider()` đưa **nguyên object** factory trả về làm option của module. Nếu factory trả `{ secret, expiresIn }` (phẳng) thay vì `{ secret, signOptions: { expiresIn } }`, TypeScript **không báo lỗi** (mọi field của `JwtModuleOptions` đều optional, và kiểm tra "thừa field" không áp dụng khi object không phải literal viết tại chỗ) — nhưng `expiresIn` bị bỏ qua âm thầm và token không bao giờ hết hạn. Dùng `asProvider()` thì factory phải có đúng hình dạng option của module đích.

Đã đối chiếu: docs.nestjs.com/techniques/configuration (mục Configuration namespaces: `@Inject(databaseConfig.KEY) private dbConfig: ConfigType<typeof databaseConfig>` ; `TypeOrmModule.forRootAsync(databaseConfig.asProvider())`). Kiểu `ConfigType` và `registerAs(...): TFactory & ConfigFactoryKeyHost` đọc trong `dist/types/config.type.d.ts` , `dist/utils/register-as.util.d.ts` của bản 4.0.4. Docs còn cảnh báo: config đăng ký qua `ConfigModule.forFeature()` có thể chưa sẵn ở constructor của provider khác vì "the order of module initialization is indeterminate" — đọc nó trong `onModuleInit()` nếu gặp `undefined` .

14.6 Dùng config để cấu hình module khác — `forRootAsync` / `registerAsync`

`TypeOrmModule.forRoot({...})` nhận object ngay lúc decorator `@Module()` chạy — lúc đó DI container chưa tồn tại, không inject được `ConfigService`. Biến thể `...Async` giải quyết bằng cách đăng ký 1 **factory provider**: Nest chờ tới khi resolve được mọi thứ trong `inject`, gọi `useFactory(...)` với đúng các instance đó, rồi mới dùng kết quả làm option. `AuthModule` của project đang làm đúng kiểu này:

```
JwtModule.registerAsync({
  inject: [ConfigService],
  useFactory: (config: ConfigService) => ({
    secret: config.getOrThrow<string>('JWT_SECRET'),
    signOptions: { expiresIn: '1h' },
  }),
}),
```

Không cần `imports: [ConfigModule]` vì `isGlobal: true`. Việc tiếp theo là làm điều tương tự cho TypeORM (đang hard-code `database: 'db.sqlite3'`, `synchronize: true`):

```
TypeOrmModule.forRootAsync({
  inject: [ConfigService],
  useFactory: (config: ConfigService<EnvironmentVariables, true>) => ({
    type: 'better-sqlite3',
    database: config.get('DB_PATH', { infer: true }),
    autoLoadEntities: true,
    synchronize: config.get('NODE_ENV', { infer: true }) !== Environment.Production,
  }),
}),
```

Bẫy `expiresIn` đọc từ config: đổi `'1h'` thành `config.getOrThrow<string>('JWT_EXPIRES_IN')` sẽ gặp lỗi compile (đã chạy thử với `@nestjs/jwt` 12.0.2): `Type 'string' is not assignable to type 'number | StringValue | undefined'`. Lý do: `@types/jsonwebtoken` khai `expiresIn?: StringValue | number`, trong đó `StringValue` (từ `@types/ms`) là template literal type như `` `${number}${Unit}` `` — `string` bất kỳ không khớp. Cách sạch nhất: lưu **số giây** (`JWT_EXPIRES_IN=3600`), validate bằng `@IsInt()` ở 14.3, đọc bằng `getOrThrow<number>(...)`.

> **Đối chiếu docs.nestjs.com/data/typeorm (mục Async configuration)**: ví dụ chính thức dùng `useFactory: (configService: ConfigService) => ({...})` + `inject: [ConfigService]` — khớp mẫu trên; trang này cũng cảnh báo "Don't use `synchronize: true` in production; otherwise, you can lose production data" — lý do dòng `synchronize` phụ thuộc `NODE_ENV`. Chữ ký `forRootAsync(options: TypeOrmModuleAsyncOptions)` (có `useFactory`, `inject`, `useClass`, `useExisting`, `imports`) kiểm tra trong `@nestjs/typeorm` 12.0.1; `JwtModuleAsyncOptions` tương tự trong `@nestjs/jwt` 12.0.2.

14.7 `process.env.X` trực tiếp hay `ConfigService` — khi nào dùng cái nào

`main.ts` của project đang đọc `process.env.PORT ?? 3000`, còn `AuthModule` / `JwtStrategy` thì đọc qua `ConfigService`. Cả hai đều ra giá trị, nên người mới hay hỏi: vậy cần gì `ConfigService` cho dài dòng? Câu hỏi quyết định: **giá trị này có cần được validate, ép kiểu, và thay thế được khi test không?**

Cách 1 — đọc thẳng `process.env`

```
const port = process.env.PORT ?? 3000;
```

Dùng được ở mọi nơi, kể cả trước khi app được tạo (chọn file `.env` theo `NODE_ENV`, mục 14.2) và bên trong factory của `registerAs`/hàm `validate`. Nhưng: kiểu luôn là `string | undefined`; không qua validate; đọc ở top-level file có thể ra `undefined` (mục 14.1); và muốn đổi giá trị trong unit test phải sửa biến toàn cục `process.env` — ảnh hưởng mọi test khác chạy cùng process.

Cách 2 — `ConfigService` (hoặc `ConfigType` của namespace)

```
constructor(private readonly config: ConfigService<EnvironmentVariables, true>) {}

const ttl = this.config.get('JWT_EXPIRES_IN', { infer: true });
```

Đi qua DI: nhận giá trị **đã validate và ép kiểu** (số là số), có kiểu suy ra từ class env, gõ sai tên key là lỗi compile, và trong test chỉ cần `{ provide: ConfigService, useValue: { get: jest.fn() } }` — không đụng biến toàn cục.

> **Đối chiếu docs.nestjs.com/techniques/configuration:** tài liệu chính thức định vị `ConfigService` là cách chuẩn để đọc cấu hình trong provider, và chỉ đọc `process.env` bên trong factory của `registerAs` (ví dụ `host: process.env.DATABASE_HOST`) hoặc trong hook `ConfigModule.envVariablesLoaded` — không có ví dụ nào đọc `process.env` trong Service/Controller. Chi tiết "`get()` trả bản đã validate trước `process.env`" kiểm tra trong source 4.0.4 (mục 14.3).

Nhu cầu	Cách 1: `process.env`	Cách 2: `ConfigService`
Chọn file `.env` theo `NODE_ENV` trong `app.module.ts`	✅ Bắt buộc (ConfigService chưa tồn tại)	❌ Chưa có
Bên trong factory `registerAs` / hàm `validate`	✅ Đúng chỗ của nó	❌ Không inject được vào đây
Trong Service/Controller/Guard/Strategy	❌ Không validate, không kiểu, khó mock	✅ Luôn dùng cách này
Giá trị số/boolean (port, TTL, cờ bật/tắt)	❌ Luôn là string, phải tự ép	✅ Ra đúng kiểu nếu đã validate
Cấu hình module khác (`TypeOrmModule`, `JwtModule`)	Chạy được nhưng dễ dính bẫy top-level	✅ Qua `forRootAsync` / `registerAsync`

Quy tắc thực dụng: `process.env` chỉ xuất hiện ở đúng 3 chỗ — hàm `validate`, factory của `registerAs`, và dòng chọn `envFilePath` theo `NODE_ENV`. Mọi chỗ khác đọc qua `ConfigService`. Đây chính là quy tắc "chỉ gọi `env()` trong `config/*.php`" của Laravel, phiên bản NestJS.

14.8 Secret: không commit, không log, lộ thì thay

Việc	Làm thế nào
Kiểm tra `.env` chưa từng bị commit	`git ls-files | grep env` phải rỗng; `git log --all -- .env` không ra commit nào (project hiện đạt cả hai)
Lỡ commit `.env`	`git rm --cached .env` chỉ gỡ khỏi các commit **sau** — secret vẫn nằm trong lịch sử. Coi như đã lộ: **tạo secret mới** (mọi JWT cũ hết hiệu lực), rồi mới tính chuyện dọn lịch sử
Sinh `JWT_SECRET` đủ mạnh	`node -e "console.log(require('crypto').randomBytes(32).toString('hex'))"` — 64 ký tự hex, qua được `@MinLength(32)`

Không log secret	Không `console.log(config)` / `console.log(process.env)` ; log lỗi validate thì in tên biến, không in giá trị
Mỗi môi trường 1 secret riêng	Secret dev lộ không được mở khoá production
Production	Biến môi trường thật trên hosting/CI; `synchronize` tắt; cân nhắc `ignoreEnvFile: true` để chắc chắn không có file `.env` nào "vô tình" được đọc

```
# .env.example — commit file này, giá trị giả
NODE_ENV=development
PORT=3000
JWT_SECRET=replace-with-64-hex-chars-from-crypto-randomBytes
JWT_EXPIRES_IN=3600
DB_PATH=db.sqlite3
```

Phiên bản đã kiểm tra — đọc kỹ trước khi mở docs: project cài `@nestjs/config` **4.0.4** (peerDependencies `@nestjs/common ^10.0.0 || ^11.0.0` — khớp `@nestjs/common` 11.2.3 đang cài). Bản mới nhất trên npm là **12.0.1** (tải về đọc `.d.ts`): peer `@nestjs/common ^11 || ^12` , **ESM-only** (`"type": "module"`), `validationSchema` đổi sang chuẩn Standard Schema (dùng được Zod; Joi phải từ v18), thêm option `override` (cho `.env` thắng biến môi trường thật) và `parser` . Trang docs.nestjs.com/techniques/configuration **hiện mô tả bản 12** (đi cùng NestJS 12) — các ví dụ Zod, `override` , `validationOptions.libraryOptions` trên đó **không có** trong 4.0.4. Course này nhắm **NestJS 11 + `@nestjs/config` 4.0.4** đúng như project đang cài: không cần và không nên `npm i @nestjs/config@latest` trong lúc học; cũng đừng chép đuôi `.js` trong import hay top-level `await` từ code mẫu ESM của docs vào project CommonJS này (Part P.10). Mọi code trong Part này (`validate` , `registerAs` , `ConfigType` , `asProvider` , `get` / `getOrThrow` với `infer`) có chữ ký giống nhau ở cả 4.0.4 và 12.0.1.

Việc cần làm:

1. Tạo `.env.example` (commit) liệt kê đủ `NODE_ENV` , `PORT` , `JWT_SECRET` , `JWT_EXPIRES_IN` , `DB_PATH` ; bổ sung các biến mới vào `.env` của bạn; xác nhận `git ls-files | grep env` chỉ ra `.env.example`
2. Viết `src/config/env.validation.ts` theo mục 14.3 (Cách A), gắn vào `ConfigModule.forRoot({ isGlobal: true, cache: true, validate })` . Thử xoá `JWT_SECRET` khỏi `.env` → app phải từ chối khởi động với thông báo nêu đúng tên biến. Thử `PORT=abc` → cũng phải chết ngay
3. Sửa `main.ts` : bỏ `process.env.PORT` , đọc port qua `app.get(ConfigService)` với `{ infer: true }` ; log ra `typeof port` để thấy nó là `number`
4. Chuyển `TypeOrmModule.forRoot` sang `forRootAsync` (mục 14.6): `database` đọc từ `DB_PATH` , `synchronize` chỉ bật khi không phải production
5. Đưa `expiresIn` của `JwtModule` ra config: thử trước với `getOrThrow<string>` để tự thấy lỗi TS2322, rồi sửa đúng bằng số giây
6. Tạo `src/config/jwt.config.ts` bằng `registerAs('jwt', ...)` , đưa vào `load` , rồi đổi `JwtStrategy` sang inject `ConfigType<typeof jwtConfig>` qua `@Inject(jwtConfig.KEY)` thay vì chuỗi `'JWT_SECRET'` — import `ConfigType` bằng `import type` , nếu quên sẽ gặp lại TS1272
7. Tạo `.env.test` với `DB_PATH=test.sqlite3` , cấu hình `envFilePath` theo `NODE_ENV` , chạy `NODE_ENV=test npm run start` và xác nhận file DB mới được tạo ra thay vì ghi vào `db.sqlite3`

PART 15

Serialization: định dạng dữ liệu trả về

> **Mục tiêu:** Không bao giờ để entity quyết định API trả gì. Có một tầng riêng trả lời câu hỏi "client được thấy những field nào, dưới dạng nào" — tương đương API Resource của Laravel (Part 06 bên Laravel). Cuối Part này, thêm 1 cột nhạy cảm vào entity mà quên xử lý cũng không làm nó lộ ra API.

15.1 Vì sao không trả thẳng entity — và response thực sự được tạo ra thế nào

Khi handler `return post`, chuỗi xử lý bên dưới là: kết quả đi ngược qua các Interceptor (Part 07) → Nest gọi `res.json(result)` của Express → `JSON.stringify` duyệt **mọi own enumerable property** của object. Không có bước nào "lọc bớt" theo ý bạn. Ba hệ quả giống hệt bên Laravel: (1) **rò rỉ dữ liệu** — thêm cột mới vào entity là API tự phơi ra; (2) **vỡ hợp đồng API** — đổi tên cột DB là frontend hỏng theo; (3) **không kiểm soát định dạng** — field tính toán, đổi tên, gộp quan hệ.

Với project này, rủi ro cụ thể nằm ở entity `User`:

```
@Entity()
export class User {
  @PrimaryGeneratedColumn() id: number;
  @Column({ unique: true }) email: string;
  @Column() password: string;   // bcrypt hash
}
```

Hiện tại chưa lộ vì code đang **tự nhặt field bằng tay** ở từng chỗ: `AuthService.register()` trả `{ id: user.id, email: user.email }`, `JwtStrategy.validate()` trả `{ userId, email }` kèm comment "KHÔNG trả password hash". Cách này đúng nhưng mong manh: chỉ cần 1 route mới `GET /users/:id` viết `return this.usersService.findById(id)`, hoặc sau này thêm quan hệ `Post.author` rồi `relations: { author: true }`, là hash password đi thẳng ra JSON — không có lỗi nào báo trước.

Laravel (Part 06)	NestJS	Ghi chú
`JsonResource::toArray()`	Response DTO / hàm mapper (mục 15.6)	Tường minh: liệt kê field được trả
`$hidden = ['password']` trên Model	`@Exclude()` trên entity + `ClassSerializerInterceptor`	Khai 1 lần ở entity, áp mọi nơi entity xuất hiện
`$this->when(...)`, `whenLoaded()`	`groups` + `@SerializeOptions` (mục 15.5)	Field có điều kiện
Wrapping `{ data: ... }`	`TransformInterceptor` của Part 07	Project đã có

> **Bẫy "đặt `private` là giấu được":** `private password: string` chỉ chặn ở compile time (Part P.2) — lúc chạy vẫn là property thường, `JSON.stringify` vẫn in ra. Cũng đừng trông vào `whitelist` của `ValidationPipe`: nó chỉ lọc dữ liệu **đi vào**, không liên quan gì tới dữ liệu đi ra.

15.2 `ClassSerializerInterceptor` — cơ chế bên trong & thứ tự với `TransformInterceptor`

`ClassSerializerInterceptor` có sẵn trong `@nestjs/common`, dựa trên `class-transformer` (project đã cài). Đọc source `@nestjs/common/serializer/class-serializer.interceptor.js` (bản 11.2.3), toàn bộ logic chỉ có mấy bước:

```
intercept(context, next)
  options = { ...defaultOptions (lúc new),
              ...@SerializeOptions() của handler/controller }
  next.handle().pipe(map(res => serialize(res, options)))

serialize(res)
  ├─ res không phải object (string, number, null) hoặc là StreamableFile → trả nguyên
  ├─ res là mảng → transformToPlain từng phần tử
  └─ còn lại     → transformToPlain(res)

transformToPlain(x)
  ├─ không có options.type            → classToPlain(x)          (= instanceToPlain)
  ├─ x đã là instance của options.type → classToPlain(x)
  └─ ngược lại → plainToInstance(options.type, x) rồi classToPlain
```

`classToPlain` duyệt object, gặp property nào thuộc class có decorator `@Exclude` / `@Expose` / `@Transform` thì áp dụng — rồi trả về object thường. Sau đó Express mới `JSON.stringify`.

Đăng ký global trong `main.ts` — chú ý **thứ tự** so với `TransformInterceptor` đang có:

```ts
import { ClassSerializerInterceptor, ValidationPipe } from '@nestjs/common';
import { NestFactory, Reflector } from '@nestjs/core';

app.useGlobalInterceptors(
  new TransformInterceptor(),
  new ClassSerializerInterceptor(app.get(Reflector)),
);
```

```
Request ─► TransformInterceptor ─► ClassSerializerInterceptor ─► handler
                                                                  │ return user
Response ◄─ { success, data } ◄── object thường, KHÔNG password ◄┘
           (map của Transform     (map của Serializer chạy TRƯỚC
            chạy SAU CÙNG)         vì nó nằm TRONG)
```

Interceptor đứng **đầu** danh sách là lớp **ngoài cùng**: chạy trước tiên lúc request vào, nhưng `map()` của nó chạy **sau cùng** lúc response ra. Xếp `ClassSerializerInterceptor` sau `TransformInterceptor` nghĩa là serializer nhìn thấy đúng thứ handler trả về (entity, mảng entity, hoặc `{ data, meta }` của pagination), chưa bị bọc `{ success, data }`.

> **Đã kiểm chứng thứ tự bằng source `@nestjs/core` 11.2.3** (docs.nestjs.com/interceptors không nói rõ thứ tự khi xếp chồng): `helpers/context-creator.js` ghép danh sách theo thứ tự **global → controller → method**; `interceptors-consumer.js` gọi `interceptors[0].intercept()` trước, và `next.handle()` của nó mới gọi tới `interceptors[1]` — nên phần tử đầu là lớp ngoài cùng. Ngoài ra, interceptor đăng ký bằng provider `APP_INTERCEPTOR` được thêm vào lúc `NestFactory.create()` (`applyApplicationProviders()` trong `nest-factory.js`), tức là **trước** mọi interceptor thêm bằng `app.useGlobalInterceptors()` sau đó → nếu đăng ký serializer qua `APP_INTERCEPTOR` (như docs gợi ý), nó thành lớp ngoài cùng và nhận response **đã bọc** `{ success, data }`. Với option mặc định vẫn chạy đúng (15.4), nhưng hỏng khi dùng `type` / `excludeExtraneousValues` (15.5).

15.3 `@Exclude`, `@Expose`, `@Transform` — khai trên entity

```
import { Exclude } from 'class-transformer';

@Entity()
export class User {
  @PrimaryGeneratedColumn()
  id: number;

  @Column({ unique: true })
  email: string;

  @Exclude()
  @Column()
  password: string;
}
```

`@Exclude()` không xoá gì khỏi object — `user.password` vẫn còn nguyên trong RAM, nên `bcrypt.compare(dto.password, user.password)` ở `AuthService.login()` vẫn chạy bình thường. Nó chỉ nói với `classToPlain`: "khi biến instance này thành object thường, bỏ field này". TypeORM không đọc decorator của class-transformer, nên đặt chung 2 loại decorator trên 1 field không xung đột.

```
import { Expose, Transform } from 'class-transformer';

export class Post {
  ...
  @Expose()
  get excerpt(): string {
    return this.body.slice(0, 100);
  }
}

export class Product {
  ...
  @Transform(({ value }) => value?.name, { toPlainOnly: true })
  @ManyToOne(() => Category, (category) => category.products, { onDelete: 'CASCADE' })
  category: Category;
}
```

Decorator	Tác dụng khi serialize	Dùng cho

`@Exclude()` trên field	Bỏ field khỏi output	password, token, cột nội bộ
`@Exclude()` trên class + `@Expose()` từng field	Mặc định bỏ hết, chỉ trả field được `@Expose`	Entity nhạy cảm: thêm cột mới không tự lộ (whitelist thay vì blacklist)
`@Expose()` trên getter	Thêm field tính toán (getter không phải own property nên mặc định không có trong JSON)	`excerpt`, `fullName`
`@Expose({ name: 'createdAt' })`	Đổi tên key trong output	Giữ hợp đồng API khi đổi tên cột
`@Transform(fn, { toPlainOnly: true })`	Thay giá trị bằng kết quả `fn({ value, key, obj, type, options })`, chỉ theo chiều ra	Rút gọn quan hệ thành tên, format tiền

Bẫy `@Transform` gặp quan hệ chưa load: `ProductsService.findAll()` hiện dùng `leftJoin` (không `AndSelect`) nên `product.category` là `undefined`. Viết `value.name` (không có `?.`) sẽ ném `TypeError` ngay trong interceptor → 500 cho cả danh sách. Mọi `@Transform` trên quan hệ phải chịu được `undefined`.

15.4 Bẫy lớn nhất: decorator chỉ có tác dụng trên *instance của class*

`@Exclude()` là metadata gắn vào **class** `User`. `classToPlain` chỉ tìm thấy nó khi object đang xử lý thật sự là instance của `User` (có `constructor === User`). Object thường có đủ field y hệt nhưng không phải instance → không có metadata nào để áp. Kết quả thử thật với class-transformer 0.5.1 (User có `@Exclude()` `password`):

Handler trả về	Output	Kết luận
`user` (entity từ `findOneBy`)	`{"id":1,"email":"..."}`	✅ An toàn
`[user1, user2]`	từng phần tử đã bỏ password	✅ Interceptor map từng phần tử
`{ data: [user], meta }` (kiểu `Paginated` của Part 12)	`{"data":[{"id":1,...}],"meta":{...}}`	✅ Với option mặc định, class-transformer đi đệ quy và nhận ra instance lồng bên trong
`{ ...user }` hoặc `{ ...user, extra }`	`{"id":1,"email":"...","password":"HASH"}`	❌ **Lộ** — spread tạo object thường, mất class
Kết quả `getRawMany()` / object tự dựng	nguyên văn	❌ Không có class nào để đọc decorator
Route dùng `@Res()` rồi tự `res.json(user)`	nguyên văn	❌ Docs: "Response mapping doesn't work with the library-specific response strategy"

Code thật trong project đang dính mẫu nguy hiểm này: `CategoriesService.findAll()` (Part 9.4) viết `rows.map((category) => ({ ...category, productsCount: ... }))`. Hiện `Category` không có field nhạy cảm nên chưa có hậu quả — nhưng mọi `@Exclude` / `@Expose` / `@Transform` bạn thêm vào `Category` sau này sẽ **không có tác dụng** ở endpoint danh sách. Sửa giữ nguyên instance: `rows.map((c) => Object.assign(c, { productsCount: ... }))` (`Object.assign` sửa trên chính instance, không tạo object mới).

Docs vs code thật — ghi rõ chỗ lệch: docs.nestjs.com/techniques/serialization cảnh báo nguyên văn: "The handler must return an instance of the class. A plain JavaScript object, such as `{ user: new UserEntity() }`, is not serialized correctly." Chạy thử với class-transformer **0.5.1** (bản đang cài, cũng là bản mới nhất trên npm): với option mặc định, `instanceToPlain({ user })` ra `{"user":{"id":1}}` — instance lồng trong object thường **vẫn được xử lý đúng**. Nó chỉ hỏng khi bật `strategy: 'excludeAll'` (ra `{}`), `excludeExtraneousValues: true` (ra `{"user":{}}`) hoặc option `type` (15.5). Theo nguyên tắc của tài liệu này, giữ phát hiện từ code thật — nhưng coi cảnh báo của docs là quy tắc an toàn: trả instance (hoặc mảng / `{ data, meta }` chứa instance), đừng bao giờ spread entity.

15.5 `groups` và `@SerializeOptions()` — field có điều kiện

`@SerializeOptions(options)` gắn option của class-transformer cho 1 handler hoặc cả controller (source dùng `reflector.getAllAndOverride()`: handler ưu tiên hơn controller), trộn đè lên option mặc định của interceptor. Dùng nhiều nhất là `groups`: field chỉ hiện khi response được serialize với group tương ứng.

```
export class User {
  @PrimaryGeneratedColumn() id: number;

  @Expose({ groups: ['self'] })
  @Column({ unique: true }) email: string;

  @Exclude()
  @Column() password: string;
}

@Controller('users')
export class UsersController {
  @Get(':id')
  findOne(@Param('id', ParseIntPipe) id: number) {
    return this.usersService.findOneOrFail(id);      // → { id }
  }

  @UseGuards(JwtAuthGuard)
  @SerializeOptions({ groups: ['self'] })
  @Get('me/profile')
  profile(@Req() req: Request) {
    const { userId } = req.user as { userId: number };
    return this.usersService.findOneOrFail(userId);      // → { id, email }
  }
}
```

`Request` ở đây import bằng `import type { Request } from 'express'` như `auth.controller.ts` (Part P.5). `req.user` có kiểu `Express.User` rỗng nên phải khai lại đúng hình dạng mà `JwtStrategy.validate()` trả về.

Đã thử thật: không truyền group → `{"id":1}`; `groups: ['self']` → `{"id":1,"email":"a@b.c"}`. Field không có decorator nào thì luôn hiện (chiến lược mặc định là `exposeAll`).

> **Route tĩnh phải khai trước route động:** trong ví dụ trên, nếu đặt `@Get('me')` **sau** `@Get(':id')`, request `/users/me` sẽ khớp `:id` trước và `ParseIntPipe` trả 400 cho chữ "me". Hoặc khai route tĩnh trước, hoặc dùng đường dẫn không trùng dạng như `me/profile`. `groups` tĩnh theo route; muốn group theo **vai trò người dùng** (admin thấy thêm field) phải tự viết interceptor kế thừa `ClassSerializerInterceptor` và override `getContextOptions()` (method `protected` trong `.d.ts`) — ngoài phạm vi Part này.

Option `type`: `@SerializeOptions({ type: UserEntity })` bảo interceptor `plainToInstance` response thành class đó trước khi serialize — dùng khi handler trả object thường. Docs: "converts responses into instances of the specified class before they are serialized, so the class's class-transformer decorators apply even when the handler returns a plain object."

> **Hai bẫy của `type` (đã chạy thử đúng logic `transformToPlain` của source):** (1) `type` áp lên **toàn bộ** giá trị interceptor nhận được. Nếu serializer là lớp ngoài cùng (đăng ký qua `APP_INTERCEPTOR`, 15.2) hoặc handler trả `{ data, meta }`, nó sẽ ép cả wrapper thành `UserEntity`: kèm `excludeExtraneousValues: true` thì response ra `{}` — trống trơn. `type` chỉ dùng cho handler trả đúng 1 object hoặc 1 mảng, và serializer phải nằm trong `TransformInterceptor`. (2) Response DTO chỉ khai `@Expose()` từng field mà **không** bật `excludeExtraneousValues: true` (hoặc `@Exclude()` ở mức class) thì field lạ vẫn đi qua: `type: UserDto` áp lên `{ id, email, password }` vẫn ra đủ cả `password`.

15.6 Cách thay thế: response DTO / hàm mapper tường minh — và lớp phòng thủ ở DB

Cách thứ hai không dùng decorator nào: viết hẳn một hàm biến entity thành đúng hình dạng API. Đây gần với `JsonResource::toArray()` của Laravel nhất.

```
export interface UserResponse {
  id: number;
  email: string;
}

export function toUserResponse(user: User): UserResponse {
  return { id: user.id, email: user.email };
}

@Get(':id')
async findOne(@Param('id', ParseIntPipe) id: number): Promise<UserResponse> {
  return toUserResponse(await this.usersService.findOneOrFail(id));
}
```

Kiểu trả về `Promise<UserResponse>` biến hợp đồng API thành thứ compiler kiểm tra được: lỡ `return user` thì không lỗi (User có đủ field của UserResponse — TS so khớp theo hình dạng), nhưng khi đọc code, 1 dòng `toUserResponse(...)` nói rõ "đây là field được phép ra ngoài". Không phụ thuộc interceptor, không phụ thuộc "có phải instance không", không bị ảnh hưởng bởi thứ tự interceptor. Nếu sau này dùng `@nestjs/swagger`, đổi interface thành class để tài liệu API tự sinh.

Lớp phòng thủ thứ ba — ẩn ngay từ câu SQL: `@Column({ select: false }) password` làm mọi `find*` và QueryBuilder **không SELECT** cột này (option `select` trong `ColumnOptions.d.ts` của typeorm

1.1.1: "Indicates if column is always selected by QueryBuilder and find operations. Default value is true"). Cái giá: chỗ thật sự cần (login) phải xin tường minh:

```
findByEmailWithPassword(email: string): Promise<User | null> {
  return this.usersRepository.findOne({
    where: { email },
    select: { id: true, email: true, password: true },
  });
}
```

15.7 Decorator trên entity + `ClassSerializerInterceptor` hay response DTO/mapper riêng — chọn thế nào

Cả hai đều chặn được việc lộ password. Người mới hay hỏi "vậy dùng cái nào cho chuẩn?", và trên mạng thì mỗi bài nói một kiểu. Câu hỏi quyết định: **hình dạng response của bạn có khác nhiều so với hình dạng bảng trong DB không?**

Cách 1 — Decorator trên entity + `ClassSerializerInterceptor` global

Khai `@Exclude()` 1 lần ở `User`, bật interceptor 1 lần ở `main.ts` — mọi nơi entity xuất hiện (trả trực tiếp, nằm trong mảng, lồng trong `{ data, meta }`, lồng trong quan hệ `post.author`) đều được lọc. Thử thật: `Post` có `author: User` → output `{"title":"...","author":{"id":1}}`. Bạn được một "lưới an toàn" mặc định, rất ít code. Đổi lại: hình dạng API dính chặt vào entity; hỏng âm thầm khi lỡ spread (15.4); và thứ tự interceptor bắt đầu quan trọng khi dùng `type`.

Cách 2 — Response DTO / hàm mapper tường minh (mục 15.6)

Mỗi endpoint quyết định chính xác field nào đi ra. Đổi tên cột DB chỉ phải sửa hàm mapper, API không đổi. Không phụ thuộc cơ chế runtime nào. Đổi lại: nhiều code hơn, và quên gọi mapper ở 1 route mới là lộ — không có lưới an toàn mặc định.

> **Đối chiếu docs.nestjs.com/techniques/serialization:** tài liệu chính thức chỉ trình bày Cách 1 (`ClassSerializerInterceptor` + `@Exclude` / `@Expose` / `@Transform` + `@SerializeOptions`), đăng ký bằng `@UseInterceptors(ClassSerializerInterceptor)` hoặc global qua `APP_INTERCEPTOR` ; ghi chú "Neither interceptor serializes `StreamableFile` responses" khớp với nhánh `instanceof StreamableFile` trong source. Trang docs hiện còn mô tả thêm `StandardSchemaSerializerInterceptor` (`@SerializeOptions({ schema })`) — interceptor này **chỉ có từ `@nestjs/common` 12** (đã tải 12.1.1 kiểm tra: có file `standard-schema-serializer.interceptor.js`); bản 11.2.3 đang cài **không có**. Logic `transformToPlain` của `ClassSerializerInterceptor` ở 12.1.1 giống hệt 11.2.3, nên mọi điều ở Part này vẫn đúng nếu sau này nâng cấp. Lưu ý chung: docs.nestjs.com hiện hiển thị NestJS 12 (code mẫu kiểu ESM, import có đuôi `.js`) — course này nhắm **NestJS 11**, CommonJS, đúng như project đang cài; đừng chép đuôi `.js` hay top-level `await` từ docs vào project.

Nhu cầu	Cách 1: decorator + interceptor	Cách 2: DTO/mapper
Chặn lộ field nhạy cảm ở MỌI nơi entity xuất hiện, kể cả lồng trong quan hệ	✅ Khai 1 lần ở entity	❌ Phải nhớ gọi mapper ở từng route, từng quan hệ
Response gần giống bảng DB, chỉ bớt vài field	✅ Gọn nhất	Được nhưng lặp code

Response khác hẳn DB (gộp nhiều bảng, đổi tên nhiều, tính toán)	❌ Entity đầy `@Expose` / `@Transform`, khó đọc	✅ Hàm mapper rõ ràng
Giữ hợp đồng API ổn định khi refactor DB	❌ Đổi entity là đổi API	✅ Chỉ sửa mapper
Dữ liệu là object thường (`getRawMany`, spread, tự dựng)	❌ Decorator không có tác dụng	✅ Không quan tâm instance hay không
Field theo ngữ cảnh (chủ tài khoản thấy email)	✅ `groups` + `@SerializeOptions`	✅ Hai hàm mapper khác nhau

Quy tắc thực dụng: luôn bật Cách 1 làm lưới an toàn — `@Exclude()` cho mọi field không bao giờ được ra ngoài (password, token), `ClassSerializerInterceptor` global xếp **sau** `TransformInterceptor`. Chuyển sang Cách 2 cho endpoint nào có response khác đáng kể so với bảng DB hoặc cần hợp đồng API ổn định. Hai cách không loại trừ nhau: object thường do mapper trả ra đi qua serializer vẫn nguyên vẹn.

> **Việc cần làm:**
>
> 1. Tái hiện lỗi trước khi sửa: thêm `findOneOrFail(id)` vào `UsersService` (ném `NotFoundException` khi không có) và route `GET /users/:id` trong `UsersController` đang trống, `return` thẳng entity. Gọi thử, xác nhận thấy hash `$2b$10$...` trong response
> 2. Thêm `@Exclude()` vào `User.password`, đăng ký `ClassSerializerInterceptor` trong `main.ts` **sau** `TransformInterceptor` (15.2). Gọi lại: password biến mất, response vẫn có dạng `{ success, data }`. Login vẫn phải chạy được (chứng minh `@Exclude` không xoá dữ liệu trong RAM)
> 3. Thí nghiệm 15.4: tạm đổi route thành `return { ...user }` → xác nhận password lộ lại. Hoàn tác. Sau đó sửa `CategoriesService.findAll()` dùng `Object.assign` thay spread
> 4. Thêm `@Expose({ groups: ['self'] })` cho `email`, route `GET /users/me/profile` (có `JwtAuthGuard`, `@SerializeOptions({ groups: ['self'] })`) — so sánh output với `GET /users/:id`
> 5. Thêm getter `excerpt` có `@Expose()` vào `Post`; xác nhận nó xuất hiện cả ở `GET /posts/:id` lẫn trong `data` của `GET /posts` (pagination)
> 6. Viết `toProductResponse()` (Cách 2) cho `GET /products/:id`: trả `{ id, name, price, categoryName }` — tự trả lời vì sao endpoint này hợp với mapper hơn là thêm `@Transform` vào entity
> 7. (Tuỳ chọn) Thêm `@Column({ select: false })` cho `password`, sửa `AuthService.login()` dùng hàm tìm có `select` tường minh (15.6); chạy lại register/login/me để chắc chắn không hỏng luồng auth

PART 16

Chuẩn hoá REST API cho production

> **Mục tiêu:** Biến blog API từ "chạy được trên máy mình" thành "đưa người khác dùng được": status code đúng nghĩa, lọc/sắp xếp không mở cửa cho SQL injection, chống spam bằng rate limit, cấu hình CORS/Helmet/prefix/version trong `main.ts`, tự sinh tài liệu Swagger, và một checklist trước khi deploy.

Part 01–13 lo cho API **đúng chức năng**. Part này lo cho **hợp đồng** giữa API và người gọi nó (frontend, mobile app, đối tác): gọi sai thì nhận mã lỗi gì, được gọi bao nhiêu lần/phút, URL có version không, tài liệu ở đâu. Phần lớn chỉ là cấu hình 1 lần trong `main.ts` / `AppModule` — nhưng cấu hình sai là ảnh hưởng toàn bộ route cùng lúc, nên cần hiểu từng dòng đang làm gì bên trong.

16.1 Status code — dùng đúng nghĩa, đừng để mọi thứ là 200

Client (và các công cụ như interceptor của axios, retry của mobile app) đọc **status code trước**, body sau. Trả 200 kèm `{ error: 'not found' }` nghĩa là client phải tự đoán. Nest có sẵn quy ước mặc định, bạn chỉ cần biết chỗ nào phải chỉnh tay:

Code	Nghĩa	Dùng khi	Trong Nest
200	OK	GET, PUT/PATCH thành công	Mặc định của mọi method **trừ POST**
201	Created	POST tạo mới thành công	Mặc định của POST — kể cả POST *không tạo gì* như `/auth/login`
204	No Content	DELETE thành công, không cần body	`@HttpCode(HttpStatus.NO_CONTENT)`
400	Bad Request	Body/query/param sai định dạng	Mặc định của `ValidationPipe`, `ParseIntPipe`
401	Unauthorized	Chưa đăng nhập, token thiếu/sai/hết hạn	`JwtAuthGuard` (Passport) tự trả
403	Forbidden	Đã đăng nhập nhưng không đủ quyền	Guard trả `false` → 403 `"Forbidden resource"`; hoặc `ForbiddenException` (Part 17)
404	Not Found	Resource không tồn tại (hoặc cố tình giấu — Part 17)	`NotFoundException`; URL không khớp route nào → Nest tự 404
409	Conflict	Trùng dữ liệu, xung đột trạng thái	`ConflictException` — đã dùng ở `AuthService.register`
422	Unprocessable Content	Đúng cú pháp nhưng sai nghiệp vụ / validation kiểu Laravel	`UnprocessableEntityException` hoặc `errorHttpStatusCode` (bên dưới)
429	Too Many Requests	Vượt rate limit	`ThrottlerGuard` (mục 16.3)

500	Internal Server Error	Lỗi phía server	Mọi exception **không** kế thừa `HttpException` (Part 06)

Hai chỗ trong project cần chỉnh tay:

```
import { HttpCode, HttpStatus } from '@nestjs/common';

@Post('login')
@HttpCode(HttpStatus.OK)
login(@Body() dto: LoginDto) {
  return this.authService.login(dto);
}

@UseGuards(JwtAuthGuard)
@Delete(':id')
@HttpCode(HttpStatus.NO_CONTENT)
async remove(@Param('id', ParseIntPipe) id: number): Promise<void> {
  await this.postsService.remove(id);
}
```

`login` là POST nhưng không tạo resource nào → 200 mới đúng nghĩa. `remove` hiện đang trả `{ message: 'Post ... deleted' }` với 200 — cũng chấp nhận được, nhưng nếu chọn 204 thì hàm trả `void` : HTTP 204 **không được phép có body**, Node/Express tự bỏ body đi nên mọi thứ bạn return (kể cả lớp `{ success, data }` của `TransformInterceptor`) đều không tới client. Chọn 1 quy ước và áp dụng cho **mọi** DELETE.

> **Bẫy `@Res()` :** nếu handler nhận `@Res() res` để tự `res.json(...)` , Nest coi như bạn đã chọn "cách của Express" và **tắt** luôn cơ chế chuẩn cho route đó — `@HttpCode()` không còn tác dụng, Interceptor không bọc response được. Nếu chỉ cần chạm vào response (set cookie, header) thì dùng `@Res({ passthrough: true })` .

> **401 hay 403 — mẹo nhớ:** 401 = "tôi không biết bạn là ai" (client nên đi đăng nhập lại). 403 = "tôi biết bạn là ai, nhưng bạn không được phép" (đăng nhập lại cũng vô ích). Trả nhầm 401 cho trường hợp thiếu quyền sẽ khiến frontend tự logout người dùng một cách khó hiểu.

Lỗi validation: 400 hay 422?

Laravel trả **422** khi validation sai; Nest trả **400**. Không bên nào "sai" — 400 nghĩa rộng là "request không hợp lệ", 422 cụ thể hơn là "hiểu request nhưng dữ liệu không xử lý được". Nếu frontend đã quen hợp đồng 422 (hoặc dùng chung với backend Laravel), đổi đúng 1 option:

```
app.useGlobalPipes(
  new ValidationPipe({
    whitelist: true,
    forbidNonWhitelisted: true,
    transform: true,
    errorHttpStatusCode: HttpStatus.UNPROCESSABLE_ENTITY,
  }),
);
```

> **Đã kiểm chứng trên `@nestjs/common` 11.2.3 (node_modules của project):** trong `pipes/validation.pipe.js` có dòng `this.errorHttpStatusCode = errorHttpStatusCode || HttpStatus.BAD_REQUEST` — tức mặc định 400, và exception được tạo qua bảng `HttpErrorByCode` (kiểu `ErrorHttpStatusCode` có sẵn `UNPROCESSABLE_ENTITY`). Chạy thử app mẫu: `POST` body rỗng → `422 {"message":["name should not be empty",...],"error":"Unprocessable Entity"}`. Lưu ý: `ParseIntPipe` có option `errorHttpStatusCode` **riêng**, mặc định vẫn 400 — nên `GET /posts/abc` vẫn là 400 dù đã đổi ValidationPipe. Khớp docs.nestjs.com/techniques/validation: *"By default it throws `BadRequestException`"*.

16.2 Filter & sort an toàn — luôn whitelist tên cột

Part 12 đã chặn trần `limit` (`@Max(100)`). Bước tiếp theo client hay xin là **sắp xếp**: `GET /products?sortBy=price&order=ASC`. Cách viết "nhanh" nhất cũng là cách nguy hiểm nhất:

```
// SAI — đưa thẳng chuỗi client gửi vào orderBy
qb.orderBy(`product.${query.sortBy}`, query.order);
```

Tham số `:search` trong `andWhere('product.name LIKE :search', {...})` được TypeORM gửi xuống DB dạng **parameter** (an toàn), nhưng `orderBy(string)` thì **ghép chuỗi thô** vào SQL — không có cơ chế parameter nào cho tên cột.

> **Đã chạy thử trên TypeORM 1.1.1 + better-sqlite3:** với `sortBy = "title, (SELECT email FROM user LIMIT 1)"`, `getSql()` sinh ra nguyên văn `... ORDER BY "post"."title", (SELECT email FROM user LIMIT 1) DESC` và `getMany()` chạy thành công — tức client đã chèn được 1 subquery tuỳ ý vào câu SQL của bạn (đọc được cột của bảng khác qua kỹ thuật blind injection). Ngược lại, Repository API `find({ order: { [sortBy]: 'ASC' } })` với tên cột không tồn tại thì ném `EntityPropertyNotFoundError` — không bị injection, nhưng client nhận **500** thay vì 400. Kết luận: dùng cách nào cũng phải whitelist ở tầng DTO.

Whitelist bằng `@IsIn()` ngay trong DTO — sai giá trị thì `ValidationPipe` chặn từ cửa, request không bao giờ tới Service:

```
import { IsIn, IsOptional } from 'class-validator';

export const PRODUCT_SORTABLE = ['id', 'name', 'price'] as const;
export type ProductSortColumn = (typeof PRODUCT_SORTABLE)[number];

export class ProductQueryDto extends PaginationQueryDto {

  @IsOptional()
  @IsIn(PRODUCT_SORTABLE)
  sortBy: ProductSortColumn = 'id';

  @IsOptional()
  @IsIn(['ASC', 'DESC'])
  order: 'ASC' | 'DESC' = 'DESC';
}
```

```
const { page, limit, search, categoryId, sortBy, order } = query;

const qb = this.productsRepository
  .createQueryBuilder('product')
  .leftJoin('product.category', 'category')
  .orderBy(`product.${sortBy}`, order)
  .skip((page - 1) * limit)
  .take(limit);
```

`as const` biến mảng thành tuple chỉ-đọc, nhờ đó `ProductSortColumn` là union `'id' | 'name' | 'price'` — TypeScript cũng biết `sortBy` chỉ có thể là 1 trong 3 giá trị đó. **Một nguồn sự thật duy nhất** cho cả validate lúc chạy và kiểu lúc compile. Gửi `?sortBy=password` → `"sortBy must be one of the following values: id, name, price"` (đã chạy thử).

> **Ba lỗi hay gặp ở endpoint danh sách:** (1) không chặn trần `limit` → `?limit=999999` kéo sập DB (Part 12 đã xử lý); (2) sort theo tên cột client gửi mà không whitelist → SQL injection qua `orderBy`; (3) khai báo field lọc trong DTO nhưng quên dùng trong Service — `ProductQueryDto` của project đang có `minPrice` / `maxPrice` nhưng `ProductsService.findAll()` chưa đọc tới, client gửi lên sẽ bị âm thầm bỏ qua. Ngoài ra `@IsIn(['ASC','DESC'])` phân biệt hoa thường: `?order=desc` sẽ bị từ chối — hoặc ghi rõ trong tài liệu API, hoặc chuẩn hoá bằng `@Transform(({ value }) => String(value).toUpperCase())` của `class-transformer`.

16.3 Rate limiting với `@nestjs/throttler`

Không có rate limit, 1 script vòng lặp có thể thử 10.000 mật khẩu/phút vào `/auth/login`, hoặc spam `POST /posts` tới khi đầy ổ đĩa. `@nestjs/throttler` là 1 **Guard** (Part 07): trước khi vào handler, nó đếm số request của "người gọi" (mặc định theo IP) trong 1 cửa sổ thời gian, vượt ngưỡng thì throw `ThrottlerException` (429).

```
npm i @nestjs/throttler
```

```
import { APP_GUARD } from '@nestjs/core';
import { ThrottlerGuard, ThrottlerModule, seconds } from '@nestjs/throttler';

@Module({
  imports: [
    ThrottlerModule.forRoot({
      throttlers: [{ ttl: seconds(60), limit: 100 }],
    }),
    // ...ConfigModule, TypeOrmModule, các feature module như cũ
  ],
  providers: [
    AppService,
    { provide: APP_GUARD, useClass: ThrottlerGuard },
  ],
})
export class AppModule {}
```

`{ provide: APP_GUARD, useClass: ThrottlerGuard }` đăng ký guard **toàn cục thông qua DI container** — khác với `app.useGlobalGuards(new ThrottlerGuard(...))` trong `main.ts` (tạo tay bằng `new`, nằm

ngoài DI nên không tự inject được options/storage/Reflector mà guard cần). Quy tắc chung: guard toàn cục mà có dependency → luôn dùng `APP_GUARD`.

Siết riêng route nhạy cảm, nới/bỏ qua route không cần:

```
import { SkipThrottle, Throttle, minutes } from '@nestjs/throttler';

@Throttle({ default: { limit: 5, ttl: minutes(1) } })
@Post('login')
@HttpCode(HttpStatus.OK)
login(@Body() dto: LoginDto) { /* ... */ }

@SkipThrottle()
@Controller('health')
export class HealthController { /* ... */ }
```

API	Chữ ký ở v6 (đang dùng)	Ghi chú
`ThrottlerModule.forRoot`	`{ throttlers: [{ ttl, limit, name? }] }` hoặc mảng `[{ ... }]`	`ttl` tính bằng **mili-giây**; có helper `seconds()`, `minutes()`, `hours()`, `days()`
`@Throttle`	`@Throttle({ default: { limit, ttl } })`	Key `default` = tên throttler không đặt tên; có thể kèm `blockDuration`, `getTracker`
`@SkipThrottle`	`@SkipThrottle()` = `{ default: true }`	Đặt ở class rồi mở lại 1 method bằng `@SkipThrottle({ default: false })`
Nhiều throttler	`[{ name: 'short', ttl: 1000, limit: 3 }, { name: 'long', ttl: 60000, limit: 100 }]`	Request phải qua **tất cả**; header có hậu tố `-short`, `-long`

Đã kiểm chứng `@nestjs/throttler` 6.7.1 (bản `latest` trên npm, peerDependencies `@nestjs/common ^7 || … || ^11 || ^12` — cài thẳng được vào project Nest 11): đọc `throttler.decorator.d.ts` → `Throttle: (options: Record<string, ThrottlerMethodOrControllerOptions>)`, `SkipThrottle: (skip?: Record<string, boolean>)`; `utilities.js` → `seconds(n) = n * 1000`. Để so sánh, bản 4.2.1 có chữ ký cũ `Throttle(limit?: number, ttl?: number)` với `ttl` tính bằng **giây** — đây là lý do rất nhiều tutorial trên mạng viết `@Throttle(5, 60)` giờ báo lỗi compile, và `ttl: 60` chép từ bài cũ nay chỉ còn là 60 **mili-giây** (rate limit gần như vô hiệu). Chạy thử app mẫu với `limit: 2`: lần 3 nhận `429 {"statusCode":429,"message":"ThrottlerException: Too Many Requests"}` kèm header `Retry-After: 60` (giây); các response bình thường có `X-RateLimit-Limit`, `X-RateLimit-Remaining`, `X-RateLimit-Reset`. Khớp docs.nestjs.com/security/rate-limiting.

Bẫy khi lên production: (1) **Storage mặc định là bộ nhớ trong process** — chạy 3 instance (PM2 cluster, nhiều container) thì mỗi instance đếm riêng, limit thực tế nhân 3, và restart là mất bộ đếm; nhiều instance cần storage dùng chung (docs gợi ý Redis storage của cộng đồng). (2) **Chạy sau reverse proxy** (Nginx, load balancer): mọi request đều đến từ IP của proxy → toàn bộ người dùng chung 1 bộ đếm, 100 request/phút cho cả hệ thống. Phải bật `app.set('trust proxy', 'loopback')` (cần `NestFactory.create<NestExpressApplication>(...)` để có method `set`) để Express lấy IP thật từ `X-Forwarded-For`. (3) `ThrottlerException` kế thừa `HttpException` nên đi qua `HttpExceptionFilter` của Part 06 — response 429 sẽ có format `{ success: false, ... }` như mọi lỗi khác.

16.4 `main.ts` cho production: CORS, Helmet, global prefix, versioning

CORS — cho phép frontend ở domain khác gọi API

Trình duyệt chặn JavaScript ở `http://localhost:5173` đọc response từ `http://localhost:3000` (khác origin) trừ khi server trả header `Access-Control-Allow-Origin` cho phép. Với request "không đơn giản" (có header `Authorization`, method `PUT/DELETE`...), trình duyệt còn gửi trước 1 request `OPTIONS` (preflight) để hỏi. `app.enableCors()` gắn middleware của package `cors` (có sẵn qua `@nestjs/platform-express`) để trả lời những câu hỏi đó.

```
app.enableCors({
  origin: ['http://localhost:5173', 'https://blog.example.com'],
  methods: ['GET', 'HEAD', 'PUT', 'PATCH', 'POST', 'DELETE'],
});
```

Cấu hình	Kết quả khi origin lạ `https://evil.com` gọi (đã chạy thử)	Đánh giá
`enableCors()` không option	`Access-Control-Allow-Origin: *`	OK cho API public không dùng cookie
`{ origin: [danh sách] }`	Không có header `Access-Control-Allow-Origin` → trình duyệt chặn đọc response	✅ Nên dùng ở production
`{ origin: true, credentials: true }`	`Access-Control-Allow-Origin: https://evil.com` + `Allow-Credentials: true`	❌ Nguy hiểm: phản chiếu MỌI origin kèm cookie
`{ origin: '*', credentials: true }`	`*` + `Allow-Credentials: true`	❌ Trình duyệt từ chối tổ hợp này — cấu hình vô nghĩa

CORS không phải là cơ chế bảo mật phía server. Request từ origin bị "chặn" vẫn **tới server và vẫn được xử lý** — trình duyệt chỉ không cho JavaScript đọc kết quả. `curl`, Postman, script Node không hề quan tâm CORS. Bảo vệ dữ liệu là việc của Guard (Part 10, 17), không phải CORS. Và vì project gửi JWT qua header `Authorization` (không phải cookie), bạn **không cần** `credentials: true` — option đó chỉ dành cho cookie/session.

Helmet — header bảo mật mặc định

```
npm i helmet
```

```
import helmet from 'helmet';

app.use(helmet());
```

`helmet()` là 1 Express middleware đặt khoảng chục header phòng thủ cho mọi response: `Content-Security-Policy`, `X-Content-Type-Options: nosniff`, `Strict-Transport-Security`... và **gỡ** header `X-Powered-By: Express` (không quảng cáo cho kẻ tấn công biết bạn chạy gì). Đã chạy thử: sau khi bật,

response có `x-content-type-options: nosniff`, CSP bắt đầu bằng `default-src 'self'`, và không còn `x-powered-by`.

> **Đã kiểm chứng `helmet` 8.3.0 (không có peerDependencies, `engines.node >=18`) và docs.nestjs.com/security/helmet:** docs nhấn mạnh thứ tự — *"Helmet registered after a route doesn't apply to that route"* — nên `app.use(helmet())` phải là 1 trong các dòng **đầu tiên** sau `NestFactory.create`, trước mọi `app.use()` khác và trước `SwaggerModule.setup`. Docs cũng xác nhận CSP mặc định của Helmet chạy được với Swagger UI của `@nestjs/swagger` (chỉ cần nới khi tự nhúng thêm script qua `customJs`).

Global prefix — gom mọi route dưới `/api`

```
import { RequestMethod } from '@nestjs/common';

app.setGlobalPrefix('api', {
  exclude: [{ path: 'health', method: RequestMethod.GET }],
});
```

Tách rõ URL của API khỏi những thứ khác cùng domain (trang web, file tĩnh). `exclude` dành cho route cần URL cố định mà hạ tầng gọi tới (health check của load balancer).

API versioning — phát hành v2 mà không phá client v1

Khi API đã có người dùng, đổi tên/xoá 1 field là **breaking change**. Nguyên tắc: không sửa hợp đồng v1, mà mở v2 song song. Nest hỗ trợ 4 kiểu versioning (URI, Header, Media Type, Custom); URI (`/api/v1/posts`) là kiểu dễ thấy và dễ test nhất:

```
import { VersioningType, Version, VERSION_NEUTRAL } from '@nestjs/common';

app.enableVersioning({
  type: VersioningType.URI,
  defaultVersion: '1',
});

@Controller({ path: 'posts', version: '1' })
export class PostsController {
  @Get()
  findAll() { /* v1 */ }

  @Version('2')
  @Get()
  findAllV2() { /* v2: format mới */ }
}

@Controller({ path: 'health', version: VERSION_NEUTRAL })
export class HealthController {}
```

```
GET /api/v1/posts/5
    |    |    |    └── route param (@Param('id'))
    |    |    └─────── path của @Controller('posts')
    |    └──────────── version: tiền tố 'v' (mặc định) + '1'
    └───────────────── setGlobalPrefix('api')

Thứ tự luôn là: prefix → version → controller → route
VERSION_NEUTRAL: không có đoạn version → /api/health
```

URL (controller `posts` không khai version)	Không có `defaultVersion`	`defaultVersion: '1'`
`/api/posts`	200	404
`/api/v1/posts`	404	200
`/api/health` (`VERSION_NEUTRAL`)	200	200

> **Đã chạy thử trên `@nestjs/core` 11.2.3 — docs và code thật lệch nhau:** docs.nestjs.com/techniques/versioning viết *"If versioning is enabled for the application but a controller or route does not specify a version (and no global default version is set), any request to that controller or route receives a `404` "*. Với `VersioningType.URI` , bảng trên cho thấy thực tế khác: controller không khai version vẫn chạy bình thường ở URL **không có** `/v1` . Hệ quả nguy hiểm hơn 404: bật versioning xong mà quên khai version cho vài controller, API của bạn lẫn lộn 2 kiểu URL (`/api/v1/posts` và `/api/auth/login`) mà không có lỗi nào báo. Cách chắc chắn nhất: luôn đặt `defaultVersion` để mọi route đều có version; route nào thật sự không cần version thì khai `VERSION_NEUTRAL` tường minh. Các phần khác khớp docs: tiền tố mặc định `'v'` (đổi bằng option `prefix` , tắt bằng `prefix: false`), version đứng sau global prefix và trước path controller.

Ghép lại, `main.ts` của project sẽ có dạng (Swagger ở mục 16.5):

```
async function bootstrap() {
  const app = await NestFactory.create<NestExpressApplication>(AppModule);
  const port = process.env.PORT ?? 3000;

  app.use(helmet());
  app.enableCors({ origin: ['http://localhost:5173'] });
  app.setGlobalPrefix('api', {
    exclude: [{ path: 'health', method: RequestMethod.GET }],
  });
  app.enableVersioning({ type: VersioningType.URI, defaultVersion: '1' });

  app.useGlobalPipes(
    new ValidationPipe({ whitelist: true, forbidNonWhitelisted: true, transform: true
}),
  );
  app.useGlobalFilters(new HttpExceptionFilter());
  app.useGlobalInterceptors(new TransformInterceptor());
  app.enableShutdownHooks();

  setupSwagger(app);
  await app.listen(port);
}
```

Bẫy sau khi thêm prefix/version: mọi URL cũ đổi hết — Postman collection, e2e test (Part 11: `request(app.getHttpServer()).get('/posts')`), và frontend đều phải sửa sang `/api/v1/...`. Riêng e2e test tạo app bằng `Test.createTestingModule` thì **không** chạy qua `main.ts` — phải gọi lại `app.setGlobalPrefix(...)` / `app.enableVersioning(...)` trên app test trước `app.init()`, nếu không test vẫn "xanh" với URL cũ trong khi production đã đổi. Mẹo: tách phần cấu hình thành 1 hàm `configureApp(app)` dùng chung cho cả `main.ts` và e2e.

16.5 Swagger / OpenAPI với `@nestjs/swagger`

OpenAPI là 1 file JSON mô tả toàn bộ API (route, body, response, auth). `@nestjs/swagger` sinh file đó **từ chính metadata mà code bạn đã có**: decorator `@Controller/@Get/@Body`, kiểu DTO (nhờ `emitDecoratorMetadata` — Part 00), rồi phục vụ kèm giao diện Swagger UI để bấm thử API ngay trên trình duyệt.

```
npm i @nestjs/swagger@^11
```

⚠ Bẫy phiên bản — đừng gõ `npm i @nestjs/swagger` trần: tại thời điểm audit, bản `latest` là **12.0.2** với peerDependencies `@nestjs/common ^12.0.0`, trong khi project đang ở `@nestjs/common` 11.2.3. Đã chạy thử: `npm i @nestjs/swagger@latest` trên project Nest 11 → `npm error code ERESOLVE ... peer @nestjs/common@"^12.0.0" from @nestjs/swagger@12.0.2`. Đừng "chữa" bằng `--force` / `--legacy-peer-deps` (cài được nhưng chạy 2 bản Nest lệch major). Ghim `@^11` → hiện là 11.4.7, peerDependencies `@nestjs/common ^11.0.1`. Cùng lý do: docs.nestjs.com hiện viết cho Nest 12 — ví dụ ở đó dùng `import ... from './app.module.js'` và `await bootstrap()` ở top-level (cú pháp ESM); project này biên dịch ra CommonJS, cứ giữ kiểu `bootstrap().catch(...)` như `main.ts` hiện tại.

```
import { INestApplication } from '@nestjs/common';
import { DocumentBuilder, SwaggerModule } from '@nestjs/swagger';

export function setupSwagger(app: INestApplication) {
  if (process.env.NODE_ENV === 'production') return;

  const config = new DocumentBuilder()
    .setTitle('Blog API')
    .setDescription('Posts, Comments, Categories, Products')
    .setVersion('1.0')
    .addBearerAuth()
    .build();

  const documentFactory = () => SwaggerModule.createDocument(app, config);
  SwaggerModule.setup('docs', app, documentFactory);
}
```

Gọi `setupSwagger(app)` **sau** `enableVersioning()` (docs yêu cầu, để path trong tài liệu có đúng `/v1`). Truyền **hàm** `documentFactory` thay vì document có sẵn: tài liệu chỉ được dựng khi có người mở lần đầu, không làm chậm lúc khởi động. Trên controller, thêm nhóm và ổ khoá:

```ts
import { ApiBearerAuth, ApiTags } from '@nestjs/swagger';

@ApiTags('posts')
@ApiBearerAuth()
@Controller({ path: 'posts', version: '1' })
export class PostsController { /* ... */ }
```

`addBearerAuth()` thêm nút **Authorize** trên Swagger UI — dán `access_token` lấy từ `/auth/login` vào là mọi request "Try it out" tự kèm header `Authorization: Bearer ...`.

> **Đã kiểm chứng `@nestjs/swagger` 11.4.7 (đọc `dist/swagger-module.js` + chạy thử):** `SwaggerModule.setup('docs', ...)` phục vụ UI ở `/docs` và JSON ở `/docs-json` (YAML ở `/docs-yaml`). Đường dẫn này **không** tự thêm global prefix: đã có `setGlobalPrefix('api')` mà vào `/api/docs-json` vẫn 404 — trừ khi truyền option `{ useGlobalPrefix: true }` (mặc định `false`). Ngược lại, các path *bên trong* tài liệu thì đã đủ prefix + version: `/api/v1/products`, `/api/v2/products`, `/health` (route bị `exclude`).

`@ApiProperty` và chuyện đổi import `PartialType` (nhắc lại ghi chú Part 05)

Swagger đọc được *kiểu* của từng field nhờ metadata TypeScript, nhưng không biết field nào bắt buộc, ví dụ giá trị là gì, độ dài tối đa bao nhiêu... — những thứ đó khai bằng `@ApiProperty()`:

```ts
import { ApiProperty } from '@nestjs/swagger';

export class CreatePostDto {
  @ApiProperty({ example: 'Học NestJS trong 30 ngày', maxLength: 120 })
  @IsString()
  @IsNotEmpty()
  @MaxLength(120)
  title: string;
}
```

Và đây là lúc ghi chú ở Part 05 có hiệu lực: `UpdatePostDto` / `UpdateCategoryDto` đang `extends PartialType(...)` import từ `@nestjs/mapped-types` — phải đổi sang import từ `@nestjs/swagger`:

```ts
import { PartialType } from '@nestjs/swagger';

export class UpdatePostDto extends PartialType(CreatePostDto) {}
```

> **Đã chạy thử để thấy "side effect" mà docs nhắc tới là gì:** cùng 1 `CreatePostDto` có `@ApiProperty`, `PartialType` của `@nestjs/mapped-types` 12.0.0 (bản project đang cài) sinh schema `{"type":"object","properties":{}}` — Swagger UI hiển thị body PUT **rỗng**; bản của `@nestjs/swagger` 11.4.7 sinh đủ `title`, `body` (đều optional). Lý do: bản của swagger gọi lại chính các hàm `inheritValidationMetadata` / `inheritPropertyInitializers` của mapped-types (validation vẫn giữ nguyên) **và** copy thêm metadata Swagger; bản mapped-types không biết Swagger tồn tại. Docs.nestjs.com/openapi/mapped-types ghi rõ cả `PartialType`, `PickType`, `OmitType`, `IntersectionType` đều import từ `@nestjs/swagger`.

Bẫy "tài liệu nói dối": `TransformInterceptor` (Part 07/12) bọc mọi response thành `{ success: true, data }` — Swagger không biết điều đó, nên schema response nó hiển thị là object "trần" (vd `Post`), khác với JSON client thực sự nhận. Ít nhất hãy ghi rõ format bọc trong `setDescription()`; cách làm chuẩn chỉnh (schema generic với `@ApiExtraModels` + `getSchemaPath`) để dành khi API đã ổn định.

16.6 Swagger CLI plugin hay `@ApiProperty` viết tay — chọn cách nào

Nhìn ví dụ `CreatePostDto` ở trên: `@IsString() @IsNotEmpty() @MaxLength(120)` đã nói hết "field này là string, bắt buộc, tối đa 120" — vậy mà vẫn phải viết lại `@ApiProperty({ maxLength: 120 })`. Với vài chục DTO, việc khai 2 lần vừa mệt vừa dễ lệch (sửa validation quên sửa tài liệu). Nest có 1 plugin compiler tự làm phần lặp lại đó. Câu hỏi quyết định: **bạn muốn tài liệu tự đi theo code DTO, hay cần kiểm soát tường minh từng dòng mô tả?**

Cách 1 — Bật CLI plugin, để compiler tự suy ra

```
{
  "$schema": "https://json.schemastore.org/nest-cli",
  "collection": "@nestjs/schematics",
  "sourceRoot": "src",
  "compilerOptions": {
    "deleteOutDir": true,
    "plugins": [
      {
        "name": "@nestjs/swagger",
        "options": { "classValidatorShim": true, "introspectComments": true }
      }
    ]
  }
}
```

Plugin chạy lúc `nest build` / `nest start`: đọc AST của các file DTO rồi **chèn sẵn** vào class một hàm static chứa metadata Swagger — bạn không viết thêm dòng nào. Nó suy ra: `required` từ dấu `?`, `type` / `enum` từ kiểu TypeScript, `default` từ giá trị khởi tạo (`page = 1`), các ràng buộc từ decorator `class-validator` (`classValidatorShim`), và `description` từ comment JSDoc (`introspectComments`).

```
export class CreatePostDto {
  /** Tiêu đề bài viết */
  @IsString()
  @IsNotEmpty()
  @MaxLength(120)
  title: string;
}
```

Cách 2 — Viết `@ApiProperty()` tay trên từng field

Như ví dụ ở mục 16.5: mọi thông tin tài liệu nằm ngay trên field, đọc code là thấy, không phụ thuộc bước build nào. Đổi lại phải lặp lại những gì `class-validator` đã nói, và field nào quên `@ApiProperty` thì **biến mất** khỏi tài liệu (Swagger chỉ liệt kê property có metadata).

> **Đã chạy thử plugin của `@nestjs/swagger` 11.4.7** (gọi `require('@nestjs/swagger/plugin').before(...)` như transformer của TypeScript compiler — đúng cơ chế `nest build` dùng): với DTO trên, output JS có thêm `static _OPENAPI_METADATA_FACTORY() { return { title: { required: true, type: () => String, description: "Tiêu đề bài viết", maxLength: 120 }, ... } }`. Mặc định trong `dist/plugin/merge-options.js`: `dtoFileNameSuffix: ['.dto.ts', '.entity.ts']`, `classValidatorShim: true`, `introspectComments: false`. **Bẫy riêng của project này:** file `users/enities/user.enity.ts` và `comments/enities/comment.enity.ts` (gõ thiếu chữ `t`) **không** khớp hậu tố `.entity.ts` — đã thử, plugin bỏ qua hoàn toàn file đó. Docs.nestjs.com/openapi/cli-plugin nhắc thêm: đổi option plugin thì phải xoá `dist/` rồi build lại; plugin chỉ chạy qua Nest CLI — Jest (`ts-jest`) không chạy nó, e2e test muốn có metadata phải cấu hình transformer riêng.

Nhu cầu	Cách 1: CLI plugin	Cách 2: `@ApiProperty` tay
Nhiều DTO, không muốn khai báo 2 lần	✅ Validation là nguồn sự thật duy nhất	❌ Lặp lại, dễ lệch khi sửa
Thêm field mới là tự có trong tài liệu	✅ Tự động	❌ Quên decorator là field biến mất
Cần `example`, mô tả enum, format đặc biệt	Phải bổ sung `@ApiProperty` cho riêng field đó	✅ Khai đầy đủ ngay tại chỗ
Hiểu tài liệu chỉ bằng cách đọc source	❌ Metadata sinh lúc build, không thấy trong code	✅ Nhìn là thấy
File không theo hậu tố `.dto.ts` / `.entity.ts`	❌ Bị bỏ qua (hoặc phải cấu hình `dtoFileNameSuffix`)	✅ Không phụ thuộc tên file

Quy tắc thực dụng: bật plugin (Cách 1) làm nền cho toàn project — nó lo 90% việc nhàm chán — và chỉ viết `@ApiProperty({ example, enum, description })` tay cho những field cần thông tin mà code không tự nói được. Tự viết tay 100% (Cách 2) chỉ hợp lý khi project nhỏ vài DTO, hoặc build không đi qua Nest CLI.

16.7 Checklist trước khi deploy

Việc	Vì sao	Trong project này
`synchronize: false` + migrations	`synchronize: true` tự `ALTER` / `DROP` cột theo entity — đổi tên 1 field là mất dữ liệu cột cũ	`AppModule` đang để `true` — chỉ giữ cho môi trường dev
`JWT_SECRET` dài, ngẫu nhiên, chỉ nằm trong biến môi trường	Lộ secret = ai cũng tự ký được token admin	Đã dùng `config.getOrThrow('JWT_SECRET')`; `.env` đã có trong `.gitignore`
Build rồi chạy JS: `npm ci` → `npm run build` → `npm run start:prod`	`start:dev` (watch + ts) chậm, tốn RAM, không dành cho production	Script `start:prod` = `node dist/main` đã có sẵn

`ValidationPipe` có `whitelist` + `forbidNonWhitelisted` + `transform`	Chặn field lạ (mass assignment — Part 17), ép kiểu query	`main.ts` hiện **thiếu** `transform: true` — xem lại Part 05/12
Response lỗi 500 không lộ chi tiết nội bộ	Stack trace/câu SQL lộ cấu trúc DB	`HttpExceptionFilter` chỉ bắt `HttpException`; lỗi khác rơi về handler mặc định của Nest (chỉ trả "Internal server error")
Helmet + CORS whitelist + rate limit	Mục 16.3, 16.4	Việc cần làm bên dưới
`trust proxy` khi chạy sau Nginx/load balancer	Không bật: rate limit đếm chung 1 IP cho mọi người	`app.set('trust proxy', 'loopback')`
HTTPS (thường làm ở reverse proxy)	JWT gửi qua HTTP thường là JWT bị lộ	Cấu hình ở Nginx/Caddy, không trong Nest
Swagger tắt hoặc được bảo vệ ở production	Tài liệu đầy đủ = bản đồ tấn công miễn phí	`setupSwagger` ở 16.5 đã `return` khi `NODE_ENV=production`
`app.enableShutdownHooks()`	Khi bị dừng (deploy mới, `SIGTERM`), Nest gọi hook đóng module/kết nối DB gọn gàng	Thêm 1 dòng trong `main.ts`
File database/log không nằm trong git	Lộ dữ liệu thật	`*.sqlite3` đã có trong `.gitignore`

Việc cần làm:

1. Rà status code toàn bộ route: `POST /auth/login` → `@HttpCode(HttpStatus.OK)`; chọn 1 quy ước cho DELETE (204 không body, hoặc 200 kèm message) và áp dụng cho mọi DELETE. Nhân tiện kiểm tra `ProductsController`: route đang khai `@Get('id')` (thiếu dấu `:`) nên `GET /products/1` luôn 404 — sửa thành `@Get(':id')`, xác nhận `/products/1` → 200, `/products/999` → 404.
2. Thêm `sortBy` / `order` với `@IsIn()` vào `ProductQueryDto`, dùng trong QueryBuilder của `ProductsService.findAll()`; đồng thời xử lý `minPrice` / `maxPrice` đang khai báo mà chưa dùng. Gọi thử `?sortBy=password` → phải nhận 400 (hoặc 422 nếu bạn đổi `errorHttpStatusCode`).
3. `npm i @nestjs/throttler`, cấu hình global 100 request/phút qua `APP_GUARD`, siết `/auth/login` còn 5/phút. Gọi login 6 lần liên tiếp → lần 6 nhận 429, kiểm tra header `Retry-After` và format lỗi đi qua `HttpExceptionFilter`.
4. `npm i helmet`; trong `main.ts` thêm `helmet()` (dòng đầu tiên), `enableCors` với danh sách origin cụ thể, `setGlobalPrefix('api')`, `enableVersioning` URI với `defaultVersion: '1'`. Cập nhật e2e test và Postman sang `/api/v1/...`.
5. `npm i @nestjs/swagger@^11` (không phải `latest`). Viết `setupSwagger()` với `addBearerAuth()`, gắn `@ApiTags` / `@ApiBearerAuth` cho các controller, bật CLI plugin trong `nest-cli.json`, đổi mọi import `PartialType` sang `@nestjs/swagger`. Xoá `dist/`, chạy lại, mở `/docs` xác nhận schema của `UpdatePostDto` không còn rỗng.
6. Trên Swagger UI: gọi `/auth/login`, dán token vào nút Authorize, tạo 1 post bằng "Try it out" — xác nhận nhận 201 và response thật có dạng `{ success, data }` (khác schema tài liệu, như bẫy ở 16.5).

7. Đi qua bảng 16.7, ghi kết quả từng dòng vào README (mục "Deploy checklist") kèm danh sách endpoint (method, path, auth, status code thành công).

PART 17

Authorization chuyên sâu: Roles & Policy

Mục tiêu: Trả lời đúng chỗ câu hỏi "ai được làm gì": role (admin/user) kiểm tra bằng Guard + decorator tự viết, quyền trên từng bản ghi ("chỉ tác giả được sửa bài của mình") kiểm tra trong Service, danh sách thì lọc ngay từ query thay vì trả 403; biết khi nào cần tới CASL; đặt các Guard đúng thứ tự và viết test cho chúng.

Part 10 dừng ở "đã đăng nhập chưa": có token hợp lệ là được tạo/sửa/xoá **mọi** post — kể cả post của người khác. Đó là lỗ hổng phổ biến nhất trong API thật (OWASP gọi là *Broken Object Level Authorization*). Part này thêm tầng thứ hai: **được phép làm gì**.

17.1 Authentication vs Authorization — 3 câu hỏi, 3 chỗ trả lời

Câu hỏi	Loại	Ai trả lời	Sai thì trả
"Bạn là ai?"	Authentication	`JwtAuthGuard` + `JwtStrategy` (Part 10) — gắn `request.user`	401
"Vai trò của bạn có được vào route này không?" (không cần biết bản ghi nào)	Authorization theo role	`RolesGuard` đọc `@Roles()` (17.2)	403
"Bạn có được sửa **đúng bài viết #5** này không?"	Authorization theo bản ghi (ownership)	Service, sau khi đã load bản ghi (17.4)	403 (hoặc 404 để giấu)

```
Request → Middleware → JwtAuthGuard → RolesGuard → Pipes → Controller → Service
                          |               |                      |
                  "là ai?" → 401   "role đủ?" → 403        load post #5 từ DB
                  gắn request.user  đọc request.user.role  post.authorId === user.userId ?
                                                              không → 403 / 404
```

Thứ tự trên là bắt buộc: `RolesGuard` đọc `request.user` — thứ chỉ tồn tại **sau khi** `JwtAuthGuard` chạy xong. Và kiểm tra ownership phải chờ tới Service vì chỉ ở đó mới có bản ghi thật trong tay.

> **Về nguồn tài liệu chính thức (kiểm tra ngày 30/09/2026):** trang `docs.nestjs.com/security/authorization` vừa được viết lại hoàn toàn (commit docs `5498a5e`, 29/09/2026) quanh 1 package mới `@nestjs/authorization` (`@Policy()`, `@Can()`, `AuthorizationService`). Đã kiểm tra npm: `@nestjs/authorization` mới có bản **0.0.1** (tạo ngày 29/09/2026) với peerDependencies `@nestjs/common ^12.0.0` — **không cài được** vào project Nest 11 này, và còn quá non để dùng thật. Nội dung RBAC + CASL dưới đây theo bản trước của chính trang đó (commit `96f330a`, 22/09/2026 — `Reflector`, `RolesGuard`, `getAllAndOverride`, `createMongoAbility`), mọi API đều đã được đối chiếu lại với `@nestjs/core` 11.2.3 trong node_modules của project và chạy thử. Tư tưởng không đổi giữa 2 bản — trang mới cũng khẳng định: authentication chạy trước vì *"its guard sets request.user, which authorization reads"*, và kiểm tra theo bản ghi nằm trong service sau khi load bản ghi.

17.2 Role-based: enum `Role`, decorator `@Roles()`, `RolesGuard`

Bước 1 — lưu role vào User và đưa vào `request.user`:

```
export enum Role {
  User = 'user',
  Admin = 'admin',
}

@Entity()
export class User {
  // id, email, password như cũ
  @Column({ type: 'simple-enum', enum: Role, default: Role.User })
  role: Role;
}

export type AuthUser = { userId: number; email: string; role: Role };
```

```
async validate(payload: JwtPayload): Promise<AuthUser> {
  const user = await this.usersService.findById(payload.sub);
  if (!user) throw new UnauthorizedException();
  return { userId: user.id, email: user.email, role: user.role };
}
```

`simple-enum` là kiểu TypeORM lưu enum thành text trên những DB không có kiểu ENUM thật như SQLite (đã chạy thử với better-sqlite3: user mới không truyền role nhận `'user'`). Role được đọc **từ DB** trong `validate()` (project đã query user ở đây rồi) thay vì nhét vào payload JWT: hạ quyền 1 admin thì có hiệu lực ngay request tiếp theo, không phải chờ token cũ hết hạn sau 1 giờ.

Bước 2 — decorator gắn metadata "route này cần role gì". Có 2 cách viết, cả 2 đều có trong API hiện hành:

```
import { Reflector } from '@nestjs/core';

export const Roles = Reflector.createDecorator<Role[]>();

@Roles([Role.Admin])
@Delete(':id')
remove() {}
```

```ts
import { SetMetadata } from '@nestjs/common';

export const ROLES_KEY = 'roles';
export const Roles = (...roles: Role[]) => SetMetadata(ROLES_KEY, roles);

@Roles(Role.Admin)
@Delete(':id')
remove() {}
```

	`Reflector.createDecorator<T>()`	`SetMetadata(KEY, value)` **tự bọc**
Key metadata	Tự sinh chuỗi ngẫu nhiên 21 ký tự (`Roles.KEY`) — không bao giờ trùng	Tự đặt (`'roles'`) — 2 thư viện cùng dùng `'roles'` là đè nhau
Kiểu khi đọc lại trong Guard	✅ `reflector.getAllAndOverride(Roles, ...)` tự suy ra `Role[]`	Phải tự ghi generic `getAllAndOverride<Role[]>(ROLES_KEY, ...)`
Cách gọi	1 tham số: `@Roles([Role.Admin])`	Tuỳ bạn thiết kế, thường variadic: `@Roles(Role.Admin)`
Decorator không tham số (`@Public()`)	Gượng — vốn thiết kế cho decorator mang giá trị	✅ Tự nhiên: `() => SetMetadata(IS_PUBLIC_KEY, true)`

> **Đã kiểm chứng trong node_modules của project (`@nestjs/core` 11.2.3):** `services/reflector.service.d.ts` khai `static createDecorator<TParam>(options?: CreateDecoratorOptions<TParam>): ReflectableDecorator<TParam>`, với `ReflectableDecorator = ((opts?: TParam) => CustomDecorator) & { KEY: string }`; file `.js` cho thấy bên trong nó vẫn gọi `SetMetadata(metadataKey, value)` với `metadataKey = options.key ?? uid(21)` — tức `createDecorator` chỉ là lớp bọc có kiểu quanh `SetMetadata`, không phải cơ chế khác. Docs.nestjs.com/fundamentals/execution-context dùng đúng dạng `Reflector.createDecorator<string[]>()` + `@Roles(['admin'])`.

Bước 3 — Guard đọc metadata và so với `request.user`:

```ts
import { CanActivate, ExecutionContext, Injectable } from '@nestjs/common';
import { Reflector } from '@nestjs/core';
import type { AuthUser } from '../auth/auth-user';
import { Roles } from './roles.decorator';

@Injectable()
export class RolesGuard implements CanActivate {
  constructor(private readonly reflector: Reflector) {}

  canActivate(context: ExecutionContext): boolean {
    const required = this.reflector.getAllAndOverride(Roles, [
      context.getHandler(),
      context.getClass(),
    ]);
    if (!required || required.length === 0) return true;

    const user = context.switchToHttp().getRequest<{ user?: AuthUser }>().user;
    if (!user) return false;
    return required.includes(user.role);
  }
}
```

`getAllAndOverride(Roles, [handler, class])` tìm metadata lần lượt trên method rồi tới class, lấy **giá trị đầu tiên khác undefined**: đặt `@Roles([Role.User])` ở controller và `@Roles([Role.Admin])` ở 1 method → method đó chỉ cần `admin`. Nếu muốn **cộng dồn** (method cần cả 2) thì dùng `getAllAndMerge` → `['admin', 'user']`. Route không có `@Roles` → `required` là `undefined` → cho qua: Guard này chỉ chặn khi được yêu cầu chặn. Trả `false` → Nest tự trả `403 {"message":"Forbidden resource"}`; muốn message riêng thì `throw new ForbiddenException('...')`.

> **Bẫy `isolatedModules` — project đã từng gặp với `Request`:** `tsconfig.json` của project bật cả `isolatedModules` lẫn `emitDecoratorMetadata`. Khi 1 **type** (không phải class) như `AuthUser` xuất hiện trong chữ ký có decorator, ví dụ `update(@CurrentUser() user: AuthUser)`, mà import bằng `import { AuthUser }` thường → compile lỗi `TS1272: A type referenced in a decorated signature must be imported with 'import type'...` (đã tái hiện). Luôn dùng `import type { AuthUser }` — giống hệt lý do `auth.controller.ts` đang viết `import type { Request } from 'express'`.

17.3 Thứ tự Guard: JWT phải chạy trước `RolesGuard`

Guard chạy theo thứ tự: **global** (`APP_GUARD`) → **controller** (`@UseGuards` trên class) → **route** (`@UseGuards` trên method); trong cùng 1 `@UseGuards(A, B)` thì từ trái sang phải. Có 2 cách lắp đúng:

Cách A — cả 2 Guard gắn theo route (giữ kiểu project đang dùng)

```ts
@UseGuards(JwtAuthGuard, RolesGuard)
@Roles([Role.Admin])
@Delete(':id')
remove(@Param('id', ParseIntPipe) id: number) {}
```

Cách B — cả 2 global, route public thì đánh dấu `@Public()`

```
providers: [
  { provide: APP_GUARD, useClass: ThrottlerGuard },
  { provide: APP_GUARD, useClass: JwtAuthGuard },
  { provide: APP_GUARD, useClass: RolesGuard },
],
```

```
export const IS_PUBLIC_KEY = 'isPublic';
export const Public = () => SetMetadata(IS_PUBLIC_KEY, true);

@Injectable()
export class JwtAuthGuard extends AuthGuard('jwt') {
  constructor(private readonly reflector: Reflector) {
    super();
  }

  canActivate(context: ExecutionContext) {
    const isPublic = this.reflector.getAllAndOverride<boolean>(IS_PUBLIC_KEY, [
      context.getHandler(),
      context.getClass(),
    ]);
    if (isPublic) return true;
    return super.canActivate(context);
  }
}
```

Cách B "an toàn mặc định": quên gắn guard cho route mới thì route đó **bị khoá** (401) thay vì **bị mở**. Đổi lại phải nhớ gắn `@Public()` cho `/auth/register`, `/auth/login`, `GET /posts` ... Các `APP_GUARD` chạy theo thứ tự khai báo trong mảng `providers` — gom hết vào **1 mảng trong** `AppModule` để thứ tự nhìn thấy được.

Cách lắp (đã chạy thử từng cách)	Không token → `@Roles([Admin])`	Token admin → `@Roles([Admin])`	Kết luận
Global: `JwtAuthGuard` rồi `RolesGuard`	401	200	✅ Đúng
Global: `RolesGuard` rồi `JwtAuthGuard`	403 (lẽ ra 401)	**403**	❌ Admin bị chặn — `request.user` chưa có
`RolesGuard` global + `@UseGuards(JwtAuthGuard)` ở controller	403	**403**	❌ Global luôn chạy trước controller
`@UseGuards(JwtAuthGuard, RolesGuard)`	401	200	✅ Đúng

Bẫy dễ dính nhất khi "nâng cấp": project đang gắn `@UseGuards(JwtAuthGuard)` từng route. Nếu chỉ thêm `{ provide: APP_GUARD, useClass: RolesGuard }` vào `AppModule` (dòng 3 của bảng), **mọi** route có `@Roles` trả 403 cho cả admin — vì guard toàn cục chạy trước guard của controller, lúc đó chưa ai gắn `request.user`. Chọn hẳn 1 trong 2 cách A/B, đừng trộn.

Đối chiếu docs.nestjs.com/recipes/passport (mục "Enable authentication globally"): đoạn `IS_PUBLIC_KEY` + `Public = () => SetMetadata(...)` + `JwtAuthGuard.canActivate` ở trên khớp nguyên văn docs. Bảng kết quả được đo bằng app mẫu (`@nestjs/core` 11.2.3, `@nestjs/passport` 12.0.0, `@nestjs/jwt` 12.0.2 — cùng phiên bản project) gọi qua `supertest` .

17.4 Ownership — "chỉ tác giả được sửa bài của mình": đặt trong Service

Người mới thường muốn viết thêm 1 Guard `IsPostOwnerGuard` cho gọn. Guard không phải chỗ tốt cho việc này, vì 3 lý do cụ thể:

1. **Guard chạy TRƯỚC Pipes** (Part 00): lúc guard chạy, `ParseIntPipe` chưa chạy — `request.params.id` vẫn là chuỗi `"5"`, chưa được validate. Guard phải tự parse/validate lại.
2. **Query 2 lần:** guard phải load post từ DB để biết tác giả, rồi Service lại load post lần nữa để sửa (hoặc phải nhét post vào `request` — 1 kênh truyền dữ liệu ngầm khó theo dõi).
3. **Đây là luật nghiệp vụ** ("admin sửa được mọi bài, tác giả sửa bài mình, bài đã khoá thì không ai sửa") — đặt trong Service thì unit test được bằng 1 lời gọi hàm, không cần dựng HTTP request.

Bước 1 — Post biết tác giả của nó. Khai cả khoá ngoại dạng cột `authorId` lẫn quan hệ `author`, dùng chung 1 cột qua `@JoinColumn`:

```
@Column({ default: false })
published: boolean;

@Column()
authorId: number;

@ManyToOne(() => User, { onDelete: 'CASCADE' })
@JoinColumn({ name: 'authorId' })
author: User;
```

Có `authorId` là 1 cột thường thì so sánh quyền không cần `relations: { author: true }` — mọi lần `findOneBy` đều có sẵn số id. Đã chạy thử trên TypeORM 1.1.1: lưu bằng `{ author: userEntity }` hay `{ authorId: 3 }` đều ghi đúng cùng 1 cột.

Bước 2 — lấy user hiện tại gọn gàng bằng param decorator:

```
import { createParamDecorator, ExecutionContext } from '@nestjs/common';
import type { AuthUser } from './auth-user';

export const CurrentUser = createParamDecorator(
  (_data: unknown, ctx: ExecutionContext): AuthUser =>
    ctx.switchToHttp().getRequest<{ user: AuthUser }>().user,
);
```

Bước 3 — Controller chỉ chuyển user xuống, Service quyết định:

```
@UseGuards(JwtAuthGuard)
@Put(':id')
update(
  @Param('id', ParseIntPipe) id: number,
  @Body() dto: UpdatePostDto,
  @CurrentUser() user: AuthUser,
): Promise<PostEntity> {
  return this.postsService.update(id, dto, user);
}
```

```
async update(id: number, dto: UpdatePostDto, user: AuthUser): Promise<PostEntity> {
  const post = await this.findOne(id);
  this.assertCanModify(post, user);
  Object.assign(post, dto);
  return this.postsRepository.save(post);
}

private assertCanModify(post: PostEntity, user: AuthUser): void {
  if (user.role === Role.Admin) return;
  if (post.authorId !== user.userId) {
    throw new ForbiddenException('Bạn không phải tác giả bài viết này');
  }
}
```

Tạo post thì gán tác giả từ token, **không bao giờ** từ body: `this.postsRepository.create({ ...dto, authorId: user.userId })`. `remove()` dùng lại đúng `assertCanModify` — 1 luật, 1 chỗ.

Bẫy mass assignment: `Object.assign(post, dto)` copy **mọi** field có trong DTO. Nếu lỡ thêm `authorId` (hay `role` vào DTO của user) "cho tiện", người dùng gửi `{ "authorId": 1 }` là chuyển được bài của mình sang tên người khác — hoặc tự phong admin. `ValidationPipe({ whitelist: true, forbidNonWhitelisted: true })` (Part 05) chỉ bảo vệ bạn khi DTO **không** khai field đó. Quy tắc: field liên quan tới quyền (`authorId` , `role` , `published` nếu chỉ admin được duyệt) không bao giờ nằm trong DTO mà client gửi lên.

403 hay 404 khi không phải chủ sở hữu? Với bài đã public, 403 là trung thực (ai cũng thấy bài tồn tại). Với tài nguyên riêng tư (bản nháp, đơn hàng, tin nhắn), trả 403 tiết lộ "id này có tồn tại" — kẻ dò quét có thể đếm được có bao nhiêu đơn hàng. Lúc đó throw `NotFoundException` thay cho `ForbiddenException` (tương đương `Response::denyAsNotFound()` của Laravel).

Danh sách: lọc theo quyền ngay trong query, thay vì trả 403

Với 1 bản ghi thì "không được xem → 403/404". Với **danh sách** thì không ai trả 403 cả — mà trả đúng những gì người đó được thấy. Cách sai là load hết rồi `.filter()` trong JS (vỡ pagination: trang 1 có thể còn 3 bài thay vì 10, và `total` sai). Cách đúng là đưa điều kiện quyền vào chính câu `WHERE` :

```
async findAllVisible(query: PaginationQueryDto, user?: AuthUser):
Promise<Paginated<PostEntity>> {
  const { page, limit } = query;
  const where: FindOptionsWhere<PostEntity> | FindOptionsWhere<PostEntity>[] =
    !user ? { published: true }
    : user.role === Role.Admin ? {}
    : [{ published: true }, { authorId: user.userId }];

  const [data, total] = await this.postsRepository.findAndCount({
    where,
    order: { id: 'DESC' },
    skip: (page - 1) * limit,
    take: limit,
  });
  return { data, meta: { total, page, limit, totalPages: Math.ceil(total / limit) } };
}
```

Truyền **mảng** object vào `where` = các nhóm điều kiện nối bằng **OR** (trong mỗi object vẫn là AND): "đã publish **HOẶC** do tôi viết". Đã chạy thử trên TypeORM 1.1.1: user A thấy bài public của B + bản nháp của chính A, không thấy bản nháp của B.

Route public nhưng "biết" người đang xem: `GET /posts` là `@Public()`, nên `JwtAuthGuard` ở 17.3 `return true` sớm, **không** đọc token → `request.user` luôn `undefined`, kể cả khi client có gửi token. Muốn "khách thì thấy bài public, tác giả thì thấy thêm nháp của mình", thay vì check `isPublic` trong `canActivate`, luôn gọi `super.canActivate()` và override `handleRequest(err, user, info, context)`: route public → `return user || undefined` (không throw), route thường → `if (err || !user) throw err || new UnauthorizedException()`. Đã chạy thử: route public không token → `user = null`, token hợp lệ → có user, token rác → vẫn 200 với `user = null`; route thường giữ nguyên 401.

17.5 Policy nâng cao với CASL (`@casl/ability`)

Khi luật quyền nhiều lên — "user đọc bài public + bài của mình; sửa/xoá bài của mình; **không** xoá bài đã publish; moderator sửa mọi bài nhưng không xoá; admin làm tất cả" — chuỗi `if` trong từng Service bắt đầu lặp và lệch nhau. CASL gom toàn bộ luật về **1 chỗ** khai báo dạng "ai *can/cannot* làm hành động gì trên đối tượng nào, với điều kiện gì", rồi mọi nơi chỉ hỏi `ability.can(action, subject)`.

```
npm i @casl/ability
```

```ts
import { Injectable } from '@nestjs/common';
import {
  AbilityBuilder, createMongoAbility, ExtractSubjectType, InferSubjects, MongoAbility,
} from '@casl/ability';

export enum Action {
  Manage = 'manage',
  Create = 'create',
  Read = 'read',
  Update = 'update',
  Delete = 'delete',
}

type Subjects = InferSubjects<typeof PostEntity> | 'all';
export type AppAbility = MongoAbility<[Action, Subjects]>;

@Injectable()
export class CaslAbilityFactory {
  createForUser(user: AuthUser): AppAbility {
    const { can, cannot, build } = new AbilityBuilder<AppAbility>(createMongoAbility);

    if (user.role === Role.Admin) {
      can(Action.Manage, 'all');
    } else {
      can(Action.Read, PostEntity, { published: true });
      can(Action.Read, PostEntity, { authorId: user.userId });
      can([Action.Update, Action.Delete], PostEntity, { authorId: user.userId });
      cannot(Action.Delete, PostEntity, { published: true });
    }

    return build({
      detectSubjectType: (item) => item.constructor as ExtractSubjectType<Subjects>,
    });
  }
}
```

Đăng ký `CaslAbilityFactory` trong 1 `CaslModule` (`providers` + `exports`), import module đó vào `PostsModule` , rồi trong Service thay `assertCanModify` bằng:

```ts
const post = await this.findOne(id);
const ability = this.caslAbilityFactory.createForUser(user);
if (ability.cannot(Action.Delete, post)) {
  throw new ForbiddenException();
}
```

Cơ chế bên trong: `AbilityBuilder` gom các rule thành mảng; `build()` tạo 1 `MongoAbility` đánh index rule theo (action, subject type). Khi hỏi `can(Action.Delete, post)` , CASL dùng `detectSubjectType` để biết `post` thuộc loại `PostEntity` (qua `post.constructor`), lấy các rule khớp, rồi so điều kiện (`{ authorId: 1 }` , cú pháp giống query MongoDB — tên "Mongo" chỉ nói về *cú pháp điều kiện*, không liên quan database bạn dùng) với chính các field của object. Rule khai **sau** được ưu tiên: `cannot(Delete, { published: true })` ghi đè `can(Delete, { authorId })` — đã chạy thử: tác giả xoá bản nháp của mình → được, xoá bài đã publish của mình → 403, admin → được.

Đã kiểm chứng `@casl/ability` 7.0.1 (bản `latest`; không phụ thuộc Nest nên không có vấn đề peerDependencies; dependency duy nhất `@ucast/mongo2js`). CHANGELOG của bản 7.0.0 (21/05/2026) ghi breaking change: *"renames PureAbility to Ability and drops default options. In order, to get the previous behavior of Ability class, clients are expected to use `createMongoAbility` and `MongoAbility` type"*. Đã chạy thử: code kiểu tutorial cũ `new AbilityBuilder(Ability)` kèm điều kiện ném lỗi lúc `build()`: `Cannot restrict access by conditions without a "conditionsMatcher" option ... or use "createMongoAbility" instead`. Code ở trên (theo bản docs.nestjs.com trước ngày 29/09/2026) biên dịch sạch với `tsconfig` y hệt project và chạy đúng như mô tả.

4 bẫy CASL đã tái hiện được: (1) **Object mất class:** `detectSubjectType` dựa vào `item.constructor` — entity load bằng Repository là instance thật của `PostEntity` nên chạy đúng, nhưng `ability.can(Action.Update, { ...post })` (spread) hay kết quả `getRawOne()` là object thường → CASL không nhận ra loại → luôn `false`. (2) **`ForbiddenError` thành 500:** `ForbiddenError.from(ability).throwUnlessCan(...)` ném lỗi của CASL, **không** kế thừa `HttpException` → client nhận 500 (đã chạy thử); hoặc tự `throw new ForbiddenException()` như trên, hoặc viết 1 filter `@Catch(ForbiddenError)` (Part 06). (3) **Điều kiện lồng không có kiểu:** `{ 'author.id': user.userId }` bị TypeScript từ chối (`'author.id' does not exist in type MongoQuery<PostEntity>`) — thêm lý do để có cột `authorId` ở 17.4. (4) **`moduleResolution` cũ:** v7 chỉ khai kiểu qua trường `exports` của package.json; project dùng `"moduleResolution": "nodenext"` nên ổn, nhưng project cũ để `"node"` sẽ báo `TS7016: Could not find a declaration file for module '@casl/ability'`.

17.6 `RolesGuard` + ownership trong Service, hay CASL — chọn cách nào

Sau 17.2–17.5 bạn có 2 cách tổ chức quyền hoàn chỉnh, và câu hỏi tự nhiên là "project mình nên đi đường nào". Cả 2 đều đúng về kỹ thuật; khác nhau ở chỗ **luật quyền nằm ở đâu và tăng trưởng ra sao**. Câu hỏi quyết định: **luật quyền của bạn có đếm được trên 1 bàn tay (vài role + "chủ sở hữu"), hay nó là 1 ma trận role × hành động × điều kiện sẽ còn lớn lên?**

Cách 1 — `@Roles()` + `RolesGuard` cho route, `assertCanModify()` trong Service cho bản ghi

Code ở 17.2–17.4. Mỗi luật là vài dòng TypeScript thuần, đọc là hiểu, debug bằng breakpoint bình thường, không thêm thư viện. Nhận được: 401/403 đúng chỗ, ownership kiểm tra sau khi load bản ghi, test bằng Jest không cần mock gì đặc biệt.

Cách 2 — CASL: mọi luật khai trong `CaslAbilityFactory`, mọi nơi hỏi `ability.can()`

Code ở 17.5. Toàn bộ "ai được làm gì" nằm ở 1 file; thêm role mới là thêm vài dòng `can/cannot` ở đúng 1 chỗ. Nhận thêm: rule có thể serialize (`ability.rules` là mảng JSON) để gửi xuống frontend ẩn/hiện nút bấm cho khớp backend. Đổi lại: thêm 1 thư viện với khái niệm riêng (subject, `detectSubjectType`, thứ tự ưu tiên rule), và các bẫy ở cuối mục 17.5.

Nguồn: bản docs.nestjs.com/security/authorization trước 29/09/2026 tự gọi ví dụ `RolesGuard` là "basic" vì *"it checks roles only at the route handler level ... you have to check roles somewhere within your business logic, which is harder to maintain because no single place associates permissions with specific actions"* — đó chính là vấn đề CASL giải quyết; và giới thiệu CASL là thư viện *"designed to be incrementally adoptable"*. API CASL đã đối chiếu với 7.0.1 như mục 17.5.

Nhu cầu	Cách 1: RolesGuard + Service	Cách 2: CASL

2–3 role + luật "chủ sở hữu" (như blog này)	✅ Đủ, ít khái niệm	Chạy được nhưng thừa
Chặn cả route theo role (vd `/admin/*`)	✅ `@Roles([Role.Admin])`	Cần thêm `PoliciesGuard` + `@CheckPolicies` tự viết
Luật có điều kiện trên field (published, status, tenant...) ở nhiều module	❌ `if` rải nhiều Service, dễ lệch nhau	✅ Khai 1 chỗ, hỏi mọi nơi
Frontend cần biết quyền để ẩn/hiện nút	❌ Phải tự nhân bản luật sang frontend	✅ Gửi `ability.rules` xuống, frontend dùng lại CASL
Người mới đọc code hiểu ngay	✅ TypeScript thuần	❌ Phải học mô hình của CASL

Quy tắc thực dụng: bắt đầu bằng Cách 1 — nó là nền tảng không bỏ đi được (kể cả khi dùng CASL, bạn vẫn cần `JwtAuthGuard` chạy trước và vẫn phải load bản ghi rồi mới hỏi quyền). Chỉ chuyển sang CASL khi thấy cùng 1 luật quyền bị viết lại ở từ 3 chỗ trở lên, hoặc khi frontend cần dùng chung luật — không phải vì "project chuyên nghiệp thì phải có CASL".

17.7 Test guard và luật quyền

Unit test Guard — không cần dựng app: tạo `ExecutionContext` giả chỉ với đúng những method guard dùng, và dùng `Reflector` thật để đọc metadata do `@Roles` gắn lên method thật:

```
import { ExecutionContext } from '@nestjs/common';
import { Reflector } from '@nestjs/core';

class FakeController {
  @Roles([Role.Admin])
  adminOnly() {}
  open() {}
}

const contextFor = (handler: Function, user?: AuthUser) =>
  ({
    getHandler: () => handler,
    getClass: () => FakeController,
    switchToHttp: () => ({ getRequest: () => ({ user }) }),
  }) as unknown as ExecutionContext;

describe('RolesGuard', () => {
  const guard = new RolesGuard(new Reflector());
  const admin: AuthUser = { userId: 1, email: 'a@x.com', role: Role.Admin };
  const member: AuthUser = { userId: 2, email: 'b@x.com', role: Role.User };

  it('cho admin vào route @Roles([Admin])', () => {
    expect(guard.canActivate(contextFor(FakeController.prototype.adminOnly,
admin))).toBe(true);
  });
  it('chặn user thường', () => {
    expect(guard.canActivate(contextFor(FakeController.prototype.adminOnly,
member))).toBe(false);
  });
  it('route không có @Roles thì cho qua', () => {
    expect(guard.canActivate(contextFor(FakeController.prototype.open))).toBe(true);
  });
});
```

Metadata được gắn lên **hàm** `FakeController.prototype.adminOnly` — đúng cái mà `context.getHandler()` trả về khi chạy thật. Ba case trên đã chạy thử với `@nestjs/core` 11.2.3: `true`, `false`, `true`.

Unit test luật ownership trong Service — dùng lại cách mock Repository của Part 11:

```
it('user khác sửa post nhận ForbiddenException', async () => {
  postsRepository.findOne.mockResolvedValue({ id: 5, authorId: 1, title: 't' });
  await expect(
    service.update(5, { title: 'hack' }, { userId: 2, email: 'b@x.com', role: Role.User
}),
  ).rejects.toThrow(ForbiddenException);
  expect(postsRepository.save).not.toHaveBeenCalled();
});
```

Dòng cuối quan trọng không kém dòng throw: nó chứng minh dữ liệu **không** bị ghi khi bị từ chối. Cuối cùng, 1 bài **e2e** (supertest) cho đủ 4 tình huống giống Laravel Part 19.5 — đây là tầng duy nhất bắt được lỗi thứ tự guard ở 17.3, vì unit test từng guard riêng lẻ đều "xanh":

Tình huống	Kỳ vọng
Không token → `PUT /api/v1/posts/5`	401
Tác giả → `PUT /api/v1/posts/5`	200
User khác → `PUT /api/v1/posts/5`	403
Admin → `PUT /api/v1/posts/5` và `GET` route `@Roles([Role.Admin])`	200

Việc cần làm:

1. Tạo `Role` enum + type `AuthUser`; thêm cột `role` (`simple-enum`, mặc định `user`) vào `User`; sửa `JwtStrategy.validate()` trả thêm `role`. Dùng DB viewer đổi 1 user thành `admin` để test.
2. Viết `Roles` bằng `Reflector.createDecorator<Role[]>()` và `RolesGuard`; thêm 1 route `GET /admin/stats` (đếm user/post) chỉ cho admin. Chọn cách lắp guard A hoặc B ở 17.3 — nếu chọn B thì viết `@Public()` và gắn cho register/login/các route GET công khai.
3. Thêm `authorId` + quan hệ `author` (và `published`) vào `Post`; viết `CurrentUser` decorator; `create()` gán `authorId` từ token; `update()` / `remove()` gọi `assertCanModify()`. Kiểm tra DTO không chứa `authorId`. (Post cũ trong `db.sqlite3` chưa có tác giả — xoá file DB hoặc chạy lại `npm run seed` sau khi sửa seed.)
4. Áp dụng luật tương tự cho `Comment`: chỉ tác giả comment **hoặc** tác giả post được xoá comment.
5. Đổi `GET /posts` sang `findAllVisible()` (khách: bài public; user: public + nháp của mình; admin: tất cả) — dùng `handleRequest` như ghi chú ở 17.4 để route public vẫn đọc được token. Kiểm tra `meta.total` khớp với số bài thấy được.
6. Viết unit test cho `RolesGuard` (3 case) và cho `PostsService.update()` (tác giả OK, người khác `ForbiddenException` + `save` không được gọi), cùng 1 e2e cho 4 tình huống ở bảng 17.7.
7. (Tuỳ chọn) `npm i @casl/ability`, viết `CaslAbilityFactory` cho `PostEntity`, thay `assertCanModify` bằng `ability.cannot(...)`; thử cố ý truyền `{ ...post }` để tự thấy bẫy "object mất class". So sánh độ dài/độ dễ đọc với Cách 1 rồi quyết định giữ cách nào theo bảng 17.6.

PART 18

Queue & Jobs với BullMQ

Mục tiêu: Đưa việc chậm (gửi mail, xử lý ảnh, gọi API bên thứ 3, tính thống kê) ra khỏi request HTTP một cách **bền vững** — job sống sót qua crash/deploy, tự retry khi lỗi tạm thời, quan sát được trạng thái. Hiểu vì sao trong Node "cứ `async` là xong" KHÔNG phải là queue, và vận hành worker đúng cách.

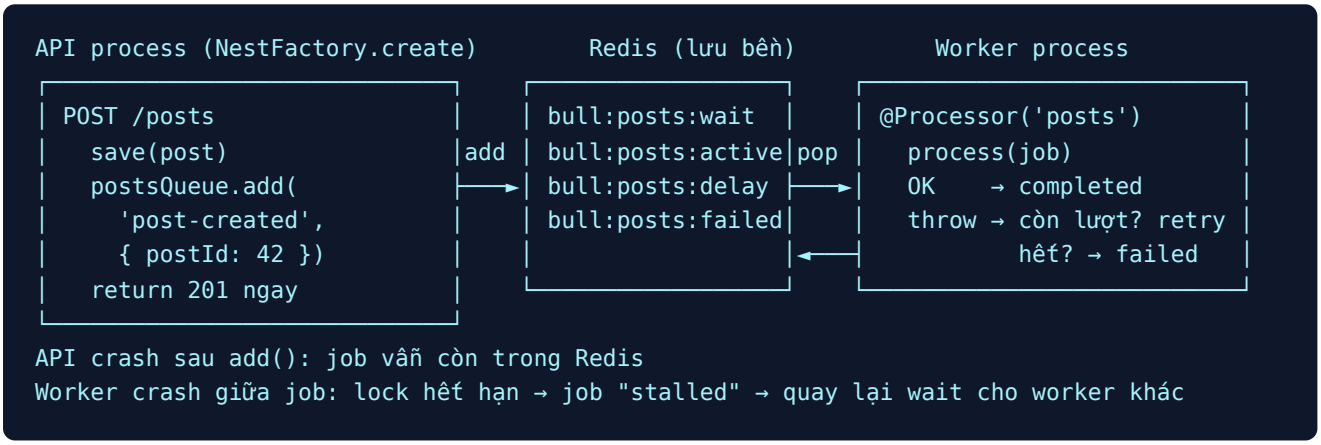
18.1 Vì sao cần queue — "Node có async rồi mà?"

Người đến từ PHP hay nghĩ: Node có event loop, cứ gọi hàm chậm mà không `await` là "chạy nền" rồi. Về kỹ thuật thì đúng là request trả về ngay — nhưng đó là **fire-and-forget trong RAM**, không phải queue:

```ts
async create(dto: CreatePostDto) {
  const post = await this.postsRepository.save(dto);
  this.mailService.notifyFollowers(post.id); // không await — "chạy nền"?
  return post;
}
```

- **Promise chỉ tồn tại trong RAM của process.** Deploy (restart), crash, `Ctrl+C`, container bị kill giữa chừng → mọi Promise đang chạy dở biến mất, không ai biết mail chưa gửi, không có cách chạy lại.
- **Không retry.** SMTP timeout 1 lần là mất luôn — muốn thử lại phải tự viết vòng lặp, tự lưu trạng thái.
- **Vẫn chung 1 event loop.** Nếu việc "nền" tốn CPU (resize ảnh, tạo PDF, hash hàng loạt), nó **chặn** mọi request khác của cùng process — Node chỉ có 1 luồng chạy JavaScript.
- **Không quan sát được, không giới hạn được.** 10.000 request cùng lúc = 10.000 lời gọi API bên thứ 3 cùng lúc; không có chỗ nào xem "còn bao nhiêu việc đang chờ, bao nhiêu việc đã fail".

Queue giải quyết bằng cách **ghi mô tả công việc ra ngoài process** (vào Redis) — một process khác (worker) lấy ra chạy, ghi lại kết quả/lỗi, retry theo cấu hình. Process web chết thì job vẫn nằm trong Redis; worker chết giữa job thì job bị đánh dấu "stalled" và được trả về hàng đợi cho worker khác.



Tiêu chí	Fire-and-forget (Promise không await)	Queue (BullMQ + Redis)
Sống sót khi restart/crash	❌ Mất sạch	✅ Job nằm trong Redis
Retry tự động, có backoff	❌ Tự viết	✅ `attempts` + `backoff`
Việc tốn CPU	❌ Chặn event loop của API	✅ Chạy ở process khác
Scale	Scale cùng web	✅ Thêm worker độc lập với web
Quan sát (đang chờ / fail)	❌	✅ `getJobCounts()`, event `failed`
Độ phức tạp	Không cần gì thêm	Cần Redis + process worker

So với Laravel (Part 14 bên Laravel): ở PHP-FPM, queue là cách *duy nhất* để có "nền"; ở Node, bạn có "nền" miễn phí nhưng **không bền**. Quy tắc rút ra: việc nào mất đi là có người phàn nàn (mail, thanh toán, webhook, export) → queue. Việc nào mất cũng không sao (log phụ, đếm view gần đúng) → fire-and-forget/event trong process là đủ (Part 19).

18.2 Redis & cài đặt `@nestjs/bullmq`

```
docker run -d --name blog-redis -p 6379:6379 redis:7-alpine
docker exec -it blog-redis redis-cli ping        # PONG

npm i @nestjs/bullmq bullmq ioredis
```

> **Đã kiểm chứng trong code package (bullmq 6.3.10, @nestjs/bullmq 12.0.0):** docs.nestjs.com/application/queues (trang cũ `/techniques/queues` vẫn trỏ tới cùng nội dung) chỉ ghi `npm install @nestjs/bullmq bullmq` . Nhưng từ BullMQ 6 (phát hành 30/07/2026), `ioredis` đã chuyển thành **optional peerDependency** — npm KHÔNG tự cài optional peer. Khi bạn cấu hình `connection: { host, port }` , BullMQ gọi `require('ioredis')` lúc khởi tạo và ném lỗi *"BullMQ could not load the optional 'ioredis' package. Install it with `npm install ioredis` ..."* (đọc tại `dist/cjs/classes/redis-connection.js` , hàm `loadIORedis()`). Vì vậy lệnh cài ở trên có thêm `ioredis` . Peer của @nestjs/bullmq 12.0.0: `@nestjs/common ^10 || ^11 || ^12` , `bullmq ^3 → ^6` — khớp project đang dùng @nestjs/common 11. @nestjs/bullmq 12 là package ESM giống @nestjs/typeorm 12 mà project đã dùng, chạy bình thường trên Node 24. Đã thử `npm i @nestjs/bullmq bullmq ioredis` trên bản sao `package.json` + `package-lock.json` của project: resolve sạch, `@nestjs/common` / `@nestjs/core` vẫn 11.2.3, không kéo theo gói nào đòi Nest 12.
> **Lưu ý phiên bản:** từ 09/2026 npm `latest` của `@nestjs/common` là 12.1.1 và docs.nestjs.com đã chuyển mẫu code sang kiểu Nest 12 ESM (import có đuôi `.js` như `'./app.controller.js'` , `await bootstrap()` ở top-level). Course này nhắm **Nest 11 + CommonJS** (đúng project): mọi code trong Part 18–20 đã bỏ đuôi `.js` và không dùng top-level await — đừng chép 2 chi tiết đó từ docs.

Đăng ký kết nối Redis **một lần** ở `AppModule` (dùng `ConfigService` vì project đã có `ConfigModule.forRoot({ isGlobal: true })`), rồi mỗi feature module tự đăng ký queue của mình:

```
// src/posts/posts.queue.ts
export const POSTS_QUEUE = 'posts';
export interface PostCreatedJob { postId: number }
```

```
// src/app.module.ts
import { BullModule } from '@nestjs/bullmq';

BullModule.forRootAsync({
  inject: [ConfigService],
  useFactory: (config: ConfigService) => ({
    connection: {
      host: config.get('REDIS_HOST', 'localhost'),
      port: Number(config.get('REDIS_PORT', 6379)),
    },
    defaultJobOptions: {
      attempts: 3,
      backoff: { type: 'exponential', delay: 1000 },
      removeOnComplete: 100,
      removeOnFail: 500,
    },
  }),
}),
```

```
// src/posts/posts.module.ts
@Module({
  imports: [
    TypeOrmModule.forFeature([Post]),
    BullModule.registerQueue({ name: POSTS_QUEUE }),
  ],
  controllers: [PostsController],
  providers: [PostsService, PostsProcessor],
})
export class PostsModule {}
```

Ngoài queue `posts` dùng xuyên suốt Part này, các Part sau dùng thêm đúng 2 queue nữa trên cùng kết nối Redis: `mail` (gửi mail nền) và `maintenance` (việc dọn dẹp chạy theo lịch). Mỗi queue được đăng ký trong module dùng nó, cú pháp y hệt — `registerQueue()` nhận nhiều object cùng lúc:

```
BullModule.registerQueue({ name: 'mail' }, { name: 'maintenance' }),
```

`forRoot` / `forRootAsync` = cấu hình dùng chung (kết nối, `prefix` mặc định `bull`, `defaultJobOptions`). `registerQueue({ name })` tạo queue và đăng ký nó thành provider — tên queue chính là **injection token** (bên trong là chuỗi `BullQueue_posts`, lấy bằng `getQueueToken('posts')`). Đặt tên queue trong 1 hằng số (`POSTS_QUEUE`) để producer và consumer không bao giờ lệch nhau 1 ký tự. Tên queue không được chứa dấu `:` (BullMQ ném `Queue name cannot contain :` — ký tự này dùng để ghép khoá Redis `bull:posts:...`).

Bẫy biến môi trường: mọi giá trị đọc từ `.env` là **chuỗi**. `config.get('REDIS_PORT')` trả `"6379"`, không phải số — bọc `Number(...)` như trên. Và nhớ thêm `REDIS_HOST` / `REDIS_PORT` vào `.env` bên cạnh `PORT`, `JWT_SECRET` đang có.

18.3 Producer — đẩy job vào queue với `@InjectQueue` + `queue.add()`

```
import { InjectQueue } from '@nestjs/bullmq';
import { Queue } from 'bullmq';

@Injectable()
export class PostsService {
  constructor(
    @InjectRepository(PostEntity)
    private readonly postsRepository: Repository<PostEntity>,
    private readonly dataSource: DataSource,
    @InjectQueue(POSTS_QUEUE)
    private readonly postsQueue: Queue<PostCreatedJob>,
  ) {}

  async create(dto: CreatePostDto): Promise<PostEntity> {
    const post = await this.postsRepository.save(this.postsRepository.create(dto));
    await this.postsQueue.add(
      'post-created',
      { postId: post.id },
      { attempts: 3, backoff: { type: 'exponential', delay: 1000 }, removeOnComplete:
100 },
    );
    return post;
  }
}
```

`add(name, data, opts)`: `name` để consumer phân biệt loại job, `data` là object bất kỳ **serialize được** (sẽ lưu vào Redis dạng JSON), `opts` điều khiển cách chạy. `add()` trả Promise — **phải** `await`: nếu Redis đang sập, bạn muốn biết ngay (request lỗi 500) thay vì âm thầm mất job. Để ý `create()` giờ bắt buộc `async` vì cần `post.id` đã resolve (đúng quy tắc ở mục 8.3).

Option	Ý nghĩa	Ghi chú
`attempts`	Tổng số lần thử (kể cả lần đầu)	Mặc định 1 = không retry
`backoff`	`number` hoặc `{ type: 'fixed' | 'exponential', delay, jitter? }`	Đơn vị **ms**. Exponential 1000 → chờ ~1s, ~2s, ~4s...
`delay`	Chờ bao nhiêu ms mới được xử lý	Job nằm ở trạng thái `delayed`
`priority`	0 → 2.097.151; số nhỏ chạy trước	0 (mặc định) = không ưu tiên, chạy trước job có priority
`jobId`	Tự đặt ID (chuỗi)	ID đã tồn tại → job mới **bị bỏ qua** (chống trùng). KHÔNG được là số nguyên (kể cả chuỗi `'42'`) và không chứa `:` → dùng dạng `` `post-created-${post.id}` ``

`removeOnComplete` / `removeOnFail`	`true` / số job giữ lại / `{ age, count }`	Mặc định GIỮ mãi → Redis phình to

⚠ **Bẫy kinh điển — truyền nguyên entity vào job data:** `queue.add('post-created', { post })` trông tiện, nhưng job được lưu dạng JSON. Đã chạy thử: truyền 1 instance class có field `createdAt: Date` và method `summary()` → trong worker nhận được `createdAt` là **chuỗi**, `summary` là `undefined`, `instanceof` sai. Tệ hơn: dữ liệu là **bản chụp lúc add** — nếu post bị sửa/xoá trước khi worker chạy (có thể vài phút sau nếu queue đang dồn), worker xử lý dữ liệu cũ. Laravel tự làm hộ việc này bằng `SerializesModels` (chỉ lưu ID); BullMQ thì **không** — bạn tự truyền `{ postId }` và worker tự query lại DB.

Đối chiếu docs.nestjs.com/application/queues (mục "Job options"): danh sách option và đơn vị ms khớp bảng trên. Docs cũng nhấn mạnh: BullMQ **không còn nhận option `repeat`** trong `Queue.add()` — muốn job lặp theo cron phải dùng `queue.upsertJobScheduler()` (đã thấy method này trong `bullmq/dist/esm/classes/queue.d.ts`, bản 6.3.10). Tutorial cũ trên mạng dùng `add(name, data, { repeat: {...} })` là API đã bỏ.

18.4 Consumer — `@Processor` + `extends WorkerHost` + `process(job)`

```
import { Processor, WorkerHost, OnWorkerEvent } from '@nestjs/bullmq';
import { Job, UnrecoverableError } from 'bullmq';

@Processor(POSTS_QUEUE)
export class PostsProcessor extends WorkerHost {
  private readonly logger = new Logger(PostsProcessor.name);

  constructor(
    @InjectRepository(PostEntity)
    private readonly postsRepository: Repository<PostEntity>,
  ) {
    super();
  }

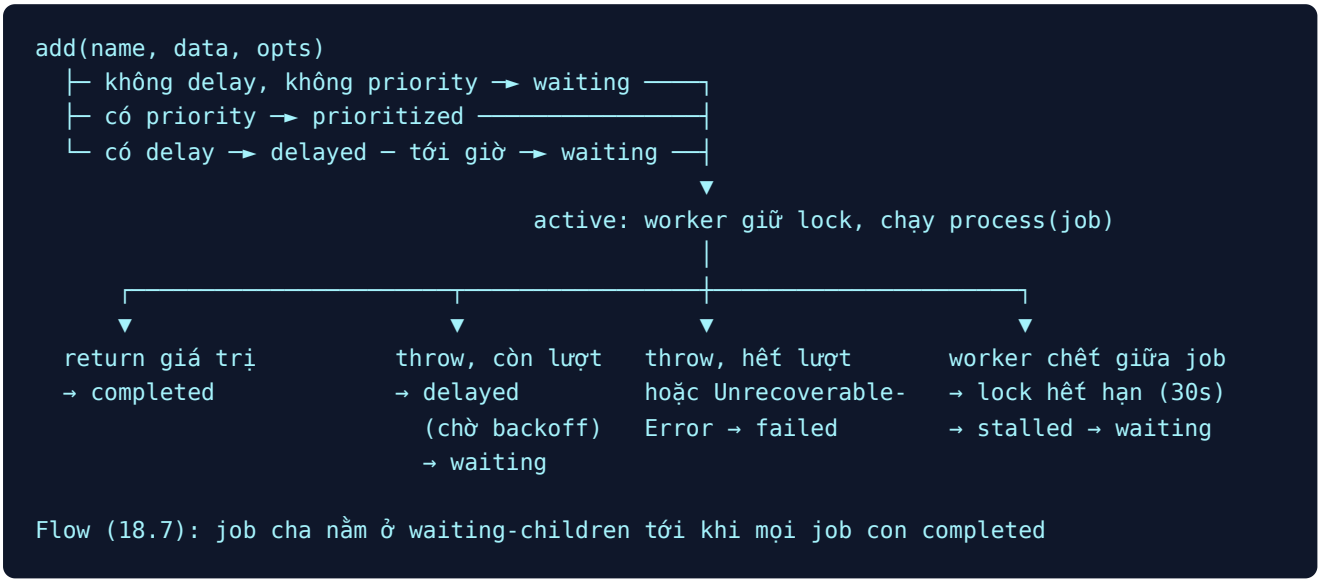
  async process(job: Job<PostCreatedJob>): Promise<unknown> {
    switch (job.name) {
      case 'post-created':
        return this.handlePostCreated(job);
      default:
        throw new UnrecoverableError(`Không có handler cho job ${job.name}`);
    }
  }

  private async handlePostCreated(job: Job<PostCreatedJob>) {
    const post = await this.postsRepository.findOneBy({ id: job.data.postId });
    if (!post) {
      throw new UnrecoverableError(`Post ${job.data.postId} đã bị xoá`);
    }
    await new Promise((r) => setTimeout(r, 3000));
    const words = post.body.split(/\s+/).length;
    return { words };
  }
}
```

- **Consumer phải nằm trong `providers`** của một module (như `PostsModule` ở 18.2) — Nest quét provider có metadata `@Processor` để tạo `Worker` của BullMQ. Quên đăng ký → job nằm ở `waiting` mãi, không có lỗi nào.
- Consumer là provider bình thường → **DI hoạt động đầy đủ** (inject Repository, Service khác). Vì `extends WorkerHost` nên constructor phải gọi `super()`.
- Giá trị `process()` trả về được lưu vào job (`job.returnvalue`) và truyền cho event `completed`. `throw` = lần thử đó thất bại.
- `UnrecoverableError`: lỗi mà thử lại cũng vô ích (dữ liệu sai, bản ghi đã xoá). Đã chạy thử với `attempts: 5`: ném `UnrecoverableError` → job vào `failed` ngay sau lần 1, không retry.
- Chạy song song nhiều job trong 1 worker: `@Processor(POSTS_QUEUE, { concurrency: 5 })` (tham số thứ 2 là `WorkerOptions` của BullMQ). Chỉ hữu ích cho việc chờ I/O; việc tốn CPU thì tăng concurrency không giúp gì vì vẫn 1 luồng JS.

> **Đối chiếu docs + `@nestjs/bullmq/dist/hosts/worker-host.class.d.ts` (12.0.0):** `WorkerHost` là abstract class với đúng 1 method bắt buộc `abstract process(job: Job, token?: string): Promise<any>`, và getter `worker` trả về `Worker` của BullMQ. Cú pháp `@Process('transcode')` đặt trên từng method là API của package CŨ `@nestjs/bull` — docs ghi rõ *"This doesn't work with BullMQ"*, thay bằng `switch (job.name)` trong `process()`. Nếu tutorial bạn đọc có `@Process()` thì đó là Bull, không phải BullMQ.

18.5 Vòng đời một job, retry & event `@OnWorkerEvent`



Trạng thái (`JobState`)	Nghĩa
`waiting`	Đang xếp hàng chờ worker rảnh
`prioritized`	Chờ, nhưng có `priority` — xếp theo độ ưu tiên
`delayed`	Chưa tới giờ chạy — do `delay` hoặc đang chờ backoff giữa 2 lần retry
`active`	Worker đang giữ lock và chạy `process()`
`completed`	Xong, có `returnvalue`
`failed`	Hết lượt thử (hoặc `UnrecoverableError`), có `failedReason` + stacktrace
`waiting-children`	Job cha trong flow, chờ các job con xong

Kết quả chạy thật với `attempts: 3, backoff: { type: 'exponential', delay: 500 }`, job cố tình lỗi 2 lần đầu: lần 1 ở 0.1s → lỗi → lần 2 ở 0.7s → lỗi → lần 3 ở 1.7s → completed. Khoảng chờ 500ms rồi 1000ms — đúng cấp số nhân.

```
@OnWorkerEvent('completed')
onCompleted(job: Job, result: unknown) {
  this.logger.log(`${job.name}#${job.id} xong: ${JSON.stringify(result)}`);
}

@OnWorkerEvent('failed')
onFailed(job: Job | undefined, err: Error) {
  if (!job) return;
  const isFinal = job.attemptsMade >= (job.opts.attempts ?? 1);
  this.logger.warn(`${job.name}#${job.id} lần ${job.attemptsMade} lỗi:
${err.message}`);
  if (isFinal) {
    this.logger.error(`${job.name}#${job.id} THẤT BẠI HẲN`);
  }
}
```

⚠ **Bẫy: event `failed` bắn ở MỖI lần thử lỗi, không chỉ lần cuối.** Đã chạy thử: job `attempts: 3` lỗi 2 lần → `onFailed` được gọi 2 lần (`attemptsMade` = 1 rồi 2) dù cuối cùng job `completed`. Nếu bạn gửi mail cảnh báo "job lỗi!" trong `onFailed` mà không kiểm tra `attemptsMade >= opts.attempts`, bạn sẽ báo động giả cho mọi lỗi tạm thời. Ngoài ra kiểu tham số là `Job | undefined` (xem `WorkerListener` trong `bullmq/dist/esm/classes/worker.d.ts`) — phải kiểm tra `undefined`.

Event khác hay dùng của `WorkerListener`: `active`, `progress` (khi gọi `job.updateProgress(n)`), `stalled` (job bị worker bỏ dở), `error`, `drained` (queue hết việc). `@OnWorkerEvent` chỉ nghe được event của **worker trong process này**. Muốn nghe event của cả queue (từ mọi worker ở mọi máy) thì dùng class riêng `@QueueEventsListener(POSTS_QUEUE)` `extends QueueEventsHost` với `@OnQueueEvent('completed')` — tham số khi đó chỉ là `{ jobId, ... }`, không phải object Job.

Retry chỉ có ích với lỗi tạm thời (mạng, API ngoài quá tải, deadlock). Lỗi logic (dữ liệu sai) retry 3 lần vẫn sai — ném `UnrecoverableError` để dừng ngay. Và vì job CÓ THỂ chạy lại (retry, stalled), `process()` phải **idempotent**: chạy 2 lần cho cùng `postId` không được gửi 2 mail / cộng 2 lần.

18.6 Chạy worker: chung process với API hay process riêng

Với cấu hình ở 18.2, `PostsProcessor` nằm trong `PostsModule` → khi chạy `npm run start:dev`, **cùng một process** vừa phục vụ HTTP vừa chạy worker. Tiện để học, nhưng mất 1 nửa lợi ích của queue (việc tốn CPU vẫn chặn request). Production nên tách:

```
// src/worker.module.ts
@Module({
  imports: [
    ConfigModule.forRoot({ isGlobal: true }),
    TypeOrmModule.forRoot({ type: 'better-sqlite3', database: 'db.sqlite3',
autoLoadEntities: true }),
    TypeOrmModule.forFeature([Post]),
    BullModule.forRoot({ connection: { host: 'localhost', port: 6379 } }),
    BullModule.registerQueue({ name: POSTS_QUEUE }),
  ],
  providers: [PostsProcessor],
})
export class WorkerModule {}

// src/worker.ts
async function bootstrap() {
  const app = await NestFactory.createApplicationContext(WorkerModule);
  app.enableShutdownHooks();
}
bootstrap();
```

```
// package.json — giống script "seed" project đang có
"worker": "nest start --watch --entryFile worker"

npm run start:dev     # terminal 1: API (bỏ PostsProcessor khỏi PostsModule.providers)
npm run worker        # terminal 2: worker, không mở cổng HTTP
```

`createApplicationContext()` dựng DI container mà **không** tạo HTTP server — đúng thứ worker cần. Kết nối Redis của `Worker` giữ process sống.

	Cùng process với API	Process worker riêng
Số lệnh phải chạy	1	2 (web + worker)
Việc tốn CPU ảnh hưởng request	❌ Có	✅ Không
Scale	Web và worker scale cùng nhau	✅ Thêm worker mà không thêm web (và ngược lại)
Deploy code mới	Restart 1 lần	Phải restart CẢ worker, nếu không worker chạy code cũ
Phù hợp	Học, app nhỏ, job chỉ chờ I/O	Production, job nặng

> **Đã đọc code @nestjs/bullmq 12.0.0 (`dist/bull.explorer.js`):** worker được đóng trong hook `onApplicationShutdown()` bằng `worker.close()` — theo `worker.d.ts` của bullmq 6.3.10, `close()` *"waits for current jobs to finalize before returning"*. Nhưng Nest chỉ gọi hook này khi nhận SIGTERM nếu bạn đã gọi `app.enableShutdownHooks()` — thiếu dòng đó, deploy sẽ giết worker giữa job, job chỉ được cứu nhờ cơ chế stalled (chờ hết `lockDuration` , mặc định 30.000ms) và có thể chạy lại lần 2. Docs còn giới thiệu "sandboxed processor" (`registerQueue({ processors: [đường dẫn file] })`) chạy job trong process con — nhưng ở đó **không có DI**, phải tự tạo mọi dependency; chỉ dùng cho job CPU thuần.

18.7 Flow — job cha chờ nhiều job con

Ví dụ: "tạo bản tin tuần" = phân tích 20 bài viết (20 job con chạy song song) rồi gộp kết quả (1 job cha). BullMQ gọi đây là **flow**: job cha nằm ở `waiting-children` cho tới khi mọi con `completed` , rồi đọc kết quả con bằng `job.getChildrenValues()` .

```
BullModule.registerFlowProducer({ name: 'post-flows' }),

constructor(@InjectFlowProducer('post-flows') private readonly flow: FlowProducer) {}

await this.flow.add({
  name: 'weekly-digest',
  queueName: POSTS_QUEUE,
  children: postIds.map((postId) => ({ name: 'post-created', queueName: POSTS_QUEUE,
data: { postId } })),
});

// trong PostsProcessor.process()
case 'weekly-digest': {
  const results = await job.getChildrenValues();
  return Object.values(results).reduce((sum: number, r: any) => sum + r.words, 0);
}
```

Đã chạy thử: 2 job con trả `10` và `20` , job cha nhận `getChildrenValues()` = object dạng `{ "bull:posts:<id con>": 10, ... }` — key là khoá Redis của job con, nên dùng `Object.values()` .

> **Bẫy:** docs ghi rõ `defaultJobOptions` khai ở `forRoot()` **không áp dụng** cho job tạo qua `FlowProducer` — muốn job con có `attempts` / `backoff` phải khai `opts` cho từng con. Flow là tính năng nâng cao; nắm vững 18.3–18.5 trước đã.

18.8 Test — không cần Redis

Unit test không nên phụ thuộc Redis thật. Queue là provider với token `getQueueToken(tên)` → thay bằng object giả y như cách `posts.service.spec.ts` đang thay Repository bằng `getRepositoryToken(Post)` :

```
import { getQueueToken } from '@nestjs/bullmq';

const queue = { add: jest.fn() };

const module = await Test.createTestingModule({
  providers: [
    PostsService,
    { provide: getRepositoryToken(Post), useValue: repo },
    { provide: DataSource, useValue: {} },
    { provide: getQueueToken(POSTS_QUEUE), useValue: queue },
  ],
}).compile();

it('create đẩy job post-created với đúng postId', async () => {
  repo.create.mockReturnValue({ title: 'A', body: 'B' });
  repo.save.mockResolvedValue({ id: 7, title: 'A', body: 'B' });

  await service.create({ title: 'A', body: 'B' });

  expect(queue.add).toHaveBeenCalledWith('post-created', { postId: 7 },
expect.any(Object));
});
```

Lỗi bạn SẼ gặp ngay khi thêm `@InjectQueue`: spec cũ chưa khai queue → `Nest can't resolve dependencies of the PostsService (..., ?). Please make sure that the argument "BullQueue_posts" at index [N] is available in the RootTestModule module.` (đã tái hiện với 1 service tối giản; `N` là vị trí tham số queue trong constructor, chuỗi `BullQueue_posts` chính là `getQueueToken('posts')`). Cách sửa: thêm provider giả như trên. Tương tự `DataSource` mà `PostsService` đang inject từ Part 13.

Test consumer: gọi thẳng `process()` với 1 object job giả — không cần worker, không cần Redis. Ví dụ repo giả trả `null` → `await expect(processor.process({ name: 'post-created', data: { postId: 1 } } as Job)).rejects.toThrow(UnrecoverableError)`.

18.9 `@nestjs/bullmq` hay `@nestjs/bull` — chọn package nào hôm nay

Tìm "nestjs queue" trên mạng bạn sẽ gặp 2 package gần như trùng tên, cùng export `BullModule`, `Processor`, `InjectQueue` — IDE auto-import rất dễ chọn nhầm. Cả hai đều do cùng 1 đội (Taskforce) viết thư viện lõi, đều dùng Redis. Câu hỏi quyết định: **bạn đang bắt đầu dự án mới, hay đang bảo trì code đã chạy Bull?**

Cách 1 — `@nestjs/bullmq` + `bullmq`

```
BullModule.forRoot({ connection: { host: 'localhost', port: 6379 } });

@Processor('posts')
export class PostsProcessor extends WorkerHost {
  async process(job: Job) {
    switch (job.name) { case 'post-created': ... }
  }
}
```

Được: thư viện đang phát triển tích cực (bullmq 6.x), viết bằng TypeScript, có flow cha-con, job scheduler (`upsertJobScheduler`), `UnrecoverableError`, deduplication; tên job xử lý bằng `switch`.

Cách 2 — `@nestjs/bull` + `bull` (thế hệ cũ)

```
BullModule.forRoot({ redis: { host: 'localhost', port: 6379 } });

@Processor('posts')
export class PostsConsumer {
  @Process('post-created')
  async handle(job: Job) { ... }
}
```

Được: mỗi tên job 1 method riêng (`@Process('name')`), decorator event có sẵn kiểu `@OnQueueFailed()` , `repeat` trực tiếp trong `add()`. Nhiều tutorial cũ viết theo cách này.

> **Đã kiểm chứng (30/09/2026):** docs.nestjs.com/application/queues: *"Bull is in maintenance mode (the team only fixes bugs), while BullMQ is actively developed"*; README của repo OptimalBits/bull cũng ghi *"Bull is currently in maintenance mode, we are only fixing bugs. For new features check BullMQ"*. Trên npm: `bull` bản cuối 4.16.5 phát hành 18/12/2024 (gần 2 năm không có bản mới); `bullmq` 6.3.10 phát hành 29/09/2026. Cả `@nestjs/bull` 12.0.0 (peer `bull ^3.3 || ^4`) lẫn `@nestjs/bullmq` 12.0.0 (peer `bullmq ^3 → ^6`) đều hỗ trợ @nestjs/common 11 — nên tương thích không phải lý do chọn, mà là hướng phát triển. Cấu hình khác nhau ở 1 chữ: `connection` (bullmq) vs `redis` (bull).

Nhu cầu	Cách 1: @nestjs/bullmq	Cách 2: @nestjs/bull
Dự án mới, bài tập của course này	✅ Chọn cái này	❌ Đang ở chế độ bảo trì
Flow cha-con, job scheduler, deduplication	✅ Có	❌ Không
Mỗi tên job 1 method riêng bằng decorator	❌ Dùng `switch (job.name)`	✅ `@Process('name')`
Codebase đang chạy ổn định với Bull	Migrate khi có lý do (cần tính năng mới)	✅ Giữ nguyên được, docs gọi là "battle-tested"
Theo được tutorial cũ trên mạng	Phải "dịch" `@Process` → `switch`	✅ Chép gần nguyên

Quy tắc thực dụng: code mới → luôn `@nestjs/bullmq`. Khi đọc tutorial, nhìn 3 dấu hiệu để biết nó viết cho Bull cũ: import từ `'@nestjs/bull'` / `'bull'`, decorator `@Process()`, cấu hình `redis: {...}` — gặp 1 trong 3 là phải dịch lại, đừng chép thẳng.

Việc cần làm:

1. Chạy Redis bằng lệnh `docker run` ở 18.2, `npm i @nestjs/bullmq bullmq ioredis`, thêm `REDIS_HOST` / `REDIS_PORT` vào `.env`, cấu hình `BullModule.forRootAsync` trong `AppModule` và `registerQueue({ name: POSTS_QUEUE })` trong `PostsModule`
2. Sửa `PostsService.create()`: sau khi save, `add('post-created', { postId })` — CHỈ truyền id. Viết `PostsProcessor` (18.4) giả lập việc chậm 3 giây, log số từ của `body`. Gọi `POST /posts`, xác nhận response về ngay (không đợi 3s) và log worker xuất hiện sau đó
3. Chứng minh job bền: tắt app (`Ctrl+C`) ngay sau khi `POST /posts`, bật lại → job vẫn được xử lý. Rồi thử phiên bản fire-and-forget (không queue) cùng kịch bản → việc bị mất
4. Cho `handlePostCreated` ném lỗi khi `job.attemptsMade < 2`, thêm `@OnWorkerEvent('failed')` có phân biệt lần cuối (18.5) — quan sát 3 lần thử và khoảng cách backoff trong log. Sau đó xoá post rồi đẩy job với `postId` đó → phải vào `failed` ngay nhờ `UnrecoverableError`
5. Viết endpoint `GET /posts/queue/stats` trả `postsQueue.getJobCounts('waiting', 'active', 'completed', 'failed', 'delayed')`; tạo 5 post liên tiếp và gọi endpoint để nhìn job di chuyển giữa các trạng thái
6. Tách worker ra process riêng (18.6): tạo `src/worker.ts` + `WorkerModule`, script `"worker"`, gỡ `PostsProcessor` khỏi `PostsModule`; chạy 2 terminal. Nhớ `enableShutdownHooks()`
7. Cập nhật `posts.service.spec.ts`: thêm mock `getQueueToken(POSTS_QUEUE)` + `DataSource`, viết test "create đẩy job đúng postId" (18.8); `npm test` phải xanh mà KHÔNG cần Redis đang chạy

---

PART 19

Events: EventEmitter & TypeORM Subscribers

Mục tiêu: Tách "chuyện gì vừa xảy ra" khỏi "ai cần phản ứng" để thêm hành vi mới (thống kê, xoá cache, ghi audit) mà không sửa lại service gốc. Hiểu chính xác listener chạy **ở đâu và khi nào** (cùng process, đồng bộ theo mặc định), phân biệt event của ứng dụng với event của ORM, và tránh bẫy bắn event trước khi transaction commit.

19.1 Vì sao cần event — và bức tranh so với Laravel

Không có event, `PostsService.create()` sẽ phình ra theo thời gian: lưu post → xoá cache danh sách (Part 20) → đẩy job phân tích (Part 18) → ghi audit log → tăng bộ đếm thống kê... Mỗi tính năng mới lại sửa hàm này, và `PostsService` phải inject đủ thứ không liên quan tới "bài viết". Với event, service chỉ tuyên bố **"post vừa được tạo"** rồi xong; mỗi listener tự lo phần của mình, thêm/bớt listener không đụng tới service.

Laravel (Part 15)	NestJS	Ghi chú

Class Event (`PostPublished`)	Class payload thường (`PostCreatedEvent`) + tên event dạng chuỗi `'post.created'`	Nest định danh event bằng **chuỗi**, không phải bằng class
Listener + event discovery	Method có `@OnEvent('post.created')` trong 1 provider	Được quét lúc `onApplicationBootstrap`
`ShouldQueue` trên listener	Không có sẵn — listener tự `queue.add()` (Part 18)	`{ async: true }` KHÔNG phải queue (19.3)
Model events / Observer	Entity listener (`@AfterInsert`) / `EntitySubscriberInterface`	19.4 – 19.5
`ShouldDispatchAfterCommit`	Không có sẵn — tự emit SAU khi transaction resolve	19.6

19.2 Cài đặt, payload class, `emit` và `@OnEvent`

```
npm i @nestjs/event-emitter
```

```
// src/app.module.ts
import { EventEmitterModule } from '@nestjs/event-emitter';

imports: [
  EventEmitterModule.forRoot({ wildcard: true }),
  ...
]
```

```
// src/posts/events/post-created.event.ts
export const POST_CREATED = 'post.created';

export class PostCreatedEvent {
  constructor(
    public readonly postId: number,
    public readonly title: string,
  ) {}
}
```

```ts
// src/posts/posts.service.ts
import { EventEmitter2 } from '@nestjs/event-emitter';

constructor(
  ...,
  private readonly eventEmitter: EventEmitter2,
) {}

async create(dto: CreatePostDto): Promise<PostEntity> {
  const post = await this.postsRepository.save(this.postsRepository.create(dto));
  this.eventEmitter.emit(POST_CREATED, new PostCreatedEvent(post.id, post.title));
  return post;
}
```

```ts
// src/posts/listeners/post-stats.listener.ts
@Injectable()
export class PostStatsListener {
  private readonly logger = new Logger(PostStatsListener.name);

  @OnEvent(POST_CREATED)
  handlePostCreated(event: PostCreatedEvent) {
    this.logger.log(`Post #${event.postId} "${event.title}" vừa được tạo`);
  }

  @OnEvent('post.*')
  handleAnyPostEvent(event: unknown) {
    this.logger.debug(`post.* → ${JSON.stringify(event)}`);
  }
}
```

- `EventEmitterModule` là **global module mặc định** — inject `EventEmitter2` ở bất kỳ đâu mà không cần import lại. `EventEmitter2` import từ `'@nestjs/event-emitter'`.
- Listener class phải là provider đã đăng ký (thêm `PostStatsListener` vào `providers` của `PostsModule`). Nest quét mọi provider tìm `@OnEvent` trong hook `onApplicationBootstrap` — event bắn ra **trước** thời điểm đó (trong constructor, `onModuleInit`) có thể bị mất.
- Payload nên là class có field `readonly` chứa **id và dữ liệu tối thiểu** — không phải entity nguyên khối. Tên event gom vào hằng số (`POST_CREATED`) để không gõ sai chuỗi: gõ sai tên event không báo lỗi gì, listener đơn giản là không bao giờ chạy.
- Wildcard phải bật `wildcard: true` ở `forRoot()`. `'post.*'` khớp đúng **1 đoạn** (`post.created`, `post.deleted`) nhưng KHÔNG khớp `post.comment.created` — muốn nhiều cấp dùng `'**'`.

> **Đối chiếu docs.nestjs.com/application/events (trang cũ `/techniques/events`) và type của `@nestjs/event-emitter` 12.0.1:** `OnEvent(event: string | symbol | Array<string | symbol>, options?: OnEventOptions)`, với `OnEventOptions = OnOptions & { prependListener?: boolean; suppressErrors?: boolean }` (`OnOptions` của `eventemitter2` 6.4.9 — chứa `async`, `promisify`, `nextTick`...). `EventEmitterModuleOptions extends ConstructorOptions` của eventemitter2 thêm `global?` và `inheritRequestContextId?`. Peer: `@nestjs/common ^11 || ^12` — khớp project (Nest 11). Package là ESM như @nestjs/typeorm 12 đang dùng. Mẫu code trên docs hiện viết theo Nest 12 ESM (import đuôi `.js`) — trong project Nest 11 CommonJS thì import không có đuôi như ở đây.

19.3 Bên trong: listener chạy CÙNG process, ĐỒNG BỘ theo mặc định

Đây là chỗ hiểu sai phổ biến nhất: "event = chạy nền". Không phải. `emit()` của EventEmitter2 là một vòng `for` gọi lần lượt từng listener **ngay tại chỗ**, trong cùng call stack của request. Đã đo thực tế với listener bận CPU 200ms:

```
before emit
  sync-listener start        ← emit() gọi listener NGAY, request đứng chờ
  sync-listener end
  wildcard listener chạy
after emit returned — 201ms  ← request chậm đúng bằng thời gian listener
  async-listener ran         ← { async: true } chạy SAU khi emit() trả về`
```

Còn lỗi thì sao? Nest bọc mỗi listener trong 1 hàm `async` có `try/catch` (đọc tại `event-subscribers.loader.js`, hàm `wrapFunctionInTryCatchBlocks`): lỗi bị **log với context `[Event]` rồi nuốt mất** nếu `suppressErrors` không đặt (mặc định `true`). Kết quả đã chạy thử cho từng tổ hợp:

Cách bắn / cấu hình listener	Request có chờ listener?	Listener `throw` thì...
`emit()` + listener mặc định	✅ Chờ hết phần đồng bộ của listener	Log lỗi, `emit()` không throw, request vẫn trả 201 — **lỗi "im lặng"**
`emit()` + `{ async: true }`	❌ Listener chạy sau khi `emit()` trả về	Như trên — log rồi nuốt
`emit()` + `{ suppressErrors: false }`	✅ Chờ phần đồng bộ	⚠ **Unhandled promise rejection** (đã tái hiện) — Node ≥15 mặc định **crash cả process**
`await emitAsync()` + `{ suppressErrors: false }`	✅ Chờ cả phần `await` bên trong listener	✅ Promise reject về phía người gọi → `try/catch` được
`await emitAsync()` + listener mặc định	✅ Chờ	Log rồi nuốt; `emitAsync` resolve bình thường

⚠ **Bẫy "`{ async: true }` = chạy nền":** nó chỉ **hoãn** listener tới lượt sau của event loop — vẫn cùng process, cùng luồng JS. Listener tốn CPU vẫn chặn mọi request khác; server restart giữa chừng thì listener đang chờ biến mất; không có retry. Nó giải quyết đúng 1 việc: "đừng bắt request hiện tại đợi". Muốn bền vững → Part 18 (queue). Tương tự: listener là method `async` có `await` bên trong thì phần sau `await` cũng đã chạy sau khi `emit()` trả về.

⚠ **Bẫy `suppressErrors: false` + `emit()`:** nghe hợp lý ("tôi muốn thấy lỗi"), nhưng `emit()` là hàm đồng bộ không chờ Promise của listener → lỗi thành unhandled rejection, cùng loại bug với "quên await ở Controller" trong mục 8.3. Nếu người gọi thật sự cần biết listener lỗi → dùng `await this.eventEmitter.emitAsync(...)` kèm `suppressErrors: false`. Còn nếu người gọi cần kết quả của việc đó để trả response, thì đó không nên là event — gọi thẳng service.

19.4 Entity listener của TypeORM — `@BeforeInsert`, `@AfterUpdate`...

Ngoài event của ứng dụng, TypeORM có event ở tầng ORM — tương đương "model events" của Eloquent. Cách đơn giản nhất: đặt decorator lên method ngay trong entity:

```ts
@Entity()
export class Post {
  ...
  @BeforeInsert()
  @BeforeUpdate()
  normalizeTitle() {
    this.title = this.title?.trim();
  }
}
```

Có đủ cặp `Before/After` + `Insert/Update/Remove/SoftRemove/Recover` và `@AfterLoad` (xem thư mục `node_modules/typeorm/decorator/listeners/`). Chúng hợp với việc **chỉ đụng tới chính entity đó** (chuẩn hoá dữ liệu, tính field phụ) vì method nằm trong entity — không có DI, không inject được service nào.

> ⚠ **Listener chỉ chạy khi đi qua `save()` / `remove()` trên instance.** Đã chạy thử với typeorm 1.1.1 + better-sqlite3: `repository.save(post)` → `@BeforeInsert` / `@AfterInsert` chạy; `repository.update(id, { title })` → `@AfterUpdate` của entity **KHÔNG chạy** (không có instance nào để gọi method). Giống cảnh báo mass update của Laravel (15.5). Đó cũng là lý do `PostsService.update()` của project dùng `findOne → Object.assign → save` — giữ nguyên cách đó nếu bạn dựa vào entity listener.

19.5 `EntitySubscriberInterface` — "Observer" có DI

Subscriber là class riêng nghe event của 1 entity (hoặc mọi entity), tương đương Observer của Laravel. Vấn đề: TypeORM tự `new` các class khai trong option `subscribers` — không qua DI container của Nest, nên không inject được gì. Docs Nest dùng mẹo đảo ngược: để **Nest** tạo subscriber (như 1 provider thường, DI đầy đủ), rồi subscriber tự đăng ký mình vào `DataSource`:

```ts
import { DataSource, EntitySubscriberInterface, EventSubscriber, InsertEvent,
UpdateEvent } from 'typeorm';

@EventSubscriber()
export class PostSubscriber implements EntitySubscriberInterface<PostEntity> {
  private readonly logger = new Logger(PostSubscriber.name);

  constructor(dataSource: DataSource) {
    dataSource.subscribers.push(this);
  }

  listenTo() {
    return PostEntity;
  }

  afterInsert(event: InsertEvent<PostEntity>) {
    this.logger.log(`INSERT post #${event.entity.id}`);
  }

  afterUpdate(event: UpdateEvent<PostEntity>) {
    this.logger.log(`UPDATE post: ${JSON.stringify(event.entity)}`);
  }
}

// posts.module.ts
providers: [PostsService, PostSubscriber],
```

	Entity listener (`@AfterInsert`)	**Subscriber (`EntitySubscriberInterface`)**
Viết ở đâu	Method trong entity	Class riêng, provider của Nest
DI (inject service)	❌	✅ Với mẹo `dataSource.subscribers.push(this)`
`repository.update()` / QueryBuilder update	❌ Không chạy	✅ `afterUpdate` chạy — nhưng `event.entity` chỉ chứa field vừa đổi, `event.databaseEntity` = `undefined` (đã chạy thử)
Event transaction	❌	✅ `beforeTransactionCommit`, `afterTransactionCommit`, `afterTransactionRollback`...
Hợp với	Chuẩn hoá dữ liệu của chính entity	Audit log, đồng bộ sang hệ thống khác, việc cần service

⚠ Đừng vừa push vừa khai trong `subscribers: [...]` của `TypeOrmModule.forRoot()`. Đã tái hiện: TypeORM tự `new PostSubscriber()` KHÔNG truyền tham số → constructor chạy `dataSource.subscribers.push` với `dataSource = undefined` → app không khởi động được: `Cannot read properties of undefined (reading 'subscribers')`. Chọn 1 trong 2: khai trong `subscribers` (không DI) HOẶC làm provider + push (có DI) — course này dùng cách thứ 2. Docs cũng cảnh báo subscriber **không được request-scoped**.

> **Đã kiểm chứng trong `node_modules/typeorm` 1.1.1 của project:**
> `subscriber/EntitySubscriberInterface.d.ts` có đủ hook `afterLoad`, `before/afterInsert`, `before/afterUpdate`, `before/afterRemove`, `before/afterSoftRemove`, `before/afterRecover`, `beforeQuery/afterQuery`, `before/afterTransactionStart|Commit|Rollback`; `DataSource.subscribers` là mảng public. Trong `DataSource.js`, TypeORM chỉ tự tạo subscriber từ `options.subscribers` (không quét decorator toàn cục) — vì vậy subscriber dạng provider BẮT BUỘC phải tự push. Mẫu code khớp docs.nestjs.com/data/typeorm mục "Subscribers"; đã chạy thử trong 1 app Nest với @nestjs/typeorm 12.0.1: subscriber nhận được `EventEmitter2` qua constructor và `afterInsert` chạy đúng.

19.6 Bẫy lớn: event/job bắn ra TRONG transaction (nối tiếp Part 13)

Laravel có hẳn mục 13.5 và 15.4 cho bài học này, và NestJS dính y hệt — chỉ khác là **không có** `afterCommit()` dựng sẵn. Kịch bản với `createWithWelcomeComment()` của project:

```
// ❌ SAI
return this.dataSource.transaction(async (manager) => {
  const post = await manager.save(PostEntity, dto);
  this.eventEmitter.emit(POST_CREATED, new PostCreatedEvent(post.id, post.title));
  await this.postsQueue.add('post-created', { postId: post.id });
  await manager.save(Comment, { content: 'chào mừng...', post });
  return post;
});
```

```
BEGIN
  INSERT post #1                ← chưa commit
  emit('post.created') ──► listener chạy NGAY (19.3): đọc post #1,
                           gửi mail "bài của bạn đã được đăng"
  queue.add(...)       ──► job đã nằm trong Redis, worker có thể nhặt ngay
  INSERT comment → LỖI
ROLLBACK                        ← post #1 không còn tồn tại
Kết quả: mail đã gửi cho 1 bài không tồn tại; worker thì findOneBy trả null
         (hoặc tệ hơn: ID #1 được cấp lại cho bài sau, worker xử lý nhầm bài)
```

Đã tái hiện trong app Nest + better-sqlite3: emit bên trong callback rồi `throw` → listener in ra "post #1 FOUND → gửi mail", sau đó rollback, bảng `post` còn **0** dòng. Với SQLite (1 kết nối), listener dùng repository thường vẫn **đọc thấy** dòng chưa commit — nên bug càng khó phát hiện lúc dev. Với Postgres/MySQL (pool nhiều connection), listener/worker dùng connection khác thường sẽ **không thấy** dòng đó → lỗi "not found" chập chờn tuỳ tốc độ. Cả 2 đều sai. Và cả entity listener lẫn subscriber `afterInsert` cũng đã chạy cho bản ghi bị rollback (đã chạy thử) — "after insert" ≠ "after commit".

```
// ✅ ĐÚNG — transaction chỉ lo dữ liệu, side effect chạy sau khi nó resolve
async createWithWelcomeComment(dto: CreatePostDto): Promise<PostEntity> {
  const post = await this.dataSource.transaction(async (manager) => {
    const post = await manager.save(PostEntity, dto);
    await manager.save(Comment, { content: 'chào mừng bài viết đầu tiên!', post });
    return post;
  });

  this.eventEmitter.emit(POST_CREATED, new PostCreatedEvent(post.id, post.title));
  await this.postsQueue.add('post-created', { postId: post.id });
  return post;
}
```

`await this.dataSource.transaction(...)` chỉ resolve khi `COMMIT` xong; nếu rollback thì nó `throw` → dòng `emit` không bao giờ được chạy. Đã chạy thử cả 2 nhánh: thất bại → listener không chạy; thành công → listener chạy và thấy post. Với `QueryRunner` (13.3): đặt side effect **sau** `commitTransaction()`, không đặt trong `try` trước nó.

> **Giới hạn còn lại — đối chiếu docs.nestjs.com/reliability/outbox:** cách "emit sau commit" vẫn còn 1 khe hở: process crash đúng giữa `COMMIT` và `queue.add()` → post đã lưu nhưng job mất. Docs gọi đây là *dual-write problem* và nói thẳng *"`ClientProxy.emit()`, `EventEmitter2` and the CQRS `EventBus` all publish from memory, so they all have it"*. Giải pháp triệt để là **transactional outbox**: ghi "tin nhắn" vào 1 bảng outbox trong CÙNG transaction, một relay đọc bảng đó để publish (Nest vừa ra package `@nestjs/outbox` 0.0.1 ngày 25/09/2026). Docs cũng xác nhận `queue.add()` của BullMQ 6 không thể tham gia transaction của bạn. Với course này: "emit/add sau khi transaction resolve" là mức bắt buộc; outbox là bước tiếp theo khi hệ thống không chấp nhận mất sự kiện.

19.7 EventEmitter (trong process) hay BullMQ queue (bền vững) — dùng cái nào cho side effect

Sau Part 18 và mục 19.3, người mới hay phân vân: "gửi thông báo khi có bài mới" nên viết `@OnEvent` hay `@Processor`? Cả hai đều tách được logic khỏi service. Câu hỏi quyết định: **nếu việc này bị mất giữa chừng (server restart, lỗi mạng), có ai phàn nàn không?**

Cách 1 — EventEmitter: listener chạy trong process

```
this.eventEmitter.emit(POST_CREATED, new PostCreatedEvent(post.id, post.title));

@OnEvent(POST_CREATED, { async: true })
async invalidateCache(event: PostCreatedEvent) {
  await this.cache.del('posts:list:p1:l10');
}
```

Được: không cần hạ tầng gì thêm, nhanh (gọi hàm trực tiếp, không qua mạng), có thể chạy đồng bộ khi cần (listener thấy đúng trạng thái hiện tại), dễ test. Mất: không bền (restart là mất), không retry, lỗi bị log rồi nuốt, dùng chung CPU với request.

Cách 2 — BullMQ: job trong Redis, worker xử lý

```
await this.postsQueue.add('notify-followers', { postId: post.id }, { attempts: 5,
backoff: { type: 'exponential', delay: 2000 } });

@Processor(POSTS_QUEUE)
export class PostsProcessor extends WorkerHost {
  async process(job: Job) { ... }
}
```

Được: sống sót qua restart, retry có backoff, quan sát được (`failed` set), chạy ở process khác. Mất: cần Redis + worker, dữ liệu phải serialize (chỉ truyền id), có độ trễ, job có thể chạy lại nên phải idempotent.

Kết hợp cả hai là mẫu phổ biến nhất: service chỉ `emit` 1 event; các listener nhẹ (xoá cache, log) làm việc ngay; listener nào có việc quan trọng thì bên trong nó chỉ làm đúng 1 việc — `queue.add()`. Service không cần biết có queue.

```
@OnEvent(POST_CREATED)
async enqueueNotification(event: PostCreatedEvent) {
  await this.postsQueue.add('notify-followers', { postId: event.postId });
}
```

> **Nguồn đã đối chiếu:** docs.nestjs.com/application/events mô tả `@nestjs/event-emitter` là *"an observer implementation"* dựa trên `eventemitter2` (in-process); docs.nestjs.com/application/queues mô tả queue để *"provide a reliable communication channel... When queue producers or consumers fail, their state is preserved"*. Hành vi đồng bộ/nuốt lỗi của EventEmitter ở bảng dưới lấy từ thí nghiệm ở 19.3 (@nestjs/event-emitter 12.0.1, eventemitter2 6.4.9); hành vi retry/bền của BullMQ lấy từ thí nghiệm Part 18 (bullmq 6.3.10, Redis 7).

Nhu cầu	Cách 1: EventEmitter	Cách 2: BullMQ
Xoá cache, ghi log phụ, cập nhật bộ đếm gần đúng	✅ Dùng cái này	Thừa — thêm độ trễ và hạ tầng
Gửi mail/thông báo, gọi webhook, gọi API bên thứ 3	❌ Restart là mất, không retry	✅ Dùng cái này
Việc tốn CPU (xử lý ảnh, PDF, export)	❌ Chặn event loop (kể cả `async: true`)	✅ Worker process riêng
Listener phải chạy trong cùng luồng và thấy ngay kết quả	✅ `emit` / `emitAsync`	❌ Có độ trễ, chạy ở process khác
Cần biết việc đã fail hẳn để xử lý tay	❌ Chỉ có 1 dòng log	✅ Set `failed` + event
Không muốn chạy Redis	✅	❌

Quy tắc thực dụng: event dùng để **tách code**, queue dùng để **bảo đảm việc được làm**. Mặc định cho mọi side effect: `emit` 1 event sau khi transaction resolve (19.6). Rồi với từng listener hỏi câu "mất có ai phàn nàn không?" — có → listener đó chỉ `queue.add()`; không → làm luôn trong listener.

19.8 Test service bắn event & test listener

```
const eventEmitter = { emit: jest.fn() };
const dataSource = { transaction: jest.fn() };

providers: [
  PostsService,
  { provide: getRepositoryToken(Post), useValue: repo },
  { provide: DataSource, useValue: dataSource },
  { provide: EventEmitter2, useValue: eventEmitter },
  { provide: getQueueToken(POSTS_QUEUE), useValue: { add: jest.fn() } },
]

it('create bắn post.created với đúng id', async () => {
  repo.save.mockResolvedValue({ id: 7, title: 'A', body: 'B' });
  await service.create({ title: 'A', body: 'B' });
  expect(eventEmitter.emit).toHaveBeenCalledWith(POST_CREATED, new PostCreatedEvent(7,
'A'));
});

it('rollback thì KHÔNG bắn event', async () => {
  dataSource.transaction.mockRejectedValue(new Error('db down'));
  await expect(service.createWithWelcomeComment(dto)).rejects.toThrow('db down');
  expect(eventEmitter.emit).not.toHaveBeenCalled();
});
```

Listener là class thường → test bằng cách `new PostStatsListener()` rồi gọi thẳng `handlePostCreated(new PostCreatedEvent(1, 'A'))`, không cần emitter. Test thứ 2 ở trên chính là "hợp đồng" của mục 19.6 — giữ nó để không ai vô tình chuyển `emit` vào trong callback transaction.

Việc cần làm:

1. `npm i @nestjs/event-emitter`, thêm `EventEmitterModule.forRoot({ wildcard: true })` vào `AppModule`. Tạo `PostCreatedEvent` + hằng `POST_CREATED`, bắn từ `PostsService.create()`; viết `PostStatsListener` log ra console, đăng ký trong `PostsModule`
2. Tự đo mục 19.3: cho listener bận 500ms (vòng `while` theo `Date.now()`), gọi `POST /posts` và xem `LoggerMiddleware` (Part 07) báo thời gian request. Đổi sang `{ async: true }`, đo lại. Rồi cho listener `throw` — xác nhận response vẫn 201 và lỗi chỉ nằm trong log `[Event]`
3. Thêm `@BeforeInsert` / `@BeforeUpdate` trim `title` trong entity `Post`. Gọi `PUT /posts/:id` (đi qua `save()`) → thấy trim; viết tạm 1 lệnh `repository.update()` → chứng minh listener không chạy
4. Viết `PostSubscriber` theo mẫu 19.5 (provider + `dataSource.subscribers.push(this)`), log `afterInsert` / `afterUpdate`; so sánh `event.entity` khi update qua `save()` và qua `update()`
5. Tái hiện bug 19.6 trên `createWithWelcomeComment()`: emit bên trong callback, sau đó `throw new Error('test rollback')` → listener vẫn log "post #N", DB không có post. Sửa bằng cách emit sau khi transaction resolve, chạy lại cả 2 nhánh
6. Áp dụng quy tắc 19.7: viết listener `enqueueNotification` chỉ làm `queue.add('notify-followers', { postId })` (dùng queue của Part 18). Viết 2 test ở 19.8 (bắn đúng event; rollback thì không bắn)

PART 20

Cache & Lock

> **Mục tiêu:** Giảm tải database bằng cache đúng cách với `@nestjs/cache-manager` (và biết khi nào cache gây hại), nắm chắc đơn vị TTL, xoá cache đúng lúc khi dữ liệu đổi, chuyển sang Redis khi chạy nhiều instance, chống cache stampede, và dùng distributed lock để chống race condition giữa nhiều process.

20.1 Cache trong Node khác PHP thế nào — và khi nào chưa nên cache

Laravel Part 16 mở đầu bằng "PHP mỗi request một process, RAM không sống qua request, nên cache phải nằm ngoài process". Node thì ngược lại: process sống mãi, một `Map` khai ở module scope **có** sống qua mọi request. Vì vậy store mặc định của Nest là **in-memory** và dùng được ngay. Cái giá phải trả xuất hiện khi bạn chạy nhiều hơn 1 process:

```
              ┌─ instance A (RAM riêng): posts:list = [bài 1, 2, 3]
Load balancer ┤
              └─ instance B (RAM riêng): posts:list = [bài 1, 2, 3]

PUT /posts/2 rơi vào A → A xoá cache của A → B vẫn trả bản CŨ tới khi hết TTL
Mỗi instance tự query DB để làm đầy cache riêng → N instance = N lần tính lại
```

Store	Đặc điểm	Dùng khi
In-memory (mặc định, Keyv + `Map`)	Nhanh nhất, mất khi restart, mỗi process 1 bản riêng	1 instance, dev, test
Redis (`@keyv/redis`)	Dùng chung giữa mọi instance, sống qua restart, tốn 1 round-trip mạng	Production nhiều instance / pm2 cluster / nhiều container
Nhiều tầng (`stores: [memory, redis]`)	Store đầu là mặc định, các store sau là dự phòng	Khi đã đo và thấy Redis round-trip là nút thắt

Đừng cache quá sớm — cùng lời khuyên với Laravel 16.4. Cache thêm một loại bug mới ("sửa rồi mà vẫn thấy cũ"). Thứ tự đúng: (1) sửa query thừa và N+1, chỉ lấy cột cần (Part 08–09), (2) đo thời gian bằng `LoggerMiddleware` (Part 07), (3) chỉ cache chỗ đọc nhiều – ghi ít và thật sự chậm. Và **không bao giờ** cache dữ liệu riêng theo user mà key không chứa user id (xem 20.4).

20.2 Cài đặt, `CacheModule` và bẫy đơn vị TTL

```
npm i @nestjs/cache-manager cache-manager
```

```
// src/app.module.ts
import { CacheModule } from '@nestjs/cache-manager';

imports: [
  CacheModule.register({
    isGlobal: true,
    ttl: 60_000,
  }),
  ...
]
```

`isGlobal: true` để mọi feature module inject được mà không phải import `CacheModule` lại (giống `ConfigModule` đang dùng). Không đặt `ttl` thì entry **không bao giờ hết hạn** (tương đương `ttl: 0`).

⚠ Bẫy kinh điển: TTL tính bằng MILLI-GIÂY. `ttl: 60` nghĩa là 60ms — cache gần như vô dụng mà không có lỗi nào báo. Đã chạy thử: `cache.set('k', 'v', 300)` → sau 400ms đọc lại được `undefined`; trên Redis, `PTTL` của key đặt 10.000 trả ~9.885 (ms). Nguồn gốc của nhầm lẫn: `cache-manager` v4 dùng **giây** (README 4.1.0: `ttl: 10/*seconds*/`), từ v5 đổi sang ms (README 5.7.6: `ttl: 10 * 1000 /*milliseconds*/`) — tutorial NestJS viết thời v4 vẫn còn đầy trên mạng. Viết `60_000` (dấu `_` là phân cách số hợp lệ trong TS) để mắt thấy ngay đơn vị.

Đã kiểm chứng (docs.nestjs.com/data/caching — trang cũ `/techniques/caching`; type trong package): @nestjs/cache-manager 12.0.0 có peer `cache-manager >=6`, `keyv >=5`, `@nestjs/common ^9 → ^12`; bản cài được là cache-manager 7.2.9 + keyv 5.6.0 (đã thử resolve trên bản sao `package.json` / `package-lock.json` của project — không xung đột, `@nestjs/common` vẫn 11.2.3; course nhắm Nest 11 CommonJS dù docs đã chuyển mẫu sang Nest 12 ESM với import đuôi `.js`). Từ v6, cache-manager chạy trên **Keyv**: cấu hình store kiểu cũ `store: redisStore` (package `cache-manager-redis-store`) không còn dùng, thay bằng mảng `stores` chứa instance Keyv (20.6). Docs ghi: miss thì `get()` trả `undefined` (v6 trả `null`) — đã chạy thử với 7.2.9: `undefined`. Kiểu `Cache` import từ `'@nestjs/cache-manager'` (khai trong `dist/cache.module.d.ts` là `ReturnType<typeof createCache>`, dùng được làm type lẫn token).

20.3 Cache-aside thủ công với `CACHE_MANAGER`: `get` / `set` / `del` / `wrap`

```
import { CACHE_MANAGER, Cache } from '@nestjs/cache-manager';

@Injectable()
export class ProductsService {
  constructor(
    @InjectRepository(Product)
    private readonly productsRepository: Repository<Product>,
    @Inject(CACHE_MANAGER)
    private readonly cache: Cache,
  ) {}

  async findOne(id: number): Promise<Product> {
    const key = `product:${id}`;
    const cached = await this.cache.get<Product>(key);
    if (cached) return cached;

    const product = await this.loadProduct(id);
    await this.cache.set(key, product, 60_000);
    return product;
  }

  private async loadProduct(id: number): Promise<Product> {
    const product = await this.productsRepository.findOne({ where: { id }, relations: {
category: true } });
    if (!product) throw new NotFoundException(`Product ${id} not found`);
    return product;
  }
}
```

Đây là mẫu **cache-aside** (giống `Cache::remember` của Laravel): đọc cache → có thì trả → không có thì hỏi DB → cất vào cache → trả. `cache-manager` có sẵn `wrap()` làm đúng việc đó trong 1 dòng:

```
findOne(id: number): Promise<Product> {
  return this.cache.wrap(`product:${id}`, () => this.loadProduct(id), 60_000);
}
```

Đã chạy thử: hàm tính bị `throw` (ví dụ `NotFoundException`) → `wrap()` ném lại lỗi và **không** cất gì vào cache, lần sau vẫn hỏi DB. `del(key)` xoá 1 key, `clear()` xoá **toàn bộ** cache (không dùng để "xoá nhóm posts").

> ⚠ **Bẫy "dev chạy, production hỏng" — kiểu dữ liệu lấy ra từ cache:** đã chạy thử lưu 1 instance class có field `Date` và method:
>
> - **In-memory (mặc định):** `get()` trả về **chính object đã `set`** (cùng tham chiếu, `instanceof` đúng, `Date` vẫn là `Date`). Hệ quả: sửa object lấy từ cache (`product.name = ...`) là **sửa luôn bản trong cache** cho mọi request sau.
> - **Redis:** dữ liệu đi qua JSON → nhận lại object thường: `Date` thành chuỗi, method mất, `instanceof` sai.
>
> Code dựa vào `product.createdAt.getTime()` chạy ngon ở dev (memory) và nổ ở production (Redis). Quy tắc: chỉ cất dữ liệu JSON-an-toàn và luôn đối xử với giá trị lấy ra như object thường, không sửa trực tiếp.

> **Chỗ docs và thực tế lệch nhau:** docs.nestjs.com/data/caching ghi *"The default in-memory store is a Keyv instance, which serializes values to JSON before storing them"*. Thí nghiệm với @nestjs/cache-manager 12.0.0 + cache-manager 7.2.9 + keyv 5.6.0 cho thấy store mặc định trả lại **cùng tham chiếu** (`got === orig` là `true` , sửa `got.a` thì lần `get` sau thấy giá trị mới). Theo nguyên tắc của tài liệu này, hành vi đo được trên đúng phiên bản package được giữ; lời khuyên của docs ("chỉ lưu giá trị JSON-serializable") vẫn đúng và càng quan trọng vì bạn sẽ chuyển sang Redis.

20.4 `CacheInterceptor` + `@CacheKey` / `@CacheTTL` — cache tự động theo URL

```
import { CacheInterceptor, CacheTTL } from '@nestjs/cache-manager';

@Controller('categories')
@UseInterceptors(CacheInterceptor)
@CacheTTL(30_000)
export class CategoriesController {
  @Get()
  findAll(@Query() query: PaginationQueryDto) { ... }

  @Get(':id')
  @CacheTTL(60_000)
  findOne(@Param('id', ParseIntPipe) id: number) { ... }
}
```

Bên trong (đọc `dist/interceptors/cache.interceptor.js`): `trackBy()` lấy **URL của request làm key** (gồm cả query string) → `cache.get(key)` → có thì trả luôn bằng `of(value)` , handler **không chạy** → không có thì chạy handler rồi `set` kết quả. Interceptor còn gắn header `X-Cache: HIT` / `MISS` — mở tab Network là thấy. Kết quả chạy thử:

```
GET  /posts        → MISS  {"hits":1}  handler chạy
GET  /posts        → HIT   {"hits":1}  handler KHÔNG chạy
GET  /posts?page=2 → MISS  {"hits":2}  query khác = key khác
POST /posts        → (—)   {"hits":3}  POST không bao giờ được cache...
GET  /posts        → HIT   {"hits":1}  ...và KHÔNG xoá cache của GET → dữ liệu cũ
GET  /posts/5      → MISS; sau 1.1s với @CacheTTL(1000) → MISS lại (TTL tính ms)
```

Giới hạn của CacheInterceptor	**Hệ quả**
Chỉ cache `GET` (`allowedMethods = ['GET']`)	Đúng ý — nhưng ghi dữ liệu không tự làm mất cache
Key = URL, không biết dữ liệu nào nằm trong response	Khi post #2 đổi, không biết phải xoá những URL nào (`/posts` , `/posts?page=2&limit=5` , `/posts/2` ...)
Key không chứa user	⚠ Lộ dữ liệu giữa các user (xem bẫy bên dưới)
Route dùng `@Res()`	Không dùng được (docs)

Cache kết quả handler, không phải response cuối	`TransformInterceptor` global (Part 07) vẫn bọc `{ success, data }` cho cả HIT lẫn MISS vì nó nằm ngoài; nhưng đừng bind CacheInterceptor ở chỗ nó chạy ngoài TransformInterceptor rồi mong 2 thứ khớp nhau

⚠ **Bẫy bảo mật — cache route theo user:** project có `GET /auth/me` (JWT). Nếu route này bị `CacheInterceptor` bao phủ (ví dụ bạn bind global bằng `APP_INTERCEPTOR` "cho nhanh"), mọi user cùng một URL → cùng một key. Đã tái hiện: user 1 gọi → MISS `{"userId":1}`; user 2 gọi → **HIT** `{"userId":1}` — user 2 thấy hồ sơ của user 1. Cách sửa: không cache route đó, hoặc kế thừa và override `trackBy` đưa user id vào key (đã chạy thử: user 2 nhận MISS rồi đúng dữ liệu của mình):

```
@Injectable()
export class UserCacheInterceptor extends CacheInterceptor {
  protected async trackBy(context: ExecutionContext): Promise<string | undefined> {
    const base = await super.trackBy(context);
    const user = context.switchToHttp().getRequest().user;
    return base && user ? `user:${user.userId}:${base}` : undefined;
  }
}
```

Guard chạy trước Interceptor (Part 07) nên lúc này `request.user` đã có. Trả `undefined` = không cache.

Đối chiếu docs.nestjs.com/data/caching mục "Auto-caching responses" / "Adjust tracking": *"By default, only `GET` endpoints are cached (an explicit `@CacheKey()` on a route handler bypasses this check)"* — khớp code: khi có `@CacheKey`, `trackBy` trả thẳng key đó mà không gọi `isRequestCacheable`. `@CacheTTL()` đặt ở controller áp cho mọi route, ở method thì ưu tiên hơn; nhận số (ms) hoặc hàm `(ctx) => number`. Kiểu trong `cache.interceptor.d.ts` 12.0.0: `protected trackBy(context): Promise<string | undefined | null> | string | undefined | null`.

20.5 Xoá cache khi dữ liệu đổi (cache invalidation)

"Cất thế nào" dễ; "xoá khi nào" mới khó. Với 1 bản ghi thì đơn giản — sau khi ghi thành công, xoá đúng key của nó:

```
async update(id: number, dto: UpdatePostDto): Promise<PostEntity> {
  const post = await this.findOne(id);
  Object.assign(post, dto);
  const saved = await this.postsRepository.save(post);
  await this.cache.del(`post:${id}`);
  await this.bumpListVersion();
  return saved;
}
```

Danh sách có phân trang (Part 12) mới là vấn đề: `posts:list:p1:l10`, `p2:l10`, `p1:l5`... bạn không liệt kê hết được để `del`. Mẹo đơn giản và đúng: **version key** — nhét 1 "số phiên bản" vào mọi key danh sách, muốn xoá cả nhóm thì chỉ đổi số đó; key cũ không ai đọc nữa và tự hết hạn theo TTL:

```
private async listVersion(): Promise<number> {
  return (await this.cache.get<number>('posts:list:ver')) ?? 0;
}

private async bumpListVersion(): Promise<void> {
  await this.cache.set('posts:list:ver', Date.now(), 0);
}

async findAll(query: PaginationQueryDto): Promise<Paginated<PostEntity>> {
  const key = `posts:list:v${await this.listVersion()}:p${query.page}:l${query.limit}`;
  return this.cache.wrap(key, () => this.loadPage(query), 30_000);
}
```

Chiến lược	Cách làm	Đánh đổi
TTL ngắn	Chỉ đặt `ttl` 30–60s, không xoá gì	Đơn giản nhất; chấp nhận dữ liệu trễ tối đa bằng TTL
Xoá đúng key	`del('post:2')` sau khi ghi	Chính xác cho 1 bản ghi; không dùng được cho danh sách nhiều biến thể
Version key	Đổi `posts:list:ver` → mọi key danh sách cũ bị bỏ	Xoá "cả nhóm" chỉ bằng 1 lệnh; tốn thêm 1 lần `get` mỗi request

Xoá cache ở đâu? Nếu logic ghi nằm rải rác nhiều service, gom việc xoá cache vào 1 listener `@OnEvent('post.*')` (Part 19) — và bắn event **sau khi transaction commit** (19.6). Xoá cache *trước* commit là tự mở cửa cho race: request khác đọc DB (vẫn dữ liệu cũ) rồi cất lại vào cache ngay trước khi bạn commit. Dùng `Date.now()` làm version (thay vì đọc rồi `+1`) để 2 request cùng bump không ghi đè nhau về cùng 1 số.

20.6 Redis store với `@keyv/redis` — khi chạy nhiều instance

```
npm i @keyv/redis
```

```
import { createKeyv } from '@keyv/redis';

CacheModule.registerAsync({
  isGlobal: true,
  inject: [ConfigService],
  useFactory: (config: ConfigService) => ({
    ttl: 60_000,
    stores: [
      createKeyv(`redis://${config.get('REDIS_HOST',
'localhost')}:${config.get('REDIS_PORT', 6379)}`, {
        namespace: 'blog',
      }),
    ],
  }),
}),
```

Dùng lại đúng Redis đã chạy cho BullMQ ở Part 18. `namespace` thêm tiền tố cho key — đã chạy thử: `set('posts:list', ...)` tạo key Redis `blog::posts:list`. Nên đặt, để key cache không lẫn với key `bull:*` của queue và dễ soi bằng `redis-cli --scan --pattern 'blog::*'`. Không có namespace thì key nằm trần (`/posts` , `product:1` ...).

> **Đã kiểm chứng:** `@keyv/redis` 5.1.6 (peer `keyv ^5.6.0` , dùng client `@redis/client` v5 — KHÁC client `ioredis` của BullMQ, 2 kết nối riêng là bình thường): `createKeyv(connect?: string | RedisClientOptions | RedisClientType, options?: KeyvRedisOptions): Keyv` , `KeyvRedisOptions` có `namespace` , `keyPrefixSeparator` ... Cấu hình `stores: [...]` khớp docs.nestjs.com/data/caching mục "Using alternative Cache stores" (*"The first store in the array is the default store, and the rest are fallback stores"*). Đã chạy toàn bộ thí nghiệm 20.3–20.4 lại trên Redis 7: kết quả HIT/MISS giống hệt, TTL đúng ms.

20.7 Cache stampede — 100 request cùng lúc thấy cache hết hạn

Trang chủ được cache 60s. Đúng giây thứ 60 key hết hạn, 100 request đang đến cùng lúc đều MISS → cả 100 cùng chạy query nặng → DB quá tải đúng lúc cache lẽ ra phải bảo vệ nó. Ba lớp phòng thủ:

- `wrap()` **gộp lời gọi trùng trong 1 process.** Đã chạy thử: 10 lời gọi `wrap('w', fn)` đồng thời → `fn` chỉ chạy **1 lần**, 9 lời gọi kia chờ chung kết quả (code dùng `coalesceAsync` với key `${cacheId}::${key}`). Viết tay `get → if → set` như đầu mục 20.3 thì KHÔNG có bảo vệ này — thêm 1 lý do để dùng `wrap()` . Nhưng nó chỉ gộp trong **một** process: 4 instance vẫn tính 4 lần.
- **Stale-while-revalidate bằng** `refreshThreshold` (tương đương `Cache::flexible` của Laravel 16.3): `wrap(key, fn, 60_000, 10_000)` — khi TTL còn dưới 10s, vẫn trả giá trị cũ ngay và âm thầm tính lại trong nền, nên key hầu như không bao giờ "hết hạn trước mặt user".
- **TTL lệch nhau (jitter)**: `60_000 + Math.floor(Math.random() * 10_000)` để hàng loạt key tạo cùng lúc không hết hạn cùng lúc. Với việc tính cực đắt trên nhiều instance: dùng lock (20.8) để chỉ 1 instance tính lại.

> **Nguồn:** `cache-manager` 7.2.9 — chữ ký `wrap<T>(key, fnc, ttl?, refreshThreshold?)` trong `dist/index.d.cts` ; README mục `wrap` : *"If `refreshThreshold` is set and the remaining TTL is less than `refreshThreshold` , the system will update the value asynchronously. In the meantime, the system will return the old value until expiration."* Việc gộp lời gọi đọc trực tiếp trong `dist/index.cjs` (hàm `wrap` bọc trong `coalesceAsync` của `@cacheable/utils`) và đã đo thực tế.

20.8 Distributed lock — chống race condition giữa nhiều process

Tình huống: endpoint `POST /products/reindex` chạy 1 việc nặng 30 giây (dựng lại chỉ mục tìm kiếm). Admin bấm 2 lần, hoặc 2 instance cùng nhận request → 2 lần chạy song song, ghi đè lẫn nhau. Transaction (Part 13) không giải quyết được (việc này không nằm gọn trong 1 transaction), một biến `let running = false` trong RAM cũng không (mỗi instance có RAM riêng — lại 20.1). Cần 1 khoá nằm ở chỗ **mọi instance cùng thấy**: Redis.

```ts
import Redis from 'ioredis';
import { randomUUID } from 'node:crypto';

const RELEASE_SCRIPT = `if redis.call('GET', KEYS[1]) == ARGV[1] then return
redis.call('DEL', KEYS[1]) end return 0`;

@Injectable()
export class RedisLockService implements OnModuleDestroy {
  private readonly redis = new Redis({ host: 'localhost', port: 6379 });

  async acquire(key: string, ttlMs: number): Promise<string | null> {
    const token = randomUUID();
    const ok = await this.redis.set(`lock:${key}`, token, 'PX', ttlMs, 'NX');
    return ok === 'OK' ? token : null;
  }

  async release(key: string, token: string): Promise<void> {
    await this.redis.eval(RELEASE_SCRIPT, 1, `lock:${key}`, token);
  }

  async onModuleDestroy() {
    await this.redis.quit();
  }
}
```

```ts
// ProductsController
@Post('reindex')
async reindex() {
  const token = await this.locks.acquire('products:reindex', 60_000);
  if (!token) throw new ConflictException('Đang reindex, thử lại sau');
  try {
    return await this.productsService.reindex();
  } finally {
    await this.locks.release('products:reindex', token);
  }
}
```

- `SET key token PX 60000 NX` là **1 lệnh nguyên tử**: chỉ ghi nếu key chưa tồn tại (`NX`) và tự hết hạn sau 60s (`PX`). Đã chạy thử 2 lời gọi `acquire` song song: 1 bên nhận token, bên kia nhận `null`. Tự hết hạn là bắt buộc — process crash khi đang giữ khoá thì khoá tự nhả, không kẹt vĩnh viễn.
- **Token ngẫu nhiên + nhả bằng Lua**: chỉ người giữ đúng token mới được xoá khoá. Đã chạy thử: nhả với token sai → Redis trả `0` (không xoá); token đúng → `1` . Nếu nhả bằng `DEL` trần: A giữ quá TTL, khoá hết hạn, B lấy được, rồi A chạy xong gọi `DEL` → xoá mất khoá của B → C vào được, lại 2 người cùng chạy.
- `finally` bảo đảm nhả khoá kể cả khi việc bên trong `throw` . Trả **409 Conflict** (không phải 500) khi không lấy được khoá — đây là tranh chấp, không phải lỗi server.

Giới hạn phải biết: TTL phải dài hơn thời gian việc chạy lâu nhất. Nếu việc chạy quá TTL (hoặc process bị treo vì GC/mạng), khoá hết hạn và người khác vào được trong khi bạn vẫn tưởng mình đang giữ — khoá kiểu này là "gần đúng", đủ cho chống bấm trùng / chống chạy trùng việc nặng, **chưa đủ** cho tiền bạc. Những chỗ tuyệt đối không được ghi trùng cần thêm ràng buộc ở DB (unique index, điều kiện trong `UPDATE ... WHERE`) hoặc fencing token.

Đã kiểm chứng: đoạn code trên chạy với `ioredis` (bản npm tự chọn cho project là 5.11.1 — vì typeorm 1.1.1 khai optional peer `ioredis ^5.0.4`; ép cài `ioredis@6` sẽ gặp lỗi `ERESOLVE`, đã tái hiện trong thư mục thử nghiệm) trên Redis 7. Lệnh `SET NX PX` và kịch bản Lua kiểm tra-rồi-xoá trùng với store Redis mẫu trong docs.nestjs.com/reliability/locks. Trang đó giới thiệu package chính thức mới **`@nestjs/locks`** (0.0.1, phát hành 25/09/2026, peer `@nestjs/common ^11 || ^12`): `Locks.withLock(key, fn, { ttl, wait })` ném `LockNotAcquiredError` khi không lấy được khoá, decorator `@OnOneInstance()` / `@WithoutOverlapping()` cho cron job, tự gia hạn lease và có fencing token — nhưng package **không kèm store dùng chung** (bạn tự viết `LockStore`, mặc định in-memory chỉ đúng cho 1 process và bị từ chối khởi động khi `NODE_ENV=production` trừ khi đặt `allowInMemoryStorage`). Docs cũng giải thích đúng lỗ hổng ở khung vàng trên: *"A `SET NX` key in Redis at the top of the job works until its holder pauses for longer than the key's TTL"*. Với course này: hiểu cơ chế `SET NX PX` trước; khi cần cho cron job nhiều instance thì cân nhắc `@nestjs/locks` — lưu ý nó mới ở bản 0.0.x.

20.9 `CacheInterceptor` (tự động) hay cache-aside trong service (thủ công)

Hai cách ở 20.3 và 20.4 đều cho ra "lần 2 nhanh hơn lần 1", nên người mới hay chọn `CacheInterceptor` vì chỉ tốn 1 dòng decorator. Câu hỏi quyết định: **khi dữ liệu này thay đổi, bạn có cần nó hiện đúng ngay (phải xoá cache chủ động) không?**

Cách 1 — `CacheInterceptor` ở controller

```
@Controller('categories')
@UseInterceptors(CacheInterceptor)
@CacheTTL(30_000)
export class CategoriesController { ... }
```

Được: 1 dòng, không đụng service, cache cả response của handler, có header `X-Cache`. Mất: key là URL nên không xoá chính xác được khi dữ liệu đổi (chỉ trông vào TTL), chỉ GET, dễ lộ dữ liệu giữa user nếu quên `trackBy`, chỉ cache được ở tầng HTTP (service gọi từ worker/listener không hưởng).

Cách 2 — cache-aside bằng `CACHE_MANAGER` trong service

```
findOne(id: number) {
  return this.cache.wrap(`product:${id}`, () => this.loadProduct(id), 60_000);
}
async update(id: number, dto) {
  ...save...
  await this.cache.del(`product:${id}`);
}
```

Được: bạn đặt tên key theo **dữ liệu** (`product:1`) nên xoá đúng key khi ghi, dùng được version key cho danh sách, chống stampede bằng `wrap()`, mọi nơi gọi service (controller, worker Part 18, listener Part 19) đều hưởng cache. Mất: thêm code trong service, phải tự nhớ xoá cache ở mọi đường ghi.

Nguồn đối chiếu: docs.nestjs.com/data/caching (CacheInterceptor dùng URL làm key, chỉ GET, `trackBy` để đổi cách tạo key; cảnh báo *"Never cache actions that perform business operations rather than only querying data"*); hành vi HIT/MISS, POST không xoá cache, lộ dữ liệu giữa user và `wrap()` gộp lời gọi — đo thực tế ở 20.3, 20.4, 20.7 (@nestjs/cache-manager 12.0.0, cache-manager 7.2.9, memory và Redis 7).

Nhu cầu	Cách 1: CacheInterceptor	Cách 2: cache-aside trong service
Dữ liệu công khai, ít đổi, trễ 30–60s không sao (danh mục, cấu hình)	✅ Dùng cái này	Chạy được, nhưng tốn code hơn cần thiết
Sửa xong phải thấy ngay (post, product)	❌ Không biết xoá URL nào	✅ `del` đúng key / version key
Response phụ thuộc user đăng nhập	⚠ Chỉ khi override `trackBy`	✅ Key tự chứa user id
Worker / listener cũng cần dữ liệu đã cache	❌ Chỉ ở tầng HTTP	✅
Chống stampede	❌ Tự `get` rồi chạy handler, không gộp	✅ `wrap()` + `refreshThreshold`

Quy tắc thực dụng: mặc định cache trong service bằng `wrap()` với key đặt theo dữ liệu, và xoá key ngay sau khi ghi (sau commit). Chỉ dùng `CacheInterceptor` cho route GET công khai, không theo user, mà bạn chấp nhận hiển thị cũ trong đúng khoảng TTL — và nếu phải hỏi "cái này sửa xong bao lâu mới thấy?" mà câu trả lời "phải thấy ngay", thì đó không phải chỗ cho `CacheInterceptor`.

Việc cần làm:

1. `npm i @nestjs/cache-manager cache-manager`, `CacheModule.register({ isGlobal: true, ttl: 60_000 })`. Thử cố tình đặt `ttl: 60`, gọi 2 lần liên tiếp và thấy cache "không ăn" — rồi sửa lại, ghi nhớ đơn vị ms
2. Cache `ProductsService.findOne()` bằng `wrap()` key `product:${id}` (lưu ý route hiện là `@Get('id')` trong `ProductsController` — thiếu dấu `:`, sửa thành `':id'` trước). Đo thời gian lần 1/lần 2 qua `LoggerMiddleware`
3. Cache `PostsService.findAll()` theo version key (20.5); bump version + `del('post:${id}')` trong `update()` / `remove()`, hoặc trong listener `@OnEvent('post.*')` của Part 19. Chứng minh: sửa post → `GET /posts` thấy ngay bản mới
4. Gắn `CacheInterceptor` + `@CacheTTL(30_000)` cho `CategoriesController`, xem header `X-Cache`. Tạo category mới bằng `POST` → xác nhận `GET /categories` vẫn trả bản cũ tới khi hết TTL (giới hạn của Cách 1). Tái hiện bẫy 20.4 trên `GET /auth/me` với 2 token khác nhau, rồi sửa bằng `UserCacheInterceptor` (hoặc bỏ cache route đó)
5. `npm i @keyv/redis`, chuyển sang `CacheModule.registerAsync` với `createKeyv(..., { namespace: 'blog' })` dùng chung Redis của Part 18. Soi key bằng `docker exec -it blog-redis redis-cli --scan --pattern 'blog::*'` và `PTTL`. Chạy lại bước 2 — kiểm tra có chỗ nào dựa vào `Date`/method của object lấy từ cache không (20.3)
6. Viết `RedisLockService` (20.8) và endpoint `POST /products/reindex` giả lập việc 10 giây. Gọi 2 lần gần như cùng lúc (`curl ... &` hai lần) → 1 lần 201, 1 lần 409. Thử nhả khoá bằng token sai và xác nhận khoá vẫn còn (`redis-cli GET lock:products:reindex`)
7. Viết unit test cho `ProductsService` với `{ provide: CACHE_MANAGER, useValue: { wrap: jest.fn((key, fn) => fn()), del: jest.fn() } }`: assert `wrap` được gọi với key `product:1` và `del` được gọi sau khi ghi

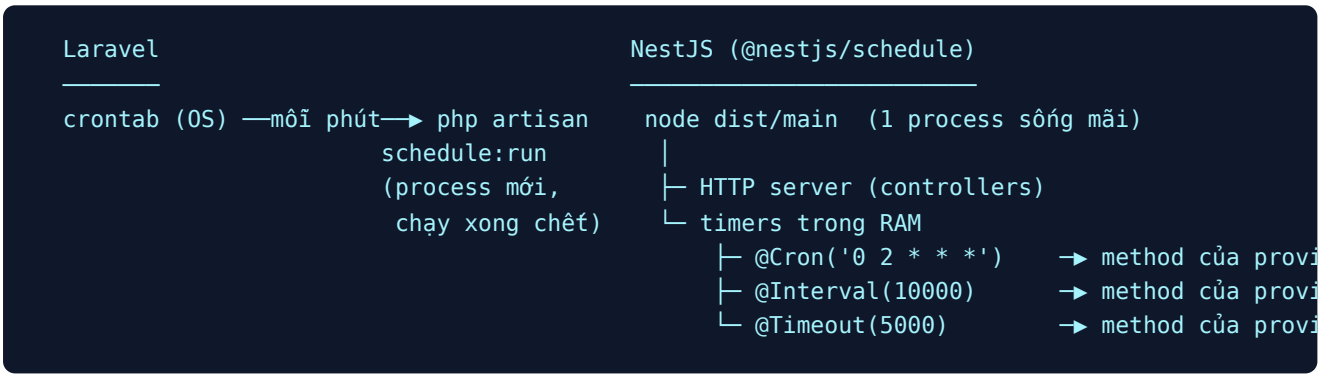
PART 21

Task Scheduling (cron)

> **Mục tiêu:** Chạy việc định kỳ (publish bài hẹn giờ, dọn dữ liệu cũ, gửi báo cáo) ngay trong code NestJS bằng `@nestjs/schedule`; hiểu nó chạy ở đâu, vì sao chạy nhiều instance là nhân đôi công việc, và khi nào phải chuyển sang BullMQ.

21.1 Cron trong Nest chạy ở đâu — khác hẳn Laravel

Ở Laravel (Part 17 bên đó), server có đúng 1 dòng crontab gọi `php artisan schedule:run` mỗi phút — mỗi lần là 1 process PHP mới, chạy xong thì chết. NestJS thì ngược lại: app là **1 process Node sống liên tục**, nên lịch được giữ ngay trong RAM của process đó dưới dạng các timer (`setTimeout`). Không có crontab, không có lệnh nào phải gọi từ ngoài — app đang chạy thì job chạy, app tắt thì job tắt theo.



Cơ chế bên trong: lúc app khởi động, `ScheduleModule` quét **mọi provider và controller** (qua `DiscoveryService` của `@nestjs/core`), tìm method nào có metadata `@Cron` / `@Interval` / `@Timeout`, rồi ở hook `onApplicationBootstrap` mới tạo timer thật. Khi app shutdown (`beforeApplicationShutdown`) toàn bộ timer bị huỷ. Vì vậy method đặt lịch **phải nằm trong 1 class đã đăng ký là provider** của một module nào đó — class "tự do" không ai đăng ký thì không bao giờ được quét thấy.

```
npm i @nestjs/schedule
```

```
import { ScheduleModule } from '@nestjs/schedule';

@Module({
  imports: [
    ConfigModule.forRoot({ isGlobal: true }),
    ScheduleModule.forRoot(),
    TypeOrmModule.forRoot({ ... }),
  ],
})
export class AppModule {}
```

Đã đối chiếu docs.nestjs.com/techniques/task-scheduling + mã nguồn `@nestjs/schedule@12.0.2` (bản `latest` trên npm lúc viết): `peerDependencies` là `@nestjs/common ^11.0.0 || ^12.0.0` → cài thẳng được vào project này (đang chạy `@nestjs/common 11.2.3`). Package phụ thuộc cứng `cron@4.4.0`. Bản 12 là package ESM (`"type": "module"`) — giống hệt `@nestjs/jwt 12` và `@nestjs/typeorm 12` mà project đang dùng, nên nạp được bình thường (đã thử chạy Jest của project: các package ESM v12 đó nạp được, chỉ cần script `NODE_OPTIONS=--experimental-vm-modules` có sẵn trong `package.json`). **Lưu ý phiên bản:** npm hiện đã có Nest 12 (`@nestjs/common 12.1.1` là `latest`) và docs.nestjs.com đang hiển thị code kiểu Nest 12/ESM (import có đuôi `.js`, `await bootstrap()` ở top-level). Course này bám Nest 11 CommonJS của project: không chép đuôi `.js` hay top-level await từ docs, và luôn kiểm tra `peerDependencies` trước khi `npm i` (ví dụ `@nestjs/serve-static` ở Part 22). Docs cảnh báo nguyên văn: *"Call `forRoot()` in one module only. Each additional import registers every `@Cron()`, `@Interval()` and `@Timeout()` handler in your app again."* Trong source, `forRoot()` trả về module `global: true` — không cần import lại ở feature module để inject `SchedulerRegistry`.

Bẫy "job không bao giờ chạy": (1) quên đăng ký class chứa `@Cron` vào `providers` của module nào đó; (2) đặt `@Cron` trong provider **request-scoped** (hoặc provider phụ thuộc vào 1 provider request-scoped) — source của `ScheduleExplorer` chỉ log cảnh báo *"Cannot register cron job ... because it is defined in a non static provider"* rồi bỏ qua; (3) import `ScheduleModule.forRoot()` 2 lần → ngược lại, job chạy **2 lần** mỗi tick.

21.2 `@Cron` — cú pháp 6 trường, `CronExpression`, múi giờ

```
import { Injectable, Logger } from '@nestjs/common';
import { Cron, CronExpression } from '@nestjs/schedule';
import { InjectRepository } from '@nestjs/typeorm';
import { IsNull, LessThanOrEqual, Repository } from 'typeorm';
import { Post as PostEntity } from './enities/post.entity';

@Injectable()
export class PostsScheduler {
  private readonly logger = new Logger(PostsScheduler.name);

  constructor(
    @InjectRepository(PostEntity)
    private readonly postsRepository: Repository<PostEntity>,
  ) {}

  @Cron(CronExpression.EVERY_MINUTE, {
    name: 'publish-scheduled-posts',
    timeZone: 'Asia/Ho_Chi_Minh',
    waitForCompletion: true,
  })
  async publishScheduledPosts(): Promise<number> {
    const now = new Date();
    const result = await this.postsRepository.update(
      { scheduledAt: LessThanOrEqual(now), publishedAt: IsNull() },
      { publishedAt: now },
    );
    const count = result.affected ?? 0;
    if (count > 0) this.logger.log(`Published ${count} scheduled post(s)`);
    return count;
  }
}
```

Chú ý cách viết job: **1 câu `UPDATE ... WHERE` có điều kiện** thay vì "đọc danh sách → lặp → save từng bài". Câu UPDATE có điều kiện `publishedAt IS NULL` là **idempotent** — chạy 2 lần liên tiếp thì lần 2 không đổi gì. Đây là thói quen quan trọng nhất khi viết cron, vì (như mục 21.5) sẽ có lúc job bị chạy trùng mà bạn không ngờ tới.

Cú pháp cron — có thêm trường GIÂY ở đầu

Vị trí	Trường	Giá trị
1 (tuỳ chọn)	giây	0–59
2	phút	0–59
3	giờ	0–23
4	ngày trong tháng	1–31
5	tháng	1–12 (hoặc `jan` – `dec`)
6	thứ trong tuần	0–7 (0 và 7 đều là Chủ nhật, hoặc `sun` – `sat`)

Thư viện `cron` bên dưới chấp nhận **cả 5 lẫn 6 trường**: nếu bạn viết 5 trường, nó hiểu là thiếu trường giây và tự điền `giây = 0`. Ký hiệu giống cron Linux: `*` (mọi giá trị), `1-5` (khoảng), `1,3,5` (danh sách), `*/10` (bước nhảy).

Biểu thức	Số trường	Nghĩa thật
`'0 2 * * *'`	5	02:00:00 mỗi ngày
`'0 2 * * * *'`	6	**Mỗi giờ**, ở phút 02 giây 00 (không phải 2h sáng!)
`'* * * * *'`	5	Mỗi phút (giây 0)
`'* * * * * *'`	6	**Mỗi giây** — 86.400 lần/ngày
`'0 */15 9-17 * * 1-5'`	6	Mỗi 15 phút, 9h–17h, thứ 2–thứ 6

Bẫy lệch 1 trường: copy biểu thức 5 trường từ crontab Linux rồi "thêm số 0 cho chắc" ở cuối → thành 6 trường và toàn bộ ý nghĩa dịch sang trái 1 bậc (dòng 1 và 2 bảng trên). Ưu tiên dùng enum `CronExpression` cho lịch phổ biến; lịch tự viết thì luôn kiểm tra bằng `job.nextDate()` (mục 21.3) trước khi deploy.

Đã đọc source `cron@4.4.0` (`dist/time.js`, `dist/constants.js`): parser tách chuỗi theo khoảng trắng, báo *"Too few fields"* nếu < 5 trường và *"Too many fields"* nếu > 6; khi chỉ có 5 trường thì trường giây lấy giá trị mặc định `'0'`. Ràng buộc: tháng `[1, 12]`, thứ `[0, 7]`. Enum `CronExpression` của `@nestjs/schedule@12.0.2` trộn cả 2 dạng: `EVERY_MINUTE = "*/1 * * * *"` (5 trường) nhưng `EVERY_5_MINUTES = "0 */5 * * * *"` (6 trường), `EVERY_30_SECONDS = "*/30 * * * * *"` — đều đúng vì parser chấp nhận cả hai.

Các option của `@Cron`

Option	Tác dụng
`name`	Đặt tên để lấy lại job qua `SchedulerRegistry` (mục 21.3). Không đặt thì Nest sinh UUID ngẫu nhiên.
`timeZone`	Múi giờ để hiểu biểu thức, vd `'Asia/Ho_Chi_Minh'`. Múi giờ sai → throw lúc khởi động.
`utcOffset`	Thay cho `timeZone` (kiểu type không cho dùng cả 2 cùng lúc).
`waitForCompletion`	Lần chạy trước chưa xong → bỏ qua tick này (mục 21.4).
`disabled`	`true` → job được đăng ký nhưng không chạy.
`initialDelay`	Chờ N ms sau khi app khởi động mới bắt đầu lịch (hữu ích khi job cần DB/cache "ấm" trước).
`threshold`	Mặc định 250 ms: tick bị trễ quá ngưỡng (máy bận, event loop bị chặn) sẽ bị **bỏ qua** kèm cảnh báo, không chạy bù.

Bẫy múi giờ: không truyền `timeZone` thì lịch hiểu theo múi giờ của **process Node**. Máy dev của bạn là giờ Việt Nam, nhưng container Docker/VPS thường là UTC → `'0 2 * * *'` chạy lúc 2h sáng UTC = **9h sáng** giờ Việt Nam, đúng lúc đông người dùng nhất. Job nào mang ý nghĩa "giờ địa phương" thì luôn ghi `timeZone` tường minh.

21.3 `@Interval`, `@Timeout` và job động qua `SchedulerRegistry`

```
@Interval('refresh-stats', 30_000)
async refreshStats() { ... }

@Timeout(5_000)
warmUpCache() { ... }
```

Decorator	Bên dưới là	Dùng khi
`@Cron(expr)`	Job của thư viện `cron`, tính theo đồng hồ + múi giờ	"Vào lúc X" — 2h sáng, đầu mỗi giờ, thứ 2 hàng tuần
`@Interval(ms)`	`setInterval` thuần	"Cứ mỗi N giây" tính từ lúc app khởi động, không quan tâm giờ đồng hồ
`@Timeout(ms)`	`setTimeout` thuần, chạy đúng 1 lần	Việc 1 lần sau khi khởi động (warm-up cache, kiểm tra cấu hình)

`SchedulerRegistry` là provider (global) giữ tham chiếu tới mọi job/interval/timeout đã đăng ký. Inject nó để xem, dừng, bật lại, thêm hay xoá job lúc runtime:

```ts
import { SchedulerRegistry } from '@nestjs/schedule';
import { CronJob } from 'cron';

@Injectable()
export class JobsAdminService {
  constructor(private readonly registry: SchedulerRegistry) {}

  list() {
    return [...this.registry.getCronJobs()].map(([name, job]) => ({
      name,
      active: job.isActive,
      lastRun: job.lastDate(),
      nextRun: job.nextDate().toISO(),
    }));
  }

  async stop(name: string) {
    await this.registry.getCronJob(name).stop();
  }

  addReport(name: string, cronTime: string, run: () => Promise<void>) {
    const job = CronJob.from({
      cronTime,
      onTick: run,
      timeZone: 'Asia/Ho_Chi_Minh',
      waitForCompletion: true,
    });
    this.registry.addCronJob(name, job);
    job.start();
  }

  remove(name: string) {
    this.registry.deleteCronJob(name);
  }
}
```

Đã đọc `dist/scheduler.registry.d.ts` của `@nestjs/schedule@12.0.2` và `dist/job.d.ts` của `cron@4.4.0`: registry có `getCronJob`, `getCronJobs(): Map`, `addCronJob`, `deleteCronJob`, `doesExist('cron' | 'timeout' | 'interval', name)` và bộ tương ứng cho interval/timeout. `CronJob` có `CronJob.from(params)`, `start()`, `stop()` (trả `Promise | undefined`), `lastDate(): Date | null`, `nextDate()` trả **Luxon `DateTime`** (nên phải `.toISO()`, không phải `Date`), getter `isActive`. `getCronJob(name)` / `deleteCronJob(name)` **throw** nếu không tồn tại tên đó. `addCronJob` KHÔNG tự `start()` — phải gọi tay như docs. Import `CronJob` từ `'cron'` dùng được vì `cron` là dependency của `@nestjs/schedule`; nếu import trực tiếp trong code của bạn, nên `npm i cron@4.4.0` để khai báo tường minh đúng bản mà `@nestjs/schedule` dùng.

Job động sống trong RAM: job thêm bằng `addCronJob` **mất sạch khi restart/deploy**. Nếu lịch do người dùng tạo (vd "gửi báo cáo cho tôi mỗi thứ 2") thì lịch phải lưu ở DB, lúc khởi động đọc DB để đăng ký lại — hoặc tốt hơn giao cho BullMQ job scheduler (mục 21.6) vốn lưu lịch trong Redis.

21.4 Chạy chồng (overlap) và lỗi trong job

Overlap — lần chạy trước chưa xong thì tick mới đã tới

Job `async` chạy mỗi phút nhưng hôm nay DB chậm, 1 lần chạy mất 90 giây → phút tiếp theo tick mới vẫn bắt đầu, 2 lần chạy song song cùng sửa 1 tập dữ liệu. Với `@Cron`, option `waitForCompletion: true` xử lý đúng việc này: đang chạy thì tick mới bị **bỏ qua hẳn** (không xếp hàng chờ).

> **Đã đọc `CronJob.fireOnTick()` trong `cron@4.4.0`:** dòng đầu là `if (this.waitForCompletion && this._isCallbackRunning) return;`, sau đó đặt cờ `_isCallbackRunning = true`, `await` callback, và reset cờ trong `finally`. Khi `waitForCompletion` là `false` (mặc định) thì Promise của callback **không được await** — tick sau chạy chồng lên tick trước. Docs Nest mô tả option này: *"If true, no additional instances of cronjob will run until the current onTick callback has completed."*

`@Interval` **không có** option tương đương (bên dưới chỉ là `setInterval`). Muốn chống chồng thì tự viết cờ — nhớ reset trong `finally`, nếu không 1 lần lỗi sẽ khoá job vĩnh viễn:

```
private running = false;

@Interval('refresh-stats', 30_000)
async refreshStats() {
  if (this.running) return;
  this.running = true;
  try {
    await this.doRefresh();
  } finally {
    this.running = false;
  }
}
```

> **Cả `waitForCompletion` lẫn cờ tự viết chỉ có hiệu lực TRONG 1 PROCESS.** Cờ nằm trong RAM của instance này; instance thứ hai có cờ riêng của nó. Chống chồng giữa nhiều instance là bài toán khác — mục 21.5.

Job ném lỗi thì sao?

Nest tự bọc mọi method `@Cron` / `@Interval` / `@Timeout` trong `try/catch`: lỗi được **log ra** (context `Scheduler`), app **không sập**, và tick sau vẫn chạy bình thường. Nhưng cũng chỉ có vậy: **không retry**, không lưu lại lỗi ở đâu, không ai được báo. Job cần "thất bại thì thử lại 3 lần, vẫn lỗi thì lưu để xem" → đó là việc của queue (BullMQ, Part 18), không phải của `@nestjs/schedule`.

> **Đã đọc `dist/schedule.explorer.js` của `@nestjs/schedule@12.0.2`:** hàm `wrapFunctionInTryCatchBlocks` bọc handler thành `async (...args) => { try { await methodRef.call(instance, ...args); } catch (error) { this.logger.error(error); } }` cho cả 3 loại decorator. Job thêm động qua `SchedulerRegistry.addCronJob()` cũng được bọc tương tự (registry bọc lại `job.fireOnTick`). Docs xác nhận: *"Every method annotated with `@Cron()` is automatically wrapped in a `try-catch` block, so if an exception occurs, it's logged to the console."*

Lỗ hổng của lưới try/catch: nó chỉ bắt được lỗi của Promise mà handler **trả về / await**. Nếu trong handler bạn "bắn rồi quên" (`this.mailService.send(...)` không `await` , hoặc `items.forEach(async item => ...)`), Promise con đó reject bên ngoài lưới → thành *unhandled rejection*; từ Node 15 hành vi mặc định là **crash cả process** — kéo sập luôn HTTP server. Trong job luôn `await` , và lặp async thì dùng `for...of` + `await` (hoặc `await Promise.all(...)`), không dùng `forEach(async ...)` .

21.5 Bẫy lớn nhất: chạy nhiều instance = job chạy nhiều lần

Lịch nằm trong RAM của từng process (mục 21.1). Khi bạn scale lên 3 replica (Docker/Kubernetes, PM2 cluster mode, 2 server sau load balancer), **cả 3 process đều có đủ bộ timer** — 2h sáng, cả 3 cùng chạy job "gửi email tổng kết ngày" → mỗi người dùng nhận 3 email. Laravel có `->onOneServer()` sẵn; `@nestjs/schedule` thì **không có gì tương đương** trong chính nó.



Cách xử lý	Cách làm	Ưu / nhược
1. Chỉ 1 instance bật cron	Bật/tắt cron bằng biến môi trường `RUN_CRON` qua `forRootAsync` (code ngay dưới bảng) — chỉ 1 deployment (hoặc 1 container "worker" riêng) đặt `RUN_CRON=true`	Đơn giản nhất, không cần hạ tầng mới. Nhược: instance đó chết thì không ai chạy cron cho tới khi nó sống lại.
2. Job idempotent	Viết job sao cho chạy 2–3 lần cũng ra cùng kết quả (UPDATE có điều kiện như 21.2)	Luôn nên làm, nhưng không cứu được việc có "tác dụng phụ ra ngoài" (gửi mail, gọi API thanh toán).
3. Distributed lock	Trước khi chạy, instance phải giành 1 khoá chung (Redis `SET key NX PX ttl` , hoặc 1 dòng trong DB); ai không giành được thì bỏ qua tick. Nest có package chính chủ mới `@nestjs/locks` với `@OnOneInstance()` / `@WithoutOverlapping()` đặt cạnh `@Cron` .	Mọi replica đều chạy được cron, chết 1 cái vẫn còn cái khác. Nhược: cần Redis/DB làm nơi giữ khoá, và phải hiểu TTL của khoá.
4. Chuyển sang BullMQ job scheduler	Lịch lưu trong Redis, Redis sinh đúng 1 job mỗi lần tới giờ, worker nào rảnh thì nhận (mục 21.6)	Có retry, lịch sử, không trùng. Nhược: cần Redis + worker (Part 18).

```
ScheduleModule.forRootAsync({
  inject: [ConfigService],
  useFactory: (config: ConfigService) => ({
    cronJobs: config.get('RUN_CRON') === 'true',
    intervals: config.get('RUN_CRON') === 'true',
  }),
}),
```

`ConfigModule` của project đã `isGlobal: true` nên không cần khai `imports`. Giá trị trong `.env` luôn là **chuỗi** — so sánh với `'true'`, đừng viết `!!config.get('RUN_CRON')` (chuỗi `'false'` cũng là truthy).

> **Về `@nestjs/locks` (đã tải `@nestjs/locks@0.0.1` và đọc docs.nestjs.com/reliability/locks):** đây là bản **0.0.1** — mới phát hành, peer `@nestjs/common ^11 || ^12`. Package **không kèm sẵn store nào ngoài bộ nhớ**; docs viết nguyên văn: *"The package doesn't ship one. It defines the contract, the `LockStore` interface, and you implement it in a provider of your application with the database it already has."* Store trong RAM chỉ khoá được trong 1 process (vô nghĩa với bài toán nhiều instance), và theo `locks-module-options.interface.d.ts`: khi `NODE_ENV=production` mà chưa đăng ký `LockStore` thì app **từ chối khởi động** trừ khi đặt `allowInMemoryStorage: true`. Comment của interface `LockStore` mô tả đúng cơ chế cần tự cài: `acquire()` phải nguyên tử (*"`SET NX PX` in a Lua script, `INSERT ... ON CONFLICT DO UPDATE ... WHERE expired`"*). Với người mới: biết là có công cụ này, còn trước mắt dùng cách 1 + cách 2 là đủ.

> **Bẫy "dev chạy được, production nhân đôi":** máy dev chỉ có 1 process nên không bao giờ thấy lỗi này. PM2 `-i max` hay `replicas: 3` được thêm vào lúc deploy, nhiều tuần sau khi viết job — và lỗi chỉ lộ ra khi khách hàng phàn nàn nhận mail trùng. Mỗi khi viết 1 `@Cron`, tự hỏi ngay: "nếu 3 bản app cùng chạy dòng này thì sao?".

21.6 `@nestjs/schedule` hay BullMQ job scheduler — chọn chỗ đặt lịch

Sau Part 18 bạn đã có BullMQ, và BullMQ cũng biết chạy job theo lịch cron. Hai công cụ trùng một phần chức năng nên rất dễ phân vân. Khác biệt cốt lõi: `@nestjs/schedule` giữ lịch **trong RAM của từng process**; BullMQ giữ lịch **trong Redis, dùng chung cho mọi process**. Câu hỏi quyết định: **nếu job này chạy trùng, hoặc thất bại mà không ai thử lại, thì có sao không?**

Cách 1 — `@Cron` trong process (in-process)

```
@Cron(CronExpression.EVERY_DAY_AT_3AM, {
  name: 'prune-read-notifications',
  timeZone: 'Asia/Ho_Chi_Minh',
  waitForCompletion: true,
})
async pruneReadNotifications() {
  const cutoff = new Date(Date.now() - 30 * 24 * 3600 * 1000);
  await this.notificationsRepository.delete({ readAt: LessThan(cutoff) });
}
```

Được: không cần hạ tầng gì thêm, code ngắn, đọc là hiểu. Mất: chạy N lần khi có N instance, lỗi chỉ được log, không retry, không có lịch sử chạy.

Cách 2 — BullMQ job scheduler (`upsertJobScheduler`)

```
import { InjectQueue, Processor, WorkerHost } from '@nestjs/bullmq';
import { Job, Queue } from 'bullmq';

@Injectable()
export class MaintenanceSchedules implements OnApplicationBootstrap {
  constructor(@InjectQueue('maintenance') private readonly queue: Queue) {}

  async onApplicationBootstrap() {
    await this.queue.upsertJobScheduler(
      'daily-digest',
      { pattern: '0 8 * * *', tz: 'Asia/Ho_Chi_Minh' },
      { name: 'send-daily-digest', opts: { attempts: 3, backoff: { type: 'exponential',
delay: 60_000 } } },
    );
  }
}

@Processor('maintenance')
export class MaintenanceProcessor extends WorkerHost {
  async process(job: Job): Promise<void> {
    if (job.name === 'send-daily-digest') {
      ...
    }
  }
}
```

Được: dù có 5 replica cùng gọi `upsertJobScheduler('daily-digest', ...)` lúc khởi động, Redis vẫn chỉ có **1** scheduler tên đó → mỗi lần tới giờ sinh đúng 1 job, 1 worker nhận; job lỗi được retry theo `attempts` / `backoff` ; job lỗi hẳn nằm lại trong danh sách failed để xem. Mất: phải có Redis + worker đang chạy, thêm 1 lớp để hiểu và giám sát.

> **Đã đọc type của `bullmq@6.3.10` và `@nestjs/bullmq@12.0.0` (peer `@nestjs/common ^10 || ^11 || ^12` , `bullmq ^3 ... ^6`):** API hiện hành là `Queue.upsertJobScheduler(jobSchedulerId, repeatOpts, jobTemplate?: { name?, data?, opts? })` ; `repeatOpts` nhận `pattern` (cron) hoặc `every` (ms, không dùng chung với `pattern`), `tz` , `limit` , `immediately` . Đi kèm `removeJobScheduler(id)` , `getJobSchedulers()` . Comment trong `types/job-options.d.ts` ghi rõ: *" `repeat` is no longer a valid option for `Queue.add` ; use `Queue.upsertJobScheduler` to schedule repeating jobs."* — docs.nestjs.com/techniques/queues cũng viết: BullMQ *"no longer accepts a `repeat` option in `Queue.add()` "*. Tutorial cũ trên mạng dùng `queue.add(name, data, { repeat: { cron: ... } })` là API đã bị bỏ. Docs BullMQ (docs.bullmq.io/guide/job-schedulers) giải thích vì sao dùng "upsert": *"It ensures the scheduler is updated or created without duplications."*

Nhu cầu	Cách 1: `@Cron`	Cách 2: BullMQ scheduler
App chỉ chạy 1 instance, job dọn dẹp idempotent	✅ Đủ dùng, không thêm hạ tầng	Thừa
Nhiều instance, job có tác dụng phụ (gửi mail, gọi API ngoài, trừ tiền)	❌ Chạy N lần (trừ khi tự thêm lock / `RUN_CRON`)	✅ Đúng 1 job mỗi lần tới giờ

Job lỗi phải tự thử lại, cần xem lịch sử lỗi	❌ Chỉ log	✅ `attempts`, `backoff`, danh sách failed
Việc chạy rất dày (mỗi vài giây) và nhẹ, chỉ tác động bộ nhớ của chính process (làm mới cache cục bộ)	✅ Đúng chỗ — mỗi process PHẢI tự làm cho mình	❌ Sai — job chỉ tới 1 worker, các process khác không được làm mới
Lịch do người dùng tạo lúc runtime, phải sống qua restart	❌ Registry nằm trong RAM	✅ Lịch lưu trong Redis
Job nặng (xuất báo cáo lớn) không được làm chậm API	❌ Chạy chung event loop với HTTP server	✅ Worker có thể tách thành process riêng

Quy tắc thực dụng: job nhỏ, idempotent, không đụng tới thế giới bên ngoài → `@Cron`. Job mà "chạy 2 lần" hoặc "lỗi không ai biết" là có hậu quả thật (email, tiền, API bên thứ ba) → để BullMQ scheduler tạo job, còn `@Cron` không dùng cho việc đó. Một kiểu kết hợp hay gặp: `@Cron` chỉ làm đúng 1 việc nhỏ là `queue.add(...)` với `jobId` cố định theo ngày — nhưng khi đã có BullMQ thì `upsertJobScheduler` gọn hơn.

21.7 Test job định kỳ — gọi thẳng method, đừng chờ đồng hồ

`@Cron` chỉ là metadata gắn lên method; bản thân method vẫn là 1 hàm bình thường. Unit test **không import** `ScheduleModule` → không có timer nào được tạo, bạn gọi thẳng method và kiểm tra kết quả, không cần chờ tới 2h sáng:

```
describe('PostsScheduler', () => {
  const repo = { update: jest.fn() };
  let scheduler: PostsScheduler;

  beforeEach(async () => {
    jest.clearAllMocks();
    const moduleRef = await Test.createTestingModule({
      providers: [PostsScheduler, { provide: getRepositoryToken(PostEntity), useValue:
repo }],
    }).compile();
    scheduler = moduleRef.get(PostsScheduler);
  });

  it('chỉ publish bài đã tới giờ và chưa publish', async () => {
    repo.update.mockResolvedValue({ affected: 2 });
    await expect(scheduler.publishScheduledPosts()).resolves.toBe(2);
    const [where] = repo.update.mock.calls[0];
    expect(where).toHaveProperty('publishedAt');
    expect(where).toHaveProperty('scheduledAt');
  });
});
```

Muốn "đóng băng" thời gian để kiểm tra đúng mốc `now` được truyền vào câu UPDATE: dùng fake timers của Jest (`jest.useFakeTimers().setSystemTime(new Date('2026-01-01T02:00:00Z'))`, nhớ `jest.useRealTimers()` ở `afterEach`).

Bẫy e2e test: test e2e import cả `AppModule` → `ScheduleModule` bật timer thật trong lúc test chạy (job có thể ghi vào DB giữa chừng, Jest báo "open handles"). Tắt cron khi test bằng đúng cơ chế ở mục 21.5: `cronJobs: c.get('RUN_CRON') === 'true'` và không đặt biến đó trong môi trường test. `ScheduleModuleOptions` có 3 cờ `cronJobs`, `intervals`, `timeouts` (mặc định đều `true` — đã đọc `schedule.module.js`); khi cờ là `false`, explorer bỏ qua không đăng ký loại job đó.

Việc cần làm:

1. `npm i @nestjs/schedule`, thêm `ScheduleModule.forRootAsync(...)` **đúng 1 lần** trong `AppModule`, bật/tắt cron bằng biến `RUN_CRON` trong `.env` (mục 21.5)
2. Thêm 2 cột nullable `scheduledAt` và `publishedAt` (`type: 'datetime'`) vào entity `Post`; cho `CreatePostDto` nhận `scheduledAt` tuỳ chọn (`@IsOptional() @IsDateString()`)
3. Viết `PostsScheduler` như mục 21.2 (đăng ký vào `providers` của `PostsModule`), chạy `npm run start:dev`, tạo post có `scheduledAt` 2 phút tới và quan sát log khi nó được publish; `GET /posts` chỉ trả bài đã có `publishedAt`
4. Viết `JobsAdminController` (guard JWT) với `GET /admin/jobs` (tên, `nextRun`, `lastRun`) và `POST /admin/jobs/:name/stop` dùng `SchedulerRegistry`; thử stop rồi xác nhận job không chạy nữa
5. Cố ý `throw new Error('boom')` trong job → xác nhận log lỗi có context `Scheduler`, app vẫn sống và phút sau job chạy lại. Sau đó thử 1 Promise KHÔNG await ném lỗi trong job để thấy khác biệt (rồi sửa lại)
6. Chạy 2 bản app cùng lúc (`PORT=3000` và `PORT=3001`, cùng `RUN_CRON=true`, cùng file SQLite) → thấy job chạy 2 lần; giải thích vì sao câu UPDATE có điều kiện vẫn cho kết quả đúng; sau đó tắt `RUN_CRON` ở 1 bản
7. Viết unit test cho `PostsScheduler` như mục 21.7 (mock repository, gọi thẳng method)

PART 22

File Upload & Storage

Mục tiêu: Cho phép upload ảnh sản phẩm an toàn: nhận file qua Multer, kiểm tra kích thước + loại file THẬT (magic number), tự đặt tên file, lưu qua 1 lớp trừu tượng "storage" để sau này đổi sang S3 không phải sửa service, phục vụ file ra ngoài, và biết khi nào nên cho client upload thẳng lên S3.

22.1 Upload đi qua Nest như thế nào

File upload không nằm trong body JSON — trình duyệt/Postman gửi request dạng `multipart/form-data`: body chia thành nhiều "part", mỗi part là 1 field text hoặc 1 file. `ValidationPipe` và `express.json()` không đọc được định dạng này. Người đọc nó là **Multer** — middleware Express kinh điển, được `@nestjs/platform-express` đóng gói sẵn thành các **Interceptor**.

```
POST /products/7/image   (Content-Type: multipart/form-data; boundary=...)
    |
    ▼  Guard (JwtAuthGuard)          ← chặn TRƯỚC khi đọc byte nào của file
    ▼  FileInterceptor('image')      ← gọi multer.single('image'): đọc stream, gom file
    |     └─ gán req.file = { fieldname, originalname, mimetype, size, buffer | path }
    ▼  ParseFilePipe                 ← validate size / loại file (22.3)
    ▼  handler(@UploadedFile() file: Express.Multer.File)
```

Thứ tự trên là request lifecycle của Part 00 áp vào upload: Guard chạy trước Interceptor, nên đặt `JwtAuthGuard` thì request không có token bị trả 401 trước khi Multer kịp đọc file vào bộ nhớ. `@UploadedFile()` chỉ là param decorator lấy `req.file` mà Multer đã gắn.

```
npm i -D @types/multer
```

Interceptor	Nhận gì	Lấy ra bằng
`FileInterceptor('image', options?)`	1 file ở field `image`	`@UploadedFile()` → `Express.Multer.File`
`FilesInterceptor('photos', maxCount?, options?)`	Nhiều file cùng 1 field	`@UploadedFiles()` → `Express.Multer.File[]`
`FileFieldsInterceptor([{ name: 'avatar', maxCount: 1 }, { name: 'gallery', maxCount: 5 }], options?)`	Nhiều field, mỗi field giới hạn số file riêng	`@UploadedFiles()` → `{ avatar?: File[], gallery?: File[] }`
`AnyFilesInterceptor()`	File ở field bất kỳ	`@UploadedFiles()` — tránh dùng, không kiểm soát được client gửi gì
`NoFilesInterceptor()`	Form multipart chỉ có field text	`@Body()`; gửi kèm file → 400

Đã kiểm tra trong `node_modules` của project: `@nestjs/platform-express@11.2.3` khai báo dependency cứng `multer@2.2.0` — **không cần cài multer riêng**. Chỉ thiếu type: `@types/multer` chưa có trong project (bản mới nhất trên npm: `2.3.0`, khai báo namespace global `Express.Multer.File`), docs.nestjs.com/techniques/file-upload hướng dẫn đúng lệnh `npm i -D @types/multer`. Đọc `multer/multer.utils.js`: lỗi của Multer được dịch sang HttpException — `LIMIT_FILE_SIZE` → `PayloadTooLargeException` (413), `LIMIT_UNEXPECTED_FILE` / `LIMIT_FILE_COUNT` ... → `BadRequestException` (400) — nên `HttpExceptionFilter` của Part 06 format được chúng như mọi lỗi khác.

Bẫy tên field: `FileInterceptor('image')` mà client gửi file ở field `file` → 400 *"Unexpected field - file"*. Tên trong interceptor phải khớp đúng key của form-data. Ngoài ra, các field text đi kèm trong multipart đều đến dưới dạng **chuỗi** — DTO nhận chúng cần `@Type(() => Number)` như Part 12.

22.2 `memoryStorage` hay `diskStorage` — file nằm ở đâu trong lúc xử lý

Multer phải để file ở đâu đó trước khi handler chạy. Có 2 "storage engine":

	`memoryStorage` (mặc định)	`diskStorage`

Kích hoạt khi	Không truyền `storage` lẫn `dest`	Truyền `dest: './tmp'` hoặc `storage: diskStorage({...})`
File nằm ở	`file.buffer` (toàn bộ file trong RAM)	`file.path` trên đĩa, **không có** `file.buffer`
Tên file	Không có (chưa ghi đi đâu)	Mặc định: 16 byte ngẫu nhiên dạng hex, **không có đuôi**
Hợp với	File nhỏ (ảnh ≤ vài MB), cần đọc nội dung ngay (kiểm tra magic number, resize, đẩy lên S3)	File lớn (video, file zip) — không chiếm RAM
Nguy hiểm	50 người cùng upload file 200MB = 10GB RAM → process bị OOM kill	Rác trên đĩa nếu validate fail mà quên xoá; `FileTypeValidator` không đọc được (22.3)

Với ảnh sản phẩm, `memoryStorage` là lựa chọn đúng — **với điều kiện** luôn đặt `limits.fileSize`. Đây là "cầu dao" thật sự: Multer/busboy đếm byte khi đang đọc stream và **dừng ngay** khi vượt ngưỡng (413), thay vì đọc hết 2GB vào RAM rồi mới để validator phía sau từ chối.

```
const MAX_IMAGE_BYTES = 2 * 1024 * 1024;

@UseInterceptors(
  FileInterceptor('image', {
    limits: { fileSize: MAX_IMAGE_BYTES, files: 1, fields: 5 },
  }),
)
```

> **Đã đọc `multer@2.2.0/index.js`:** `if (options.storage) ... else if (options.dest) this.storage = diskStorage(...) else this.storage = memoryStorage()` — mặc định đúng là bộ nhớ, khớp docs Nest (*"Default behavior stores files in memory"*). `storage/disk.js` sinh tên mặc định bằng `crypto.randomBytes(16)` và ghép đường dẫn bằng `path.join(destination, filename)`. Interface `MulterOptions` của `@nestjs/platform-express` ghi chú `limits.fileSize` mặc định *"Infinity"* — tức là không đặt thì không có giới hạn nào. Có thể đặt option chung cho cả app bằng `MulterModule.register({...})` / `registerAsync()`, option truyền vào từng interceptor sẽ ghi đè (source: `multer({ ...options, ...localOptions })`).

> **Bẫy tên file tiếng Việt:** option `defParamCharset` của Multer mặc định là `'latin1'` — file tên `ảnh-sản-phẩm.png` sẽ có `originalname` bị vỡ dấu. Nếu cần lưu/hiển thị tên gốc cho người dùng, đặt `defParamCharset: 'utf8'`. (Tên gốc chỉ để **hiển thị** — không bao giờ dùng làm tên file lưu trữ, xem 22.4.)

22.3 Validate file: `ParseFilePipe`, `MaxFileSizeValidator`, `FileTypeValidator`

`ParseFilePipe` là pipe (Part 06) chuyên cho file: nhận danh sách validator, chạy **lần lượt theo thứ tự**, validator đầu tiên fail thì throw ngay. `ParseFilePipeBuilder` là cách viết gọn hơn cho cùng việc đó:

```
@Post(':id/image')
@UseGuards(JwtAuthGuard)
@UseInterceptors(FileInterceptor('image', { limits: { fileSize: MAX_IMAGE_BYTES, files:
1 } }))
uploadImage(
  @Param('id', ParseIntPipe) id: number,
  @UploadedFile(
    new ParseFilePipeBuilder()
      .addMaxSizeValidator({ maxSize: MAX_IMAGE_BYTES })
      .addFileTypeValidator({ fileType: /^image\/(png|jpeg|webp)$/, overrideMimeType:
true })
      .build({ errorHttpStatusCode: HttpStatus.UNPROCESSABLE_ENTITY }),
  )
  file: Express.Multer.File,
) {
  return this.productsService.attachImage(id, file);
}
```

Viết theo kiểu không dùng builder (tương đương):

```
new ParseFilePipe({
  validators: [
    new MaxFileSizeValidator({ maxSize: MAX_IMAGE_BYTES }),
    new FileTypeValidator({ fileType: /^image\/(png|jpeg|webp)$/ }),
  ],
  fileIsRequired: true,
})
```

`FileTypeValidator` thật sự kiểm tra cái gì?

`file.mimetype` là chuỗi do **client tự khai** trong header của part — kẻ tấn công đổi tên `shell.php` thành `cat.png` và khai `image/png` là xong. Vì vậy bản `@nestjs/common` đang cài **không tin chuỗi đó**: nó đọc vài byte đầu của `file.buffer` ("magic number" — PNG luôn bắt đầu bằng `89 50 4E 47`, JPEG bằng `FF D8 FF`) bằng thư viện `file-type`, rồi so kiểu phát hiện được với `fileType` của bạn.

Option	Tác dụng (bản 11.2.3)
`fileType`	Chuỗi hoặc RegExp để so với kiểu MIME **phát hiện từ nội dung**
`overrideMimeType`	`true` → ghi đè `file.mimetype` bằng kiểu phát hiện được; sau pipe, `file.mimetype` đáng tin để chọn đuôi file
`fallbackToMimetype`	Không nhận diện được magic number → quay về tin chuỗi client gửi. **Mở lại đúng lỗ hổng** — chỉ bật cho loại file không có chữ ký (CSV, text)
`skipMagicNumbersValidation`	Bỏ hẳn kiểm tra nội dung, chỉ so chuỗi `mimetype` (hành vi cũ)

Đã đọc source `@nestjs/common/pipes/file/file-type.validator.js` bản 11.2.3 đang cài, và chạy thử trực tiếp: `isValid()` gọi `fileTypeFromBuffer(file.buffer)` (package `file-type@21.3.4`, dependency của `@nestjs/common`) và so kiểu phát hiện được. Kết quả chạy thử: PNG thật khai `image/png` → `true`; chuỗi `<?php ... ?>` khai `image/png` → `false`; file **không có `buffer`** (tức là dùng `diskStorage`) → `false` với thông báo *"file buffer is not available; file type validation could not be performed"*; cùng file PHP đó nhưng bật `fallbackToMimetype: true` → `true` (lọt!). Lịch sử phiên bản (đã tải từng bản để so): `11.0.0` – `11.0.15` chỉ so chuỗi `file.mimetype`; magic number có từ `11.0.16`; `fallbackToMimetype` có từ `11.1.0`. Docs hiện tại: *"By default, it determines the MIME type from the file content's magic number."* — tutorial/StackOverflow cũ nói "FileTypeValidator chỉ check mimetype" là đúng với bản cũ, sai với bản bạn đang cài.

3 bẫy của validator: (1) **`diskStorage` + `FileTypeValidator` luôn fail** — không có `buffer` để đọc magic number. Dùng `memoryStorage` cho file cần kiểm tra loại, hoặc tự đọc vài byte đầu từ `file.path`. (2) `fileType` dạng **chuỗi** bị `String#match` biến thành RegExp: `'png'` khớp cả `'image/png'` lẫn `'image/apng'`; `'image/svg+xml'` chứa ký tự `+` của regex. Luôn dùng RegExp có neo `^...$` như ví dụ. (3) `MaxFileSizeValidator` so **nhỏ hơn hẳn** (`file.size < maxSize`) và chỉ chạy **sau khi** Multer đã đọc xong file — nó là lớp báo lỗi đẹp, không phải lớp bảo vệ RAM. Lớp bảo vệ RAM là `limits.fileSize` (22.2). Đặt cả hai.

Jest và `FileTypeValidator`: `file-type` là package ESM, `@nestjs/common` nạp nó bằng dynamic import. Source có sẵn cảnh báo: nếu nạp thất bại trong Jest thì *"run it with NODE_OPTIONS=\"--experimental-vm-modules\""* — và validator trả `false` cho MỌI file (test upload hợp lệ cũng fail khó hiểu). Script `test` / `test:e2e` trong `package.json` của project đã có sẵn flag này — đừng xoá nó.

22.4 Đặt tên và lưu file an toàn — một lớp "storage" để sau đổi sang S3

Không bao giờ dùng `originalname` làm tên file lưu trữ

- **Trùng tên:** 2 người cùng upload `image.png` → người sau ghi đè ảnh người trước.
- **Đuôi nguy hiểm:** tên `x.html` hoặc `x.svg` chứa JavaScript, phục vụ lại qua static server cùng domain → **stored XSS**, chạy script trong phiên đăng nhập của người xem.
- **Path traversal:** tên kiểu `../../src/main.ts`. Bên trong Multer, busboy cắt phần thư mục khỏi tên file khi `preservePath` là `false` (mặc định) — nhưng lớp bảo vệ đó mất ngay khi ai đó bật `preservePath: true`, và không tồn tại ở những chỗ tên/đường dẫn đến từ nơi khác (key client gửi lên ở luồng S3 22.6, endpoint download nhận tên file qua URL).

Quy tắc: **server tự sinh tên** (UUID), **đuôi lấy từ kiểu MIME đã được xác minh** (không lấy từ tên gốc), và DB chỉ lưu **key tương đối** (`products/3f2a....webp`) — không lưu URL tuyệt đối, không lưu đường dẫn máy chủ.

```ts
export abstract class FileStorage {
  abstract put(key: string, data: Buffer, contentType: string): Promise<void>;
  abstract delete(key: string): Promise<void>;
  abstract url(key: string): string;
}

@Injectable()
export class LocalFileStorage extends FileStorage {
  private readonly root = resolve(process.cwd(), 'uploads');

  async put(key: string, data: Buffer): Promise<void> {
    const fullPath = this.resolveSafe(key);
    await mkdir(dirname(fullPath), { recursive: true });
    await writeFile(fullPath, data, { flag: 'wx' });
  }

  async delete(key: string): Promise<void> {
    await rm(this.resolveSafe(key), { force: true });
  }

  url(key: string): string {
    return `/uploads/${key}`;
  }

  private resolveSafe(key: string): string {
    const fullPath = resolve(this.root, key);
    if (!fullPath.startsWith(this.root + sep)) {
      throw new BadRequestException('Invalid file key');
    }
    return fullPath;
  }
}
```

```ts
@Module({
  providers: [{ provide: FileStorage, useClass: LocalFileStorage }],
  exports: [FileStorage],
})
export class StorageModule {}
```

Đây là DI của Part 03 dùng đúng chỗ: service chỉ phụ thuộc vào `FileStorage` (abstract class — vẫn là 1 giá trị runtime nên dùng làm token được, khác với `interface` bị xoá khi compile). Ngày chuyển sang S3 chỉ đổi `useClass: S3FileStorage`, `ProductsService` không sửa dòng nào — giống vai trò "disk" của Laravel (Part 18 bên đó). `resolveSafe` là chốt chặn path traversal cuối cùng: `resolve()` chuẩn hoá mọi `..`, rồi kiểm tra đường dẫn kết quả vẫn nằm trong thư mục gốc.

```ts
const IMAGE_EXT: Record<string, string> = {
  'image/png': 'png',
  'image/jpeg': 'jpg',
  'image/webp': 'webp',
};

async attachImage(id: number, file: Express.Multer.File) {
  const product = await this.findOne(id);
  const key = `products/${randomUUID()}.${IMAGE_EXT[file.mimetype]}`;
  await this.storage.put(key, file.buffer, file.mimetype);

  const oldKey = product.imageKey;
  product.imageKey = key;
  try {
    await this.productsRepository.save(product);
  } catch (err) {
    await this.storage.delete(key);
    throw err;
  }
  if (oldKey) await this.storage.delete(oldKey);
  return { ...product, imageUrl: this.storage.url(key) };
}
```

File và DB không cùng 1 transaction: `DataSource.transaction()` của Part 13 không rollback được file đã ghi. Thứ tự an toàn: ghi file → lưu DB → (DB lỗi thì xoá file vừa ghi) → chỉ xoá file CŨ sau khi DB đã lưu key mới. Làm ngược (xoá file cũ trước) mà DB lỗi thì sản phẩm trỏ vào 1 file không còn tồn tại. File "mồ côi" vẫn có thể sót lại khi process chết giữa chừng — dọn định kỳ bằng 1 job ở Part 21.

Đối chiếu: `randomUUID` là API của `node:crypto` (có trong `@types/node` của project); `resolve`, `dirname`, `sep` từ `node:path`; `mkdir`, `writeFile`, `rm` từ `node:fs/promises` — flag `'wx'` làm `writeFile` thất bại nếu file đã tồn tại (không bao giờ ghi đè âm thầm). `file.mimetype` đáng tin để tra `IMAGE_EXT` **chỉ vì** validator ở 22.3 đã bật `overrideMimeType: true` và regex chỉ cho qua đúng 3 kiểu có trong bảng.

22.5 Phục vụ file ra ngoài: static (công khai) hay `StreamableFile` (có kiểm tra quyền)

File công khai — `useStaticAssets`

```ts
import { NestExpressApplication } from '@nestjs/platform-express';

async function bootstrap() {
  const app = await NestFactory.create<NestExpressApplication>(AppModule);
  app.useStaticAssets(join(process.cwd(), 'uploads'), { prefix: '/uploads/', dotfiles:
'deny' });
  ...
}
```

Express phục vụ thẳng file tĩnh, không đi qua controller/guard/interceptor. Hợp với ảnh sản phẩm (ai cũng xem được). Thư mục `uploads/` phải nằm **ngoài** `src/` và `dist/`, và thêm vào `.gitignore`.

> **Đã đọc `nest-express-application.interface.d.ts` (`@nestjs/platform-express@11.2.3`):** có overload `useStaticAssets(path: string, options?: ServeStaticOptions)`, `ServeStaticOptions` có `prefix` và `dotfiles`; phải truyền generic `NestExpressApplication` cho `NestFactory.create` thì mới có method này. Cách khác là package `@nestjs/serve-static` — nhưng **chú ý phiên bản**: bản `latest` hiện là `12.0.0` với peer `@nestjs/common ^12.0.0`, không cài được sạch vào project Nest 11; phải ghim `npm i @nestjs/serve-static@5` (bản `5.0.5`, peer `^11.0.2`). Với nhu cầu của project này, `useStaticAssets` đủ dùng và không thêm package.

File riêng tư — stream qua controller

```
type AuthRequest = Request & { user: { userId: number; email: string } };

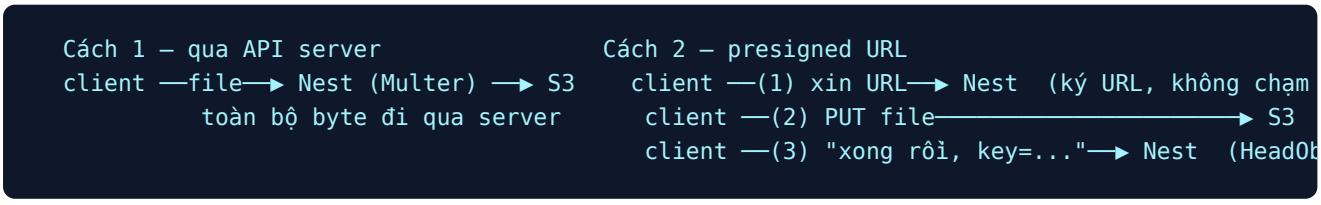
@Get(':id/invoice')
@UseGuards(JwtAuthGuard)
async downloadInvoice(@Param('id', ParseIntPipe) id: number, @Req() req: AuthRequest) {
  const invoice = await this.invoicesService.findOwned(id, req.user.userId);
  return new StreamableFile(createReadStream(invoice.filePath), {
    type: 'application/pdf',
    disposition: `attachment; filename="invoice-${invoice.id}.pdf"`,
  });
}
```

`StreamableFile` bọc 1 `Readable` stream (hoặc `Buffer`); Nest thấy handler trả về kiểu này thì `pipe` stream thẳng vào response thay vì `JSON.stringify` — file 500MB cũng không phải nạp hết vào RAM. Tên file trong `disposition` cũng do server đặt, không lấy nguyên văn từ input.

> **Bẫy với `TransformInterceptor` của Part 07:** interceptor global đó bọc MỌI kết quả thành `{ success: true, data: payload }` — kể cả `StreamableFile`. Adapter Express chỉ stream khi `body instanceof StreamableFile`; bị bọc vào object rồi thì nó rơi xuống nhánh `response.json(body)` → client nhận 1 cục JSON vô nghĩa thay vì file PDF. Sửa interceptor: `if (payload instanceof StreamableFile) return payload;` trước khi bọc. (Đã đọc `ExpressAdapter.reply()` trong `@nestjs/platform-express@11.2.3`: đúng thứ tự kiểm tra `isNil` → `instanceof StreamableFile` → `isObject ? response.json : response.send`.)

22.6 Upload qua API server (Multer) hay upload thẳng lên S3 bằng presigned URL

Khi app lên production nhiều instance, đĩa cục bộ của từng container không còn dùng được (ảnh upload vào container A, request xem ảnh rơi vào container B) — file phải nằm ở object storage dùng chung (Amazon S3, hoặc S3-compatible: Cloudflare R2, MinIO, DigitalOcean Spaces). Lúc đó có 2 kiến trúc upload, và câu hỏi quyết định là: **server có cần đọc/kiểm tra nội dung file ngay trong request không, hay chỉ cần biết "file đã nằm trên S3"?**



Cách 1 — Multer nhận file rồi server đẩy lên S3

Giữ nguyên mọi thứ từ 22.1–22.4, chỉ thay `LocalFileStorage` bằng `S3FileStorage` (dùng `PutObjectCommand` với `Body: data`). Được: validate magic number, resize ảnh, quét virus ngay trong request; client đơn giản (1 request multipart). Mất: mọi byte đi qua server 2 lần (vào và ra), chiếm băng thông + RAM của API, file lớn làm request dài và dễ timeout.

Cách 2 — Server chỉ ký URL, client PUT thẳng lên S3

```
npm i @aws-sdk/client-s3 @aws-sdk/s3-request-presigner
```

```
import { DeleteObjectCommand, HeadObjectCommand, PutObjectCommand, S3Client } from
'@aws-sdk/client-s3';
import { getSignedUrl } from '@aws-sdk/s3-request-presigner';

async createUploadUrl(productId: number, contentType: string) {
  const key = `products/${productId}/${randomUUID()}.${IMAGE_EXT[contentType]}`;
  const command = new PutObjectCommand({ Bucket: this.bucket, Key: key, ContentType:
contentType });
  const uploadUrl = await getSignedUrl(this.s3, command, { expiresIn: 300 });
  return { uploadUrl, key };
}

async confirmUpload(productId: number, key: string) {
  if (!key.startsWith(`products/${productId}/`)) throw new BadRequestException('Invalid
key');
  const head = await this.s3.send(new HeadObjectCommand({ Bucket: this.bucket, Key: key
}));
  if ((head.ContentLength ?? 0) > MAX_IMAGE_BYTES) {
    await this.s3.send(new DeleteObjectCommand({ Bucket: this.bucket, Key: key }));
    throw new BadRequestException('File too large');
  }
  ...
}
```

`contentType` đến từ body của request xin URL — DTO của endpoint đó phải chặn bằng `@IsIn(['image/png', 'image/jpeg', 'image/webp'])`, nếu không client xin được URL cho `text/html`. Client nhận `uploadUrl`, gửi `PUT` với body là file và header `Content-Type` **đúng bằng** giá trị đã ký, rồi gọi endpoint confirm. Được: file không đi qua API server, upload file lớn không tốn RAM/băng thông của app, scale tốt. Mất: server không thấy nội dung lúc upload (không kiểm tra magic number đồng bộ được — muốn kiểm tra phải tải về trong 1 job nền, Part 18), luồng 3 bước phức tạp hơn ở client, và phải tự lo "key đã ký nhưng client không bao giờ confirm" (dọn bằng lifecycle rule của bucket hoặc job Part 21).

> **Đã đọc type của `@aws-sdk/client-s3@3.1143.0` và `@aws-sdk/s3-request-presigner@3.1143.0` (cùng phiên bản, presigner phụ thuộc đúng `client-s3` bản đó):** `getSignedUrl(client, command, options?)` trả `Promise<string>`; README của presigner: `expiresIn` tính bằng **giây**, *"If not set, it's default at 900"*. `PutObjectRequest` có `Bucket`, `Key`, `ContentType`, `ContentLength`; có `HeadObjectCommand`, `DeleteObjectCommand`; cấu hình client có `forcePathStyle` (cần `true` khi dùng MinIO local). Đây là SDK độc lập với Nest (không có peer Nest) — bọc nó trong 1 provider như `S3FileStorage`.

Bẫy presigned PUT: (1) URL PUT **không giới hạn được kích thước** file client đẩy lên — phải kiểm tra lại bằng `HeadObject` ở bước confirm như trên, hoặc dùng presigned **POST** (package `@aws-sdk/s3-presigned-post`, điều kiện `content-length-range`) nếu cần chặn cứng từ phía S3. (2) Key phải do **server** sinh và bước confirm phải kiểm tra key thuộc đúng sản phẩm/người dùng — nếu không, client gửi key của người khác để "chiếm" file. (3) Bucket phải cấu hình CORS cho domain frontend, nếu không trình duyệt chặn request PUT.

Nhu cầu	Cách 1: qua Multer	Cách 2: presigned URL
Ảnh nhỏ (≤ vài MB), cần kiểm tra loại file thật ngay lúc upload	✅ Magic number + resize trong request	❌ Server không thấy nội dung, phải kiểm tra bất đồng bộ
File lớn (video, file dữ liệu hàng trăm MB)	❌ Chiếm RAM/băng thông API, dễ timeout	✅ Byte đi thẳng tới S3
Chạy local, chưa có S3	✅ `LocalFileStorage` là đủ	❌ Cần S3 hoặc MinIO
Client đơn giản (1 request)	✅	❌ 3 bước: xin URL → PUT → confirm
Giới hạn kích thước chặt chẽ	✅ `limits.fileSize` dừng stream ngay	Chỉ kiểm tra được sau khi upload xong (hoặc dùng presigned POST)

Quy tắc thực dụng: bắt đầu bằng Cách 1 với lớp `FileStorage` trừu tượng — ảnh sản phẩm nhỏ và cần kiểm tra thật. Chỉ chuyển sang presigned URL khi file lớn thực sự làm API server chậm/tốn RAM; lúc đó chỉ thêm 2 endpoint mới, `FileStorage` vẫn giữ nguyên cho phần xoá/lấy URL.

22.7 Test upload với supertest `.attach()`

```
const PNG_1x1 = Buffer.from(
  '89504E470D0A1A0A0000000D49484452000000010000000108060000000' +
  '1F15C4890000000D49444154789C6360000002000154A24F5D0000000049454E44AE426082',
  'hex',
);

beforeEach(async () => {
  const moduleRef = await Test.createTestingModule({ imports: [AppModule] })
    .overrideProvider(FileStorage)
    .useValue({ put: jest.fn(), delete: jest.fn(), url: (k: string) => `/uploads/${k}`
})
    .compile();
  app = moduleRef.createNestApplication();
  await app.init();
});

it('nhận ảnh PNG thật', () =>
  request(app.getHttpServer())
    .post(`/products/${productId}/image`)
    .set('Authorization', `Bearer ${token}`)
    .attach('image', PNG_1x1, { filename: 'a.png', contentType: 'image/png' })
    .expect(201));

it('từ chối file PHP đội lốt PNG', () =>
  request(app.getHttpServer())
    .post(`/products/${productId}/image`)
    .set('Authorization', `Bearer ${token}`)
    .attach('image', Buffer.from('<?php echo 1; ?>'), { filename: 'evil.png',
contentType: 'image/png' })
    .expect(422));
```

`.attach(field, Buffer | đường dẫn file, { filename, contentType })` tự chuyển request sang `multipart/form-data`. Dùng `Buffer` dựng sẵn trong test thay vì file thật trên đĩa để test không phụ thuộc thư mục. `overrideProvider(FileStorage)` thay storage thật bằng mock — test không ghi rác vào `uploads/`, và có thể `expect(storage.put).toHaveBeenCalled()`. Thêm 1 case không gửi file (→ 422 *"File is required"*) và 1 case file quá `limits.fileSize` (→ 413).

> **Đã đọc type `@types/superagent` (dùng bởi `supertest@7.2.2` của project):** `attach(field: string, file: MultipartValueSingle, options?: string | { filename?: string; contentType?: string })`. Chuỗi hex PNG 1×1 ở trên đã được chạy thử qua `FileTypeValidator` của project → nhận diện đúng `image/png`. `ParseFilePipe`: `fileIsRequired` mặc định `true`, lỗi dùng `errorHttpStatusCode` đã cấu hình (ở đây 422) — đọc từ `parse-file.pipe.js` bản 11.2.3.

Việc cần làm:

1. `npm i -D @types/multer`; thêm cột `imageKey` (`type: 'varchar', nullable: true`) vào entity `Product`; thêm `uploads/` vào `.gitignore`
2. Tạo `StorageModule` với `FileStorage` (abstract) + `LocalFileStorage` như 22.4, import vào `ProductsModule`

3. Viết `POST /products/:id/image` (guard JWT, `FileInterceptor` có `limits.fileSize` 2MB, `ParseFilePipeBuilder` với regex neo `^...$` + `overrideMimeType`) và `ProductsService.attachImage()` theo đúng thứ tự ghi file → lưu DB → xoá file cũ
4. Bật `useStaticAssets` cho `/uploads/` trong `main.ts`; `GET /products/:id` trả thêm `imageUrl` (null nếu chưa có ảnh). Thử bằng Postman: ảnh hợp lệ, file `.txt` đổi tên thành `.png`, file 5MB — ghi lại status code mỗi trường hợp
5. Sửa `TransformInterceptor` để bỏ qua `StreamableFile` (22.5), rồi viết thử 1 endpoint tải ảnh gốc bằng `StreamableFile` để xác nhận file tải về đúng
6. Viết e2e test như 22.7: PNG hợp lệ (201), PHP đội lốt PNG (422), không gửi file (422), không token (401)
7. (Tuỳ chọn) Chạy MinIO bằng Docker, viết `S3FileStorage` (`forcePathStyle: true`), đổi `useClass` và xác nhận `ProductsService` không phải sửa

---

PART 23

Mail & Notifications

Mục tiêu: Gửi email (chào mừng khi đăng ký, báo có comment mới) qua 1 `MailService` tự viết trên nodemailer, luôn gửi qua queue BullMQ thay vì trong request, xem mail lúc dev mà không gửi thật, lưu thông báo trong app vào bảng `notifications` (có `readAt`), và unit test bằng mock.

23.1 Nest không có module mail chính chủ — chọn gì?

Khác Laravel (có sẵn `Mail`, `Notification`, kênh `mail` / `database` ngay trong framework — Part 20 bên đó), NestJS **không có package `@nestjs/mail` hay `@nestjs/notifications` nào**. Docs.nestjs.com không có trang về email. Bạn tự chọn thư viện gửi mail và tự bọc nó thành provider. Hai lựa chọn thực tế:

	nodemailer + `MailService` tự viết	**`@nestjs-modules/mailer` (cộng đồng)**
Là gì	Thư viện gửi mail chuẩn của Node; bạn bọc vào 1 provider ~30 dòng	Module Nest bọc sẵn nodemailer + template engine (handlebars, pug, ejs, mjml, nunjucks, liquid)
Phiên bản (lúc viết)	`nodemailer@10.0.13`, đã viết lại bằng TypeScript, **kèm sẵn type**	`2.3.7` (06/2026), vẫn phát hành đều trong 2026
Tương thích Nest 11	Không phụ thuộc Nest	peer `@nestjs/common >=7.0.9`, `nodemailer >=8.0.5`
Cài kéo theo	Chỉ nodemailer	`optionalDependencies` gồm ejs, handlebars, pug, mjml (bản alpha), nunjucks, liquidjs, preview-email — npm **cài hết mặc định** dù bạn chỉ dùng 1
Hợp khi	Học để hiểu cơ chế; mail ít, template đơn giản	Nhiều template HTML phức tạp, cần layout/partials, i18n

> **Đã tải và đọc `nodemailer@10.0.13` và `@nestjs-modules/mailer@2.3.7`:** CHANGELOG nodemailer 10.0.0 (09/2026): *"migrate to TypeScript with ES module and CommonJS builds"*, breaking change duy nhất là yêu cầu Node ≥ 20. Bản CJS export có tên `createTransport`, `createTestAccount`, `getTestMessageUrl` và type `Transporter`, `SendMailOptions` — **không cần `@types/nodemailer`** (package đó giờ chỉ còn cho bản cũ). `@nestjs-modules/mailer`: peer **bắt buộc** chỉ có `@nestjs/common >=7.0.9`, `@nestjs/core >=7.0.9`, `nodemailer >=8.0.5` (không có cận trên — Nest 11 và 12 đều khớp); các peer còn lại (`handlebars`, `ejs`, `pug`, `mjml`, `nunjucks`, `liquidjs`, `preview-email`, `@nestjs/event-emitter`, `@nestjs/terminus`, `bullmq`) đều đánh dấu `optional` trong `peerDependenciesMeta`. Peer khai `nodemailer >=8.0.5` nhưng devDependencies (bản họ tự test) là `nodemailer 8.0.7` và `@nestjs/common 11.1.21` — dùng với nodemailer 10 là chưa được chính họ kiểm chứng; import adapter theo đường dẫn `@nestjs-modules/mailer/adapters/handlebars.adapter`; có `MailerModule.forRoot/forRootAsync`, `MailerService.sendMail({ template, context, ... })` và `MailerTestModule` (dùng `streamTransport`) cho test. Package được duy trì, không bị bỏ rơi — nhưng với project học này, nodemailer trực tiếp ít phụ thuộc hơn và cho bạn thấy rõ cơ chế.

Course này đi theo cột trái: `MailService` tự viết. Mọi phần còn lại (queue, test, notifications) giống hệt nếu sau này bạn đổi sang `@nestjs-modules/mailer` — chỉ ruột của `MailService` thay đổi.

23.2 `MailService`: nodemailer + `ConfigService` + template

```
npm i nodemailer
```

```
MAIL_FROM="Blog API <no-reply@blog.local>"
MAIL_TRANSPORT=smtp
SMTP_HOST=localhost
SMTP_PORT=1025
SMTP_SECURE=false
SMTP_USER=
SMTP_PASS=
```

```ts
import { Injectable } from '@nestjs/common';
import { ConfigService } from '@nestjs/config';
import { createTransport, Transporter } from 'nodemailer';

export interface MailMessage {
  to: string;
  subject: string;
  text: string;
  html: string;
}

@Injectable()
export class MailService {
  private readonly transporter: Transporter;
  private readonly from: string;

  constructor(config: ConfigService) {
    this.from = config.getOrThrow<string>('MAIL_FROM');
    const user = config.get<string>('SMTP_USER');
    this.transporter =
      config.get('MAIL_TRANSPORT') === 'json'
        ? createTransport({ jsonTransport: true })
        : createTransport({
            host: config.getOrThrow<string>('SMTP_HOST'),
            port: Number(config.getOrThrow<string>('SMTP_PORT')),
            secure: config.get('SMTP_SECURE') === 'true',
            auth: user ? { user, pass: config.getOrThrow<string>('SMTP_PASS') } :
undefined,
          });
  }

  async send(message: MailMessage): Promise<void> {
    await this.transporter.sendMail({ from: this.from, ...message });
  }
}
```

Transporter tạo **1 lần** trong constructor (provider là singleton — Part 00), không tạo mới mỗi lần gửi. `MailModule` khai `providers: [MailService]`, `exports: [MailService]`.

Bẫy biến môi trường: mọi giá trị trong `.env` là **chuỗi**. `config.get<number>('SMTP_PORT')` *không* đổi kiểu — generic chỉ là lời hứa với TypeScript, runtime vẫn là `'1025'`. Tương tự `secure: config.get('SMTP_SECURE')` với giá trị `'false'` là truthy → nodemailer bật TLS ngay từ đầu và treo/lỗi khi nói chuyện với Mailpit. Luôn `Number(...)` và so sánh `=== 'true'` tường minh.

Template — hàm TypeScript thuần là đủ cho vài email

```
const escapeHtml = (value: string) =>
  value
    .replace(/&/g, '&amp;')
    .replace(/</g, '&lt;')
    .replace(/>/g, '&gt;')
    .replace(/"/g, '&quot;')
    .replace(/'/g, '&#39;');

export function newCommentMail(to: string, postTitle: string, commentBody: string):
MailMessage {
  return {
    to,
    subject: `Bài "${postTitle}" có bình luận mới`,
    text: `Bài "${postTitle}" vừa có bình luận:\n\n${commentBody}`,
    html: `<p>Bài <b>${escapeHtml(postTitle)}</b> vừa có bình luận:</p>
<blockquote>${escapeHtml(commentBody)}</blockquote>`,
  };
}
```

Bẫy HTML injection trong email: nội dung comment do người dùng gõ. Nhét thẳng vào `html` không escape → ai đó comment `<a href="https://phishing...">Đăng nhập lại</a>` và email **từ domain của bạn** gửi link lừa đảo tới tác giả bài viết. Mọi dữ liệu người dùng đưa vào HTML đều phải qua `escapeHtml`. (Template engine như handlebars tự escape với cú pháp `{{ }}` — một lý do để dùng nó khi số template lớn.) Luôn gửi kèm bản `text`: một số client mail chỉ hiện text, và nó giảm điểm spam.

23.3 Không gửi mail trong request — đẩy vào queue (Part 18)

Gửi 1 email qua SMTP mất từ vài trăm ms tới vài giây (bắt tay TLS, server mail chậm), và **có thể lỗi** (server mail bảo trì, bị rate limit). Nếu `AuthService.register()` `await mailService.send(...)` trực tiếp: người dùng phải chờ thêm vài giây mới thấy "đăng ký thành công", và SMTP lỗi thì request đăng ký trả 500 — dù user đã được lưu vào DB. Cách đúng: request chỉ **xếp 1 job vào queue** (vài ms), worker gửi mail sau, lỗi thì BullMQ tự thử lại.

```
@Module({
  imports: [BullModule.registerQueue({ name: 'mail' }), UsersModule],
  providers: [MailService, MailProcessor],
  exports: [BullModule],
})
export class MailModule {}
```

```ts
@Injectable()
export class AuthService {
  constructor(
    private readonly usersService: UsersService,
    private readonly jwtService: JwtService,
    @InjectQueue('mail') private readonly mailQueue: Queue,
  ) {}

  async register(dto: RegisterDto) {
    ...
    const user = await this.usersService.create(dto.email, hash);
    await this.mailQueue.add(
      'welcome',
      { userId: user.id },
      {
        jobId: `welcome-${user.id}`,
        attempts: 5,
        backoff: { type: 'exponential', delay: 10_000 },
        removeOnComplete: 1000,
        removeOnFail: 5000,
      },
    );
    return { id: user.id, email: user.email };
  }
}
```

```ts
@Processor('mail')
export class MailProcessor extends WorkerHost {
  constructor(
    private readonly mail: MailService,
    private readonly users: UsersService,
  ) {
    super();
  }

  async process(job: Job<{ userId: number; commentId?: number }>): Promise<void> {
    const user = await this.users.findById(job.data.userId);
    if (!user) return;

    switch (job.name) {
      case 'welcome':
        return this.mail.send(welcomeMail(user.email));
      default:
        throw new Error(`Unknown mail job: ${job.name}`);
    }
  }
}
```

Mỗi loại mail là 1 `case` trong cùng 1 processor (1 queue `mail` cho mọi email). Job `new-comment` ở mục 23.6 thêm 1 case tương tự: đọc comment + bài viết theo id rồi gọi `newCommentMail(...)`. `default` ném lỗi để job gõ sai tên bị đánh dấu failed thay vì âm thầm "thành công".

Đã đối chiếu type `bullmq@6.3.10` + `@nestjs/bullmq@12.0.0` và docs.nestjs.com/techniques/queues: `@InjectQueue(name)`, `@Processor(queueName)`, `abstract class WorkerHost { abstract process(job: Job, token?: string): Promise<any> }` đúng như trên. `JobsOptions` có `attempts`, `backoff: number | { type: 'fixed' | 'exponential', delay }`, `removeOnComplete` / `removeOnFail` (boolean | số job giữ lại), `jobId` — comment của `jobId`: *"If you attempt to add a job with an id that already exists, it will not be added."* Source `classes/job.js` còn chặn 2 kiểu `jobId`: *"Custom Id cannot be integers"* và *"Custom Id cannot contain :"* — vì vậy viết `welcome-42`, không phải `welcome:42` hay `'42'`. Comment của `addBulk` nhắc lại: dữ liệu job được **serialize JSON**, instance class mất method khi tới worker.

4 bẫy khi gửi mail qua queue: (1) **Job chỉ chứa id, không chứa entity.** Data được lưu JSON trong Redis — nhét cả `user` vào là lưu luôn hash password vào Redis, và worker nhận dữ liệu cũ nếu user đổi email trong lúc job chờ. Worker tự đọc lại từ DB. (2) **Enqueue sau khi commit.** Nếu tạo user trong `DataSource.transaction()` (Part 13) mà `add()` job ngay bên trong callback, worker có thể chạy trước khi transaction commit → `findById` trả `null` (hoặc transaction rollback mà mail vẫn được gửi). Luôn `add()` sau khi `transaction()` đã resolve. (3) **Retry = có thể gửi trùng.** Nếu `send()` đã thành công nhưng code sau nó trong `process()` ném lỗi, BullMQ chạy lại cả job → gửi mail lần 2. Để `send()` là bước **cuối cùng** của job. (4) **`jobId` chống trùng chỉ khi job còn tồn tại.** `removeOnComplete` xoá job cũ đi thì `add()` cùng `jobId` lần sau sẽ được thêm lại bình thường.

23.4 Gửi thẳng trong service hay đẩy vào queue — khi nào dùng cái nào

Queue không miễn phí: cần Redis, cần worker chạy, thêm 1 chỗ có thể hỏng. Với project học, gọi thẳng `mailService.send()` vẫn "chạy được" — nên phải có tiêu chí rõ ràng. Câu hỏi quyết định: **người dùng có cần biết mail đã gửi thành công NGAY trong response này không?** — gần như luôn là không.

Cách 1 — `await mailService.send()` trực tiếp trong service

```
const user = await this.usersService.create(dto.email, hash);
await this.mailService.send(welcomeMail(user.email));
return { id: user.id, email: user.email };
```

Được: đơn giản, không cần Redis, lỗi SMTP hiện ngay trong response (tiện lúc debug). Mất: response chậm thêm theo tốc độ SMTP; SMTP lỗi → client nhận 500 dù user đã tạo xong (client bấm đăng ký lại → 409 "Email already registered", rất khó hiểu); không retry.

Cách 2 — `mailQueue.add()`, worker gửi (mục 23.3)

Được: response nhanh và không phụ thuộc SMTP; lỗi tạm thời được retry có backoff; job lỗi hẳn nằm trong danh sách failed để xem lại; có thể giới hạn tốc độ gửi ở worker để không bị nhà cung cấp mail chặn. Mất: cần Redis + worker (Part 18); lỗi không còn hiện trong response — phải theo dõi log/danh sách failed.

Căn cứ: docs.nestjs.com/techniques/queues nêu đúng mục đích của queue: *"Smooth out processing peaks"* và *"Break up monolithic tasks that may otherwise block the Node.js event loop"*; retry/backoff là option `attempts` / `backoff` đã đối chiếu ở 23.3 (`bullmq@6.3.10`). Đây cũng là lựa chọn mặc định của Laravel: notification `implements ShouldQueue` (Part 20 bên đó).

Tình huống	Cách 1: gửi thẳng	Cách 2: qua queue

Mail kèm hành động của người dùng (đăng ký, có comment mới, đặt hàng)	❌ Chậm response, SMTP lỗi làm hỏng request đã thành công	✅ Mặc định nên dùng
Gửi cho nhiều người (thông báo tới mọi follower)	❌ N lần SMTP trong 1 request → timeout	✅ 1 job / người nhận, worker tự dàn đều
Chưa có Redis, đang học/prototype	✅ Chấp nhận được, biết rõ nhược điểm	❌ Chưa có hạ tầng
Script admin chạy tay / CLI (seed, gửi thử 1 mail)	✅ Muốn thấy lỗi ngay	Thừa
Mail "đặt lại mật khẩu" người dùng đang đứng chờ	Chạy được	✅ Vẫn queue, worker xử lý trong vài giây — nhưng đặt job ưu tiên cao / queue riêng để không kẹt sau hàng nghìn mail marketing

Quy tắc thực dụng: mọi email phát sinh từ request của người dùng → queue. Gửi thẳng chỉ dùng trong script/CLI hoặc khi chưa có Redis — và khi đó, bọc lời gọi trong `try/catch` + log để SMTP lỗi không biến 1 request đã thành công thành 500.

23.5 Xem mail lúc dev mà không gửi thật

Cách	Cấu hình	Xem ở đâu
`jsonTransport` của nodemailer	`MAIL_TRANSPORT=json` (nhánh đã viết ở 23.2)	`sendMail` không gửi gì, trả về `{ envelope, messageId, message }` với `message` là chuỗi JSON của email — log ra hoặc assert trong test
Mailpit (Docker)	`SMTP_HOST=localhost`, `SMTP_PORT=1025`, không auth	Giao diện web `http://localhost:8025` — hiện HTML thật, header, bản text
Ethereal (dịch vụ SMTP giả của nodemailer)	`createTestAccount()` sinh tài khoản tạm (cần internet)	Link từ `getTestMessageUrl(info)`

```
docker run -d --restart unless-stopped --name=mailpit -p 8025:8025 -p 1025:1025
axllent/mailpit
```

```
const account = await createTestAccount();
const transporter = createTransport({
  host: account.smtp.host,
  port: account.smtp.port,
  secure: account.smtp.secure,
  auth: { user: account.user, pass: account.pass },
});
const info = await transporter.sendMail({ from: 'a@test.local', to: 'b@test.local',
subject: 'Hi', text: 'Hello' });
console.log(getTestMessageUrl(info));
```

Đã chạy thử `nodemailer@10.0.13`: `createTransport({ jsonTransport: true }).sendMail(...)` trả object có đúng các key `envelope`, `messageId`, `message` (chuỗi JSON chứa `from`, `to`, `subject`, `text`, `html`). Type bundled: `createTestAccount(): Promise<TestAccount>` với `TestAccount = { user, pass, smtp: { host, port, secure }, imap, pop3, web }`, `getTestMessageUrl(info): string | false`. Lệnh Docker và cổng 1025 (SMTP) / 8025 (web) lấy nguyên văn từ mailpit.axllent.org/docs/install/docker.

Bẫy "dev gửi mail thật cho khách": copy DB production về máy dev để debug, chạy worker với cấu hình SMTP thật còn sót trong `.env` → job tồn đọng trong queue gửi mail thật tới người dùng thật. Máy dev luôn trỏ vào Mailpit hoặc `jsonTransport`; `.env` production không bao giờ nằm trên máy dev.

23.6 Thông báo trong app — bảng `notifications` (tương đương kênh "database" của Laravel)

Email chỉ là 1 kênh. Ứng dụng thường còn có "chuông thông báo" trong app: danh sách thông báo, đánh dấu đã đọc, đếm số chưa đọc. Laravel có sẵn kênh `database` + bảng `notifications`; Nest thì bạn tự dựng — thực ra chỉ là 1 entity + 1 service:

```
@Entity('notifications')
export class UserNotification {
  @PrimaryGeneratedColumn()
  id: number;

  @Column()
  userId: number;

  @ManyToOne(() => User, { onDelete: 'CASCADE' })
  @JoinColumn({ name: 'userId' })
  user: User;

  @Column()
  type: string;

  @Column({ type: 'simple-json' })
  data: Record<string, unknown>;

  @Column({ type: 'datetime', nullable: true })
  readAt: Date | null;

  @CreateDateColumn()
  createdAt: Date;
}
```

Khai `userId` thành cột riêng (cùng tên với `JoinColumn`) để lọc/cập nhật theo `userId` trực tiếp mà không phải JOIN. `readAt = null` nghĩa là chưa đọc — dùng mốc thời gian thay cho cột boolean `isRead` vì vừa biết "đã đọc chưa" vừa biết "đọc lúc nào", và job dọn dẹp (Part 21.6) xoá được "thông báo đã đọc quá 30 ngày".

```ts
@Injectable()
export class NotificationsService {
  constructor(
    @InjectRepository(UserNotification)
    private readonly repo: Repository<UserNotification>,
    @InjectQueue('mail') private readonly mailQueue: Queue,
  ) {}

  async notifyNewComment(authorId: number, postId: number, commentId: number) {
    await this.repo.save(
      this.repo.create({ userId: authorId, type: 'comment.created', data: { postId,
commentId } }),
    );
    await this.mailQueue.add('new-comment', { userId: authorId, commentId }, {
attempts: 5 });
  }

  async listForUser(userId: number, page: number, limit: number):
Promise<Paginated<UserNotification>> {
    const [data, total] = await this.repo.findAndCount({
      where: { userId },
      order: { createdAt: 'DESC' },
      skip: (page - 1) * limit,
      take: limit,
    });
    return { data, meta: { total, page, limit, totalPages: Math.ceil(total / limit) }
};
  }

  unreadCount(userId: number): Promise<number> {
    return this.repo.count({ where: { userId, readAt: IsNull() } });
  }

  async markRead(userId: number, id: number): Promise<void> {
    const result = await this.repo.update({ id, userId }, { readAt: new Date() });
    if (!result.affected) throw new NotFoundException('Notification not found');
  }

  async markAllRead(userId: number): Promise<number> {
    const result = await this.repo.update({ userId, readAt: IsNull() }, { readAt: new
Date() });
    return result.affected ?? 0;
  }
}
```

`notifyNewComment` đóng vai trò của `via()` trong Laravel: 1 chỗ duy nhất quyết định thông báo này đi những kênh nào (DB + mail). `listForUser` trả đúng dạng `Paginated` của Part 12 nên `TransformInterceptor` trải phẳng `meta` như mọi danh sách khác.

Bẫy quyền truy cập (IDOR): `POST /notifications/:id/read` mà chỉ `update({ id })` → user A đánh dấu được thông báo của user B chỉ bằng cách đổi số id. Điều kiện `where` luôn có **cả `id` lẫn `userId`** lấy từ JWT (`req.user.userId` — Part 10), và `affected === 0` thì trả 404 (không trả 403, để không lộ "id này có tồn tại").

Bẫy trùng tên `Notification`: giống bẫy `Post` ở Part 08 — `tsconfig.json` của project không khai `lib` nên TypeScript nạp kèm type của trình duyệt (DOM), trong đó có sẵn 1 class global tên `Notification`. Đặt entity tên `Notification` mà quên import ở 1 file, TypeScript không báo "không tìm thấy" mà âm thầm dùng `Notification` của trình duyệt. Tên class `UserNotification` + `@Entity('notifications')` tránh hẳn chuyện này mà tên bảng vẫn đẹp.

Đã đối chiếu `typeorm@1.1.1` trong project: `'simple-json'` có trong `SimpleColumnType` và driver SQLite xử lý nó (lưu dạng text, tự `JSON.parse` khi đọc); `'datetime'` nằm trong `supportedDataTypes` của `AbstractSqliteDriver`; `IsNull` export từ `typeorm`; `UpdateResult.affected` được driver `better-sqlite3` điền từ `raw.changes` (`BetterSqlite3QueryRunner.js`) — nên kiểm tra `affected` để phát hiện "không có dòng nào khớp" là dùng được với SQLite của project.

23.7 Unit test — mock `MailService` và queue, không gửi gì cả

Service gửi mail/enqueue job chỉ cần kiểm tra "đã gọi đúng hàm với đúng dữ liệu" — không cần SMTP, không cần Redis. Thay provider thật bằng object có các `jest.fn()`, như cách đã mock repository ở Part 11:

```
describe('AuthService.register', () => {
  const usersService = { findByEmail: jest.fn(), create: jest.fn() };
  const mailQueue = { add: jest.fn() };
  let service: AuthService;

  beforeEach(async () => {
    jest.clearAllMocks();
    const moduleRef = await Test.createTestingModule({
      providers: [
        AuthService,
        { provide: UsersService, useValue: usersService },
        { provide: JwtService, useValue: {} },
        { provide: getQueueToken('mail'), useValue: mailQueue },
      ],
    }).compile();
    service = moduleRef.get(AuthService);
  });

  it('xếp job welcome sau khi tạo user', async () => {
    usersService.findByEmail.mockResolvedValue(null);
    usersService.create.mockResolvedValue({ id: 7, email: 'a@test.local' });

    await service.register({ email: 'a@test.local', password: 'secret123' });

    expect(mailQueue.add).toHaveBeenCalledWith(
      'welcome',
      { userId: 7 },
      expect.objectContaining({ jobId: 'welcome-7' }),
    );
  });

  it('không xếp job khi email đã tồn tại', async () => {
    usersService.findByEmail.mockResolvedValue({ id: 1 });
    await expect(service.register({ email: 'a@test.local', password: 'x'
})).rejects.toThrow(ConflictException);
    expect(mailQueue.add).not.toHaveBeenCalled();
  });
});
```

Test cho `MailProcessor` cũng vậy: mock `MailService.send` và `UsersService.findById`, gọi thẳng `processor.process({ name: 'welcome', data: { userId: 7 } } as Job)` rồi kiểm tra `send` được gọi với `to` đúng email. Muốn kiểm tra cả nội dung HTML thật (escape đúng chưa) thì test hàm template trực tiếp — nó là hàm thuần, không cần Nest.

> **Token của queue:** `@InjectQueue('mail')` inject theo token do `getQueueToken('mail')` sinh ra — `@nestjs/bullmq@12.0.0` re-export `getQueueToken` từ `@nestjs/bull-shared` (đọc trong `dist/index.d.ts`). Dùng đúng hàm này làm `provide` thì mock thay được queue thật mà không cần Redis. Với test e2e: `overrideProvider(MailService).useValue({ send: jest.fn() })` (API `overrideProvider` có trong `@nestjs/testing` của project).

Việc cần làm:

1. `npm i nodemailer` ; chạy Mailpit bằng Docker; thêm biến `MAIL_*` / `SMTP_*` vào `.env` ; viết `MailModule` + `MailService` như 23.2, kèm 2 hàm template `welcomeMail` và `newCommentMail` (có `escapeHtml`)
2. Đăng ký queue `mail` (dùng kết nối Redis của Part 18), viết `MailProcessor` ; trong `AuthService.register()` thay vì gửi thẳng thì `mailQueue.add('welcome', ...)` với `jobId` , `attempts` , `backoff`
3. Đăng ký 1 user mới → mở `http://localhost:8025` thấy mail chào mừng. Tắt Mailpit, đăng ký user khác → request vẫn 201 ngay; bật lại Mailpit → sau vài lần retry mail tới nơi
4. Tạo entity `UserNotification` + `NotificationsService` như 23.6; khi có comment mới trên bài của người khác thì gọi `notifyNewComment` (cần thêm cột tác giả cho `Post` nếu chưa có — Part 10 đã có user từ JWT)
5. Viết `NotificationsController` (guard JWT): `GET /notifications?page=&limit=` , `GET /notifications/unread-count` , `POST /notifications/:id/read` , `POST /notifications/read-all` ; thử dùng token user A đánh dấu thông báo của user B → phải nhận 404
6. Thêm job `@Cron` (Part 21) xoá thông báo đã đọc quá 30 ngày
7. Viết unit test như 23.7 cho `AuthService.register` (có/không enqueue) và cho `MailProcessor` (user không tồn tại thì không gửi)

PART 24

Kiến trúc dự án lớn: Controller mỏng, Repository, DTO, Enum, Module, Migration

Mục tiêu: Biết đặt từng loại logic ở đâu khi app phình từ 5 lên 50 module, tách được 3 "hình dạng" dữ liệu (Entity, DTO, response), tổ chức module không bị vòng lặp phụ thuộc, và bỏ `synchronize: true` để chuyển sang migration. Không có "kiến trúc đúng duy nhất". Mục tiêu là code dễ tìm, dễ test và dễ đổi.

24.1 Bản đồ "cái gì ở đâu"

Từ Part 02 tới Part 13 bạn đã dùng gần đủ mọi "khối" của NestJS. Bảng dưới gom lại **trách nhiệm duy nhất** của từng khối. Khi phân vân "viết đoạn này ở đâu", tra bảng này trước:

Khối	Trách nhiệm (chỉ việc này)	Không được làm	Trong dự án của bạn
Controller	Map HTTP → gọi service → trả kết quả. Đọc `@Param` / `@Body` / `@Query` , gắn guard/pipe	Inject Repository, viết `if` nghiệp vụ, try/catch, tính toán	`PostsController` (đã mỏng)

Service	Nghiệp vụ: kiểm tra tồn tại, quy tắc, transaction, gọi repository/service khác	Đọc `req` / `res` của Express, biết về HTTP status (ngoại trừ ném `HttpException`)	`PostsService`, `ProductsService`
Repository	Truy vấn DB (`find`, QueryBuilder). Custom repository khi query dài và dùng lại (mục 24.3)	Quy tắc nghiệp vụ, ném 404	`Repository<Post>` do TypeORM sinh
Entity	Mô tả bảng: cột, quan hệ, ràng buộc	Làm body request, làm response trả thẳng (lộ cột nhạy cảm)	`Post`, `User`, `Product`
DTO (vào)	Hình dạng + luật validate của input	Chứa logic, quan hệ DB	`CreatePostDto`, `PaginationQueryDto`
Response shape (ra)	Hình dạng client nhận được: ẩn/đổi tên field, gắn `meta`	Tự lọc tay rải rác ở từng controller	`Paginated<T>`; ẩn field bằng serialization, Part 15
Pipe	Biến đổi/validate 1 giá trị đầu vào	Truy vấn DB nặng	`ValidationPipe`, `ParseIntPipe`
Guard	Trả lời "được đi tiếp không?"	Biến đổi dữ liệu	`JwtAuthGuard`; phân quyền theo role ở Part 17
Interceptor	Bọc quanh handler: transform response, đo thời gian, cache	Quyết định pass/fail	`TransformInterceptor`
Exception Filter	Format lỗi + log lỗi tập trung	Nghiệp vụ	`HttpExceptionFilter`
Middleware	Việc giống nhau ở mọi route, không cần biết handler	Phân quyền theo handler	`LoggerMiddleware`
Module	Ranh giới đóng gói: ai được thấy provider nào	Chứa logic	`PostsModule` ...
Việc chậm / phản ứng phụ / định kỳ	Queue, Event, Cron	Chạy đồng bộ trong request	Part 18, 19, 21

```
Hướng phụ thuộc hợp lệ (mũi tên = "được phép gọi/biết đến"):

  HTTP ──► Controller ──► Service ──► Repository ──► Entity ──► DB
                │            │
                │            ├──► Service khác (qua module exports)
                │            └──► Queue / EventEmitter  (Part 18, 19)
                └──► DTO (input)          Response shape ◄── Service

  ✗ Service không import Controller
  ✗ Repository không gọi Service
  ✗ Entity không import DTO
```

24.2 Controller mỏng — và 3 "hình dạng" dữ liệu: Entity ≠ DTO ≠ Response

Controller mỏng

`PostsController` hiện tại đã đúng hướng: mỗi route 1–3 dòng, chỉ gọi `postsService`. Dấu hiệu controller bắt đầu "béo" là khi bạn thấy trong đó có `@InjectRepository`, có `if (!x) throw new NotFoundException`, hoặc có vòng lặp xử lý dữ liệu:

```
@Post(':postId/comments')
async create(@Param('postId', ParseIntPipe) postId: number, @Body() dto:
CreateCommentDto) {
  const post = await this.postsRepository.findOneBy({ id: postId });
  if (!post) throw new NotFoundException();
  const comment = this.commentsRepository.create({ ...dto, post });
  return this.commentsRepository.save(comment);
}
```

Chạy vẫn đúng. Vấn đề là đoạn nghiệp vụ này bị "khoá" vào HTTP. Khi cần tạo comment từ chỗ khác (script `seed.ts`, một job queue ở Part 18, một cron ở Part 21), bạn sẽ phải copy lại nó. Đưa nó vào `CommentsService.create(postId, dto)` như Part 09 thì mọi cổng vào (HTTP, CLI, queue, cron) gọi chung một hàm, và test nó không cần giả lập HTTP.

> **Service phình quá to thì sao?** Khi `PostsService` vượt khoảng 300–400 dòng hoặc có nhiều nhóm việc khác nhau (CRUD, publish, thống kê), tách thành nhiều provider nhỏ trong cùng module: `PostsService` (CRUD), `PostPublishingService` (luồng đăng bài), `PostStatsService` (thống kê). Mỗi class là một `@Injectable()` bình thường. NestJS không có khái niệm "Action class" riêng như Laravel, nhưng một provider chỉ có 1 method public (`PublishPostUseCase.execute()`) chính là cùng ý tưởng. Muốn đi xa hơn theo hướng này thì xem module CQRS ở Part 26.

Ba hình dạng dữ liệu

	Entity	DTO (input)	Response shape (output)
Mô tả cái gì	1 dòng trong bảng DB	Body/query client gửi lên	JSON client nhận về
Decorator	`@Entity`, `@Column`, quan hệ	`@IsString`, `@Type`...	`@Exclude` / `@Expose` (Part 15) hoặc interface + hàm map
Ví dụ `User`	`id, email, password`	`RegisterDto { email, password }`	`{ id, email }`, **không bao giờ** có `password`
Đổi khi nào	Đổi schema DB (cần migration, mục 24.6)	Đổi luật input API	Đổi hợp đồng API với client (cân nhắc versioning, Part 16)

Lý do phải tách: ba thứ này đổi vì ba lý do khác nhau. Nếu dùng chung một class, mỗi lần đổi DB là API đổi theo mà không ai để ý. Hai ví dụ có thật ngay trong dự án của bạn:

- **Lộ dữ liệu:** `User` entity có cột `password` (hash). Route nào trả thẳng `User` ra thì client nhận được luôn hash. Cách ẩn chuẩn là `ClassSerializerInterceptor` + `@Exclude()`, học ở Part 15.
- **Mass assignment:** `repository.create(dto)` an toàn là nhờ `ValidationPipe({ whitelist: true, forbidNonWhitelisted: true })` đã có trong `main.ts` gạt bỏ field lạ. Nếu một ngày ai đó bỏ

`whitelist` đi, client gửi `{"id": 1, ...}` có thể ghi đè bản ghi khác. DTO là "hàng rào" giữa HTTP và Entity.

```
export interface UserResponse {
  id: number;
  email: string;
}

export function toUserResponse(user: User): UserResponse {
  return { id: user.id, email: user.email };
}
```

Một hàm map thuần như trên là cách đơn giản nhất, không cần thư viện, và nhìn vào là biết field nào đi ra ngoài. Khi số entity tăng lên, chuyển sang cách khai báo bằng `@Exclude` / `@Expose` ở Part 15. `Paginated<T>` trong `src/common/pagination.ts` cũng là một "response shape" dùng chung, đặt đúng chỗ.

24.3 Service dùng thẳng `Repository<T>` hay tự viết Custom Repository

Khi `ProductsService.findAll()` dài 40 dòng chỉ để dựng điều kiện lọc, người mới hay hỏi: "có nên tách phần query ra một Repository riêng như tutorial mạng hay làm với `@EntityRepository` không?". Câu hỏi đúng thật ra là: **query này có được dùng lại ở nơi thứ hai không, và service có đang bị query che mất nghiệp vụ không?**

> **Đã kiểm chứng trên typeorm 1.1.1 đang cài (và Nest 11 / @nestjs/typeorm 12.0.1):** decorator `@EntityRepository` và `getCustomRepository()` **không còn tồn tại**. Grep toàn bộ `node_modules/typeorm/**/*.d.ts` không có kết quả nào (bị bỏ từ 0.3, tutorial cũ vẫn dùng nên hay gặp). Cơ chế hiện hành là `Repository.extend()` (có trong `repository/Repository.d.ts`: `extend<CustomRepository>(customs: CustomRepository & ThisType<this & CustomRepository>)`) và `Repository` có constructor public `(target, manager, queryRunner?)` nên kế thừa được. Trang `docs.nestjs.com/data/typeorm` **không có** mục custom repository. Nest chỉ dạy `@InjectRepository()`, nên cả 2 cách bên dưới là pattern cộng đồng. Cả 2 đã được chạy thử với Nest DI + better-sqlite3 và trả kết quả đúng.

Cách 1 — Service inject thẳng `Repository<T>` (đang dùng)

Như `PostsService` hiện tại: `@InjectRepository(Post) private readonly postsRepository: Repository<Post>`. Query nằm trong method của service. Đủ cho 90% trường hợp.

Cách 2 — Provider class kế thừa `Repository<T>`

```
import { Injectable } from '@nestjs/common';
import { DataSource, Repository } from 'typeorm';
import { Post } from './entities/post.entity';

@Injectable()
export class PostsRepository extends Repository<Post> {
  constructor(dataSource: DataSource) {
    super(Post, dataSource.createEntityManager());
  }

  findPublishedPage(page: number, limit: number) {
    return this.findAndCount({
      where: { status: PostStatus.Published },
      order: { id: 'DESC' },
      skip: (page - 1) * limit,
      take: limit,
    });
  }
}
```

```
@Module({
  imports: [TypeOrmModule.forFeature([Post])],
  providers: [PostsService, PostsRepository],
  exports: [PostsRepository],
})
export class PostsModule {}

constructor(private readonly postsRepository: PostsRepository) {}
```

Class này là provider bình thường. Inject bằng chính type `PostsRepository`, không cần `@InjectRepository`, và vẫn có đủ `find` / `save` ... vì nó kế thừa `Repository`. Khi test, mock bằng `{ provide: PostsRepository, useValue: mockRepo }`, giống hệt mock một service.

Biến thể: `dataSource.getRepository(Post).extend({ ...methods })` đăng ký qua `useFactory` với token tự đặt (`Symbol`). Cách này chạy được nhưng cần thêm `@Inject(TOKEN)` ở nơi dùng và type phải tự suy ra, nên với người mới Cách 2 dạng class dễ đọc hơn.

⚠ **Bẫy transaction:** `PostsRepository` ở trên gắn với `dataSource.createEntityManager()`, là manager **ngoài** transaction. Bên trong `dataSource.transaction(async (manager) => ...)` (Part 13), gọi `this.postsRepository.save()` sẽ chạy **ngoài** transaction, đúng cái bẫy Part 13 đã cảnh báo. Trong transaction phải dùng `manager.withRepository(this.postsRepository)` (method có trong `EntityManager` của typeorm 1.1.1) hoặc `manager.save(Post, ...)`.

Tình huống	Cách 1: Service + `Repository<T>`	Cách 2: Custom Repository class
CRUD + vài điều kiện lọc (Part 08.4)	✅ Đủ, ít file nhất	❌ Thêm 1 lớp chỉ để chuyển tiếp

Một query dài (QueryBuilder, `loadRelationIdAndMap`) dùng ở 2+ service	❌ Copy query, sửa 1 chỗ quên chỗ kia	✅ 1 method có tên, dùng chung
Service dài vì phần lớn là dựng query, nghiệp vụ bị che	❌ Khó đọc	✅ Service chỉ còn nghiệp vụ
Test service	✅ Mock bằng `getRepositoryToken(Post)`	✅ Mock bằng class token, gọn hơn
Dùng trong transaction	Dùng `manager` như Part 13	Phải nhớ `manager.withRepository(...)`

Quy tắc thực dụng: bắt đầu bằng Cách 1 cho mọi module. Chỉ tách Custom Repository khi **cùng một query** xuất hiện ở chỗ thứ hai, hoặc khi hơn một nửa thân service là code dựng query. Đừng tạo `XxxRepository` cho mọi entity "cho đủ bộ". Đó là lớp chuyển tiếp rỗng.

24.4 Enum cho trạng thái — hết "magic string"

Blog sắp có trạng thái bài viết: nháp, đã đăng, lưu trữ. Viết `status: string` rồi so sánh `post.status === 'publised'` (gõ sai) thì TypeScript không bắt được. Có 2 cách khai báo tập giá trị cố định trong TS:

```
export enum PostStatus {
  Draft = 'draft',
  Published = 'published',
  Archived = 'archived',
}

export const POST_STATUSES = ['draft', 'published', 'archived'] as const;
export type PostStatusValue = (typeof POST_STATUSES)[number];
```

	TS `enum` (string enum)	**String union + mảng `as const`**
Tồn tại lúc runtime	✅ Có object thật, truyền được vào `@IsEnum(PostStatus)` và `@Column({ enum: PostStatus })`	Chỉ có mảng `POST_STATUSES`. Type biến mất khi compile
Dùng với class-validator / TypeORM	✅ Trực tiếp	`@IsIn(POST_STATUSES)`, `enum: POST_STATUSES`
Gõ giá trị	`PostStatus.Published`	`'published'` (vẫn được TS kiểm tra)
Với `isolatedModules` (dự án đang bật)	✅ Enum thường OK. **Tránh** `const enum`	✅

Trong NestJS + TypeORM + class-validator, **string enum** là lựa chọn thuận nhất vì cả 3 thư viện đều nhận object enum. Luôn dùng **string** enum, không dùng numeric enum (`enum X { A, B }` lưu `0` / `1` xuống DB. Chèn thêm giá trị vào giữa là toàn bộ dữ liệu cũ đổi nghĩa).

```ts
@Column({ type: 'simple-enum', enum: PostStatus, default: PostStatus.Draft })
status: PostStatus;

export class UpdatePostStatusDto {
  @IsEnum(PostStatus)
  status: PostStatus;
}
```

Đã chạy thử trên typeorm 1.1.1 + better-sqlite3 12.11.1: `@Column({ type: 'enum', enum: [...] })` làm `DataSource.initialize()` văng ngay `DataTypeNotSupportedError: Data type "enum" in "...status" is not supported by "better-sqlite3" database`, vì `"enum"` không có trong `supportedDataTypes` của `AbstractSqliteDriver`. Phải dùng `type: 'simple-enum'`: driver đổi thành cột `varchar` kèm ràng buộc `CHECK( "status" IN ('draft','published','archived') )`. Đã thử lưu giá trị `'bogus'` thì bị DB từ chối (`SqliteError: CHECK constraint failed: status`). Khi chuyển sang Postgres (Part 25), đổi thành `type: 'enum'` để có enum type thật của Postgres. `@IsEnum(entity: object)` xác nhận trong class-validator 0.15.1.

⚠ **Thêm/bớt giá trị enum = đổi schema:** ràng buộc `CHECK` (SQLite) hoặc enum type (Postgres) nằm trong DB. Thêm `PostStatus.Scheduled` vào code mà không chạy migration (mục 24.6) thì DB vẫn từ chối giá trị mới. Trên SQLite, TypeORM còn phải tạo lại cả bảng (`temporary_post` → copy → drop → rename) để đổi CHECK.

24.5 Tổ chức module: feature folder, module dùng chung, vòng lặp phụ thuộc, dynamic module

Cấu trúc thư mục theo feature

```
src/
├── main.ts                     ← bootstrap (Part 01, 25)
├── app.module.ts               ← chỉ import module, không chứa provider nghiệp vụ
├── data-source.ts              ← DataSource cho TypeORM CLI (mục 24.6)
├── migrations/                 ← file migration sinh ra
├── config/                     ← cấu hình theo nhóm (Part 14)
├── common/                     ← thứ KHÔNG thuộc feature nào, không phụ thuộc feature nào
│   ├── dto/pagination-query.dto.ts
│   ├── filters/  interceptors/  middleware/  pipes/  decorators/
│   └── pagination.ts
├── posts/                      ← 1 feature = 1 thư mục = 1 module
│   ├── posts.module.ts
│   ├── posts.controller.ts
│   ├── posts.service.ts
│   ├── posts.repository.ts     ← chỉ khi cần (mục 24.3)
│   ├── post-status.enum.ts
│   ├── dto/  create-post.dto.ts  update-post.dto.ts
│   ├── entities/  post.entity.ts
│   └── posts.service.spec.ts
├── comments/  users/  auth/  products/  categories/
└── health/                     ← Part 25

Rất lớn (nhiều team) → monorepo mode: apps/api, apps/worker, libs/shared (Part 26)
```

⚠ **Bẫy có thật trong dự án:** các thư mục đang tên `enities/` (thiếu "t") và 2 file tên `user.enity.ts`, `comment.enity.ts`. Lúc dùng `autoLoadEntities` thì không sao, vì entity được nạp qua `forFeature()` chứ không qua tên file. Nhưng tới mục 24.6, DataSource cho CLI thường khai báo glob kiểu `'**/*.entity.{ts,js}'`, và glob đó **bỏ sót** `User` và `Comment` mà không báo lỗi gì. Migration sinh ra sẽ thiếu 2 bảng. Nên đổi tên cho đúng chuẩn (`entities/`, `*.entity.ts`) trước khi làm migration. IDE (F2 / Rename file) tự sửa các import.

Module dùng chung: `common/` không phải là "module rác"

`src/common` chứa thứ dùng ở mọi nơi nhưng **không phụ thuộc** feature nào: filter, interceptor, pipe, DTO phân trang. Phần lớn là class thường (được `new` trong `main.ts` hoặc import trực tiếp), không cần module. Chỉ khi có **provider** dùng chung (ví dụ `HashingService` bọc bcrypt, dùng ở cả `AuthModule` và `UsersModule`) thì mới tạo `SharedModule` với `exports`. `@Global()` chỉ dành cho hạ tầng thật sự toàn cục (Config, Logger, DB). `ConfigModule.forRoot({ isGlobal: true })` trong `app.module.ts` là ví dụ đúng. Lạm dụng `@Global()` cho module nghiệp vụ là xoá mất ranh giới đóng gói mà Part 00 đã nói tới.

Vòng lặp phụ thuộc (circular dependency) và `forwardRef`

Kịch bản điển hình: `UsersModule` import `PostsModule` (để đếm bài của user), rồi `PostsModule` lại import `UsersModule` (để lấy tác giả). Hai file import nhau, nên lúc file này được nạp thì class của file kia vẫn đang là `undefined`:

```
Nest cannot create the OrdersModuleX instance.
The module at index [0] of the OrdersModuleX "imports" array is undefined.
```

(Thông báo trên là lỗi thật, tái hiện bằng 2 module import vòng nhau trên Nest 11.2.3.) Tài liệu chính thức đưa ra `forwardRef()`, và phải đặt ở **cả 2 phía**:

```ts
@Module({ imports: [forwardRef(() => PostsModule)] })
export class UsersModule {}

@Injectable()
export class UsersService {
  constructor(
    @Inject(forwardRef(() => PostsService))
    private readonly postsService: PostsService,
  ) {}
}
```

> **Đối chiếu docs.nestjs.com/fundamentals/circular-dependency:** tài liệu nói vòng lặp "should be avoided where possible". Nó đưa ra `forwardRef()` (phải dùng ở cả hai phía) và cách thay thế là refactor sang `ModuleRef`. Có 3 cảnh báo: thứ tự khởi tạo *"is indeterminate"*; vòng lặp dính provider `Scope.REQUEST` *"can lead to undefined dependencies"*; và file barrel `index.ts` gom export là nguyên nhân hay gặp gây vòng lặp. `forwardRef(fn: () => any)` xác nhận trong `@nestjs/common` 11.2.3.

Vì sao cần tới `forwardRef` là "mùi thiết kế": nó nghĩa là 2 module biết quá nhiều về nhau, nên thực chất chúng không còn là 2 ranh giới riêng. `forwardRef` chữa được triệu chứng chứ không chữa nguyên nhân. Các cách chữa gốc, theo thứ tự nên thử:

Cách gỡ	Khi nào hợp

Tách phần dùng chung ra module thứ 3 mà cả 2 cùng import	Cả 2 cần cùng một logic (vd `PostStatsService` đếm bài)
Đảo chiều: chỉ 1 phía biết phía kia	Thường làm được. `PostsService` nhận `authorId` thay vì gọi `UsersService`
Dùng event (Part 19): A phát sự kiện, B lắng nghe	Phản ứng phụ ("user bị xoá thì ẩn bài"). A không cần biết B tồn tại
`forwardRef()`	Tạm thời, hoặc khi 2 phía thực sự phải gọi nhau đồng bộ. Ghi chú lý do ngay cạnh

⚠ **Bẫy barrel file:** tạo `src/posts/index.ts` kiểu `export * from './posts.service'; export * from './posts.module';` rồi import từ `'src/posts'` nghe gọn, nhưng dễ tạo vòng lặp ở cấp **file** dù về logic không có vòng lặp nào. Khi đó gặp đúng lỗi "... is undefined" ở trên. Trong dự án Nest, import thẳng từ file cụ thể.

Dynamic module: `forRoot` / `register` / `forFeature` (mức giới thiệu)

Bạn đã dùng dynamic module từ Part 08 mà có thể chưa để ý: `ConfigModule.forRoot()`, `TypeOrmModule.forRoot()` / `forFeature()`, `JwtModule.registerAsync()`. Đó là các module nhận **tham số** lúc import và trả về một cấu hình module (`DynamicModule`) tương ứng.

Tên method	Ý nghĩa (quy ước của Nest)	Ví dụ trong dự án
`forRoot()`	Cấu hình 1 lần ở root, dùng chung toàn app	`TypeOrmModule.forRoot`, `ConfigModule.forRoot`
`register()`	Mỗi nơi import tự cấu hình riêng	`JwtModule.register`
`forFeature()`	Dùng cấu hình của `forRoot` nhưng thêm phần riêng cho module này	`TypeOrmModule.forFeature([Post])`
`...Async()`	Như trên, nhưng giá trị lấy từ provider khác (thường là `ConfigService`) qua `useFactory` + `inject`	`JwtModule.registerAsync` trong `auth.module.ts`

Muốn tự viết module cấu hình được (vd `MailModule.forRootAsync(...)` ở Part 23), đừng viết tay `static forRoot()`. Dùng `ConfigurableModuleBuilder`, nó sinh sẵn cả bản sync lẫn async:

```
import { ConfigurableModuleBuilder } from '@nestjs/common';

export interface GreeterModuleOptions {
  greeting: string;
}

export const { ConfigurableModuleClass, MODULE_OPTIONS_TOKEN } =
  new ConfigurableModuleBuilder<GreeterModuleOptions>()
    .setClassMethodName('forRoot')
    .build();
```

```
import { Inject, Injectable, Module } from '@nestjs/common';
import { ConfigurableModuleClass, MODULE_OPTIONS_TOKEN } from './greeter.module-
definition';
import type { GreeterModuleOptions } from './greeter.module-definition';

@Injectable()
export class GreeterService {
  constructor(@Inject(MODULE_OPTIONS_TOKEN) private readonly options:
GreeterModuleOptions) {}
}

@Module({ providers: [GreeterService], exports: [GreeterService] })
export class GreeterModule extends ConfigurableModuleClass {}

GreeterModule.forRootAsync({
  inject: [ConfigService],
  useFactory: (config: ConfigService) => ({ greeting: config.get('GREETING') ?? 'Xin
chào' }),
});
```

> **Đã kiểm chứng trên @nestjs/common 11.2.3:** `ConfigurableModuleBuilder` có `setClassMethodName`, `setFactoryMethodName`, `setExtras`, `build()`. `build()` trả về `ConfigurableModuleClass`, `MODULE_OPTIONS_TOKEN`, `OPTIONS_TYPE`, `ASYNC_OPTIONS_TYPE`. Khớp docs.nestjs.com/fundamentals/dynamic-modules. Đoạn code trên đã chạy thật: `forRootAsync` tự sinh sau khi đổi tên thành `forRoot`, service in ra "Xin chào, Nest". **Bẫy phát hiện khi chạy:** dự án bật cùng lúc `isolatedModules` và `emitDecoratorMetadata`, nên import interface `GreeterModuleOptions` bằng `import { ... }` thường rồi dùng làm type tham số constructor có decorator sẽ báo `TS1272: A type referenced in a decorated signature must be imported with 'import type'`. Phải tách thành dòng `import type` riêng như trên.

24.6 `synchronize: true` hay Migration — 2 cách giữ schema DB khớp với Entity

`app.module.ts` đang có `synchronize: true`: mỗi lần app khởi động, TypeORM so sánh entity với DB rồi **tự sửa DB** cho khớp. Lúc học thì rất tiện, nhưng đổi tên 1 cột là TypeORM hiểu thành "xoá cột cũ, tạo cột mới", và dữ liệu cột cũ mất. Câu hỏi thực tế: **DB này có dữ liệu bạn không được phép mất không?**

Cách 1 — `synchronize: true`

Không có file nào ghi lại schema đã đổi thế nào. Không review được, không rollback được. Chạy ở mọi lần khởi động, kể cả production nếu quên tắt.

Cách 2 — Migration: mỗi thay đổi schema là 1 file có `up()` / `down()`, commit vào git

Bước 1: tạo file DataSource riêng cho CLI (CLI của TypeORM chạy **ngoài** Nest, nên không đọc được `TypeOrmModule.forRoot`):

```ts
import 'reflect-metadata';
import { DataSource, DataSourceOptions } from 'typeorm';

export const dataSourceOptions: DataSourceOptions = {
  type: 'better-sqlite3',
  database: process.env.DB_PATH ?? 'db.sqlite3',
  entities: [__dirname + '/**/*.entity.{ts,js}'],
  migrations: [__dirname + '/migrations/*.{ts,js}'],
  synchronize: false,
};

export default new DataSource(dataSourceOptions);
```

```ts
TypeOrmModule.forRoot({ ...dataSourceOptions, autoLoadEntities: true })
```

Bước 2: thêm script vào `package.json`. Các lệnh bên dưới đã được chạy thật trên một bản sao của dự án:

```json
"typeorm": "node -r ts-node/register -r tsconfig-paths/register
./node_modules/typeorm/cli.js",
"migration:generate": "npm run typeorm -- migration:generate -d src/data-source.ts",
"migration:create": "npm run typeorm -- migration:create",
"migration:run": "npm run typeorm -- migration:run -d src/data-source.ts",
"migration:revert": "npm run typeorm -- migration:revert -d src/data-source.ts",
"migration:show": "npm run typeorm -- migration:show -d src/data-source.ts"
```

```bash
npm run migration:generate -- src/migrations/Init
npm run migration:show
npm run migration:run
npm run migration:revert
npm run migration:create -- src/migrations/BackfillSlug
```

Lệnh 1 so sánh entity với DB và sinh file `src/migrations/<timestamp>-Init.ts` chứa đúng câu SQL cần chạy. Lệnh 2 in danh sách: `[ ]` là chưa chạy, `[X]` là đã chạy. Lệnh 3 chạy các migration chưa chạy, mặc định trong transaction, và ghi lại vào bảng `migrations`. Lệnh 4 hoàn tác migration gần nhất. Lệnh 5 tạo file rỗng để tự viết (vd backfill dữ liệu). Đọc file sinh ra **trước khi** run: trên SQLite, một thay đổi nhỏ cũng có thể thành "tạo `temporary_x`, copy, drop, rename".

Đã kiểm chứng trên typeorm 1.1.1 đang cài (chạy thật trên bản sao dự án, không đụng project gốc): (1) CLI có đúng các lệnh `migration:create <path>`, `migration:generate <path>`, `migration:run`, `migration:revert`, `migration:show`. `-d/--dataSource` là **bắt buộc** (`demandOption: true`) với run/revert/generate. `migration:create` thì không cần. (2) File DataSource phải export **đúng 1** instance `DataSource` (default hoặc named). Export thêm object `dataSourceOptions` thì không sao. (3) **Bẫy riêng của dự án này:** lệnh `typeorm-ts-node-commonjs` mà nhiều tutorial dùng báo `Error: Cannot find module 'src/products/enities/product.entity'`, vì code import kiểu `'src/...'` (nhờ `baseUrl`) mà ts-node không tự hiểu. Lệnh đó cũng không nhận thêm `-r`. Cách chạy được là `node -r ts-node/register -r tsconfig-paths/register ./node_modules/typeorm/cli.js` như script trên (ts-node 10.9.2, tsconfig-paths 4.2.0 đều đã có trong devDependencies). Cách khác là build trước rồi chạy trên JS: `npm run build && npx typeorm migration:run -d dist/data-source.js`. `nest build` đã tự đổi `'src/...'` thành đường dẫn tương đối trong `dist/`, cách này cũng đã chạy thử thành công. (4) `migration:generate ... --check` không ghi file. Đã chạy thử: entity lệch DB thì in "Unexpected changes in database schema were found in check mode" và thoát mã 1; khớp thì thoát mã 0. Rất hợp để đặt trong CI. (5) `EntityManager.withRepository()` có trong typeorm 1.1.1 (dùng cho bẫy transaction ở mục 24.3). Spread `{ ...dataSourceOptions, autoLoadEntities: true }` vào `TypeOrmModule.forRoot` đã typecheck sạch với @nestjs/typeorm 12.0.1. Đối chiếu docs.nestjs.com/data/typeorm: *"Don't use synchronize: true in production; otherwise, you can lose production data."* Nest không có CLI migration riêng, nó trỏ sang tài liệu migration của TypeORM.

⚠ Chuyển đổi DB đang có dữ liệu: file `db.sqlite3` hiện tại do `synchronize` tạo nên chưa có bảng `migrations`. Chạy `migration:generate` trên chính file đó sẽ ra migration rỗng ("No changes"), vì schema đã khớp. Cách làm sạch: trỏ `DB_PATH` sang một file mới, generate `Init`, run; rồi với DB cũ dùng `migration:run --fake` (đánh dấu đã chạy mà không chạy SQL). Cờ `--fake` có trong typeorm 1.1.1.

Tiêu chí	Cách 1: `synchronize: true`	Cách 2: Migration
Học, prototype, DB xoá đi tạo lại thoải mái	✅ Nhanh nhất, không file thừa	Hơi nặng nề
Đổi tên cột / đổi kiểu mà giữ dữ liệu	❌ Drop + tạo lại, mất dữ liệu	✅ Tự sửa file thành `RENAME COLUMN` hoặc viết backfill
Review thay đổi schema trong PR	❌ Không có gì để review	✅ File SQL nằm trong diff
Rollback khi deploy lỗi	❌ Không có	✅ `migration:revert`
Nhiều môi trường (dev/staging/prod) cùng version schema	❌ Mỗi nơi tự "đoán"	✅ Bảng `migrations` ghi rõ đã chạy tới đâu
Production	❌ Tài liệu chính thức cấm	✅ Bắt buộc

Quy tắc thực dụng: `synchronize: true` chỉ dùng khi DB là đồ bỏ đi (học, test e2e với DB trong bộ nhớ). Ngay khi có dữ liệu cần giữ, hoặc trước lần deploy đầu tiên (Part 25), chuyển hẳn sang migration và đặt `synchronize: false`. Không bật cả hai cùng lúc: `synchronize` sẽ âm thầm sửa DB trước, làm migration lệch.

24.7 Công cụ giữ chất lượng code

Công cụ	Làm gì	Trạng thái trong dự án	Lệnh

ESLint 9 + typescript-eslint	Bắt lỗi logic tĩnh: promise bị bỏ rơi, `any` không an toàn...	Đã có, dùng `recommendedTypeChecked`. `no-floating-promises` đang để `warn` (chính là bug quên `await` ở Part 08.3)	`npm run lint`
Prettier 3	Format thống nhất	Đã có (`singleQuote`, `trailingComma: all`)	`npm run format`
TypeScript `strict`	Bắt `null`/`undefined`, `any` ngầm	Mới bật `strictNullChecks`. `noImplicitAny: false`	`npx tsc --noEmit -p tsconfig.json --strict`
Kiểm tra migration trong CI	Chặn PR đổi entity mà quên migration	Chưa có	`npm run typeorm -- migration:generate src/migrations/Check -d src/data-source.ts --check`
husky 9 + lint-staged 17 (tuỳ chọn)	Chạy lint/format chỉ trên file đang commit	Chưa có	`npm i -D husky lint-staged` → `npx husky init`
Jest (Part 11)	Unit test service/repository, e2e	Đã có	`npm test`, `npm run test:e2e`

Đã đo trên bản sao dự án (TypeScript 5.9.3): `tsc --noEmit` với cấu hình hiện tại: 0 lỗi. Thêm `--strict`: **32 lỗi, tất cả cùng một mã TS2564** (`strictPropertyInitialization`: "Property 'email' has no initializer...") ở các field DTO/entity như `login.dto.ts`, `register.dto.ts`. Lý do là field được class-validator/TypeORM gán lúc runtime chứ không gán trong constructor. Cách xử lý phổ biến trong dự án Nest: bật `"strict": true` nhưng để `"strictPropertyInitialization": false`, hoặc thêm dấu `!` (`email!: string`). Phiên bản kiểm tra trên npm: husky 9.1.7, lint-staged 17.6.0, ESLint đang cài 9.39.5.

Việc cần làm:

1. Đổi tên thư mục `enities/` → `entities/` và 2 file `*.enity.ts` → `*.entity.ts` ở mọi module (dùng Rename của IDE). Chạy `npm run build` + `npm test` để chắc không gãy import.
2. Thêm `PostStatus` (string enum) và cột `status` (`simple-enum`, default `Draft`) vào `Post`. Thêm route `PATCH /posts/:id/status` với DTO `@IsEnum(PostStatus)`. Thử gửi `"status": "bogus"` và xác nhận nhận 400.
3. Tạo `src/data-source.ts` + các script migration ở mục 24.6. Trỏ `DB_PATH` sang file mới, `migration:generate -- src/migrations/Init`, đọc file SQL sinh ra, `migration:run`, `migration:show`. Sau đó đổi `synchronize` thành `false` trong `app.module.ts`.
4. Thêm 1 cột mới (vd `Post.slug`, nullable), generate migration thứ hai, chạy thử `migration:revert` rồi `run` lại. Chạy lệnh `--check` và xác nhận thoát mã 0.
5. Viết `toUserResponse()` (mục 24.2), dùng cho mọi route trả thông tin user. Kiểm tra bằng curl rằng không response nào còn field `password`.
6. Tìm query dài nhất trong `ProductsService`/`CategoriesService`. Tự trả lời theo bảng 24.3: có nên tách Custom Repository không? Nếu có, tách và viết lại unit test mock bằng class token.

7. Chạy `npx tsc --noEmit -p tsconfig.json --strict`, đọc 32 lỗi TS2564, quyết định giữa `strictPropertyInitialization: false` và dấu `!`. Sau đó đổi `no-floating-promises` từ `warn` thành `error` trong `eslint.config.mjs`.

PART 25

Production: build, deploy, quan sát hệ thống

Mục tiêu: Đưa blog API từ `npm run start:dev` lên một môi trường chạy thật. Build đúng cách, chọn DB phù hợp, dừng app không làm rơi request, có health check cho load balancer, log dạng JSON đọc được bằng máy, và đi hết checklist bảo mật. Worker queue và cron chạy thành process riêng.

Phạm vi phiên bản của Part này: dự án đang ở **NestJS 11** (@nestjs/common 11.2.3, CommonJS, Node 24). Từ 27/08/2026, `npm i @nestjs/xxx` không kèm version sẽ kéo **Nest 12** (tag `latest` = 12.1.1, ESM). Tag `legacy` = 11.2.6 mới là dòng 11. docs.nestjs.com hiện viết theo Nest 12, vì vậy mọi lệnh cài trong Part này đều đã kiểm tra `peerDependencies` khớp Nest 11. Chỗ nào docs có API chỉ Nest 12 mới có thì ghi rõ tại chỗ.

25.1 Từ source tới process: build, `NODE_ENV`, và vì sao production nên dùng Postgres

```
                ┌──────────── máy chủ / cluster ────────────────────────┐
Internet ──► Reverse proxy / LB ──► node dist/main   (API, N bản)      ← Part 00–17
 (HTTPS)     (Nginx, Caddy, ALB)    node dist/worker (xử lý queue)     ← Part 18
                                    scheduler (1 bản duy nhất!)        ← Part 21
                                    PostgreSQL  : dữ liệu
                                    Redis       : queue + cache + lock ← Part 18, 20
                └──────────────────────────────────────────────────────┘
Thiếu 1 mảnh: API vẫn trả 200 nhưng mail không gửi, cron không chạy → lỗi "âm thầm".
```

Việc	Development	Production
Chạy	`npm run start:dev` (`nest start --watch`: tự build + reload)	`node dist/main` (script `start:prod` đã có sẵn)
Build	Tự động	`npm run build` = `nest build` → thư mục `dist/` (`deleteOutDir: true` trong `nest-cli.json` xoá bản cũ trước)
Cài package	`npm install`	`npm ci` (cài đúng theo `package-lock.json`), sau khi build thì `npm prune --omit=dev`
`NODE_ENV`	không đặt / `development`	`production`: nhiều thư viện đổi hành vi theo biến này (Express cache view, mức log...)
Schema DB	`synchronize` hoặc migration	Chỉ migration: `npx typeorm migration:run -d dist/data-source.js` (Part 24.6)
Script phụ	`npm run seed`	`node dist/seed` (seed.ts cũng được build ra `dist/`)

⚠ **Đừng chạy `nest start` ở production:** `nest` là lệnh của `@nestjs/cli`, một **devDependency**. Sau `npm prune --omit=dev` lệnh đó không còn. Kể cả khi còn, nó compile TypeScript mỗi lần khởi động. Production chỉ chạy JS đã build: `node dist/main`.

> **Đối chiếu docs.nestjs.com/deployment:** build bằng `npm run build` (*"typically runs nest build under the hood"*), chạy `node dist/main.js`, đặt `NODE_ENV=production` vì *"some libraries in the ecosystem behave differently based on this variable"*. Trang này **không** nói về PM2, graceful shutdown hay `npm ci`. Các phần đó ở dưới được kiểm chứng từ nguồn khác và ghi rõ nguồn.

SQLite chạy tốt lúc học, nhưng production nên chuyển sang Postgres

SQLite là 1 file trên đĩa của **đúng một máy**. Có 3 giới hạn chạm vào ngay khi deploy thật:

- **Một người ghi tại một thời điểm:** sqlite.org viết *"SQLite supports an unlimited number of simultaneous readers, but it will only allow one writer at any instant in time."* Với 1 process thì ổn, vì mỗi lần ghi chỉ mất vài ms. Nhưng chạy PM2 cluster 8 process (mục 25.3) là 8 process tranh nhau lock của cùng 1 file.
- **Không chia sẻ được qua mạng:** scale ra 2 máy (hoặc 2 container ở 2 node) thì mỗi máy có file riêng, dữ liệu tách đôi. sqlite.org: *"Is the data separated from the application by a network? → choose client/server"*.
- **Container là đồ dùng một lần:** file DB nằm trong container sẽ mất khi container bị thay. Phải mount volume, và volume thì lại gắn với 1 máy.

```
npm i pg

export const dataSourceOptions: DataSourceOptions = {
  type: 'postgres',
  url: process.env.DATABASE_URL,
  entities: [__dirname + '/**/*.entity.{ts,js}'],
  migrations: [__dirname + '/migrations/*.{ts,js}'],
  synchronize: false,
};
```

Đổi DB thì migration cũ (sinh cho SQLite) không dùng lại được. Trên nhánh chuyển đổi, xoá `src/migrations`, generate lại `Init` trên một Postgres trống. Nhớ đổi `simple-enum` → `enum` (Part 24.4). Dữ liệu cũ thì export/import bằng script riêng.

> **Đã kiểm chứng trên typeorm 1.1.1:** `"postgres"` có trong `DatabaseType`. `PostgresConnectionOptions` có `url?: string` và `ssl?`. Driver nạp package `pg` lúc khởi tạo (`PlatformTools.load("pg")`), vì vậy phải cài `pg` (8.23.0 tại thời điểm kiểm tra), nếu không app văng lỗi thiếu driver ngay khi boot. Trích dẫn SQLite lấy từ sqlite.org/whentouse.html.

25.2 Graceful shutdown: `app.enableShutdownHooks()`

Khi deploy phiên bản mới, Docker/Kubernetes/PM2 gửi tín hiệu dừng (`SIGTERM`, PM2 thì gửi `SIGINT`) cho process cũ. Nếu app không bắt tín hiệu, nó bị **giết cứng** sau vài giây: request đang xử lý đứt giữa chừng, kết nối DB không đóng. NestJS mặc định **không** lắng nghe tín hiệu, phải bật:

```
const app = await NestFactory.create(AppModule);
app.enableShutdownHooks();
```

Vòng đời một provider/module có 5 hook. Hai hook đầu chạy lúc khởi động. Ba hook cuối chỉ chạy khi có tín hiệu dừng (cần `enableShutdownHooks()`) hoặc khi bạn tự gọi `app.close()`, như `seed.ts` đang làm trong `finally`:

Thứ tự	Hook	Gọi khi	Dùng để
1	`onModuleInit()`	Dependency của module đã resolve xong	Khởi tạo nhẹ (khởi động)
2	`onApplicationBootstrap()`	Mọi module đã init, trước khi `listen`	Warm-up cache, kiểm tra cấu hình
3	`onModuleDestroy()`	Sau khi nhận tín hiệu dừng	Ngừng nhận việc mới (dừng consumer queue)
4	`beforeApplicationShutdown(signal)`	Sau khi mọi `onModuleDestroy` xong	Chờ việc dang dở xong
5	`onApplicationShutdown(signal)`	Sau khi đóng kết nối, bước cuối của `app.close()`	Flush log, đóng tài nguyên còn lại

Đã chạy thử bằng Docker (image build từ Dockerfile ở mục 25.3, Nest 11.2.3): không có `enableShutdownHooks()` thì `docker stop` mất **10,8 giây** và container thoát mã **137** (bị SIGKILL sau 10s chờ mặc định của Docker). Lý do: `node` chạy làm PID 1 trong container, không có handler nào thì bỏ qua SIGTERM. **Có** `enableShutdownHooks()` thì `docker stop` mất **0,65 giây**, thoát mã **0**. Đối chiếu docs.nestjs.com/fundamentals/lifecycle-events: hook bị *"disabled by default"* vì listener tốn tài nguyên. Trên Windows *"SIGTERM never works"* (chỉ `SIGINT`, `SIGBREAK`). Hook **không** chạy cho provider request-scoped. Typings 11.2.3: `enableShutdownHooks(signals?, options?)`. Bỏ trống `signals` thì Nest nghe mọi giá trị của `ShutdownSignal` (có cả `SIGINT` PM2 dùng và `SIGTERM` Docker dùng). `options.useProcessExit` dùng khi logger bất đồng bộ (vd pino transport) cần `exit` event để flush.

⚠ **Dùng exec form trong Dockerfile:** `CMD ["node", "dist/main"]`, không dùng `CMD npm run start:prod`. Đã chạy thử với `npm run start:prod` làm PID 1: npm có chuyển tín hiệu cho node, app vẫn dừng, nhưng container thoát mã **1** (npm coi là lỗi), và có thêm 1 process trung gian không cần thiết.

25.3 PM2 hay Docker — 2 cách giữ process sống và chạy nhiều bản

`node dist/main` chết (crash, hết RAM) thì phải có thứ khởi động lại nó. Muốn tận dụng nhiều CPU thì phải chạy nhiều bản, vì Node chạy JS trên 1 thread. Có hai hướng. Câu hỏi quyết định là: **bạn deploy lên một VPS tự quản lý, hay lên nền tảng chạy container?**

Cách 1 — PM2 trên VPS (không container)

```
module.exports = {
  apps: [
    {
      name: 'api',
      script: 'dist/main.js',
      exec_mode: 'cluster',
      instances: 'max',
      kill_timeout: 5000,
      max_memory_restart: '500M',
      env_production: { NODE_ENV: 'production' },
    },
    {
      name: 'worker',
      script: 'dist/worker.js',
      exec_mode: 'fork',
      instances: 1,
      kill_timeout: 30000,
      env_production: { NODE_ENV: 'production' },
    },
  ],
};
```

```
npm i -g pm2
pm2 start ecosystem.config.js --env production
pm2 reload api
pm2 logs api
pm2 save && pm2 startup
```

Chế độ `cluster` dùng module `cluster` của Node: N process cùng nghe 1 cổng, PM2 chia request. `pm2 reload` thay từng process một, nên không có lúc nào app ngừng phục vụ. `pm2 startup` sinh lệnh để PM2 tự chạy lại khi máy khởi động lại.

> **Đã kiểm chứng trên mã nguồn pm2 7.0.4 (npm pack, chưa chạy thật trên máy):** `exec_mode` nhận `cluster|fork` (mặc định `fork`). `instances: 'max'` được đổi thành 0, và God.js đổi 0 thành số CPU. `kill_timeout` mặc định **1600 ms** (*"Time in ms before sending the final SIGKILL signal after SIGINT"*). Tức là PM2 gửi **SIGINT**, và nếu Nest dọn dẹp lâu hơn 1,6s thì bị giết, nên phải tăng lên như ví dụ. Cờ `--env <name>` chọn khối `env_<name>`. Có lệnh `reload` và `ecosystem` / `init` (sinh file mẫu).

> ⚠ **Cluster mode nhân mọi thứ lên N lần:** (1) **Cron** `@Cron()` (Part 21) nằm trong `api` sẽ chạy N lần mỗi lịch. Tách scheduler thành process riêng `instances: 1`, hoặc dùng lock (Part 20). (2) **SQLite**: N process cùng ghi 1 file → lỗi `SQLITE_BUSY` khi tải cao. Lý do thứ 2 để chuyển Postgres. (3) **State trong RAM** (cache in-memory, rate limit in-memory ở Part 16) không chia sẻ giữa các process. Cần Redis.

Cách 2 — Docker (multi-stage) + nền tảng container tự restart

```
FROM node:24-bookworm-slim AS build
WORKDIR /app
COPY package.json package-lock.json ./
RUN npm ci
COPY tsconfig.json tsconfig.build.json nest-cli.json ./
COPY src ./src
RUN npm run build && npm prune --omit=dev

FROM node:24-bookworm-slim
ENV NODE_ENV=production
WORKDIR /app
RUN mkdir -p /app/data && chown node:node /app/data
COPY --from=build /app/node_modules ./node_modules
COPY --from=build /app/dist ./dist
COPY package.json ./
USER node
ENV DB_PATH=/app/data/db.sqlite3
EXPOSE 3000
HEALTHCHECK --interval=10s --timeout=3s --retries=3 CMD node -e
"fetch('http://127.0.0.1:3000/health').then(r=>process.exit(r.ok?
0:1)).catch(()=>process.exit(1))"
CMD ["node", "dist/main"]
```

```
node_modules
dist
coverage
.env
*.sqlite3
.git
```

File thứ hai là `.dockerignore`. **Bắt buộc** có dòng `node_modules`. Stage `build` có đủ devDependencies để compile, sau đó `npm prune` bỏ chúng đi. Stage cuối chỉ copy `dist` + `node_modules` đã prune, chạy bằng user `node` (không phải root). Restart khi crash, chạy N bản, rolling update đều là việc của nền tảng: `restart: unless-stopped` trong docker compose, `replicas` trong Kubernetes/ECS/Fly/Railway.

⚠ **Native module: `better-sqlite3` và `bcrypt`** chứa file `.node` đã biên dịch cho **đúng hệ điều hành + kiến trúc CPU + phiên bản ABI của Node**. Vì thế: (1) không bao giờ copy `node_modules` từ máy bạn vào image (đó là lý do cho dòng `node_modules` trong `.dockerignore`). Phải `npm ci` **bên trong** image. (2) Stage build và stage chạy phải cùng base image (cùng glibc/musl, cùng major Node). (3) `better-sqlite3` cài bằng `prebuild-install || node-gyp rebuild`: nó tải file dựng sẵn từ GitHub, và nếu không có bản hợp (Node quá mới, kiến trúc lạ, build không có mạng) thì biên dịch tại chỗ. Lúc đó base image phải có `python3 make g++` (`apt-get install -y python3 make g++` ở stage build). Chuyển sang Postgres thì driver `pg` là JS thuần, bớt hẳn vấn đề này cho DB.

> **Đã build và chạy thật** Dockerfile trên (Docker 29.7.2) với một bản sao sạch của dự án. Thay đổi duy nhất là `database: process.env.DB_PATH ?? 'db.sqlite3'` để file DB nằm trong thư mục user `node` ghi được. Kết quả: image 488 MB, Node 24.21.0 trong image, `npm ci` tải được prebuilt cho cả 2 native module không cần công cụ build, container báo `healthy`, file DB tạo đúng ở `/app/data` với owner `node`. Lần chạy đó HEALTHCHECK trỏ vào `GET /` vì bản sao sạch chưa có `/health` (mục 25.4); cơ chế giống hệt. Image `-slim` không có `curl`, vì vậy health check dùng `fetch` có sẵn của Node. Kiểm tra release `v12.11.1` của better-sqlite3: có prebuilt `node-v137` (Node 24) cho `linux-x64/arm64`, cả glibc lẫn `linuxmusl`. `bcrypt` 6.0.0 mang sẵn `prebuilds/linux-x64/bcrypt.glibc.node` và `bcrypt.musl.node`. Dockerfile mẫu trên docs.nestjs.com/deployment là bản 1 stage (`FROM node:24`, `npm install`, `COPY . .`), chạy được nhưng image mang cả devDependencies + source TS, và `COPY . .` sẽ chép cả `node_modules` của máy bạn nếu thiếu `.dockerignore`.

Tiêu chí	Cách 1: PM2	Cách 2: Docker + orchestrator
1 VPS, tự SSH deploy, chưa biết container	✅ Cài 1 lệnh, học 5 lệnh	Phải học thêm Docker
Môi trường giống hệt nhau dev → CI → prod	❌ Phụ thuộc Node/OS của máy chủ	✅ Cùng 1 image
Dùng hết CPU trên 1 máy	✅ `cluster` + `instances: 'max'`	Chạy nhiều container / replicas
Scale ra nhiều máy, rolling update, tự thay node chết	❌ Tự lo	✅ Việc của Kubernetes/ECS/Fly...
Nền tảng PaaS (Railway, Fly, Cloud Run, Render...)	❌ Thừa (nền tảng đã giữ process)	✅ Hầu hết nhận Dockerfile trực tiếp
Native module (`better-sqlite3`, `bcrypt`)	Build trên chính máy chủ, ít rắc rối	Phải để ý base image + `.dockerignore`
Dùng chung	Không nên chạy PM2 cluster **bên trong** container: 2 lớp giám sát chồng nhau, orchestrator không thấy từng process chết. Nếu cần, dùng `pm2-runtime` (có trong pm2 7.0.4) ở chế độ 1 process.	

Quy tắc thực dụng: một VPS, một mình bạn, deploy bằng `git pull` thì dùng PM2 (Cách 1), nhớ `kill_timeout` và tách scheduler. Mọi trường hợp còn lại (có CI, có đồng đội, dùng PaaS/cloud, muốn scale nhiều máy) thì dùng Docker (Cách 2) và để nền tảng lo restart/scale. Với Docker: mỗi container 1 process Node, scale bằng số container.

25.4 Health check với `@nestjs/terminus`

Load balancer, Docker `HEALTHCHECK` và Kubernetes probe cần một URL để hỏi "instance này còn phục vụ được không?". Trả 200 cứng từ `GET /` là chưa đủ, vì app còn sống nhưng mất kết nối DB thì vẫn trả 200. Terminus kiểm tra thật từng phụ thuộc.

```
npm i @nestjs/terminus@12.1.0
```

```ts
import { Controller, Get } from '@nestjs/common';
import {
  HealthCheck,
  HealthCheckService,
  MemoryHealthIndicator,
  TypeOrmHealthIndicator,
} from '@nestjs/terminus';

@Controller('health')
export class HealthController {
  constructor(
    private readonly health: HealthCheckService,
    private readonly db: TypeOrmHealthIndicator,
    private readonly memory: MemoryHealthIndicator,
  ) {}

  @Get()
  @HealthCheck()
  check() {
    return this.health.check([
      () => this.db.pingCheck('database'),
      () => this.memory.checkHeap('memory_heap', 300 * 1024 * 1024),
    ]);
  }
}

@Module({
  imports: [TerminusModule.forRoot({ gracefulShutdownTimeoutMs: 1000 })],
  controllers: [HealthController],
})
export class HealthModule {}
```

```
{"success":true,"data":{"status":"ok","info":{"database":
{"responseTime":0,"status":"up"},"memory_heap":{"status":"up"}},"error":{},"details":
{...}}}
```

`gracefulShutdownTimeoutMs` gắn với mục 25.2. Khi nhận SIGTERM, Terminus cho `/health` trả **503** ngay, chờ 1 giây để load balancer kịp rút instance khỏi danh sách, rồi mới cho app đóng. Request mới không bị đẩy vào một instance sắp chết.

Chọn version là chỗ dễ sai nhất ở đây. Đã chạy thử cả hai trên bản sao dự án: (1) `@nestjs/terminus@11.1.1` (dòng "11", CommonJS): app boot bình thường nhưng `GET /health` trả **500** với `ERR_PACKAGE_PATH_NOT_EXPORTED: Package subpath './dist/common/typeorm.utils' is not defined by "exports"`. Nguyên nhân: `TypeOrmHealthIndicator` của bản 11 `require` sâu vào `@nestjs/typeorm/dist/...`, mà `@nestjs/typeorm` **12.0.1** (bản dự án đang cài) đã khoá lại bằng `exports`. (2) `@nestjs/terminus@12.1.0`: peer `@nestjs/common ^11.0.0 || ^12.0.0`, `@nestjs/typeorm ^11 || ^12`. Package là ESM (`"type": "module"`, engines Node `^20.19 || ^22.12 || >=24`) nhưng app CommonJS vẫn `require` được nhờ `require(esm)` của Node 24. Chạy đúng: `/health` trả 200 như trên. Sau SIGTERM, `/health` trả 503 và log ghi `"Awaiting 1000ms before shutdown"` → `"Timeout reached, shutting down now"`. Typings 12.1.0: `pingCheck(key, { timeout? })` trả về `HealthCheckAttempt` (có thêm `.withTimeout(ms)`). `MemoryHealthIndicator.checkHeap(key, bytes)`. `TypeOrmHealthIndicator` ping bằng `SELECT 1`. Tự viết indicator thì dùng `HealthIndicatorService`. Class `HealthIndicator` cũ đã `@deprecated` ở 11.x và bị xoá ở 12 (docs.nestjs.com/migration-guide).

⚠ Health check đi qua `TransformInterceptor` và `HttpExceptionFilter` của bạn: response 200 bị bọc thành `{ success, data }` (xem JSON ở trên). Load balancer chỉ nhìn status code nên không sao. Nhưng khi DB chết, Terminus ném `ServiceUnavailableException` kèm chi tiết từng indicator, và filter Part 06 rút gọn nó thành `"message": "Service Unavailable Exception"`. Chi tiết "cái gì down" bị mất (đã thấy khi chạy thử). Nếu cần giữ chi tiết đó, cho filter trả nguyên `payload` khi nó là object có field `status` / `details`, hoặc xem log. Ngoài ra nên loại `/health` khỏi log request (mục 25.5), vì LB gọi vài giây một lần.

25.5 Logging: Logger có sẵn, log JSON, và log lỗi từ Exception Filter

Ba mức, từ đơn giản đến đầy đủ:

Mức 1 — `Logger` của Nest thay cho `console.log`

```
@Injectable()
export class PostsService {
  private readonly logger = new Logger(PostsService.name);

  async remove(id: number) {
    ...
    this.logger.log(`Post ${id} deleted`);
  }
}
```

Mỗi dòng log có `context` (tên class), có level, và đi qua đúng logger mà app đang dùng. Nhờ vậy khi đổi sang JSON (mức 2, 3) thì mọi log đổi theo. `console.log` thì không đổi theo.

Mức 2 — Log JSON bằng `ConsoleLogger` (Nest 11, không cần cài gì)

```
const app = await NestFactory.create(AppModule, {
  logger: new ConsoleLogger({ json: process.env.NODE_ENV === 'production' }),
});
```

```
{"level":"log","pid":467616,"timestamp":1790757164681,"message":"Starting Nest application...","context":"NestFactory"}
```

Mức 3 — `nestjs-pino`: log mỗi request kèm `req.id`, ẩn dữ liệu nhạy cảm

```
npm i nestjs-pino pino-http
```

```
LoggerModule.forRoot({
  pinoHttp: {
    level: process.env.NODE_ENV === 'production' ? 'info' : 'debug',
    redact: ['req.headers.authorization'],
  },
  exclude: [{ method: RequestMethod.ALL, path: 'health' }],
}),

const app = await NestFactory.create(AppModule, { bufferLogs: true });
app.useLogger(app.get(Logger));
```

`Logger` ở dòng cuối import từ `'nestjs-pino'`, không phải `@nestjs/common`. `bufferLogs` giữ log lúc khởi động lại cho tới khi pino sẵn sàng. Sau đó `new Logger(PostsService.name)` trong code của bạn **không cần sửa**: nó tự đi qua pino và còn được gắn thêm thông tin request đang xử lý.

> **Đã chạy thử cả mức 2 và mức 3 trên bản sao dự án (Nest 11.2.3):** `ConsoleLogger({ json: true })` in đúng một object JSON mỗi dòng như trên (option `json` có trong `console-logger.service.d.ts` 11.2.3, docs.nestjs.com: bật `json` thì `colors` mặc định `false`). `nestjs-pino` 5.2.1 (peer `@nestjs/common ^11.0.8 || ^12.0.2`, `pino ^10`, `pino-http ^11`; có bản CommonJS; engines Node ≥ 22.12) với pino 10.3.1 + pino-http 11.0.0: mỗi request có 1 dòng JSON kèm `req.id`. Gửi `Authorization: Bearer secret-token-123` thì log ghi `"authorization":"[Redacted]"` và chuỗi token không xuất hiện ở đâu cả. `/health` không bị log. Dòng `this.logger.warn(...)` từ filter ra JSON có `"context":"HttpExceptionFilter"` + object `req` của request đó. Cách cài `npm i nestjs-pino pino-http` lấy từ README của package. `pino` là peer, npm 7+ tự cài.

> ⚠ **Hai chỗ log tự viết không đổi theo:** (1) `LoggerMiddleware` (Part 07) dùng `console.log`, nên khi chạy thử ở mức 2 và 3 vẫn in dòng thường `GET /posts/999 404 - 10ms` xen giữa các dòng JSON, và công cụ gom log sẽ không parse được. Đổi sang `new Logger('HTTP')`. Khi đã dùng pino-http thì xoá hẳn middleware này, vì pino-http đã log mỗi request kèm status + thời gian (giữ cả hai là log trùng). (2) `console.error` trong `bootstrap().catch` thì giữ lại được, vì lúc đó logger có thể chưa tồn tại.

Log lỗi ở đâu: Exception Filter (Part 06)

Lỗi **không phải** `HttpException` (lỗi code, lỗi TypeORM...) không đi qua `HttpExceptionFilter` của bạn (nó chỉ `@Catch(HttpException)`). Nest có sẵn `BaseExceptionFilter` xử lý chúng: trả `{"statusCode":500,"message":"Internal server error"}` (không lộ stack) và **tự log** kèm stack với context `ExceptionsHandler`. Đã thấy đúng như vậy khi chạy thử: log JSON có `"level":"error"`, `"context":"ExceptionsHandler"`. Phần còn thiếu là log cho các `HttpException` mà filter của bạn bắt:

```
@Catch(HttpException)
export class HttpExceptionFilter implements ExceptionFilter {
  private readonly logger = new Logger(HttpExceptionFilter.name);

  catch(exception: HttpException, host: ArgumentsHost) {
    ...
    if (status >= 500) {
      this.logger.error(`${request.method} ${request.url} → ${status}`,
exception.stack);
    } else {
      this.logger.warn(`${request.method} ${request.url} → ${status}:
${String(message)}`);
    }
    response.status(status).json({ success: false, ... });
  }
}
```

Quy tắc từ docs.nestjs.com/deployment: *"Log errors, not exceptions"*, *"Avoid sensitive data"* (không log password, token, nên mới cần `redact`), dùng correlation ID (`req.id` của pino), và tắt debug/verbose ở production. 4xx là lỗi của client: log `warn`, không kèm stack. 5xx là lỗi của bạn: log `error`, kèm stack. Tầng tiếp theo (tracing, metrics) xem Part 26. Docs Nest 12 còn giới thiệu nền tảng chính chủ `@nestjs/observe` (npm 0.3.3). Chưa kiểm tra với Nest 11.

25.6 Checklist bảo mật & cấu hình trước khi mở cổng

Hạng mục	Làm gì	Trạng thái trong dự án / ghi chú
Security headers	`npm i helmet` → `app.use(helmet())` trong `main.ts`, **trước** mọi route/middleware khác	Chưa có. Đã chạy thử helmet 8.3.0: response có `Content-Security-Policy`, `Strict-Transport-Security`, `X-Content-Type-Options: nosniff`, và mất header `X-Powered-By`. `app.useSecurityHeaders()` trên docs là API của **Nest 12.1**, không có trong 11.2.3
CORS	`app.enableCors({ origin: ['https://blog.example.com'] })`. Whitelist đọc từ env, không dùng `origin: '*'` khi có cookie/credentials	Chưa bật. Đã chạy thử: preflight từ origin trong whitelist nhận `Access-Control-Allow-Origin`, origin lạ không nhận header nào nên trình duyệt chặn
Sau reverse proxy	`NestFactory.create<NestExpressApplication>(...)` → `app.set('trust proxy', 1)`	Thiếu dòng này thì `req.ip` là IP của proxy, và rate limit (Part 16) sẽ chặn nhầm mọi người cùng lúc
Rate limit	`@nestjs/throttler`	Part 16. Chạy nhiều instance thì cần storage Redis

Secrets	Mọi secret đọc từ env / secret manager, fail-fast khi thiếu	✅ `JWT_SECRET` đã dùng `config.getOrThrow`. `.env` đã trong `.gitignore`. Validate env ở Part 14
Input	`ValidationPipe({ whitelist: true, forbidNonWhitelisted: true, transform: true })`	`main.ts` đã có `whitelist` + `forbidNonWhitelisted` nhưng **thiếu** `transform: true`, cái mà Part 05/12 yêu cầu để `@Type(() => Number)` và default `page = 1` có hiệu lực
Chi tiết lỗi	Không trả stack/SQL cho client	✅ Lỗi 500 lạ đi qua `BaseExceptionFilter` chỉ trả `"Internal server error"`. Đừng để filter tự viết đưa `exception.message` của lỗi DB ra ngoài. Muốn ẩn luôn chi tiết lỗi validate: `ValidationPipe({ disableErrorMessages: true })` (có trong 11.2.3, nhưng thường chỉ API public mới cần)
Response nhạy cảm	Không trả `password` hash	Part 24.2 + Part 15
Schema	`synchronize: false`, chạy migration khi deploy	Part 24.6
Process	Chạy bằng user không phải root, `NODE_ENV=production`, `enableShutdownHooks()`	Dockerfile mục 25.3 đã có `USER node`
Dependency	`npm audit --omit=dev` trong CI. Cập nhật patch định kỳ	Theo dõi cả thông báo bảo mật của Nest và TypeORM
Backup	Backup DB tự động và **thử restore** định kỳ	docs.nestjs.com/deployment: *"Regularly back up your data"*

25.7 Worker queue và scheduler: tách thành process riêng

Queue worker (Part 18) và cron (Part 21) nếu chạy chung process với API thì: job nặng làm chậm request, scale API lên 4 bản thì cron chạy 4 lần, và deploy API là cắt ngang job đang chạy. Cách tách dùng đúng kỹ thuật mà `src/seed.ts` đã dùng: **standalone application** (`createApplicationContext`), có đủ DI và module nhưng không mở cổng HTTP.

```
import { NestFactory } from '@nestjs/core';
import { WorkerModule } from './worker.module';

async function bootstrap() {
  const app = await NestFactory.createApplicationContext(WorkerModule);
  app.enableShutdownHooks();
}
bootstrap();
```

`WorkerModule` chỉ import những gì worker cần (TypeORM, module chứa processor/cron, Config), không import controller. Build ra `dist/worker.js`. Dev: `nest start --entryFile worker` (giống script `seed`). Prod: `node dist/worker`. Docker: **cùng một image**, khác `command`:

```
services:
  api:
    image: blog-api
    command: ["node", "dist/main"]
    deploy: { replicas: 3 }
  worker:
    image: blog-api
    command: ["node", "dist/worker"]
    stop_grace_period: 60s
  scheduler:
    image: blog-api
    command: ["node", "dist/scheduler"]
    deploy: { replicas: 1 }
```

Đã chạy thử `node dist/worker` (context không có consumer nào): process in "Worker started..." rồi **tự thoát ngay với mã 0**, vì trong event loop không còn gì giữ nó sống (better-sqlite3 là đồng bộ, không mở socket). Đây là hành vi đúng của Node chứ không phải lỗi. Worker thật sống được là nhờ kết nối Redis mà BullMQ worker giữ mở (Part 18). Scheduler sống được nhờ timer của `@nestjs/schedule` (Part 21). Đừng thêm `setInterval` "giữ process" để chữa triệu chứng này. Nếu worker thoát ngay, nghĩa là consumer chưa được đăng ký.

⚠ **Thời gian chờ khi dừng worker:** job dài (xuất file, gửi 1000 mail) cần thời gian để kết thúc hoặc trả job về queue. Đặt `stop_grace_period` (compose), `terminationGracePeriodSeconds` (Kubernetes) hoặc `kill_timeout` (PM2) **lớn hơn** thời gian chạy của job dài nhất. Nếu không, job bị giết giữa chừng mỗi lần deploy. Cùng ý tưởng với `stopwaitsecs` của Supervisor bên Laravel.

Việc cần làm:

1. Sửa `main.ts`: thêm `app.enableShutdownHooks()`, `transform: true` cho `ValidationPipe`, `app.use(helmet())`, `app.enableCors({ origin: ... })` đọc whitelist từ env. Kiểm tra header bằng `curl -I localhost:3000/posts`.
2. Đổi `database` thành `process.env.DB_PATH ?? 'db.sqlite3'`. Viết `Dockerfile` + `.dockerignore` theo mục 25.3, `docker build`, chạy với `-e JWT_SECRET=... -v blogdata:/app/data`. Đo thời gian `docker stop` khi có và khi không có `enableShutdownHooks()`.
3. Cài `@nestjs/terminus@12.1.0`, tạo `HealthModule` (DB ping + heap). Trỏ `HEALTHCHECK` của Dockerfile vào `/health`, xác nhận `docker ps` báo `healthy`.
4. Bật log JSON khi `NODE_ENV=production` (mức 2, hoặc mức 3 với nestjs-pino + `redact` header Authorization). Đổi `LoggerMiddleware` sang `Logger` (hoặc xoá nếu dùng pino-http). Thêm log `warn` / `error` vào `HttpExceptionFilter`.
5. Viết `docker-compose.yml` có `api` + `postgres`. Chuyển `data-source.ts` sang `type: 'postgres'` + `DATABASE_URL`, generate lại migration `Init`, chạy `npx typeorm migration:run -d dist/data-source.js` trong container trước khi start API.
6. Tạo `src/worker.ts` + `WorkerModule` (sẽ chứa processor của Part 18). Thêm service `worker` vào compose dùng cùng image. Chạy thử và giải thích vì sao nó thoát ngay khi chưa có consumer.

7. Viết mục "Vận hành" trong README: các process cần chạy, biến môi trường bắt buộc, lệnh deploy (build → migrate → restart), xem log ở đâu, URL health check.

PART 26 — LỘ TRÌNH

Lộ trình master NestJS — làm gì tiếp theo

Mục tiêu: Xong Part P–25 là bạn đã qua mức "biết dùng". Phần này là bản đồ để đi tiếp tới mức tự thiết kế được hệ thống nhiều dịch vụ, đọc được mã nguồn framework, và tự quyết định công nghệ thay vì làm theo tutorial.

26.1 Ba tầng năng lực

Tầng	Bạn làm được gì	Tài liệu này phủ
1. Dùng được	CRUD API có module, DTO + validation, filter/guard/interceptor, TypeORM + quan hệ, JWT auth, phân trang, transaction, test cơ bản	Part P, 00–13
2. Làm thật được	Config theo môi trường, serialization, API production (rate limit, CORS, versioning, Swagger), phân quyền, queue, event, cache + lock, cron, upload, mail; kiến trúc code khi app lớn; build, deploy, health check, log	Part 14–25
3. Master	Chia hệ thống thành nhiều dịch vụ và chọn transport; tối ưu ở tầng DB; tracing xuyên dịch vụ; đọc & mở rộng mã nguồn Nest (viết dynamic module/thư viện riêng); nâng cấp major version không sợ; mentor người khác	Lộ trình dưới đây

26.2 Chủ đề tiếp theo, theo thứ tự nên học

Kiểm tra trước khi liệt kê (30/09/2026): mọi chủ đề "Docs" bên dưới đều có trang trên docs.nestjs.com (đối chiếu với cây nội dung của repo `nestjs/docs.nestjs.com`). Hai chủ đề không thuộc Nest (Testcontainers, OpenTelemetry) ghi nguồn ngoài. **Cạm bẫy phiên bản:** docs hiện viết cho **Nest 12**. Nhiều package companion bản `latest` đã **đòi** Nest 12: `@nestjs/cqrs` 12.1.0 peer `@nestjs/common ^12.0.0`, `@nestjs/graphql` 14.0.3, `@nestjs/microservices` / `websockets` / `platform-fastify` 12.1.1 cũng vậy. Đã chạy thử `npm i @nestjs/cqrs --package-lock-only` trên bản sao `package.json` + lock của dự án: `ERESOLVE unable to resolve dependency tree ... Found: @nestjs/common@11.2.6`. Còn `npm i @nestjs/cqrs@11` thì resolve bình thường (ra `^11.0.3`). Hoặc cài đúng dòng 11 (`@nestjs/cqrs@11`, `@nestjs/microservices@11`, `@nestjs/websockets@11 @nestjs/platform-socket.io@11`, `@nestjs/graphql@13 @nestjs/apollo@13`, đã kiểm tra peer khớp `^11`), hoặc làm chủ đề số 1 trước.

#	Chủ đề	Vì sao	Docs / nguồn
1	**Nâng cấp lên Nest 12** bằng `nest upgrade`	Bài tập "nâng major version" thật, lại đúng lúc (Nest 12 ra 27/08/2026). Xem chi tiết bên dưới bảng	/migration-guide

2	SQL & hiệu năng DB: index, `EXPLAIN`, N+1, QueryBuilder nâng cao, cursor pagination	Phần lớn vấn đề hiệu năng thật nằm ở DB chứ không ở Node. Postgres (Part 25) mở ra `EXPLAIN ANALYZE`, index một phần, JSONB	typeorm.io (select-query-builder, indices), tài liệu PostgreSQL
3	Testing sâu: e2e trên DB thật bằng **Testcontainers**	Mock repository (Part 11) không bắt được lỗi SQL, migration sai hay ràng buộc unique. Testcontainers bật 1 Postgres thật trong Docker cho mỗi lần chạy test	/fundamentals/unit-testing; node.testcontainers.org (`@testcontainers/postgresql` 12.2.0)
4	AsyncLocalStorage: request context xuyên các lớp	Lấy `userId` / `requestId` ở tầng repository/log mà không phải truyền tham số qua mọi hàm, và không phải dùng provider `Scope.REQUEST` (chậm, Part 00)	/recipes/async-local-storage
5	WebSockets / Gateways	Realtime: comment mới hiện ngay, thông báo. `@WebSocketGateway` dùng lại được Guard/Pipe/Filter đã học	/websockets/gateways
6	Microservices & transports (TCP, Redis, NATS, RabbitMQ, Kafka, gRPC, MQTT)	Tách dịch vụ, giao tiếp bằng message/event. Hiểu khi nào **không nên** tách (đa số app nhỏ và vừa)	/microservices/basics và từng trang transport
7	CQRS module (CommandBus, QueryBus, EventBus, Saga)	Bước tiếp theo của "Service quá to" (Part 24.2): mỗi use case là 1 command/query + handler. Hợp với domain phức tạp, thừa với CRUD	/recipes/cqrs
8	GraphQL (code-first / schema-first, resolver, DataLoader)	Client cần chọn field linh hoạt, gom nhiều nguồn dữ liệu. Phải tự xử lý N+1 bằng DataLoader	/graphql/quick-start
9	ORM khác: Prisma, Drizzle, MikroORM	So sánh để hiểu TypeORM mạnh/yếu ở đâu. Docs Prisma hiện dùng `prisma@7` + `@prisma/adapter-better-sqlite3` (chạy được ngay trên SQLite). `@nestjs/drizzle` mới ở bản 0.0.1, còn rất sớm	/data/prisma, /data/drizzle, /data/mikroorm
10	Fastify adapter	Nhanh hơn Express khi cần throughput cao. Cái giá: middleware Express không dùng được. CORS mặc định chỉ cho `GET,HEAD,POST` (docs cảnh báo PUT/PATCH/DELETE cross-origin bị chặn ở preflight nếu không khai `methods`)	/http/performance, /security/cors

11	Monorepo mode (apps + libs)	Tách `apps/api`, `apps/worker` (Part 25.7) và `libs/shared`. `nest generate app` / `nest generate library` tự chuyển dự án sang monorepo	/cli/workspaces
12	Observability: tracing + metrics bằng **OpenTelemetry**	Log (Part 25) cho biết "có lỗi". Trace cho biết request chậm ở bước nào, qua những dịch vụ nào. OpenTelemetry là chuẩn mở, gửi được tới Jaeger/Tempo/Datadog...	opentelemetry.io (JS: `@opentelemetry/sdk-node` 0.222.0, `auto-instrumentations-node` 0.80.0). Nest 12 có thêm SDK chính chủ `@nestjs/observe`
13	Tự viết thư viện/dynamic module có test, publish lên npm	Hiểu Nest từ phía "người mở rộng" (`ConfigurableModuleBuilder`, Part 24.5)	/fundamentals/dynamic-modules

Nâng cấp lên Nest 12: những gì thật sự đổi (theo migration guide chính thức)

Thay đổi	Ảnh hưởng tới dự án này
Các package `@nestjs/*` v12 chỉ còn **ESM**	App CommonJS **vẫn chạy**, nhờ `require(esm)` (Node ≥ 20.19 / 22.12). Chuyển code của bạn sang ESM là **tuỳ chọn**, không bắt buộc. `nest upgrade` cố ý không đụng module format. Đừng copy đuôi `.js` trong import hay `await bootstrap()` top-level từ docs mới vào dự án CommonJS
Node: chạy app cần ≥ 20.19 hoặc ≥ 22.12. CLI (`nest new/generate/upgrade`) cần ≥ 22.22.3 / 24.15 / 26	Máy đang dùng Node 24.16: đủ
`nest upgrade`: nâng mọi `@nestjs/*` cùng lúc, nâng TypeScript lên v6, sửa phần cơ học, in báo cáo. Có `--dry-run`	Nâng CLI global trước, chạy `nest upgrade --dry-run`, đọc báo cáo rồi mới chạy thật
Jest chỉ load được package v12 trên Node ≥ 24.9. Dự án ESM mới mặc định dùng Vitest	Test hiện tại (Jest 30) vẫn chạy trên Node 24.16
Terminus bỏ API `HealthIndicator` cũ, chỉ còn `HealthIndicatorService`	Part 25 đã dùng terminus 12 và indicator có sẵn, không bị ảnh hưởng
`@nestjs/config`: validate env chuyển từ Joi sang Standard Schema (Zod, Valibot, ArkType...)	Xem lại phần validate env của Part 14
Lifecycle hook gọi theo cấp của cây component, thứ tự có thể đổi	Kiểm tra lại nếu code dựa vào thứ tự `onModuleInit`/shutdown giữa các provider
`@Optional()` không còn được kế thừa sang class con	Class con phải tự khai lại constructor + `@Optional()`. Nếu không, lỗi `UnknownDependenciesException`
class-validator/class-transformer **vẫn được hỗ trợ**. Có thêm `StandardSchemaValidationPipe`	DTO của bạn giữ nguyên

> **Nguồn:** docs.nestjs.com/migration-guide (bản "11 → 12", tải trực tiếp nội dung trang ngày 30/09/2026). Ngày phát hành và dist-tag lấy từ `npm view @nestjs/common time dist-tags`: 12.0.0 phát hành 27/08/2026, `latest` = 12.1.1, `legacy` = 11.2.6. Phần nâng cấp chưa được chạy thử trên dự án. Đây là bài tập để bạn tự làm trên một nhánh riêng.

26.3 Dự án nên tự xây để lên tầng 3

1. **Blog API hoàn chỉnh** (chính dự án này): tag (ManyToMany), full-text search trên Postgres, comment realtime qua WebSocket gateway, soft delete + restore (`@DeleteDateColumn`), API v2 bằng versioning (Part 16). Toàn bộ chạy bằng migration, có e2e test với Testcontainers và CI.
2. **Hệ thống đặt hàng** từ module `Products` / `Categories` có sẵn: giỏ hàng, đặt hàng trong transaction (Part 13), trừ tồn kho có lock chống bán vượt (Part 20), gửi mail hoá đơn qua queue (Part 18, 23), báo cáo doanh thu bằng cron (Part 21). Đây là bài luyện transaction + lock + queue nặng nhất.
3. **Tách 1 dịch vụ ra microservice**: đưa phần gửi thông báo thành dịch vụ riêng, nhận event qua Redis hoặc RabbitMQ transport, trong monorepo mode (`apps/api`, `apps/notifier`, `libs/contracts`). Đo và viết ra: tách xong được gì, mất gì (độ phức tạp deploy, debug xuyên dịch vụ, cần tracing).
4. **Một thư viện npm**, ví dụ `nest-audit-log`: dynamic module `AuditLogModule.forRootAsync()` viết bằng `ConfigurableModuleBuilder`, một TypeORM subscriber ghi lại mọi thay đổi entity, có unit test + e2e test với app mẫu, publish lên npm, hỗ trợ cả Nest 11 và 12 qua peerDependencies.

26.4 Thói quen của người master

- **Đọc mã nguồn framework khi thắc mắc.** Mọi "phép thuật" của Nest là JavaScript bình thường trong `node_modules/@nestjs/core/`. Nên đọc theo thứ tự: `scanner.js` (đọc metadata `@Module`, dựng cây module, Part 00.2) → `injector/instance-loader.js` + `injector/injector.js` (resolve dependency; chỗ ném `UnknownDependenciesException`, còn câu chữ "Nest can't resolve dependencies..." nằm ở `errors/messages.js`) → `router/router-explorer.js` (map `@Get()` thành route Express) → `exceptions/base-exception-filter.js` (filter mặc định trả "Internal server error" và log context `ExceptionsHandler`, Part 25.5) → `nest-application.js`. Đọc thêm `node_modules/typeorm/driver/sqlite-abstract/` sẽ hiểu vì sao `enum` không chạy trên SQLite (Part 24.4).
- **Kiểm chứng thay vì tin tutorial.** Cả tài liệu này được xây theo nguyên tắc đó: `@EntityRepository`, `loadRelationCountAndMap`, driver `sqlite`, `relations: ['x']` đều từng là "cách chuẩn" trên mạng và đều đã bị bỏ. Trước khi dùng API lạ: đọc file `.d.ts` trong `node_modules`, `npm view <pkg> peerDependencies`, và chạy thử 10 dòng.
- **Đo trước khi tối ưu:** log có thời gian (Part 25) → tìm route chậm → xem query (`logging: true` của TypeORM) → `EXPLAIN` → index → rồi mới tới cache (Part 20).
- **Test là mặc định:** service/guard/pipe tự viết đều có unit test. Luồng quan trọng có e2e. CI chạy `npm run lint`, `tsc --noEmit`, `npm test`, kiểm tra migration `--check` (Part 24.7).
- **Theo dõi release:** trang Releases của `nestjs/nest` trên GitHub, migration guide mỗi major, changelog của TypeORM. Đọc PR lớn để biết một tính năng mới hình thành thế nào.
- **Viết lại tài liệu này bằng lời của bạn** sau khi xong. Dạy lại được là cách kiểm tra hiểu chắc nhất.

26.5 Checklist tự đánh giá "đã master chưa"

Câu hỏi	Trả lời được không?

Vẽ lại đường đi của 1 request qua Middleware → Guard → Interceptor → Pipe → Handler → Interceptor, và giải thích vì sao nhánh lỗi không đi qua `map()` của Interceptor?	
"Nest can't resolve dependencies of X (?, Y)": dấu `?` nghĩa là gì, và 3 nguyên nhân hay gặp nhất?	
Provider singleton, `REQUEST`, `TRANSIENT` khác nhau thế nào, và vì sao `Scope.REQUEST` "lan" lên cả chuỗi phụ thuộc?	
Vì sao `forwardRef` là dấu hiệu thiết kế có vấn đề, và 3 cách gỡ vòng lặp phụ thuộc mà không cần nó?	
Trong `dataSource.transaction()`, gọi `this.repo.save()` thay vì `manager.save()` thì chuyện gì xảy ra? Custom repository thì xử lý thế nào?	
Vì sao `synchronize: true` bị cấm ở production, và chuyển một DB đang có dữ liệu sang migration như thế nào (`--fake`)?	
Không có `enableShutdownHooks()` thì `docker stop` mất bao lâu và thoát mã gì? Vì sao?	
Chạy PM2 cluster 4 process: cron chạy mấy lần, rate limit in-memory còn đúng không, SQLite có vấn đề gì?	
Guard và Interceptor đều có `ExecutionContext`. Khi nào dùng cái nào? Phân quyền "chỉ tác giả được sửa bài" đặt ở Guard hay Service?	
Nest 12 là ESM-only: app CommonJS của bạn có buộc phải chuyển sang ESM không? Vì sao `npm i @nestjs/cqrs` báo lỗi trên dự án Nest 11?	

Trả lời trôi chảy cả 10 câu bằng lời của bạn, kèm ví dụ từ chính dự án blog đã làm: bạn đã ở tầng 3.

Việc cần làm:

1. Tự trả lời 10 câu ở 26.5 thành văn bản (mỗi câu 3–5 dòng, có ví dụ từ dự án). Câu nào không trả lời được thì quay lại đúng Part tương ứng.
2. Chọn 1 trong 4 dự án ở 26.3, viết 1 trang thiết kế: module nào, entity nào, process nào (Part 25.1), rủi ro lớn nhất. Sau đó mới bắt đầu code.
3. Tạo nhánh `upgrade-nest-12`, chạy `nest upgrade --dry-run`, đọc báo cáo, đối chiếu với bảng ở 26.2. Chạy thật, rồi `npm run build && npm test && npm run test:e2e`. Ghi lại những gì phải sửa tay.
4. Đọc `node_modules/@nestjs/core/injector/injector.js`, tìm chỗ ném `UnknownDependenciesException`, lần sang `errors/messages.js` để thấy câu "can't resolve dependencies" được dựng ra thế nào, rồi cố tình gây lỗi đó trong dự án (bỏ 1 provider khỏi `providers`) để thấy code và thông báo khớp nhau.
5. Chọn 1 chủ đề ở 26.2 (gợi ý: Testcontainers hoặc WebSocket gateway), làm một bản nhỏ trên dự án, nhớ kiểm tra `peerDependencies` trước khi `npm i`.

PART 27 — ĐI LÀM

Refresh token xoay vòng, cookie HttpOnly

> **Mục tiêu:** Hoàn thiện đăng nhập của Part 10: access token ngắn hạn, refresh token an toàn, phát hiện token bị đánh cắp, đăng xuất thật sự.

27.1 Vấn đề của một token duy nhất

Part 10 cấp một JWT sống 1 giờ. Đặt ngắn thì người dùng liên tục phải đăng nhập lại. Đặt dài thì lộ token là mất quyền kiểm soát, vì JWT không thu hồi được trước khi hết hạn.

	Access token	**Refresh token**
Dạng	JWT, tự chứa thông tin	Chuỗi ngẫu nhiên, lưu **hash** trong database
Thời hạn	15 phút	30 ngày
Gửi đi	Header `Authorization`, mọi request	Cookie `HttpOnly`, chỉ các route `/auth`
Thu hồi được	Không, chờ hết hạn	Có, sửa database

Xoay vòng (rotation): mỗi lần refresh thì token cũ bị vô hiệu và cấp token mới. **Phát hiện dùng lại:** token đã dùng mà vẫn được gửi lại nghĩa là có hai bên đang giữ token, nên thu hồi cả "họ" token của phiên đó, buộc đăng nhập lại.

27.2 Entity

```
@Entity()
export class RefreshToken {
  @PrimaryGeneratedColumn()
  id: number;

  @Column()
  userId: number;

  @Index({ unique: true })
  @Column()
  tokenHash: string; // chỉ lưu hash: lộ database cũng không dùng được token

  @Index()
  @Column()
  familyId: string; // các token sinh ra từ cùng một lần đăng nhập

  @Column({ type: 'datetime' })
  expiresAt: Date;

  @Column({ type: 'datetime', nullable: true })
  revokedAt: Date | null;

  @CreateDateColumn()
  createdAt: Date;
}
```

Kiểu `datetime` chỉ chạy trên SQLite. Trên Postgres, TypeORM báo lỗi ngay khi khởi động: `Data type "datetime" ... is not supported`. Sang Part 28 phải đổi thành `timestamptz`.

27.3 Service

```
@Injectable()
export class RefreshTokenService {
  constructor(
    @InjectRepository(RefreshToken)
    private readonly tokens: Repository<RefreshToken>,
  ) {}

  // Token ngẫu nhiên 256 bit nên SHA-256 là đủ. bcrypt chỉ cần cho mật khẩu do người
đặt
  private hash(raw: string): string {
    return createHash('sha256').update(raw).digest('hex');
  }

  async issue(
    userId: number,
    familyId: string = randomUUID(),
  ): Promise<string> {
    const raw = randomBytes(32).toString('base64url');
    await this.tokens.save(
      this.tokens.create({
        userId,
        familyId,
        tokenHash: this.hash(raw),
        expiresAt: new Date(Date.now() + REFRESH_TTL_MS),
        revokedAt: null,
      }),
    );
    return raw; // chỉ lần này mới thấy được token gốc
  }

  async rotate(raw: string): Promise<{ userId: number; token: string }> {
    const found = await this.tokens.findOneBy({ tokenHash: this.hash(raw) });
    if (!found || found.expiresAt < new Date()) {
      throw new UnauthorizedException('Refresh token không hợp lệ');
    }

    // UPDATE có điều kiện: hai request cùng lúc thì chỉ một request giành được token
    const claimed = await this.tokens.update(
      { id: found.id, revokedAt: IsNull() },
      { revokedAt: new Date() },
    );

    if (claimed.affected !== 1) {
      // Token đã dùng rồi mà vẫn bị gửi lại: nghi bị đánh cắp, thu hồi cả họ
      await this.revokeFamily(found.familyId);
      throw new UnauthorizedException('Refresh token đã bị sử dụng lại');
    }

    return {
      userId: found.userId,
      token: await this.issue(found.userId, found.familyId),
```

```
    };
  }

  async revoke(raw: string): Promise<void> {
    const found = await this.tokens.findOneBy({ tokenHash: this.hash(raw) });
    if (found) await this.revokeFamily(found.familyId);
  }

  private async revokeFamily(familyId: string): Promise<void> {
    await this.tokens.update(
      { familyId, revokedAt: IsNull() },
      { revokedAt: new Date() },
    );
  }
}
```

Vì sao không dùng bcrypt: bcrypt cố tình chậm để chống dò mật khẩu yếu do người đặt. Token ở đây là 256 bit ngẫu nhiên, không thể dò, nên SHA-256 là đủ và nhanh hơn nhiều.

27.4 AuthService và Controller

```
async login(dto: LoginDto) {
  const user = await this.usersService.findByEmail(dto.email);
  if (!user || !(await bcrypt.compare(dto.password, user.password))) {
    throw new UnauthorizedException('Invalid credentials');
  }
  return {
    access_token: await this.signAccessToken(user.id, user.email),
    refresh_token: await this.refreshTokens.issue(user.id),
  };
}

async refresh(raw: string) {
  const { userId, token } = await this.refreshTokens.rotate(raw);
  const user = await this.usersService.findById(userId);
  if (!user) throw new UnauthorizedException();
  return {
    access_token: await this.signAccessToken(user.id, user.email),
    refresh_token: token,
  };
}

logout(raw: string) {
  return this.refreshTokens.revoke(raw);
}

private signAccessToken(id: number, email: string) {
  return this.jwtService.signAsync({ sub: id, email });
}
```

```ts
const COOKIE = 'refresh_token';
const cookieOptions: CookieOptions = {
  httpOnly: true, // JavaScript không đọc được: XSS không lấy được token
  secure: process.env.NODE_ENV === 'production', // chỉ gửi qua HTTPS
  sameSite: 'lax', // trang khác không tự gửi được bằng POST
  path: process.env.AUTH_COOKIE_PATH ?? '/auth', // phải khớp URL trình duyệt nhìn thấy
  maxAge: REFRESH_TTL_MS,
};

@Controller('auth')
export class AuthController {
  constructor(private readonly authService: AuthService) {}

  @Post('login')
  @HttpCode(200)
  async login(
    @Body() dto: LoginDto,
    @Res({ passthrough: true }) res: Response,
  ) {
    const { access_token, refresh_token } = await this.authService.login(dto);
    res.cookie(COOKIE, refresh_token, cookieOptions);
    return { access_token }; // refresh token không nằm trong body
  }

  @Post('refresh')
  @HttpCode(200)
  async refresh(
    @Req() req: Request,
    @Res({ passthrough: true }) res: Response,
  ) {
    const raw = req.cookies?.[COOKIE] as string | undefined;
    if (!raw) throw new UnauthorizedException();
    const { access_token, refresh_token } = await this.authService.refresh(raw);
    res.cookie(COOKIE, refresh_token, cookieOptions);
    return { access_token };
  }

  @Post('logout')
  @HttpCode(204)
  async logout(@Req() req: Request, @Res({ passthrough: true }) res: Response) {
    const raw = req.cookies?.[COOKIE] as string | undefined;
    if (raw) await this.authService.logout(raw);
    res.clearCookie(COOKIE, { ...cookieOptions, maxAge: undefined });
  }
```

```bash
npm i cookie-parser
npm i -D @types/cookie-parser
```

```ts
// main.ts
import cookieParser from 'cookie-parser';
app.use(cookieParser());
```

Trong `AuthModule`: đổi `expiresIn` thành `'15m'`, thêm `TypeOrmModule.forFeature([RefreshToken])` và `RefreshTokenService` vào `providers`.

> **`passthrough: true` là bắt buộc.** Thiếu nó, Nest coi như bạn tự gửi response: `return` không còn tác dụng và `TransformInterceptor` không chạy.

27.5 Đặt `Path` của cookie cho đúng

`Path` phải khớp với URL **trình duyệt nhìn thấy**, không phải URL server nhận. Nếu frontend gọi `/api/auth/refresh` qua proxy, hoặc app dùng `setGlobalPrefix('api')`, thì `Path` phải là `/api/auth`.

> **Đã chạy bằng trình duyệt thật qua proxy Vite (xem React Part 17):** để sai `Path=/auth` thì đăng nhập vẫn thành công, nhưng F5 không gửi cookie, `refresh` trả 401. Để `Path=/api/auth` thì F5 vẫn giữ được phiên.
>
> **Đã chạy test e2e:** cookie có `HttpOnly`, `Path`, `SameSite=Lax`; database không chứa token gốc; dùng lại token cũ thì trả 401 và token mới nhất cũng bị thu hồi; hai request cùng token chỉ một cái thành công; đăng xuất thì không refresh được nữa; token hết hạn trả 401.

> **Đánh đổi của phát hiện dùng lại:** nếu người dùng mở hai tab và cả hai cùng refresh, tab thua sẽ bị đăng xuất. Phía client đã giảm rủi ro bằng cách chỉ refresh một lần trong mỗi tab (React Part 17). Một số hệ thống cho thêm khoảng ân hạn vài giây để xử lý trường hợp nhiều tab.

> **Việc cần làm:**
>
> 1. Thêm entity, service, sửa `AuthService` và `AuthController`.
> 2. Viết `test/auth-refresh.e2e-spec.ts`: login, refresh, dùng lại token cũ, logout, token hết hạn.
> 3. Kiểm tra bảng `refresh_token`: không có token gốc, thấy rõ các dòng cùng `familyId`.
> 4. Thêm lệnh xoá các token đã hết hạn chạy theo lịch (Part 21).
> 5. Thêm `@nestjs/throttler` cho `/auth/login` và `/auth/refresh`.
> 6. Nối với frontend ở React Part 17.

PART 28 — ĐI LÀM

Chuyển sang Postgres và chống race condition

> **Mục tiêu:** Biết vì sao code "đúng" vẫn bán lố hàng, và chọn đúng một trong ba cách xử lý. SQLite không làm được phần này, nên phải dùng database thật.

28.1 Chuyển từ SQLite sang Postgres

Khoá dòng cần một database cho nhiều kết nối ghi đồng thời. Công việc thật cũng thường dùng Postgres hoặc MySQL.

```
docker run -d --name shop-pg -e POSTGRES_PASSWORD=pw -e POSTGRES_DB=shop -p 5432:5432
postgres:17-alpine
npm i pg
```

```
TypeOrmModule.forRootAsync({
  inject: [ConfigService],
  useFactory: (config: ConfigService) => ({
    type: 'postgres',
    url: config.getOrThrow<string>('DATABASE_URL'),
    autoLoadEntities: true,
    synchronize: false, // production dùng migration (Part 25), không để tự sửa schema
  }),
});
```

Điểm khác so với SQLite	Cần làm
Không có kiểu `datetime`	Đổi thành `timestamptz`, kể cả entity ở Part 27
Kiểm tra kiểu và khoá ngoại chặt hơn	Sửa các chỗ trước đây "vẫn chạy"
Tên bảng `user` là từ khoá	TypeORM tự đặt trong ngoặc kép. Viết SQL tay phải nhớ, hoặc đổi tên bảng thành `users`
Có `FOR UPDATE` thật	Dùng được phần dưới đây

28.2 Vấn đề: đọc rồi mới ghi

```
// SAI: nhiều request cùng đọc stock = 1, cùng thấy còn hàng, cùng trừ
async buyUnsafe(id: number): Promise<void> {
  const repo = this.dataSource.getRepository(ProductStock);
  const product = await repo.findOneByOrFail({ id });
  if (product.stock < 1) throw new ConflictException('Hết hàng');
  product.stock -= 1;
  await repo.save(product);
}
```

Dù Node đơn luồng, giữa hai `await` các request vẫn xen vào nhau (chương 7 tài liệu JS). Nhiều request cùng đọc `stock = 5`, cùng thấy còn hàng, cùng ghi.

28.3 Ba cách đúng

Cách 1: một câu lệnh atomic. Nên nghĩ đến đầu tiên.

```ts
// Một câu lệnh duy nhất: kiểm tra và trừ cùng lúc. Ưu tiên cách này khi đủ dùng
async buyAtomic(id: number): Promise<void> {
  const result = await this.dataSource
    .createQueryBuilder()
    .update(ProductStock)
    .set({ stock: () => 'stock - 1' })
    .where('id = :id AND stock > 0', { id })
    .execute();
  if (result.affected !== 1) throw new ConflictException('Hết hàng');
}
```

Cách 2: khoá dòng (pessimistic lock). Khi cần đọc giá trị rồi xử lý logic phức tạp trước khi ghi.

```ts
// Khoá dòng: request khác phải chờ đến khi transaction này kết thúc
async buyPessimistic(id: number): Promise<void> {
  await this.dataSource.transaction(async (manager) => {
    const product = await manager.findOneOrFail(ProductStock, {
      where: { id },
      lock: { mode: 'pessimistic_write' }, // SELECT ... FOR UPDATE
    });
    if (product.stock < 1) throw new ConflictException('Hết hàng');
    product.stock -= 1;
    await manager.save(product);
  });
}
```

Cách 3: kiểm tra version (optimistic lock). Khi ít xung đột và không muốn giữ khoá lâu, ví dụ sửa form lâu rồi mới lưu.

```ts
// Không khoá: cập nhật kèm điều kiện version, thua thì đọc lại và thử lại
async buyOptimistic(id: number, maxRetries = 50): Promise<void> {
  const repo = this.dataSource.getRepository(ProductStock);
  for (let attempt = 0; attempt < maxRetries; attempt++) {
    const product = await repo.findOneByOrFail({ id });
    if (product.stock < 1) throw new ConflictException('Hết hàng');

    const result = await repo
      .createQueryBuilder()
      .update()
      .set({ stock: product.stock - 1, version: () => 'version + 1' })
      .where('id = :id AND version = :version', {
        id,
        version: product.version,
      })
      .execute();
    if (result.affected === 1) return;
  }
  throw new ConflictException('Thử lại quá nhiều lần');
}
```

	Atomic	Pessimistic	Optimistic
Cơ chế	Một câu `UPDATE` có điều kiện	`SELECT ... FOR UPDATE`	`UPDATE ... WHERE version = ?`
Khi xung đột	Database tự xếp hàng	Request sau chờ	Thua thì thử lại hoặc báo lỗi
Logic phức tạp	Khó	Dễ	Dễ
Rủi ro	Ít	Deadlock, giữ khoá lâu	Thử lại nhiều khi xung đột cao
Hợp với	Trừ kho, cộng tiền, đếm	Chuyển khoản, nhiều bước	Sửa dữ liệu ít xung đột

Đã chạy thật trên Postgres 17: 20 request mua cùng lúc, kho có 5 sản phẩm.

Cách	Bán được	Tồn kho cuối
Không khoá	**20**	3 (sai cả hai số)
Atomic	5	0
Pessimistic	5	0
Optimistic	5	0

Chạy lại 3 lần, các cách đúng đều cho cùng kết quả. Với bản không khoá, con số cụ thể có thể khác giữa các lần chạy, nhưng vẫn bán lố.

Bẫy với `@VersionColumn`: chỉ thêm cột này thì `save()` **không** chặn được ghi đè. Đã thử: hai bên cùng đọc rồi cùng `save()`, không có lỗi nào, kho chỉ giảm 1 dù trừ 2 lần, và `version` còn bị đặt lại giá trị cũ. Phải tự viết `UPDATE ... WHERE version = ?` như trên.

Trong transaction, mọi truy vấn phải đi qua `manager` được truyền vào. Dùng `this.repo` là chạy trên kết nối khác, nằm ngoài transaction và không có khoá. Luôn khoá theo cùng một thứ tự để tránh deadlock, và giữ transaction thật ngắn: không gọi API bên ngoài trong đó.

Việc cần làm:

1. Chạy Postgres bằng Docker, chuyển dự án sang Postgres, sửa `timestamptz`, tạo migration.
2. Viết `StockService` với cả 4 cách, và test cho 20 request đồng thời mua 5 sản phẩm bằng `Promise.allSettled`.
3. Tự chứng minh bản không khoá bị bán lố.
4. Viết API `POST /orders` có nhiều bước (trừ kho, tạo đơn, ghi log) trong một transaction có khoá dòng.
5. Tự tạo deadlock bằng hai transaction khoá hai dòng theo thứ tự ngược nhau, đọc lỗi, rồi sửa bằng cách khoá theo thứ tự `id`.

PART 29 — ĐI LÀM

WebSocket với gateway và xác thực

Mục tiêu: Gửi thông báo realtime đến đúng người dùng, biết vì sao guard HTTP không tự bảo vệ kết nối.

29.1 Khi nào cần

	HTTP polling	SSE	WebSocket
Chiều dữ liệu	Client hỏi	Server gửi xuống	Hai chiều
Hợp với	Cập nhật chậm, đơn giản	Thông báo, tiến độ	Chat, cộng tác, game

```
# Dùng đúng major của @nestjs/core trong dự án (hiện là 11)
npm i @nestjs/websockets@11 @nestjs/platform-socket.io@11 socket.io
```

29.2 Gateway

```
@WebSocketGateway({
  namespace: 'notifications',
  cors: { origin: ['http://localhost:5173'], credentials: true },
})
export class NotificationsGateway implements OnGatewayConnection {
  private readonly logger = new Logger(NotificationsGateway.name);

  @WebSocketServer()
  private readonly server: Server;

  constructor(private readonly jwtService: JwtService) {}

  // Guard HTTP không chạy lúc bắt tay kết nối, nên phải tự xác thực ở đây
  async handleConnection(client: AppSocket) {
    try {
      const token = client.handshake.auth.token as string;
      const payload = await this.jwtService.verifyAsync<{ sub: number }>(token);
      client.data.userId = payload.sub;
      await client.join(`user:${payload.sub}`); // mỗi user một phòng, nhiều tab vẫn
nhận đủ
      client.emit('ready', { userId: payload.sub });
    } catch {
      this.logger.warn('Kết nối WebSocket bị từ chối');
      client.disconnect(true);
    }
  }

  @SubscribeMessage('ping')
  ping(
    @ConnectedSocket() client: AppSocket,
    @MessageBody() body: { n: number },
  ) {
    return { n: body.n, userId: client.data.userId }; // trả về như ack
  }

  // Service khác inject gateway rồi gọi hàm này
  notifyUser(userId: number, event: string, payload: unknown) {
    this.server.to(`user:${userId}`).emit(event, payload);
  }
}
```

Gateway là provider, nên đăng ký trong `providers` của module và import `JwtModule`. Service khác inject `NotificationsGateway` rồi gọi `notifyUser`.

Những điểm cần nhớ:

- **Xác thực lúc kết nối:** `handleConnection` chạy khi bắt tay, phải tự kiểm tra token và ngắt nếu sai.
- **Room:** mỗi user một room `user:{id}`, nên một người mở nhiều tab đều nhận được, và người khác thì không.
- **Token hết hạn:** kết nối đã mở vẫn tiếp tục sống sau khi token hết hạn. Dữ liệu nhạy cảm cần có cơ chế ngắt hoặc yêu cầu xác thực lại.

- **Nhiều server:** room chỉ tồn tại trong một process. Chạy từ hai instance trở lên phải có Redis adapter.

29.3 Client

```
import { io } from 'socket.io-client';

const socket = io('http://localhost:3000/notifications', {
  auth: { token: accessToken }, // server đọc ở client.handshake.auth.token
});
socket.on('order', (payload) => console.log(payload));
```

Trong React, tạo kết nối trong `useEffect` và **phải** `socket.disconnect()` ở cleanup, nếu không mỗi lần render lại có thể mở thêm một kết nối.

> **Đã chạy test e2e với client Socket.IO thật:** cùng một user mở hai kết nối đều nhận được, user khác không nhận được, ack của `ping` trả đúng `userId`, token sai bị ngắt kết nối.

> **Việc cần làm:**
>
> 1. Tạo gateway, xác thực bằng JWT, join room theo user.
> 2. Khi có comment mới trên bài của ai đó, gửi thông báo cho đúng người đó.
> 3. Viết test e2e với `socket.io-client`: đúng người nhận, sai người không nhận, token sai bị ngắt.
> 4. Hiển thị thông báo trong React, kiểm tra không rò rỉ kết nối khi chuyển trang.
> 5. Đọc về Redis adapter và viết ra vì sao cần nó khi có hai instance.

PART 30 — ĐI LÀM

CI với GitHub Actions và sửa bộ test

Mục tiêu: Mỗi lần push tự chạy lint, kiểm tra kiểu, test, build. Đồng thời sửa bộ test của dự án đang lỗi.

30.1 Bộ test của dự án đang lỗi: sửa trước

Đã chạy bản sao dự án của bạn:

Lệnh	Node 20.20	Node 24.22
`npm test`	**Lỗi**: `Must use import to load ES Module` tại `@nestjs/typeorm`	Qua (4 test)
`npm run test:e2e`	**Lỗi**	**Lỗi**: không tìm thấy module `src/...`
`npm run build`	Qua	Qua

Máy hiện dùng Node 20. `@nestjs/typeorm` 12 đã là ESM, và cơ chế nạp của Jest không xử lý được trên Node 20.

Sửa 1: dùng Node 24 LTS trở lên, và ghi vào `package.json` để không ai bị nhầm:

```
{ "engines": { "node": ">=24" } }
```

Sửa 2: thêm dòng ánh xạ `src/` vào `test/jest-e2e.json`:

```
{
  "moduleFileExtensions": ["js", "json", "ts"],
  "rootDir": ".",
  "testEnvironment": "node",
  "testRegex": ".e2e-spec.ts$",
  "transform": { "^.+\\.(t|j)s$": "ts-jest" },
  "moduleNameMapper": { "^src/(.*)$": "<rootDir>/../src/$1" }
}
```

30.2 Workflow

File đặt ở thư mục gốc repo: `.github/workflows/ci.yml`.

```yaml
name: NestJS CI

on:
  push:
    branches: [main]
  pull_request:

jobs:
  test:
    runs-on: ubuntu-latest
    defaults:
      run:
        working-directory: nestjs-crash-course
    services:
      postgres:
        image: postgres:17-alpine
        env:
          POSTGRES_PASSWORD: pw
          POSTGRES_DB: shop_test
        ports:
          - 5432:5432
        options: >-
          --health-cmd "pg_isready -U postgres"
          --health-interval 5s
          --health-timeout 5s
          --health-retries 10
    env:
      JWT_SECRET: ci-only-secret
      DATABASE_URL: postgres://postgres:pw@localhost:5432/shop_test
    steps:
      - uses: actions/checkout@v5
      - uses: actions/setup-node@v5
        with:
          node-version: 24
          cache: npm
          cache-dependency-path: nestjs-crash-course/package-lock.json
      - run: npm ci
      - run: npx eslint "{src,test}/**/*.ts" --max-warnings=0
      - run: npx tsc --noEmit
      - run: npm test
      - run: npm run test:e2e
      - run: npm run build
```

- `npx eslint` chạy trực tiếp, vì script `npm run lint` có `--fix`: trong CI nó sẽ tự sửa rồi báo qua, che mất lỗi.
- `--max-warnings=0` coi cảnh báo là lỗi. Hiện tại dự án còn đúng một cảnh báo `no-floating-promises` ở `src/seed.ts`, phải sửa trước (việc cần làm số 4 của Part P), nếu không CI sẽ đỏ.
- `services` tạo Postgres thật cho test. Thiết lập `health-cmd` để đợi database sẵn sàng.
- **Secret thật** đưa vào GitHub Secrets, không ghi vào file. `JWT_SECRET` ở đây chỉ dùng cho test.

Đã kiểm tra: file này qua `actionlint`, và các bước `npm ci`, `tsc --noEmit`, `npm test`, `test:e2e`, `build` đã chạy được trên Node 24. Riêng file chưa chạy trên máy chủ GitHub thật, vì repo chưa push workflow này: lần đầu push hãy xem tab Actions.

30.3 Từ CI sang CD

Khi CI đã xanh, thêm job deploy chạy với `needs: test` và chỉ khi ở nhánh `main`: build Docker image (Part 25), đẩy lên registry, cập nhật server. Bảo vệ nhánh `main` để phải qua CI mới merge được.

Việc cần làm:

1. Đổi sang Node 24, ghi `engines`. Sửa `jest-e2e.json`. Chạy `npm test` và `npm run test:e2e` đều xanh.
2. Sửa cảnh báo ở `seed.ts`.
3. Thêm workflow, push lên nhánh mới, tạo Pull Request, xem CI chạy.
4. Cố tình làm hỏng một test, xác nhận CI đỏ và PR hiện cảnh báo.
5. Bật Branch protection cho `main`, yêu cầu CI phải qua.

PART 31 — ĐI LÀM

Prisma: đọc hiểu và so sánh với TypeORM

Mục tiêu: Biết Prisma khác gì để đọc được code và trả lời phỏng vấn. Phần này chỉ để đọc hiểu.

Phần này không có code chạy thử. Tại thời điểm kiểm tra (11/2026), tag `latest` của `prisma` trên npm đang là một bản `8.0.0-rc`, chưa phải bản phát hành ổn định, nên mình không đưa cú pháp cụ thể. Khi cần dùng, hãy đọc docs của đúng bản bạn cài.

	TypeORM	Prisma
Nguồn của sự thật	Class Entity có decorator	File `schema.prisma`
Kiểu trong TypeScript	Theo class bạn viết, quan hệ chưa load vẫn có kiểu	Sinh tự động, kiểu kết quả thay đổi theo những gì bạn truy vấn
Migration	Sinh từ Entity	Sinh từ schema, rất rõ ràng
Truy vấn phức tạp	QueryBuilder	API riêng hoặc SQL thuần
Mẫu thiết kế	Active Record hoặc Data Mapper	Client sinh tự động
Dùng trong Nest	Module chính thức `@nestjs/typeorm`	Tự viết `PrismaService`
Khoá dòng	Có `lock`	Phải dùng SQL thuần

Khái niệm không đổi dù dùng ORM nào: N+1, transaction, index, migration, khoá dòng. Nắm các khái niệm này thì đổi ORM chỉ mất vài ngày.

> **Việc cần làm (tuỳ chọn):**
>
> 1. Đọc trang Getting Started của Prisma, làm một bảng nhỏ trong project riêng.
> 2. Viết ra 3 điểm bạn thích và không thích so với TypeORM. Đây là câu trả lời tốt khi phỏng vấn.